Set aside

These passages were taken out of *The Thiele Machine: No Free Insight* and are kept here unchanged. Numbered theorems and sections cited below and not found here are the book's.

Contents

1 Proofs for "What it takes to be certified" 5

2 Proofs and worked runs for "Parts I and II, run on the example machine" 9

3 Proofs, the program and the worked tables for "The universal machines" 14

4 Running it 28

4.1 Taking the coat off 29

4.2 A second machine, written by hand, to check the first 29

4.3 *U* on a real guest 30

4.4 From source programs to the host 32

4.5 What wasn't built 38

5 Constructing the quoting operation 39

5.1 Numerals and syntax as data 39

5.2 Recursion without running under a lambda 40

5.3 Quoting numerals 41

5.4 Quoting terms 41

6 Computing with counters 43
6.1 Finite programs made from two instructions 43
6.2 A tape in finitely many counters 44
6.3 Packing finitely many counters into two 45
6.4 Why halting has no decider 46
6.5 Seven descriptions of the same computation 47
6.6 Searching for a finite computation 50

7 The universal program in blocks 51
7.1 Arithmetic blocks 51
7.2 Selection, comparison and fetching 52
7.3 Registers and instruction codes 53
7.4 The fifteen blocks of U 54
7.5 Why the blocks simulate a guest 55
7.6 Adding PAY 56

8 The CHSH derivation, in full 58
8.1 Setting and notation 58
8.2 The classical bound 59
8.3 Tsirelson's bound from algebra 61
8.4 The zero-marginal 5×5 moment matrix 63
8.5 The integer check 68
8.6 Level 1+AB 70
8.7 The elliptope completion 77
8.8 The four-vector construction 80
8.9 The arcsine boundary, in both directions 84
8.10 The finite local model, in both directions 85
8.11 Scope of the constructions 87
8.12 Prior work 88

9 From "A record and a toll" 89

10 From "Why the toll isn't arbitrary" 96

11 From "Structural entitlement" 101
11.1 The count, exactly 101
11.2 When the run does the narrowing 104

12 From "Records over any base: the latch" 106
12.1 Bases from outside, and a relation I have to be honest about 106
12.2 What it takes to be complete 109

13 From "The shadow, exactly" 115
13.1 How much gets destroyed, on average 115

14 From “The diagonal” 118

15 From “What established means” 123
15.1 The guy who got closest 123
15.2 The record, the checker and the host 123

16 From “The earned-chain counter example” 125
16.1 Nothing gets forged 125
16.2 A full table traps 126
16.3 What it takes to be certified 127
16.4 Parts I and II, run on the example machine 135

17 From “Five small models” 144
17.1 One example, three levels 145

18 From “The pointer thesis” 147
18.1 What’s proved 147
18.2 What isn’t proved, and must stay that way on the page 152
18.3 What the thesis asks of the examples 152

19 From “The counter against physics” 154
19.1 The calibration is a unit conversion 154
19.2 The calorimeter protocol, as a specification 156
19.3 What this section is 160

20 From “The claims and their conditions” 161
20.1 The diagonal and the worked models 161
20.2 Bell-test vocabulary 162
20.3 Matrices 162
20.4 Thiele-complete and certified Thiele-complete, defined so a clock fails 163
20.5 The game check 164

21 From “The universal machines” 165
21.1 A simpler host, with the guest step built in 165
21.2 The diagonal, on this machine’s programs 168
21.3 The universal machine’s guest interface 176

22 Every model 178
22.1 From any base up 178
22.2 What never lifts 181
22.3 Computation is the slice 183
22.4 The same thing, in seven models 188
22.5 Why two counters 190
22.6 The lift and its instruction interface 191

23 Running it 193

24 The CHSH check 195
24.1 The game 195
24.2 Where it came from, and why anyone cared 197
24.3 Five sentences that are easy to say by accident 200
24.4 The five-by-five, and what positive semidefinite means 201
24.5 The check, in integers 203
24.6 The check, earned on the host machine 206
24.7 A rational stand-in for an irrational number 212
24.8 Two more tests, as mathematics 212
24.9 The quantum set 214
24.10 Any number of levels, and one converse that fails 215

25 The CHSH derivation, in full 217
25.1 Setting and notation 217
25.2 The classical bound 218
25.3 Tsirelson's bound from algebra 219
25.4 The zero-marginal 5×5 moment matrix 220
25.5 The integer check 221
25.6 Level 1+AB 222
25.7 The elliptope completion 223
25.8 The four-vector construction 224
25.9 The arcsine boundary, in both directions 224
25.10 The finite local model, in both directions 224
25.11 Scope of the constructions 224
25.12 Prior work 226

26 Methodology note 227

27 Rows of the claim ledger for the passages above 228

28 From the plain-text statement 244

29 From the README 247

30 From the plain-text statement and the README, second pass 248

31 Further reading 251
31.1 Structure, information and knowledge 251
31.2 Cost, records and thermodynamics 252
31.3 Computation, self-reference and checking 256
31.4 Correlations and environmental records 259

CHAPTER 1

Proofs for “What it takes to be certified”

Proof of the theorem “Certified is Thiele-complete plus certification” (book Theorem 16.3).

Proof. (1) Clause (a) is (A) together with “loaded states are clean”. Clause (c) is (C), with the three record kinds as the marks. Clause (b) says clean states have the record down, which with clean loads gives (B), and it gives the earned chain. Clause (d) gives (D): the claim’s three moves end with the record up from the loaded state where it holds and down from the one where it fails. Going back, the same pieces put the four clauses together again. (2) is (1) read left to right. (3) From any state with the record down, counting A to zero costs nothing and the first mark then raises the record: a raise for one mark from every clean state, where a certified raise from a clean state takes three (Theorem 16.5(3)). ∎

Proof of the theorem “CHECK is a unit-cost test” (book Theorem 16.4).

Proof. At a start the latch is down and the table is empty, so the check passes exactly when $\mathrm{eval}(\mathrm{Ge}(m), a)$ says yes, that is when $m \leq a$; a failing check sets the latch. The price table charges a CHECK 1. ∎

Proof of the theorem “What every certified Thiele-complete machine has” (book Theorem 16.5).

Proof. (1) A raise of the record isn’t made by a base move, by (a), so it’s a record move, which costs one by (c). Running a program on the base is induction on its steps, each matched by (a). (2) is (c), added up along the run. (3) Clause (b) finds a CHECK, a COMMIT and a CERTIFY at three different places, and (2) counts them. (4) is (d): the three moves certify exactly where the claim holds, the claim fails at some loaded state, and the check can’t pass there because (b) says a passing check means the claim holds; at the loaded state where it holds, the three moves raise the record. (5) Cut the run just after the move where the record rises. That shorter run

starts clean and ends with the record up, so clause (b) puts the record down right before a CERTIFY in it and up right after. The record never comes down, so it rises only once, and that CERTIFY is the move. (6) Every move is a base move or a record move, and (a) supplies base moves. ■

Proof of the theorem "The language first, and what a certificate then means" (book Theorem 16.6).

Proof. (1) Take the plain interface the language-first one becomes. (2) The checker's soundness and the keeping proof are the ones clause (b) already supplies. (3) Clause (b) gives the chain; the checker's soundness proof against L gives c's meaning at the state checked; L's keeping proof carries it, through states where the thing c is about is unchanged, to the state committed. ■

Proof of the observation "A clock is a Thiele machine and isn't certified Thiele–complete" (book Observation 16.7).

Proof. Every move costs one, so any raise costs at least one: the toll holds for any record. The machine's own moves are the two-counter instructions, which gives the universal base. But a certified Thiele-complete machine has a move priced 0, by Theorem 16.5(6), and this one has none. ■

Proof of the theorem "Each clause does work" (book Theorem 16.8).

Proof. (1) The base moves leave the flag alone, TICK never lowers it, the base costs 0 and TICK costs 1, which gives (a) and (c). Through any interface, (a) gives a loaded state, which is clean, so its record is down by (b), and a move that puts the record up from every state raises it alone. Clause (b) would need a CHECK, a COMMIT and a CERTIFY inside that one-move run, and one move can't be three. (2) The base gives (a) and costs 0, which with no record moves is (c). No run ends with the record up, so (b) asks nothing of a run, and a checker that always says no is sound for any meaning. Clause (d) needs a run from some loaded state that raises the record, and none does. ■

Proof of the theorem "The four clauses are independent" (book Theorem 16.9).

Proof. (a) Nothing moves a trapped state's flag, so a run that uses DROP ends with the flag down. A run from a clean state that ends with the flag up never used DROP; it's a run of this chapter's machine, and Theorem 15.11 gives its chain. DROP is priced 0 and read as a base move, so (c) holds, and (d) holds as it does for the plain machine. A state with the flag up has it down after DROP. That breaks the permanence in (a), and permanence is about the machine alone, so no interface

meets (a). (b) FORGE never lowers the flag, costs one and is read as a record move, so (a), (c) and (d) hold as before. From a clean start FORGE alone raises the flag, and a run of one move has no room for a CHECK, a COMMIT and a CERTIFY. Through any interface: (d) gives a loaded state from which the record rises, so its trap is down; (a) makes that state clean, so its record is down by (b); and FORGE alone raises it, which (b) forbids. (c) Clauses (a), (b) and (d) never mention a price, and they hold as for the plain machine. Base moves cost one, so (c) fails, and every certified Thiele-complete machine has a move priced 0 (Theorem 16.5(6)). (d) is Theorem 16.8(2). ∎

Proof of the theorem "Any unit" (book Theorem 16.10).

Proof. (1) At unit one every listed price is unchanged.

(2) Changing the size of the unit changes no state transition, claim or record. Thus the computing, earned-chain and distinguishing-claim requirements stay as they were. To multiply the prices by c, read the old ledger multiplied by c too. Every addition is then multiplied by the same amount.

In the other direction, count complete groups of c units in the ledger. There may be a remainder smaller than c at the start. A step adds either nothing or exactly one complete group, so it leaves that remainder alone and increases the group count by the divided price. The equation for that count is

$$\left\lfloor \frac{\ell + cr}{c} \right\rfloor = \left\lfloor \frac{\ell}{c} \right\rfloor + r.$$

It gives the required whole-number ledger on the same states and runs.

(3) The move for an ordinary counter instruction is a base move and costs zero at every unit. A machine charging for every move has no such move.

(4) Find the passing CHECK, matching COMMIT and raising CERTIFY in the earned chain. They occupy three different places in the run. Each costs c, and all other charges are nonnegative, giving at least $3c$. The claim was true at its check and stays true up to its commitment because what it concerns is unchanged. The record rises at that chain's CERTIFY and never comes down; changing the price unit changes none of these events.

(5) Doubling the small machine's prices gives unit two by (2). Its CHECK then costs two. At unit one every move must cost zero or one, so this same priced machine cannot qualify there through another choice of interface. ∎

Proof of the theorem "The small machine, the generic one and the sorted one are certified Thiele-complete" (book Theorem 16.11).

Proof. (1) Clause (a) is Theorem 15.3 together with the step line: INC, DEC and HALT never touch the flag, the flag never comes down, and a start is clean with its trap down. Clause (b) is Theorem 15.11 with checker soundness (Lemma 15.2), and a meaning about c keeps holding while c's value doesn't move. Clause (c) is the price table. Clause (d) is Theorem 15.20 with the claim "A is zero," which holds at $(0, b)$ and fails at $(5, b)$. (2) and (3) are the same argument with the property language left open. For (3), read a number as a list in the list code: 0 is the empty list, and $2^x(2y+1)$ is x followed by the list numbered y (every positive number splits that way in exactly one way, and y is smaller than the number, so the reading ends). The meaning of *sorted* is that each entry is at most the one after it. The checker reads the list from the front: yes on the empty list and on a list of one entry, and on x followed by a list that starts with y, yes exactly when $x \le y$ and it says yes on that rest. Induction on the length of the list shows it says yes exactly when the meaning holds, and a language with one property has an exact equality test. The sorted claim holds of the list numbered 18, which is $[1, 2]$, and fails of 20, which is $[2, 1]$. (4) is (1) of Theorem 16.5 and Section 2's floor. (5) The interface over the language is the one of (1), (2) and (3) with the meanings handed over by the language, and the checker's soundness is Lemma 15.2 and its generic form. ■

Proof of the observation "Every certified Thiele-complete machine hides its record and its ledger from its window" (book Observation 16.12).

Proof. Theorem 16.5(4) gives a loaded state and a run from it that raises the record, and (3) says that run paid at least 3. Say it ends with window (line ℓ, x, y). Two-counter instructions can steer any configuration to any other: empty A, fill it to $x+1$, empty B, fill it to y, then one DEC of A that jumps to line ℓ. Those are base moves, so from the same loaded state they reach the same window, cost nothing, and leave the record down, by (a), (b) and (c). That's two runs, one window, two records and two ledgers, which is (2). A function of the window would have to give both answers on that one window, which is (3). For (1), load $(x+1, y)$ and take just the last DEC. (4) is (3) read as a Thiele machine, whose runs are the machine's. ■

CHAPTER 2

Proofs and worked runs for “Parts I and II, run on the example machine”

Proof of the theorem “The small machine runs the eight-state machine” (book Theorem 16.13).

Proof. (1) By cases on the move. On a live state CERTIFY runs, because a committed claim is on the channel; it raises the flag, moves the line on by one and charges one. INC B moves the line on by one and charges nothing, and it touches only B and B’s version, so the fact about A and the commitment stand and the state stays live. In a jump, each COMMIT finds its fact in the table and succeeds, charging one, and the INC and the DEC charge nothing and end at line a, whose slot is a; the flag isn’t touched. (2) After each compiled move, the state is live again, so the next compiled move starts with exactly the condition just used in (1). The window agrees after each move and each move adds its own charge. At the end of the list the windows agree and the ledger has added the sum of those charges. (3) The eight-state machine’s own toll (Section 3) says a run of its moves that takes the flag from down to up is charged at least one, and (2) carries that charge across. (4) CHECK ($A \geq 0$) passes at every value of A and the COMMIT after it passes, which takes line 1 to line 3, slot 3, with two units of cost paid; then (2) applies. A raised flag was raised by a CERTIFY, and Theorem 15.11 gives the chain. ∎

Proof of the theorem “Structural entitlement represented” (book Theorem 16.14).

Proof. (1) is Theorem 9.1(1). (2) Clause (b) applied to a run from a clean state that ends with the record up gives the chain, and a passing check means the claim’s meaning holds, which keeps holding while the thing it’s about is unchanged. (3) The covering (M6) and Theorem 9.1’s count finish it, with the record moves, which

are the paid steps, in place of the charged steps. (4) is clause (c) added up. (5) is Theorem 16.5(3). ■

Proof of the theorem “A floor with no supplied tree” (book Theorem 16.15).

Proof. Write each rival’s answers on a strip, one box per CHECK in the run. One yes-or-no question gives two possible strips; a second gives four; each further question doubles the possibilities. With c questions there are at most 2^c strips. Put rivals with the same strip in the same pile. The premise limits each pile to m rivals. Hence at most $2^c m$ rivals can be on the original list.

Now $\lceil \log_2 m \rceil$ bits suffice to number m places inside a pile, and c bits suffice to name its strip. Together they number every rival using at most $c + \lceil \log_2 m \rceil$ bits. The least number needed for $|\Omega|$ rivals cannot be larger. Subtract the inside-pile allowance, clipping a negative result to zero, to get the claimed bound of c.

Each CHECK is a record move. Counting all record moves can only increase that number, and each contributes exactly one to the ledger while base moves contribute zero. This gives the two remaining comparisons. ■

Proof of the theorem “Structural entitlement, with the narrowing made by the run” (book Theorem 16.16).

Proof. (1)–(2) Start with the prior list. At each CHECK the actual run passed, run the preceding instructions from each surviving rival and ask that same question there. Keep the rivals that pass. Nothing adds a rival, and the actual start s_0 passes every one of its own tests, so it stays throughout. A missing rival has a first test that removed it. That is the required failed CHECK, at the stated place in the run.

(3) Name the successive list sizes $q_0, q_1, \ldots, q_n$, with $q_0 = |\Omega|$ and $q_n = |\Omega'|$. The before-and-after sizes for test i are consecutive members of that list. Multiply the after-sizes by q_0, and multiply the before-sizes by q_n: each side contains exactly $q_0 q_1 \cdots q_n$. This is the displayed product identity.

(4) The share premise says $q_{i-1} \le 2^k q_i$ at every test. Apply it first to q_0, then to q_1 in the resulting bound, and keep going. After n tests it gives $q_0 \le 2^{kn} q_n$. Numbering q_n final places takes $\lceil \log_2 q_n \rceil$ bits; another kn bits can number up to 2^{kn} times as many. Thus the difference in rounded index lengths is at most kn. Each of these n tests is a CHECK the run actually passed, so it is among the paid record moves. Exact prices make the number of those moves the ledger increase. Multiplying that larger count by k preserves the bound.

(5) The run starts clean and ends with its record up. The earned-record requirement therefore finds its CHECK, COMMIT and raising CERTIFY. Those three different acts each cost one, so the ledger increased by at least three.

(6) The actual start remained on every list, so every q_i is positive. The ratios $q_0/q_1, q_1/q_2, \ldots, q_{n-1}/q_n$ multiply to q_0/q_n: each intermediate size cancels once above and once below. Taking base-two logarithms turns this product into the sum of the per-test logarithms.

To account for rounding, set $u = \log_2 q_0$ and $w = \log_2 q_n$. The integer $\lceil u - w \rceil + \lceil w \rceil$ is at least $(u - w) + w = u$, so it is at least the least integer above u, namely $\lceil u \rceil$. Rearranging gives the claimed bound by $\lceil \log_2(q_0/q_n) \rceil$. Finally, rounding each per-test logarithm up and adding gives an integer at least as large as their sum. It cannot be smaller than rounding the sum up just once. This proves the last comparison. ∎

Proof of the proposition “One CHECK can take four bits” (book Proposition 16.17).

Proof. $A \geq 15$ holds at exactly one of the sixteen starts. From $(15, 0)$ the CHECK passes and files the claim, the COMMIT finds it with A’s version unchanged, and the CERTIFY raises the record. The posterior is the one rival that passes the run’s only CHECK, so the index bits are $\lceil \log_2 16 \rceil - \lceil \log_2 1 \rceil = 4$, and the run has three record moves. ∎

Proof of the theorem “What the record adds” (book Theorem 16.18).

Proof. (1) One direction is clause (b): a run from a clean start that ends with the record up contains the chain. For the other, the chain’s CERTIFY leaves the record up, which is part of what the chain is, and by the last words of clause (a) a record that’s up stays up under every move, so whatever the run does after that CERTIFY it ends with the record up. The floor of 3 is Theorem 16.5(3). (2) Read the proof of Theorem 16.14 again and mark what each conclusion used. Its (1) used (M1) and (M2), through Theorem 9.1(1), and nothing about the machine. Its (3) used (M5) and the count $|\Omega| \leq L(T) \cdot |\Omega'|$ of (M6), and (M4) to put the record moves in place of the tree’s depth; the clean start and the raised record never entered. Its (4) is clause (c) added up along the run, from whatever state the run starts at. The decoded run being shown by some survivor is in the hypotheses and, like the second half of (M3) above, feeds none of the three. The conclusions dropped, (2) and (5), are exactly the two that used the clean start and the record. ∎

Proof of the proposition “The eight-start member” (book Proposition 16.19).

Proof. From any loaded start, the first CHECK passes exactly when a is even. If it passes, the second has room in the table and passes exactly when $a \geq 2$. Nothing has changed A, so their matching COMMIT and CERTIFY then succeed. If either check

fails, the trap prevents certification. This proves the claim for every a, b. Within the eight rivals, a can only be $0, 1, 2, 3$; of those, only 2 is even and at least 2, whatever b. From $(2, 0)$ and $(2, 1)$ the COMMIT then finds its fact with A's version unchanged, nothing having written A, and the CERTIFY raises the flag; from every other start the first failing CHECK traps the machine, and a trapped machine raises nothing. So the posterior is those two. The hypotheses: the posterior narrows the eight strictly (M1); a dropped rival whose run fails a CHECK shows receipts no survivor shows (M2); the run ends with the record up and its receipts, two passes, are the survivors' (M3); the tree has depth 2 and the run 4 record moves (M4); two survivors (M5); and the fibres, the four rivals with B at 0 for the survivor $(2, 0)$ and the four with B at 1 for $(2, 1)$, cover all eight, add up to eight, and have four members each, the leaves of the tree, so $8 \leq 4 \cdot 2$ (M6). Then $\lceil \log_2 8 \rceil - \lceil \log_2 2 \rceil = 3 - 1 = 2$; the four record moves are two CHECKs, a COMMIT and a CERTIFY, and the ledger rises by exactly those four by clause (c); and the chain is in the run. For Theorem 16.15, the run's questions are its two CHECKs, and two rivals answer them alike exactly when their a's agree on being even and on reaching 2, which puts the eight starts in four classes of two, so $m = 2$ and $\lceil \log_2 8 \rceil - \lceil \log_2 2 \rceil = 2$ is the number of CHECKs. With the two CHECKs alone there's no CERTIFY, so the record stays down, the ledger rises by 2, and the index bits are still 2. ■

Proof of the theorem "A search through n bits" (book Theorem 16.20).

Proof. Each question is a CHECK of whether one register holds a given bit, which passes exactly when it does; a failing CHECK traps. So the program certifies exactly when every answer is right, which is when v begins with the answers. The counting is the count of bit strings: the 2^m fibres of 2^k values each add up to the 2^{k+m} of the prior, and none is longer than the 2^k leaves of the tree, so the representation theorem applies with every hypothesis a theorem. The cap is the fact table: it holds sixteen facts, never evicts, and a CHECK on a full table traps (Theorem 16.2), so a seventeenth CHECK before the CERTIFY ends the run in a trap. ■

Proof of the theorem "The time tax" (book Theorem 16.21).

Proof. (1) Take the starts $(m - 1, b)$ and (m, b). At the jth move, $j \leq n < m$, at most $j - 1$ pebbles have left A, so A is at least $m - j \geq 1$ in both runs, and no DEC finds A empty in either. The two runs take the same branches and differ only by the one pebble in A. They are at the same line with the same B, and they stop at the same moment if they stop at all, though $A \geq m$ holds of one start and not the other. (2) Run the ladder from $A = a$. Each of the first $\min(a, m)$ moves takes a pebble off A and jumps two lines on. If $a \geq m$ that's all m moves, ending on line $2m + 1$. If $a < m$, the run reaches line $2a + 1$ after a moves and from there goes on

one line per move (a DEC falling through, then an INC), so after m moves it's on line $a + m + 1$, below $2m + 1$. It's all DEC and INC, so it costs 0. (3) is the price table: CHECK ($A \geq m$) passes exactly when $A \geq m$, a failing CHECK traps and the flag stays down, and the three moves cost three. (4) is the same price table with one instruction: a passing CHECK goes on to line 2 and a failing one traps and leaves the line at 1. (5) is arithmetic on (1) to (4). ∎

The worked run for the time tax, with its table. Try $m = 3$ with $A = 2$. The ladder takes a pebble on line 1 and jumps to line 3, takes the second and jumps to line 5, then finds A empty and falls through to line 6, the INC: after three moves it sits on line 6, not 7. With $A = 3$ the third DEC succeeds and the run is on line 7. Two moves in, both runs are on line 5 with B untouched, which is (1).

Moves taken	Start with two pebbles	Start with three pebbles
0	line 1, $A = 2$	line 1, $A = 3$
1	line 3, $A = 1$	line 3, $A = 2$
2	line 5, $A = 0$	line 5, $A = 1$
3	line 6, $A = 0$	line 7, $A = 0$

The reader in the theorem sees the line and B, which stays unchanged through these three moves. The A entries are shown here to explain the run; that reader is not allowed to use them. Line 6 still has an instruction: the decision time is not a claim that both runs have halted.

CHAPTER 3

Proofs, the program and the worked tables for "The universal machines"

Proof of the theorem "No host fact on an exact copy of the guest's counter" (book Theorem D.1).

Proof. A fixed program mentions finitely many properties, so Q is finite, and the translation sends the infinitely many $\mathrm{Ge}(0), \mathrm{Ge}(1), \ldots$ into it: two of them land on the same member. The guest's COMMIT asks for $\mathrm{Ge}(m)$, and its table can only hold $\mathrm{Ge}(n)$, so it traps (or its CHECK already trapped). The host sees one property twice: its CHECK passes, since the guest's would, and its COMMIT finds exactly what that CHECK wrote. ∎

Proof of the theorem "U keeps pace with its guest" (book Theorem D.2).

Proof. The one-step part is a case for each thing the guest can do at its line: stop; INC; DEC, taken or not; CHECK, passing, failing, or with a full table; COMMIT, found or not; CERTIFY, passing or not. The block invariants and the case analysis in Appendix 7 show where each matching stretch lands. The relation carries the bookkeeping that makes the next case go through: each count register equals the number of slots in use for its counter, each slot in use holds (property number, checked value), a slot's fact is current exactly when the matching guest fact is, a live slot's mirror is the property's number plus one and a stale one's is 0, and unused slots, unused mirrors and the dead end are 0. At loading, the counters and guest line have their specified starting values, and both tables and channels are empty, so the matching conditions hold before any step. Now take one guest instruction. The matching stretch just described ends either at the next matching point or with both machines stopped. In the first case its slot and mirror conditions are exactly what the following instruction needs, so you can do the next stretch. Repeat this for the first m guest steps. Each running stretch takes at least one host step; if neither

machine has stopped, the accumulated host count is therefore at least m. If they stop earlier, both remain stopped with the matching final fields. That gives the whole-run statement for every m.

Proof of the theorem "Halting and answers match" (book Theorem D.3).

Proof. Theorem D.2: a halted guest is matched by a stopped host, and a host that has stopped can't be back at line 1 with the guest still running, because line 1 isn't a stop. A halted machine stays halted, so the two halting points carry the same values.

Proof of the theorem "The host's flag is the guest's, and its ledger is exact" (book Theorem D.4).

Proof. (1) Flags are equal at matching points and never come down. (2) Ledgers are equal at matching points; inside the stretch of U that runs one guest instruction, at most the one paid site of that instruction runs, so the host's ledger sits between the guest's before and after it.

Proof of the theorem "The host earns its own flag" (book Theorem D.5).

Proof. By Theorem D.4 the guest's flag rises too; find the guest step where it rises, a passing CERTIFY, and the related host point just before it. The guest's channel there names a fact, and the relation says the host's channel names the matching slot fact. The host's own flag rises at its own first passing CERTIFY, and nothing paid runs between the related point and that CERTIFY, so the host's channel hasn't moved: it names that slot fact. The host machine's version of Theorem 15.11 gives the host's CHECK and COMMIT on that slot. The record kept with the slot says what pair it held and when, and the checker's meaning gives holds(p, v). The same theorem on the guest's run, up to the step where its flag rose, gives the guest's chain on the same p and c, and the record says c held v at its CHECK.

Proof of the theorem "The host is certified Thiele-complete, and U's halting is undecidable" (book Theorem D.6).

Proof. (1) The base is registers 0 and 1, run by INC and DEC; the claims are "Slot holds of register r"; the rest is the host machine's own versions of the small machine's theorems, and for non-vacuity, register 0 loaded with 1, which is $\langle 0, 0\rangle$, the pair of Zero and 0, against register 0 loaded with 0, which is no pair. Over the claim language the interface is the same one, with the meanings handed over by the language. (2) is Theorem 16.11(4) and Theorem D.5. (3) is Theorem D.3, and

the loading map reduces the small machine’s halting question, undecidable by Theorem 15.5, to this one. ■

Proof of the theorem “One host runs every computably presented Thiele machine” (book Theorem D.7).

Proof. Two layers, each proved on its own. The bottom layer is the compiler: for every presented machine, its guest program’s run passes through points that code M’s states in order, with the guest’s ledger equal to M’s ledger plus the surcharge and the guest’s flag equal to M’s latch, and the guest’s own flag stands on an earned chain whose claim is M’s reading program applied to the coded state where the reading first said yes. Here is why. The guest keeps a few registers packed into its counters the way Minsky packed them: the coded state, a latch bit, the price still owed, and a register that counts the reading’s steps. It starts at the reading with nothing owed, and goes round one loop. Run the compiled reading on the coded state, counting its steps. If it says yes and the latch bit is 0, run CHECK, COMMIT and CERTIFY on the claim “reading program r says yes of v,” v being the packed registers with the step count as fuel, set the latch bit, and take three off what’s owed, stopping at zero. Pay what’s owed in PAYs. Run the compiled driver, and halt if it says stop. Otherwise run the compiled step and price for the move it picked, put the new coded state and that price in place, and go round again. Everything but the PAYs and the one chain is INC and DEC, at no cost. So at the point just after the PAYs and before the driver runs, once n moves have been made, the coded state is M’s state after n moves, the latch bit is M’s latch, and the ledger is M’s ledger, except that the move where the latch bit was set was paid with the chain’s three and the larger of $c - 3$ and 0 in PAYs instead of its price c (or, if the reading is yes at s_0, the start was charged the chain’s three). The CHECK passes, because the fuel is the reading’s own step count, and at such a value the fixed checker says what the reading says; the COMMIT finds that fact, since nothing wrote B in between; and the latch bit lets the chain run once. The top layer is U_P, proved the way U was in the previous theorems, with the PAY case added: it keeps pace with any guest of this kind, halting and flag matched, ledger equal at matching points, and its flag earned on a slot. Compose the two. The earned part takes one more step: the host’s chain gives a guest CHECK that passed, and a guest run that ends with its flag up has exactly one passing CHECK, the compiler’s CHECK of the reading on counter B. So the guest CHECK the host’s chain points to is that one, and its claim is M’s reading at the first state where it said yes. ■

Proof of the theorem “Exact, within two, and no better” (book Theorem D.8).

Proof. (1) and (2): the guest pays M's prices in PAYs, a unit at a time, except at the move where M's reading first says yes. There it has to run its CHECK, COMMIT and CERTIFY, three units, and it makes up the rest of the move's price in PAYs, so it pays the larger of the price c and three. The price is at least one, because the move raised a reading (the toll), so the excess is at most two, and none when $c \geq 3$. (3): a raised flag on the host machine costs at least three from a clean start, whatever the program, and M's ledger is less. (4): the guest finds the reading yes before M has made a move, runs its CHECK, COMMIT and CERTIFY there, three units on top of a ledger of nothing, and from then on its flag is up, so the reading never first says yes again and every move's price is paid in PAYs, exactly. ∎

Proof of the theorem “The host runs any guest” (book Theorem 21.1).

Proof. (1) Before any move, the guest is at $g(h)$. One untrapped guest move has budget 1, enough for every guest instruction, so it takes exactly the guest's next step. It leaves the host untrapped, which lets the same argument apply to the next move. After the first move the guest has taken one step; after the second, two; and after n, exactly n. No such move changes the guest's program. (2) One pass of U_0 is a guest step, then INC A and DEC A back to line 1, so it is (1) once per pass. (3) A host program made only of OWN moves is the small machine, core for core, and Theorem 15.4 does the rest. ∎

Proof of the observation “The copy is the record” (book Observation 21.2).

Proof. One move at a time. An OWN move touches neither the mirror nor the guest. A trapped GSTEP touches neither. An untrapped one sets the mirror to the old mirror or the crossing, and the guest's flag is a latch (Theorem 15.6), so if the old mirror equalled the old flag, the new mirror equals the new flag. The last sentence is this together with Theorem 21.1. ∎

Proof of the theorem “The host's step collects the guest's toll” (book Theorem 21.3).

Proof. (1) The guest's flag rises only on a passing CERTIFY (Theorem 15.6), which costs 1, and the budget let it run, so $b \geq 1$; the mirror was the guest's flag, so it was down, and the move sets it to the crossing. (2) is (1) at the matching move. (3) Write the host's charge and the guest's ledger increase beside every move. When a guest step runs, its price is at most the budget the host paid. When none runs,

the guest's increase is zero and the host's charge is nonnegative. Adding these comparisons along the trace gives

$$\mu(g(\text{run}(t,h))) - \mu(g(h)) \le \mu(\text{run}(t,h)) - \mu(h).$$

- Move the two starting readings to the other sides to get the stated form. ■

Proof of the theorem "No unpaid host certification" (book Theorem 21.4).

Proof. An OWN move copies the mirror unchanged, and a trapped GSTEP does too, so the rise is an untrapped GSTEP b whose guest step crossed; a crossing is a passing CERTIFY, which costs 1 and needed the budget. The own flag moves by the small
- machine's step line. ■

Proof of the theorem "The host is a Thiele machine" (book Theorem 21.5).

Proof. The mirror's half is Theorem 21.4; the own flag's half is the small machine's
- toll on the host's core. The trace form is Section 2's universal floor. ■

Proof of the theorem "Rice's theorem on the host machine" (book Theorem 21.6).

Proof. The reduction above. The prefix before y leaves every register but the two fresh ones as it found them, and empties those, so when H halts the built program ends in whatever state y ends in from the same input, with the same ledger, table, channel, trap and flag. When H doesn't halt, the built program stops on no input. You don't even need to know whether the looping program has Π: ask the decider
- about it, and its answer says which of y and n to put at the end. ■

Proof of the theorem "Rice's theorem for the small machine, from the clean start" (book Theorem 21.7).

Proof. The road of Theorem 21.6, with the input dropped. Take a two-counter program H with a start (a,b), and let y have Π and n lack it; as before, if the program that loops has Π, swap the roles of y and n. Build the program that does INC A a times and INC B b times, runs a copy of H with its jumps moved down (a jump to a line where H has no instruction, line 0 or past its end, goes to the line just after the copy), empties A and then B with a one-line DEC loop each, and then runs y with its jumps moved down. From the clean start the copy of H begins at (a,b), which is H's own start, so it stops exactly when H does. If it does, both counters are emptied and y runs from $(0,0)$. Every instruction before y is an INC or a DEC: it adds nothing to the ledger and touches no fact, channel, trap latch or flag. It does move the versions of A and B up, each by the number of writes that

counter took before y started, and the facts y records carry versions. But a CHECK or a COMMIT reads versions only to ask whether two versions of the same counter are equal, and adding one fixed amount to both keeps the answer. So y takes the same steps inside the built program as it takes from the clean start, with the same counters, trap latch, ledger and flag at every point, and with facts that differ only by that fixed shift in their versions. That shift is why the fact table and the channel are left out of "behaves the same from the clean start." If H doesn't stop, neither does the built program: the copy of H never reaches a HALT, and nothing before y can trap. So the built program behaves the same as y when H stops from (a, b), and the same as the two-line program that loops, INC A then DEC A back to line 1, when it doesn't. A decider for Π would then decide whether H stops from (a, b), with the consequence stated in Theorem 15.5; which of y and n goes at the end is the decider's own answer about the looping program. "Halts from the clean start" is carried by behaving the same from the clean start, held by the empty program and lacked by the looping one, so it's covered. ∎

Proof of the theorem "s-m-n, and a universal program for host programs" (book Theorem 21.8).

Proof. (1) The prefix writes register 2 c times and nothing else, so it moves that register's version from 0 to c. A fact records a version, so after the prefix every later version of register 2, and the version in every fact the run records about register 2, is c more than in a clean run. A CHECK or a COMMIT reads versions only to ask whether two versions of the same register are equal, and adding c to both keeps the answer, so the two runs take the same steps. The run is V's run from the stated start, moved down by c lines. (2) is the evaluator. A finite algorithm decodes a number into a program, runs it step by step on given registers, record layer included, and, searching over the number of steps, returns the output of the first halting run. It is a finite tape algorithm, and the translation of Appendix 6 turns any finite tape algorithm into a finite program of INC and DEC on finitely many counters. The host runs those instructions on registers of its own, with HALT marking the returned output. That's W. ∎

Proof of the theorem "Kleene's recursion theorem on the host machine" (book Theorem 21.9).

Proof. (1) Let E be the following program of INC and DEC, built the same way as W from one finite evaluator, with T's number t written in. On inputs x and c it computes the number of spec(c, P_c), where P_c is the program numbered c, runs T on that number to get d, and then runs the program numbered d on x. Let c_0 be the number of E, and let $e = \text{spec}(c_0, E)$. By (1) of Theorem 21.8, e on input x does

what E does on x and c_0. And E on c_0 builds the number of spec(c_0, P_{c_0}), which is the number of e, since P_{c_0} is E. So e hands its own number to T, gets d, and runs d on x. (2) and (3) are (1) with T carrying out F or f. ■

Proof of the theorem "The recursion theorem, up to the record" (book Theorem 21.10).

Proof. The construction is Theorem 21.9's, with a different evaluator. The first step is to see that a record is small. From a clean start, a host program that stops has a record I can read off five numbers, in this order: what it left in register 0; how many facts the table holds; what is left of the ledger once the facts, the trap and the flag are paid for, which I'll call the replay allowance; 1 if the flag is up and 0 if not; and 1 if the trap latch is up and 0 if not. The record obeys a short rulebook, and you can read each rule off the price table. A run stops at its first trap, so its ledger is its passing CHECKs, plus its passing COMMITs, plus its passing CERTIFYs, plus one if it trapped (only those three can fail, and the failing one is charged). Its passing CHECKs are its facts, at most sixteen. Its flag is up exactly when some CERTIFY passed. So the allowance is the passing COMMITs plus the passing CERTIFYs less one for the flag, which is never negative, and the ledger is the facts, plus that allowance, plus the flag, plus the trap. A COMMIT needs a fact behind it, so a positive allowance means at least one fact. A CERTIFY needs a commitment, so a raised flag means a passing COMMIT, and so a positive allowance. And the channel is empty exactly when no COMMIT passed, which is exactly when the allowance is zero, since a CERTIFY can't pass before a COMMIT has. The allowance counts repeated CERTIFYs as well as COMMITs; it counts the cost the replay can spend on commitments, not the original commitment events. Those are the only properties of a record the replay will use.

The second step is five blocks, one for each number. Each block reads the code of T, runs T on the number of the specialised program, runs the program that comes out on x, and returns its number. All of that happens on plain data: the blocks are programs of INC and DEC, built the way W was, from a finite evaluator translated into INC and DEC. So if T has marks of its own, they are simulated and never paid.

The third step is the payment. The evaluator puts 1 in a fresh register, which is $\langle 0, 0\rangle$, the pair of Zero and 0, so it satisfies the host's one property, and CHECKs it once for every fact; there are at most sixteen, so the table has room. It COMMITs to that claim once for every unit in the replay allowance. It CERTIFYs once if the flag is up. It copies the first number into register 0. And if the trap latch is up, it CHECKs a register that holds 0, which fails, since 0 is no pair, and the host traps. Everything before the payments is INC and DEC and costs nothing, and the payments cost what the record costs, so the ledger is exact. Specialise the evaluator to its own number, as before. ■

Proof of the theorem "No fixed point of full behaviour" (book Theorem 21.11).

Proof. Let $F(p)$ be the one-line program INC r, with r one above the number of p. To carry this out on numbers, add one to the input code, encode INC with that register, and encode the one-element instruction list. These are finite arithmetic operations of the counter construction in Appendix 6. A program's number is at least every register it names: the number of an instruction is above its register, and the code of a list is above every instruction in it. So e can't name r, the register $F(e)$ writes. On input 0 the run of $F(e)$ ends with 1 in register r, and if e stops at all, it ends with 0 there. If e doesn't stop, then it already differs from $F(e)$, which does. Either way they don't behave the same. ∎

Proof of the theorem "The diagonal on the host machine" (book Theorem 21.12).

Proof. (1) The flip, send programs that d calls good to the program lacking Π and the others to the one having it, is carried out by a host program by hypothesis. Theorem 21.9(2) gives a program e that computes what its image computes; Π holds of e exactly when it holds of the image, and the image lacks it exactly when d calls e good, so d is wrong at e. (2) From a deciding program D build a host program that computes D's flip on numbers, by the finite-simulation search of Appendix 6, reading the Boolean from the first halting run of D; then (1) applies. This one is a plain no: the flip (1) needs is built from D itself, so no premise about what can be represented is left over. ∎

Proof of the theorem "Rice's theorem for the two-counter machine, on any inputs" (book Theorem 21.13).

Proof. For every input: given a two-counter instance H starting from counters a and b, build a program that turns the input of n pebbles in bin A into $(6n+5)\cdot 2^a \cdot 3^b$, runs H inside the exponents of 2 and 3, divides out every 2 and every 3 when H stops, takes away 5, divides by 6, and then runs the program that has the property. Because $6n+5$ shares no factor with 2 or 3, the coding never mistakes it for an exponent, and the input comes back whole. If H halts, the built program behaves the same as the one with the property on every input; if not, it never stops on any input, like the program that loops. As before, if the looping program has the property, put a program that lacks it at the end instead, and the decider's own answer about the looping program says which. So a decider for the property would decide two-counter halting. ∎

Proof of the theorem "Kleene for the two-counter machine, inputs written as powers of two" (book Theorem 21.14).

Proof. (1) The host's three pieces, with two counters in place of registers. First, the universal program. Load a program's number e and its input x together as $2^x \cdot 3^e$ in bin A, with bin B empty. The evaluator of Appendix 6, the one that searches budgets and runs the program numbered e on x, is a finite counter program on finitely many counters, and packing those counters into two by the prime-exponent construction of Appendix 6.3 gives U_2: started on $2^x \cdot 3^e$ it stops with 2^y in bin A exactly when the program numbered e, started on 2^x, stops with 2^y. Second, specialising. For a number c and a program V, let spec(c, V) be c copies of a block that multiplies bin A by three, followed by V with its jumps moved down. The block is INC and DEC only, so it adds nothing to the ledger and records nothing, and started on 2^x the c copies leave $2^x \cdot 3^c$; so spec(c, V) on 2^x does what V does on $2^x \cdot 3^c$. Third, the fixed point. Let T be a program that carries out F: on 2^n, n the number of a program p, it stops with 2^m, m the number of $F(p)$. Build E from the same finite evaluator, with T's number written in, and pack it into two counters as before: on $2^x \cdot 3^c$ it computes the number of spec(c, P_c), where P_c is the program numbered c, runs T on that number to get d, and runs the program numbered d on 2^x. Let c_0 be the number of E and $e = \text{spec}(c_0, E)$. On 2^x, e does what E does on $2^x \cdot 3^{c_0}$: it builds the number of spec(c_0, P_{c_0}), which is the number of e itself since P_{c_0} is E, hands it to T, gets the number of $F(e)$, and runs $F(e)$ on 2^x. The machine is deterministic, so a packed output is unique, and for every x and y, e started on 2^x stops with 2^y exactly when $F(e)$ does. (2) is the counterexample that follows. ∎

Proof of the theorem "No plain recursion on two counters" (book Theorem 21.15).

Proof. (2) from (1): let $F(p)$ be a program that multiplies bin A by the constant c determined by the length of p. Multiplication by a fixed constant is the transfer loop of Appendix 6. Decoding a finite program, counting its lines, finding the least prime above a number and writing that loop are computable operations, so a program carries out F on encoded program numbers. A fixed point would multiply every plain input by the constant forbidden by (1).

For (1), suppose an n-line program did it. Split each state into the two bin counts and a finite *control*: the line (one of the n lines, or "stopped" for any line with no instruction), the trap, whether the channel is empty, and the table read entry by entry as a fact's property, its counter, and whether it is *current* (its version is its counter's version now) or stale. Only properties the program names can enter the table, at most n of them, so an entry is one of at most $4n$ kinds, and a table of at most sixteen entries is one of at most $\sum_{k \le 16} (4n)^k \le (4n+1)^{16}$ lists. So there are at

most $Q = 4(n+1)(4n+1)^{16}$ controls. The control and the two counts decide the next control and counts: a CHECK writes a current fact, a write to a counter makes that counter's facts stale, a COMMIT passes exactly when a current fact with its property and counter is in the table, a CERTIFY passes exactly when the channel isn't empty, and the ledger, the flag, the channel's contents and the versions themselves never decide a branch. Call the control with the two counts a *configuration*.

Pick a threshold $h \geq 1$ above every number k in a property $\mathrm{Ge}(k)$ the program names. At h or above, a count answers every question the program can put to it (is it zero, is it even, is it at least k, can a DEC take one off) the same way as any other count of the same parity at h or above. Call a control together with the parities of both counts a *pattern*; there are at most $4Q = N$ of them.

The forced run. A *stage start* is a configuration with one count below h, in the *exact* bin, and the other at h or above, in the *large* bin, which holds v. From it, run the program while pretending the large bin can never fall below h: follow the exact bin exactly, the large bin only by its parity, and keep a tally of the large bin's net change. This forced run depends on the control, the exact count and the parity of v, and not on v itself. The real run does exactly what the forced run does, with the large bin at v plus the tally, up to the first step where v plus the tally drops below h, the *crossing*. A move changes a count by at most one, so at the crossing the large bin holds exactly $h - 1$.

What a forced run does. One of three things, fixed by the stage start. (a) It stops. (b) It comes back to a step with the same control, the same exact count below h and the same parity of the large bin as an earlier step; it's deterministic in those, so it repeats that stretch forever, and the exact count stays bounded. (c) Neither. Then it has at most $2Qh$ steps with the exact count below h, one per combination, so from some step on the exact count stays at h or above for good. Among any $N+1$ consecutive steps of that tail, two have the same pattern, at steps $t_1 < t_2 \leq t_1 + N$, and between them the exact count changes by some δ, even because the parities match. If $\delta < 0$, the run from t_2 repeats the run from t_1 with the exact count lowered by $|\delta|$ each time round, so after finitely many rounds it goes below h, which the tail never does. So $\delta \geq 0$, and the stretch repeats forever, lifted by δ each time round, never dipping below h. In every case but (a), then, the forced run from some step on repeats one stretch forever, and each time round the exact bin changes by a fixed g and the large bin by a fixed amount; in (b) $g = 0$, and in (c) the stretch is at most N moves long.

What the real run does, once v is large enough for that stage start, meaning the large bin can't reach the crossing before the forced run stops or its repeating stretch has gone round once in full. If the forced run stops, the real run stops the same way, with the exact bin at a count fixed by the stage start and the large one at v plus a fixed amount. If the large bin doesn't lose each time round, the real run

never crosses and never stops. If it loses and $g = 0$, the real run crosses with both bins below a bound fixed by the stage start. The remaining case is a *transfer*: each time round the large bin loses $m > 0$ and the exact one gains $g > 0$. That's case (c), so the stretch has at most N moves, each moving one bin by at most one, and $m, g \leq N$; both are even. Now the stage rule. Start the same stage with $v + m$ in place of v. The parity is the same, so the forced run is the same, and since the crossing comes after the stretch has gone round once, the bin needs exactly one more time round to get down to $h - 1$. So the real run crosses one time round later, with the same control, the formerly large bin at $h - 1$, and the other bin larger by g. With $v + km$ it crosses k times round later, the other bin larger by kg. After a transfer, the crossing is a new stage start with the bins' jobs swapped. For example, a loop that takes two pebbles from the full bin and puts four in the other has $m = 2$ and $g = 4$: two more pebbles going in buy one more trip round the loop, and four more coming out.

Choosing the input. Apart from v, a stage start is one of finitely many things: a control, an exact count below h, a parity, and which bin is which. Take one bound $V \geq h$ that's large enough, in the sense above, for every one of them, and above every count that was called fixed or bounded above: the exact bin of a stage that stops, and both bins at a crossing with $g = 0$. The configurations with one count below h and the other below V are finitely many. If the program multiplied every input by c, no two different inputs could pass through the same configuration, because everything after it, the output included, would be the same, while $cx \neq cx'$. So only finitely many inputs pass through that finite set. Pick an input x that passes through none of it and has cx above V. Since $(x, 0)$ is itself a configuration, $x \geq V$.

Following that input. The run from $(x, 0)$ starts a stage: bin B exact at 0, bin A large at x. Every later stage start along the run has its large bin at V or above, or the run would pass through the finite set. The run stops, since the program multiplies, so no stage runs forever. No stage crosses with $g = 0$, since that crossing is in the finite set. So each stage before the last is a transfer, with its own pair (m_i, g_i), and the last one stops. It stops with its exact bin below V and its large bin at its v plus a fixed amount, and the output cx is above V, so the output bin A is the large one. There are finitely many stages, because each takes at least one step of a run that stops.

The comparison. Put $D = 2 \prod_i m_i$ and run the input $x + D$. It is x plus a multiple of m_1, so by the stage rule the first stage crosses at the same configuration except that the other bin is larger by $(D/m_1)g_1 = 2g_1 \prod_{i \geq 2} m_i$, which is a multiple of m_2. So the second stage crosses the same way, with the next bin larger by $2g_1g_2 \prod_{i \geq 3} m_i$, and so on through every stage. The last stage starts with its large bin larger by $2 \prod_i g_i$, an even amount, so it stops the same way with bin A larger by exactly that. So the outputs from $x + D$ and from x differ by $2 \prod_i g_i$. But multiplication by c

makes them differ by $cD = 2c\prod_i m_i$. Cancel two:

$$c\prod_i m_i = \prod_i g_i.$$

Empty products are 1. Every factor on the right is between 1 and N, and c is a prime above N, so it divides none of them, and a prime that divides a product divides one of its factors. So c can't divide the right side. That's the contradiction. ∎

One slot, done by hand. Do one slot by hand. The property "even" has number 1. Checking it on value 4 puts $\langle 1,4\rangle = (2\cdot 4+1)2^1 = 18$ in a separate slot. To read that pair, halve 18 until what's left is odd: one halving gives 9, so the property number is 1; subtract one from 9 and halve to recover value 4. The host checks that property of that value. A second register, the mirror, holds 2: the property number plus one. COMMIT can read this second register while leaving the checked slot untouched.

When the guest changes the counter that fact concerned, nudge the slot $18 \to 19 \to 18$ and set the mirror to zero. The number is back where it was, but the version went up twice, so the host's old fact about that slot is stale. Putting a number back doesn't put an old check back. The mirror is what tells the search to leave that slot alone, and if the search tried it anyway, the version rule would refuse it.

Act	slot value	slot edition	searchable mirror
Check evenness of encoded 4	18	v	2
Guest writes its counter: INC slot	19	$v+1$	
DEC slot; clear mirror	18	$v+2$	0

The program, register by register and job by job. **The program.** I'll describe U the way Turing would, one register at a time and then one job at a time. Registers 0 and 1 hold the guest's A and B. Register 2 holds the guest's whole program as one number: each instruction gets a number (an opcode paired with its operands, INC as 0, DEC as 1, HALT as 2, CHECK as 3, COMMIT as 4, CERTIFY as 5), and the list of them is packed with $\langle\cdot,\cdot\rangle$, the empty list as 0. Register 3 is the guest's line. Registers 4 to 13 are scratch. Registers 14 and 15 count how many CHECKs on A and on B have passed. Registers 16 to 47 are *mirrors*, sixteen for each guest counter, and registers 48 to 79 are the slots, sixteen for each guest counter. The guest's table holds sixteen facts in total, across both counters, and all sixteen can be about one counter, so the host gives each counter room for all of them. The reservation is the host's, and the guest's own capacity stays sixteen in all. Register 80 is a dead end that's never written.

Each guest step goes like this. At line 1 the program copies the program number and the guest's line into scratch, putting the originals back as it goes, because copying a counter on this machine means emptying it into two others and refilling

it. If the guest's line is 0, or past the end of the program, it stops. Otherwise it peels instructions off the front of the copy, one per line, until it holds the one at the guest's line, and splits that into opcode and operands. Then a chain of DECs on the opcode sends it to one of six handlers.

- INC c: INC the guest's counter c, nudge every slot about c and zero its mirrors, add one to the guest's line, back to line 1.
- DEC c j: DEC the guest's counter c. If it was 0, add one to the guest's line. If it wasn't, nudge every slot about c, zero its mirrors, and set the guest's line to j. Back to line 1.
- HALT: stop.
- CHECK p c: let k be the count of passed CHECKs on c. If k is below 16, build $\langle$number of p, value of $c\rangle$ in slot k of c, run CHECK Slot on that slot, set its mirror to the number of p plus one, add one to the count and to the guest's line, and go back to line 1. If k is 16, run CHECK Slot on the dead end, which holds 0, which is no pair, so it traps, just as the guest's seventeenth CHECK does.
- COMMIT p c: look through the sixteen mirrors of c for one equal to the number of p plus one. At the first match, run COMMIT Slot on that slot. With no match, run COMMIT Slot on the dead end, which traps.
- CERTIFY: run CERTIFY, add one to the guest's line, back to line 1.

Why the mirrors? COMMIT has to find the slot holding the claim, and it can't read the slot to find out, because reading would make the fact stale. So beside every slot sits a mirror the program is allowed to read: the property's number plus one while the slot's fact is live, 0 once the guest has written the counter. If a guest CHECK fails its property, the host's CHECK on the slot fails too, since the slot holds exactly the pair the guest checked. If the guest table is full across both counters, the host's own sixteen-entry table is full as well, even when this counter's slot count is below sixteen; its CHECK then traps for lack of room. So both kinds of refusal come through.

The two tally lines for the surcharge. Keep two tally lines for a guest whose first three moves each cost one and whose reading first says yes after the third. The guest's totals are 1, 2, 3. This compiler pays one PAY on each of the first two moves, then its three-act chain: the host totals are 1, 2, 5. The excess is two, added once. If the third move instead costs four, the host runs the chain and one PAY there; both totals then read 6. These are counts of charged instructions, not counts of all the work needed to decode the guest. Stopping at zero throws away what the chain paid beyond the move's price. A compiler that kept it as credit against the moves after the raise would bring the surcharge back to zero once the guest had paid three

from the raising move on, and a guest whose reading starts at yes would be matched exactly from its third unit of cost on. I keep the simpler rule, and every surcharge stated below is this compiler's.

A worked replay, with its table. Try CHECK, COMMIT, CERTIFY, CERTIFY on a true claim. The ledger is four, the table holds one fact, the flag is up and the trap is down, so the replay allowance is two. The replay pays CHECK, COMMIT, COMMIT, CERTIFY: the same ledger of four and the same summary, in a different order. The theorem claims that agreement and no more.

Run	checks	commits	certificates	total paid	final flag
Original	1	1	2	4	up
Replay	1	2	1	4	up

Each table has one fact and each channel is nonempty. The counts of the individual acts differ; the selected final summary agrees. In the replay, the trap latch is up exactly when the fifth number is 1, and the flag exactly when the fourth is positive.

A worked packing, with its table. Try input 2 while the simulated counters hold 1 and 2. The pile is $17 \cdot 2 \cdot 9 = 306$. Raising the first simulated counter doubles it to 612; lowering the second divides it by three, to 204. Strip off every factor of two and three and 17 remains; subtract five and divide by six, and the original input 2 is back.

Act	packed pile	simulated counters	input factor
Load input 2; counters 1, 2	$306 = 17 \cdot 2 \cdot 3^2$	$(1, 2)$	17
Increment first counter	$612 = 17 \cdot 2^2 \cdot 3^2$	$(2, 2)$	17
Decrement second counter	$204 = 17 \cdot 2^2 \cdot 3$	$(2, 1)$	17
Strip factors 2 and 3	17	discarded	17
Subtract 5; divide by 6	2		input recovered

CHAPTER 4

Running it

Everything so far is definitions and proofs about them. Does any of it run? This appendix is me finding out. I built the small machine twice, once straight out of the definitions and once by hand from the same description, ran both, and compared the whole state after every step. Then I ran the universal programs on guests small enough to finish, and real programs through the part of the compiler that executes.

One rule holds for the whole appendix: the runs test the builds, and the proofs carry the theorems. A run shows what a build did on the inputs it was given, and no theorem in the book leans on one: every theorem in it stands on its argument, on the page.

There are two counts to keep apart as you read. The ledger counts what the price list charges. The number of interpreter steps counts the work this particular build does to carry the run out. One guest step can take millions of host steps and add one to the ledger.

Stage	What the next stage must preserve
Source orders	Variable values and completed procedure calls
Assignments and jumps	The same branches, loops and final answer
Counter blocks	Values at block boundaries; spare counters restored
Host instructions	The same result, with the opposite DEC convention translated
Packed two-counter guest	The result as an exponent in the packed counter
Fixed interpreter U_P	Guest counters at matching points; decode the exponent to read the answer

This is a path through the compiler, not a claim that one step at each stage takes the same time. The source compiler inserts no certification chain. Running the executable uses physical hardware and energy. These runs did not measure heat per host step, and the ledger is not a heat measurement. The times only tell you which runs are practical on this build. The table shows what ran and what didn't.

What	Status
The small machine	run: the executable versions agree on every tested step
The host machines, with and without PAY	run, compared the same way
U and U_P on tiny guests	run, against a separate guest implementation
U on a guest that certifies	proved and counted, not run
U_P on a guest that certifies	proved, not run
Source programs through the compiler	run as far as the host program
The two-counter guest and U_P running it	proved, not run
The presented machine carried by U_P	proved, not run

4.1 Taking the coat off

The first executable is generated straight from the formal definitions, the way you'd strip the coat off a proof and keep the program underneath. The second was written separately, by hand, from the same description. Both implement the small machine, the host with a register for every whole number, the host with PAY and the loaders, and both run the universal programs; their instruction lists the second build reads as data printed from the definitions, so it checks them by running them, not by writing them again. The compiler below has an executable runner too.

Whole numbers matter here. A machine word in one build is 63 bits wide, and the guest codes U reads go up past 2^{100}, so ordinary fixed-width arithmetic would wrap around and lie. The mathematical machine never overflows, so its executable uses integers of any size, with the arithmetic chosen to match the definitions: subtraction stops at zero and division by zero gives zero. What sits between the definitions and the running code, the mappings, the compilers and the routines that read input and write output, is code I had to trust.

4.2 A second machine, written by hand, to check the first

Comparing two builds step by step catches one of them disagreeing with the description, on the runs I tried. They're compared on the *whole* state after *every* step: the program line, every counter and register and its version, the fact table in order, the channel, the trap latch, the ledger and the flag. The comparison covers exhaustive sets of short programs, random programs and both hosts, and they agree on every

tested step. U and U_P on tiny guests are compared at fifteen checkpoints two hundred steps apart, over the first three thousand host steps of each run, instead of after every step, and the longer runs below against a separate guest implementation, once per pass of the loop and at the stop. I also broke the machine on purpose, giving it a fact table one too small, a CHECK, COMMIT and CERTIFY that cost nothing, a commitment that ignores versions, and a write that doesn't bump the version, and each break gets caught. A second set pins the boundaries this book draws in words, and those runs come out as the words say: two runs that end in the same state with different first certifications, a check that can't fail certifying from every start, a CHECK and six commitments costing seven without a flag, a write after the commitment and a flag over a claim that's now false, sixteen copies of one CHECK filling the table, a CERTIFY costing one at a ledger already at two, and a merge between two yes-states that leaves the entering state alone.

A mistake both builds share gets past the comparison, and so does a mistake in the definitions themselves, since both builds follow them.

4.3 U on a real guest

Now the universal program, which is where I expected trouble. The guest I wanted to run first was the obvious one, a guest that goes the whole way: CHECK, COMMIT, CERTIFY, flag up, run inside U on the built machine. That isn't what ran, and the reason is the way the guest is stored. U keeps the whole guest program as one number, a number that grows like two raised to the sum of the codes for the guest's instructions (for a guest of one instruction it's exactly a power of two), and it reads that number the only way a counter can, one unit at a time. So its speed depends on the code, and the code grows fast: a guest with one CHECK in it has code 2^{24}, and a guest with the whole chain has a code near 2^{106}.

So I came down to what could finish. The table below is what ran, on the executable host, compared against a separate guest implementation at every pass of the loop and at the stop: the host stopped exactly when the guest did, and at the stop showed the guest's counters, trap latch, ledger and flag.

guest	host	host steps	time
count A down from 2	U	25,953,317	
one passing CHECK	U	402,654,059	618 s
one failing CHECK	U	201,327,155	305 s
one passing CHECK	U_P	402,654,059	617 s
one failing CHECK	U_P	201,327,155	313 s

The first row's count is pinned exactly: if the build changed so that it moved by one step, the comparison would fail. The times are from one ordinary computer and are only there to give you the scale. The counts work out to about twelve host

steps for every unit of the guest's code each time U goes round its loop, which it does once for every guest step and once more to see that the guest has stopped: the failing CHECK traps on its first pass, 12×2^{24} and a little, and the passing one takes two passes, 24×2^{24} and a little. The growth is exponential in the size of the guest. A guest with one CERTIFY in it would have code 2^{32}, about fifty billion host steps, and at the measured rate of about 650,000 steps a second it would take something like twenty-two hours, nearly a day. The guest I wanted, with the whole chain, has a code near 2^{106}, which at the same twelve steps a unit is about 10^{33} host steps for each pass; at the measured rate, with this encoding on that computer, a run of its three steps comes to around 10^{20} years. So U on a guest that certifies is proved, in Appendix D.1, and has never been run. Runs this long only fit in memory at all because the build can repack the host's register store in the middle of a run; repacking changes how the registers are stored, not what they hold.

What a run that long can be is counted. Four loop shapes do most of U's work. Count from the DEC instruction that tests the loop counter; any instructions before that test are still counted separately. You can derive each shortcut from the arrows:

- In the emptying loop, a positive counter v takes one DEC back to its own test and becomes $v-1$. At zero, one DEC exits. Thus $E(0)=1$ and $E(v+1)=1+E(v)$, giving $E(v)=v+1$.
- In the copying loop, a positive DEC jumps back one line to INC on a distinct destination, which returns to the test. Each unit moved therefore takes two steps, and the zero test takes one: $C(0)=1$, $C(v+1)=2+C(v)$, so $C(v)=2v+1$.
- With two destination INCs before the test, each positive pass takes three steps. Thus $D(0)=1$, $D(v+1)=3+D(v)$ and $D(v)=3v+1$. If the two destinations are the same register, it receives two increments per pass; the step count stays three.
- In the halving loop, two successful DECs on the source and one INC on the quotient return to the same test. After $k=\lfloor v/2 \rfloor$ such passes, $3k$ steps have removed $2k$ from the source and added k to the quotient. The remaining source is zero or one. Follow that final branch instruction by instruction.

Each successful decrement or increment also raises its register's version once. The zero test does not write. So the same counts determine the version changes as well as the counter values and next line. These loops neither charge nor change the table, channel, trap or flag. Replacing one completed loop stretch by its derived effect and count therefore preserves the whole endpoint and the number of steps. Repeating that replacement throughout a run preserves both by addition.

The numerical tally below applies those four rules and steps the other instructions individually. It agrees with the three runs above that were also stepped all the way. For the guest CHECK, COMMIT, CERTIFY, using the zero predicate on A, the passing

start $(0,0)$ ends with the flag up and ledger three. From $(5,0)$ the first check fails and the guest stops with ledger one. Appendix 7 supplies the program blocks and their guest correspondence; the displayed totals count that particular encoding.

start	host steps
$(0,0)$	3,894,222,629,847,358,066,688,933,266,198,944
$(5,0)$	973,555,646,581,507,138,873,939,791,446,643

What that table shows is a count, not a run. Nothing stepped those 10^{33} instructions, and the row in the table at the top of this section says so.

Those big counts come from this encoding and from reading it one pebble at a time. They measure the work this interpreter does, and that's all they measure. Whether some other interpreter, or some other base, could do the same job fast is a separate construction with its own running-time analysis. Nothing measured here puts a floor under every way of doing it.

4.4 From source programs to the host

Running guests one CHECK at a time doesn't tell you whether the built machine can do anything you'd recognise as a program. You'd want to hand it gcd, or a sort, written the way people write them, and get the answer out, and writing gcd by hand in INC and DEC is no way to find out. So the last piece of the build is a compiler. The source is a small language you could write by hand: whole-number variables, addition, subtraction that stops at zero, comparisons, *if*, *while*, and procedures that return several results, with the rule that a procedure can only call ones defined before it, so there's no recursion. Each translation stage has a proof that it preserves the program's result: the procedures are inlined, the result is flattened to assignments and jumps, that is compiled to Minsky's counter machine through the counter-machine compiler, that to a program for the host machine, that to a two-counter guest, and the guest to a run of U_P.

□ **Theorem 4.1** (The compiler is correct)**.** For every source program with no recursion, and every input: the program computes y exactly when the runner (the bounded evaluator of the compiled host program) halts with y for some amount of fuel; exactly when the compiled host program halts with y; exactly when the two-counter guest halts with y; and exactly when U_P running that guest halts with y, with its first counter at 0, its ledger at 0 and its flag down. So a terminating compiled program delivers its answer and certifies nothing. To certify a claim about that answer, a target program must add a CHECK, a COMMIT and a CERTIFY; this compiler does not insert that chain. The runner also takes exactly as many steps as the host

program does, with the rest of its fuel left over, and a run that doesn't halt has used all its fuel.

Proof. Six arrows and a runner, each argued below, and the statement is their composition. Four of the arrows are between machines that take steps, and for those one argument does both directions, so I give it first.

The simulation argument. Take two machines S and T, each deterministic, where S has a next state from every state inside its program, and a relation $\sim$ between their states such that: a state of S inside its program is related only to states of T inside its program, and a state of S outside it only to states of T outside it; and from any related pair with S inside its program, one step of S is matched by one or more steps of T that land related to S's next state. Then from related starts, S halts exactly when T does, and they halt related. Forward: start with the related pair. Follow the stretch of T that matches S's first step; its endpoint is the related start needed for the second stretch. Keep joining stretches until S stops. There are only finitely many steps to match, so this gives a finite run of T whose final state is related to S's final state. The latter is outside its program, so the former is outside its own. Backward: suppose T halts after k steps and S doesn't halt. Then S takes at least $k+1$ steps without leaving its program, the matched stretches give T a run of at least $k+1$ steps all of whose stretch ends are inside its program, and T, being deterministic, has only one run from its start, which halted at step k. That's a contradiction, so S halts, and by the forward direction it halts related to where T did.

Unfolding the calls. Imagine a program written on cards. A call says "carry out the orders on that other card, then come back." Here a card may call only a lower-numbered card. Card zero therefore calls nothing. Replace every call on card one by the orders from card zero; then do card two, using the already expanded lower cards, and continue. There are finitely many cards, and every replacement has already been built. The process ends with calls replaced by actual orders.

The variables need their own places too. Give each active procedure a block of numbered boxes, its *frame*. If the caller's block starts at box b and uses n boxes, put the called procedure's block immediately after it, starting at $b+n$. A call becomes four pieces: copy the arguments into that new block; set its other boxes to zero; carry out the expanded orders using those box numbers; copy the answers back into the caller's named boxes. A local variable numbered x in the called procedure is thus the global box numbered $b+n+x$. No assignment reaches below the procedure's own block.

Why does this preserve a terminating run? Begin with the lowest-numbered procedure. It has no calls, so its expanded orders are its original orders with the boxes renamed. Assignments change the same values, tests choose the same branches, and loops repeat the same bodies. Now take a procedure whose lower-

numbered calls have already been settled. Each replacement call builds exactly the arguments and zeros that the original call would receive. The expanded lower procedure returns the same answers, and the copy-back puts them in the same caller boxes. The caller's surrounding orders then continue from the same values. Working upward through the numbered procedures settles the main program.

The comparison works backward as well. Whenever an expanded call finishes, its four pieces have run in their written order. The middle piece is the expanded lower procedure, whose terminating runs I have already matched to the original lower procedure. Its preparation and copy-back are just the original call's input and output operations. Replace that finished four-piece execution by the call it implements. The surrounding assignments, branches and completed loop repetitions then give the same original execution. A finite execution in either description therefore gives a finite execution in the other, with the same answer. Neither can terminate when the other does not.

In symbols, matching a source arrangement e to the global boxes G in a frame starting at b means $G(b+x) = e(x)$ for each local box x. The comparison above preserves that equality at the beginning and end of every translated statement.

Flattening. The flat code of a statement placed at address a: an assignment is one assignment; a sequence is the first statement's code followed by the second's; *if* b is a jump on false to the else part, the then part, a jump past the else part, and the else part; and *while* b with a body of l instructions is a jump on false to $a+l+2$, the body at $a+1$, and a jump back to a. Every jump in the code of a statement lands inside that code or at its end, the address just past it. Now follow a completed structured execution. An assignment performs the same change; a sequence joins its two runs; a branch follows the same test answer. For each loop iteration, the test falls through, the body's code takes the run to $a+l+1$, and the jump returns it to a. Repeat for the remaining iterations; the final false test jumps to the end. Thus an execution from variables e to e_1 gives a flat run from address a to the end of the code with those same final variables.

Read a completed flat run in the other direction. Because no jump escapes past the code's end, a run that leaves it must visit that end first. Assignments and branches recover their structured operations immediately; a sequence splits at the second statement's first address. For a loop, the run's first step is the test. If it is false, the run jumps to the end, and the loop's derivation is the one with a false test. If it is true, the run enters the body's code, which it can leave only through its end at $a+l+1$ (the body's own backward statement), where the jump returns it to a with a strictly shorter run left. Read the next iteration the same way. This cannot continue beyond the finite run I started with: its final test must be false, giving the last piece of the structured loop execution. So the flat program at address

1 halts exactly when the structured statement has a derivation, with the same final variables.

Counter machine. Here a DEC jumps when its register is zero and goes on otherwise. Variable x lives in register $x+1$, register 0 is kept at 0, so DEC 0 j is a plain jump to j, and registers above the variables are working space. The relation: register 0 is 0, registers 1 to n hold the flat variables, and every register above is 0. Each flat instruction becomes a block built from small loops. Follow one pass and then count how many pebbles remain to be moved: Emptying a register: each pass takes one off, and the loop exits at zero. Moving s into d: each pass takes one off s and adds one to d, so $s+d$ never changes, and at exit s is 0 and d holds the old sum. Copying: move s into two registers at once, then move one of them back. Cut-off subtraction $t-u$: take one off each while both are positive; at exit one of them is 0, and the other holds the difference, or 0. Comparison: take one off each in step; whichever reaches zero first, or both together, decides $<$ and $=$, and the block empties both. Build a larger expression from these smaller jobs. A constant is a fixed number of increments into an empty working register. A variable is copied there with its original restored. For a sum or cut-off subtraction, compute the two smaller expressions in separate working registers, combine them with the loops just described, and empty the temporary registers. Starting with the innermost pieces and working outward, the final value lands in register t, the registers above t return to zero, and those below t are left alone. A test computes its comparison in the same spare registers, chooses the true or false exit, and clears the spare registers on either exit. Combining tests follows the selected exits of their smaller tests, each of which has already restored the registers. Thus a test leaves the registers as it found them and changes only which address comes next. So an assignment block computes the value into the first working register, empties register $x+1$, moves the value in, and lands related; a jump on false runs the test's code with the true exit at the next block and the false exit at the block of j; a jump is DEC 0 j. The linker places the blocks one after another and sends every jump to the start of its target's block. That's the one-step matching. The flat machine has a next state from every address in its program, and an instruction that names a variable outside the frame just advances, so the step is total. The counter machine is deterministic. The simulation argument gives both directions: the counter program from the loaded inputs leaves its code with register $out+1$ equal to y exactly when the flat program halts with y in out.

Host. The host's DEC is the other way round: it jumps when its register is positive, taking one off, and goes on at zero. An INC x stays INC x. A DEC x j at host address a becomes three instructions: DEC x $(a+3)$, then INC 0, then DEC 0 j', where j' is the host address of j's block. If register x is positive, the first takes one off and jumps to $a+3$, the start of the next block, which is what the counter machine does. If it is

0, the run goes on to INC 0, which raises register 0 to one more than it held, and DEC 0 j' finds it positive, takes the one back off and jumps to j', which is the counter machine's jump at zero, with register 0 as it was. The relation: the same registers, at corresponding addresses. Each counter-machine step is matched by one or three host steps. The host program has no CHECK, COMMIT, CERTIFY or PAY, and INC and DEC change only a register and the line, at no cost, so every host state in the run has the start's facts, channel, trap latch, ledger and flag. The counter machine's step is total, the host is deterministic, and the simulation argument gives: the host program halts with register $out + 1$ equal to y exactly when the counter program leaves its code with that register equal to y.

Guest. The host program's registers $r_0, \ldots, r_{k-1}$ are packed into the guest's counter B as $\prod_i p_i^{r_i}$, one prime p_i per register, with counter A the spare. That is the packing of Appendix 6.3, with the two counters' jobs swapped. The relation: the guest is at the start of the block for the matching instruction, B holds the code of the registers, and A is 0. An INC on register i is the stretch that multiplies B by p_i; a DEC on register i is the stretch that divides B by p_i with remainder and takes the positive branch when the remainder is 0, the zero branch otherwise, restoring B. The appendix argues each stretch: it ends, because each of its loops empties the counter it counts down; it lands with A at 0; and, by unique factorisation, it changes the exponent of p_i alone, by the right amount. The host's step is total on these programs and the guest is deterministic, so the simulation argument gives: the guest halts with B coding registers whose register $out + 1$ is y, that is, with y the exponent of that register's prime in B, exactly when the host program halts with that register at y. The guest has no record instruction, so its ledger stays 0 and its flag down.

U_P. U_P keeps pace with its guest the way U does, with the PAY case added (the proof of Theorem D.7). This guest has no PAY and no record instruction, so on it U_P runs only branches U has, and the argument of Theorems D.2 and D.3 goes through: from a freshly loaded host U_P halts exactly when the guest does, with its registers 0 and 1 holding the guest's A and B, and its ledger and flag equal to the guest's. So it halts with register 0 at 0, register 1 the code with y in it, ledger 0 and flag down.

The runner. The runner keeps the host program and the registers in binary tries, a tree indexed by the bits of a number. Read the address one bit at a time: a zero chooses one branch and a one the other. Looking up an address follows that route to its stored value, or returns the default zero if no value was stored there. Storing at an address follows the same route and changes its final value, creating missing branches as needed. Every other address either branches off before that end or names a different node, and keeps its value. Thus the tree performs the same lookup and single-address update as the register map it represents. So each round of the runner's loop does exactly what one host step does to the same registers, and the

runner stops when the line leaves the program or the fuel is spent. If it halts after k rounds, the host run of k steps is halted at the end of the program with the runner's registers, and the fuel left is the fuel given less k. If the source program has a derivation, the arrows above give a halting host run, of some length k, and the host, being deterministic, has only that run; any fuel of at least k makes the runner take those k steps and stop. If the runner hasn't halted, it used every unit of fuel on host steps that stayed inside the program, so the host is still running after that many steps, and a derivation, which would give a halting host run of some length k, would need k larger than the fuel.

Composing: the program computes y exactly when the unfolded program does, exactly when the flat program halts with y, exactly when the counter program does, exactly when the host program does, exactly when the guest halts with y coded in B, exactly when U_P halts as stated, and exactly when the runner halts with y on some fuel. ∎

The theorem says nothing about how many steps the compiled program takes next to the source; it's about answers.

The runner and the compiler are implemented, and the build has been run on a set of programs I picked to be annoying: gcd, factorial, a primality program written with procedures, an Ackermann tower, a sorting network on a list packed into one number, and a tiny interpreter for another machine written in the source language. Then come 220 random programs that halt, 160 random programs with loops that might not, programs that never halt run for a fixed budget, and programs mutated on purpose to make sure the comparison notices. Answers are compared with direct arithmetic and separate interpreters of the source and compiled programs. Ackermann's function at $(3, 5)$ is 253; the source program takes 64,768 operations to get there, and the compiled host program takes about 119 million steps. The executable runner does a few million host steps a second, so that's under a minute.

That works out to about 1,800 host steps per source operation for Ackermann, and up to about a hundred thousand for multiplication by repeated addition, because every addition copies a running total one unit at a time. On this build the work grows with the size of the numbers being handled, because every operation is one pebble at a time. How it grows with the size of the program I haven't measured. The last two stages, the two-counter guest and U_P running it, are proved and have never been run, and U_P on a compiled guest would be slow for the same reason U on a certifying guest is. Still, I like that 253. It came out of nothing but INC and DEC, and a proof that they mean what the source meant.

4.5 What wasn't built

There's no chip. The model is not a circuit, and the lack of one says nothing about it. The example was never cut into metal: there's no circuit for the small machine, no simulation of one at the level of gates, and no board ever ran any of this. The small machine exists as mathematical definitions and executable implementations that agree on every tested step. That's the whole of what "it runs" means in this book.

I'd love to see it in metal some day. But builds that can be held against each other on every step are worth a lot to me. A board with nothing to hold it up against isn't, and that's the trade I made.

CHAPTER 5

Constructing the quoting operation

The proof in Section 13 uses a program that takes the code of a term and returns the code of that code. Here's how to build it.

Draw the instruction "add one" on a card. Now make a second card describing the first card's marks, rather than carrying out the addition. The count has not risen: you have described an instruction. Describe the second card in turn and you have a description of a description. Quoting does that second job mechanically. In the language below, take the variable of index 0. Its code is a tree with three lambda nodes, then an application of variable 2 to the numeral for 0. Quoting that code builds a description of those three lambdas and that application; it does not look up variable 0 or execute the term it describes.

Nothing in the operation lets a program inspect itself by magic: its input is an ordinary tree encoded as a closed lambda term.

The construction follows Forster and Smolka (2017), whose construction for L has the same three syntax builders on Scott-encoded children, an auxiliary procedure for numerals, and a quote procedure Q with the same specification, defined by the same recursion on codes; the self-application that uses Q in Section 13 is their fixed-point lemma, after Barendregt's (1984) proof for the full lambda calculus. What follows is my rebuilding of it, with the reductions written out.

5.1 Numerals and syntax as data

For readability, the formulas below give names to bound variables. Renaming a bound variable changes nothing; the underlying syntax uses the de Bruijn indices of Definition 14.1. Applications associate to the left. Write $X \Downarrow Y$ here for a finite sequence of the weak call-by-value steps ending in the value Y.

Scott numerals are the following closed values:

$$\overline{0} = \lambda z.\lambda s.z, \qquad \overline{n+1} = \lambda z.\lambda s.s\,\overline{n}.$$

So a numeral selects either the supplied zero case or the supplied successor case, passing its predecessor to the latter. The cases below are values, so choosing one does not prematurely run the other.

The three syntax constructors, operating on already encoded children, are

$$\begin{aligned} \text{mkvar} &= \lambda n.\lambda v.\lambda a.\lambda l.(v\,n), \\ \text{mkapp} &= \lambda p.\lambda q.\lambda v.\lambda a.\lambda l.a\,p\,q, \\ \text{mklam} &= \lambda p.\lambda v.\lambda a.\lambda l.l\,p. \end{aligned}$$

Each builder is a term of the language L of Definition 14.1. For a variable of index n, an application and an abstraction, respectively, define their codes by the following resulting values:

$$\begin{aligned} \ulcorner \#n \urcorner &= \lambda v.\lambda a.\lambda l.(v\,\overline{n}), \\ \ulcorner p\,q \urcorner &= \lambda v.\lambda a.\lambda l.a \ulcorner p \urcorner \ulcorner q \urcorner, \\ \ulcorner \lambda p \urcorner &= \lambda v.\lambda a.\lambda l.l \ulcorner p \urcorner. \end{aligned}$$

In particular, $\text{mkvar}\,\overline{n}$, $\text{mkapp}\ulcorner p \urcorner \ulcorner q \urcorner$ and $\text{mklam}\ulcorner p \urcorner$ reduce to these three codes. A code is closed even when the term it describes has a free variable: the index of that variable is stored as a numeral. It is data inside the code, and the code itself has no free variable.

5.2 Recursion without running under a lambda

The quoting procedure will have to call itself on the smaller pieces of a description. There is a catch: this language works out an argument before handing it to a function. Hand over the recursive calculation itself and it starts calculating at once, before the receiving function has decided whether it needs it. That can send you straight back into the same calculation forever.

Instead, hand over a recipe waiting for an ingredient. In this language a lambda is such a recipe, and it counts as a finished value even though its body has not run. The receiving function can keep the recipe and call it when it reaches a smaller piece. In the expression below, the little recipe $\lambda a.x\,x\,f\,a$ is the part doing that job. The letter a names the argument it is waiting for; f names the work to do once the recipe and the current input have been handed over.

$$W = \lambda x.\lambda f.\lambda v.\ f\,(\lambda a.x\,x\,f\,a)\,v, \qquad \text{rec}(F) = \lambda v.W\,W\,F\,v.$$

When F and u are closed values, beta reduction gives

$$\text{rec}(F)\,u \longrightarrow W\,W\,F\,u \longrightarrow^{*} F\,(\lambda a.W\,W\,F\,a)\,u = F\,(\text{rec}(F))\,u.$$

The recursive argument is a lambda, so it's already a value. It starts work only when F applies it to an argument. That is the whole reason this construction survives the call-by-value rule.

5.3 Quoting numerals

Let $E_j = \ulcorner \#j \urcorner$ for $j = 0, 1, 2$, and let $Z = \ulcorner \overline{0} \urcorner$, obtained directly from the finite syntax of $\overline{0} = \lambda\lambda.1$. Write $\mathrm{mklam}^k(X)$ for k nested applications of the syntax builder mklam to X; it is an abbreviation for constructing k lambda nodes, and the language gains no new operation from it.

Define

$$F_N = \lambda r.\lambda n.\ n\, Z\, (\lambda k.\mathrm{mklam}^2(\mathrm{mkapp}\, E_0\, (r\, k))), \qquad Q_N = \mathrm{rec}(F_N).$$

Follow the two branches. Zero chooses the already built description Z. A number one larger than n chooses the other branch and hands it the predecessor n. That branch first asks the same procedure to describe the predecessor. Then it puts the description inside an application of variable zero and wraps two lambdas around it. Those are precisely the marks in the numeral for $n + 1$.

Zero therefore works. If the procedure describes n correctly, the wrapping just described makes it describe $n + 1$ correctly. Starting at zero and taking one more step proves the claim for every natural number. Every call on a positive number asks about a smaller one, so it reaches zero and finishes. In the notation for termination and returned value, the result is $Q_N\, \overline{n} \Downarrow \ulcorner \overline{n} \urcorner$.

Do the smallest case by hand. Call the successor case G. Then $Q_N\, \overline{1}$ steps to $F_N\, Q_N\, \overline{1}$ and on to $\overline{1}\, Z\, G$; the numeral hands its predecessor $\overline{0}$ to G, the recursive call $Q_N\, \overline{0}$ comes back as Z, and the builders return $\mathrm{mklam}(\mathrm{mklam}(\mathrm{mkapp}\, E_0\, Z))$. That is the code of $\lambda\lambda.(\#0\, \overline{0})$, which is $\overline{1}$.

5.4 Quoting terms

For a proposed recursive call r, define three handlers:

$$\begin{aligned} H_V &= \lambda n.\mathrm{mklam}^3(\mathrm{mkapp}\, E_2\, (Q_N\, n)), \\ H_A(r) &= \lambda p.\lambda q.\mathrm{mklam}^3(\mathrm{mkapp}\, (\mathrm{mkapp}\, E_1\, (r\, p))\, (r\, q)), \\ H_L(r) &= \lambda p.\mathrm{mklam}^3(\mathrm{mkapp}\, E_0\, (r\, p)). \end{aligned}$$

Now set

$$F_Q = \lambda r.\lambda t.t\, H_V\, (H_A(r))\, (H_L(r)), \qquad Q = \mathrm{rec}(F_Q).$$

The handlers are lambdas and so are values. The Scott code of the input chooses one of them and hands it the encoded children. Start at the leaves and build outward.

At each node the handler builds the syntax of that node's encoding: $Q\ulcorner t\urcorner \Downarrow \ulcorner\ulcorner t\urcorner\urcorner$. There are three kinds of node to consider:

- For $t = \#n$, the first handler constructs three lambda nodes over an application of variable 2 to the quoted numeral. That is the syntax of $\lambda\lambda\lambda.(\#2\,\overline{n})$, the code of $\#n$.
- For $t = p\,q$, quote the two children first. The second handler constructs three lambdas over variable 1 applied to those two codes. That is the syntax of the application code.
- For $t = \lambda p$, the last handler constructs three lambdas over variable 0 applied to the quoted child code. That is the syntax of the abstraction code.

Each recursive call works on a proper child of the finite input tree, or on a smaller numeral. No chain of calls can keep descending through a finite tree or decreasing a natural number forever. Each builder takes finitely many beta steps, so the construction finishes and returns the code just described.

The mkapp used in Section 13 is this builder. With Q and that builder written out, every piece of the self-referential term there is on the page.

CHAPTER 6

Computing with counters

Put a few beans in one cup and leave a second empty. Write a short list of orders: add one, take one away if there is one, go to this line, stop. A finger on the page keeps your place. Carry out the orders literally. The question in this appendix is how to give the same job to another person with different equipment (a strip of paper, stacks of cards, or a different set of cups) and get the same answer whenever the first person stops.

Agree on how the input is laid out and how to read the output before either starts. One person must not quietly solve the problem while preparing the other's cups. The reductions in this appendix concern functions on natural numbers. To compute a relation $R(\vec{x}, y)$ means to halt with output y exactly when $R(\vec{x}, y)$ holds, and to produce only numerical outputs on halting runs. The machines below are deterministic, so any relation they compute is the graph of a partial function. A translation must preserve both the output and whether the run terminates. Its input and output encodings are part of the translation.

6.1 Finite programs made from two instructions

A counter holds a natural number. INC adds one and advances to the next line. DEC subtracts one and jumps to its named line when the counter is positive; at zero, it leaves the counter alone and advances. A finite program uses finitely many counters. An unconditional jump to j is INC r followed by DEC r j: it restores r's value and always jumps. A zero test is DEC r to a positive branch that first restores r; its fall-through is the zero branch. These operations preserve values, not versions. They are used here for computation, where there are no live facts to preserve.

Try copying a cup with three beans. Put two empty cups beside it and keep a supply of spare beans. Each time you remove a bean from the first cup, put one in each of the other two. When the first is empty, move the third cup's beans back into it. Now the first and second each have three, and the third is empty. Try zero beans, then five. At every pass one more of the original beans has been counted into both other cups; that is why it works for any starting count, not just the three you tried. These are the moves behind the copy $C(x, y, t)$ in the next appendix.

Emptying a counter is a DEC loop back to its own line. Moving a pile decrements its source and increments its destination until the source is empty. Copying uses two such destinations and then restores the source from one of them. Addition, multiplication by a fixed natural number, and bounded loops are finite combinations of these moves. Division by a fixed positive number b needs only b control states: decrement the input, cycle through remainders $0, \ldots, b-1$, and increment a quotient counter whenever the cycle returns to zero. On exit the quotient is in that counter and the remainder is in the control state.

6.2 A tape in finitely many counters

Cut a strip into numbered squares and put a finger on one square. Lift the squares on its left into a stack, nearest on top, and do the same on its right. Moving your finger one place right is now two acts: put the square you were reading on top of the left stack, then take the top of the right stack as the new square. Write a new mark before moving and the stack keeps the changed square. Move left and the two stacks exchange jobs. You can walk the strip either way without losing its order. An exhausted stack supplies a fresh blank square.

Now replace each stack by a count. Use digit 1 for a blank and digit 2 for a mark, with base 3. Push a mark onto an empty stack: its code is 2. Push a blank on top: its code is $3 \cdot 2 + 1 = 7$. Count seven beans in groups of three. The one left over says “blank on top”; the two groups say the remaining stack has code 2. Pop that and the marked square comes back. Multiplication pushes and division pops.

In general, number a finite tape alphabet by $1, \ldots, q$, including its blank, and put $b = q + 1$. Encode a finite stack whose top symbol is d and whose remaining stack has code n by $bn + d$; the empty stack is 0. A push is multiplication by b followed by d increments. A pop is division by b: the remainder is the top symbol and the quotient is the remaining stack. The digits are nonzero, so no nonempty stack has code 0. Popping an empty stack gives remainder 0, which is no digit; the control reads it as the blank and leaves the stack empty.

A tape configuration is a control state, the scanned symbol, and two stacks: the symbols to the left of the head, nearest first, and the symbols to its right, nearest first. The control state and scanned symbol have finitely many possibilities, so they fit in the program’s line number. To write d and move right, push d onto the left stack and pop the right stack for the new scanned symbol. A left move does the reverse. The arithmetic just described implements each transition in a finite terminating stretch of counter instructions. Extra stored blanks do not change the represented tape. A finite number of tapes uses two stacks per tape and finitely many scratch counters. Halting states translate to halting lines. Induction over tape steps proves the simulation; termination of every arithmetic stretch proves its converse.

In the other direction a tape can hold the finitely many counter values as unary strings separated by delimiters. INC appends a mark in the selected string; DEC erases one when present and otherwise detects the empty string. The current instruction is finite control. Each scan and rewrite terminates, so halting and the output are preserved. Multiple tapes may be represented on one tape by a list of tracks with marked head positions: a complete scan reads the marked symbols, a second scan writes their changes and shifts their markers. Extend the represented interval by a blank cell when needed. A fixed-length binary code for the finite alphabet implements each scan with binary symbols and finitely many intermediate control states. So this argument also applies to the single-tape binary halting question.

6.3 Packing finitely many counters into two

Try holding two counts in one pile of eighteen beans. Count how many times you can halve eighteen exactly: once. Count how many times you can divide it exactly by three: twice. Reassemble the eighteen beans after each reading. Those are the two stored counts, 1 and 2. Double the pile to 36 and only the first stored count rises. Divide 36 by three to get 12 and only the second falls. Divide by three again: four beans, storing 2 and 0.

Try taking one more off that second count. Four beans do not divide evenly into threes. The attempted division must restore the four and take the zero branch; keeping only the quotient would corrupt the first count. This is why the procedure below remembers the remainder. Two and three share no factor, so changing the number of one kind of factor leaves the other alone. Assign another prime to each extra count. An all-zero collection is represented by one bean: the empty product is 1.

Let $p_0, \ldots, p_{k-1}$ be distinct primes. At the boundary between simulated instructions, keep

$$A = \prod_{i=0}^{k-1} p_i^{r_i}, \qquad B = 0,$$

where r_i are the simulated counter values. The simulated line is finite control. The representation is unique by prime factorization. There are enough primes for any finite program: a prime divisor of $1 + \prod_i p_i$ is not among the primes already chosen, so the list can always be extended.

To simulate INC on r_i, empty A into B, adding p_i to B per decrement, then move B back to A. The result is $p_i A_{\text{old}}$ and $B = 0$. To simulate DEC, divide A by p_i using the finite remainder cycle described above. On exit $A = 0$, $B = q$ and the control holds d, where $A_{\text{old}} = p_i q + d$ and $0 \le d < p_i$. If $d = 0$, the original number was

divisible by p_i, so $r_i > 0$. Move B to A and take the positive branch. Its exponent of p_i has decreased by one and every other exponent is unchanged. If $d > 0$, then $r_i = 0$. Restore A by moving B back with p_i increments per unit, add d, and take the zero branch. Both branches restore $B = 0$. All loops terminate because their source counter decreases. Induction gives the represented run at every simulated instruction boundary, and no simulated instruction can conceal an infinite internal run.

This is universality with an encoding. With input n in the first simulated counter and the others zero, the packed input is 2^n when $p_0 = 2$. It is supplied to the two-counter machine as its input; the construction does not claim that two counters starting at $(n, 0)$ can first manufacture 2^n. That is precisely the distinction Schroeppel's theorem in Section 15 turns on, and the undecidability used by the one-counter theorem in Appendix 22 ranges over packed starts too. For a tuple of inputs, supply its product of prime powers. If a numerical output is needed in ordinary notation, the external decoding reads the designated prime's exponent. A host with finitely many additional counters can perform this decoding itself by repeated fixed-prime division.

6.4 Why halting has no decider

Write two responses on a card: "If the predictor says STOP, keep going forever. If it says KEEP GOING, stop." Suppose somebody offers a predictor that always finishes and always gets a program's eventual behaviour right. Give it this very card, with these instructions for using its answer. Work through both possible answers. STOP makes the card keep going; KEEP GOING makes it stop. Waiting longer for the predictor does not create a third correct answer: its promise was to finish with one of the two.

The card can be a program because each response is a finite instruction, and waiting for and branching on the predictor's answer are computable operations. Here is how to include the input explicitly. Finite tape programs can be written as finite binary words, and a program can interpret such a word by looking up the next transition and maintaining the simulated tape. Invalid descriptions may be assigned one fixed halting program. Suppose a terminating algorithm $H(p, x)$ decided whether the program described by p halts on x. Build a program D which, on input p, computes $H(p, p)$, loops forever if the answer is yes, and halts if it is no. Let d describe D. Either answer to $H(d, d)$ contradicts what $D(d)$ does. So no such deciding algorithm exists.

Translate a tape program and its input by the stack construction and then prime packing. Both translations are finite algorithms, and the translated two-counter run halts exactly when the tape run halts. A decider for two-counter halting would

then give H. The same reduction, followed by Theorem 15.4, applies to the small machine. Halting instances can be enumerated by simulating every description and input for successively larger finite budgets. If the nonhalting instances could also be enumerated, interleaving those two enumerations would decide each instance. This proves the impossibility underlying the reduction formulation of Theorem 15.5.

6.5 Seven descriptions of the same computation

Here are the translations behind Theorem 22.6. Each one keeps the partial function, including where it runs forever. The constructions are the familiar ones behind the results of Minsky, Shepherdson and Sturgis, and Forster, Larchey-Wendling and their coauthors; the encodings and the reason each translation works are given here.

Binary stacks. A binary stack is a finite word with push and pop operations, with a distinct empty case. Encode the empty word as 1 and pushing $d \in \{0,1\}$ as $n \mapsto 2n + d$. Division by two pops it, with 1 tested before division. Finite counters implement these operations. Conversely two stacks hold a tape as above; encode every symbol of a larger alphabet by a fixed-length binary block. A push or pop of that block takes a fixed finite number of stack operations, and the control stores a partially read block. A tape simulates counters, so the translations connect binary stack machines and counter machines in both directions.

The two decrement conventions. Some counter machines jump on zero and fall through after a positive decrement; the convention of this book does the reverse. To translate a conditional decrement, send its two outcomes to separate short blocks that unconditionally jump to the intended destinations. In this book's convention an unconditional jump is the restoring INC/DEC pair above. Under jump-on-zero that pair falls through, because the DEC finds the counter positive; there the jump is a DEC on one extra counter that nothing ever increments, so it is always zero. So either convention implements the other's instruction, with the same final values on the original counters and one finite stretch per step, and the translation into jump-on-zero spends one counter more. Relabel each target to the beginning of its translated block. This proves the equivalence of the two Minsky instruction conventions without identifying their individual steps.

FRACTRAN. A program is an ordered finite list of positive rational numbers. At a positive integer N, multiply by the first fraction whose product with N is an integer; halt if none applies. Counter machines simulate this by fixed-denominator division, multiplication and an ordered scan of the finite list. For the reverse translation, assign distinct primes p_r to counters and q_i to control states. Represent a counter configuration by $q_i \prod_r p_r^{n_r}$. Every translated instruction uses a fresh intermediate state prime, so its state marker never cancels out of a fraction. For INC r going to j, use $t_i p_r / q_i$, followed by the transfer q_j / t_i. Try it with a counter represented

by factors of 2, a starting line marked by 5, a destination marked by 7 and an intermediate marker 11. Ten beans, $5 \cdot 2$, mean line 5 with counter value 1. The fraction $22/5$ takes ten to 44; $7/11$ takes 44 to 28. Divide the result by its line marker 7: four remains, two factors of 2. The line has changed and the counter has risen by one. The intermediate 11 keeps the two parts of the instruction separate. For DEC r with positive target j and zero target k, use, in this priority order,

$$\frac{t_i}{q_i p_r}, \qquad \frac{u_i}{q_i},$$

with transfers q_j/t_i and q_k/u_i. All state and intermediate primes are distinct from the counter primes. Exactly one state marker is present. Fractions belonging to other states cannot apply. At state i, the first decrement fraction applies exactly when $n_r > 0$; otherwise the second applies. The next fraction transfers the intermediate marker to the destination marker. A halting state has no outgoing fraction. Each counter step then takes two fraction steps, and the prime exponents preserve all counter values. Read the output as the exponent of the output counter's prime.

Partial recursive functions. Compare two orders. "Try the first ten tickets" gives you a stopping point before you begin. "Try tickets in order until one passes" does not. If each test finishes, the second order can still keep you working forever because every ticket fails. If one test itself never finishes, you never reach the next ticket. Those are the differences between a bounded loop and the search used here. Start with zero, successor and projections; close under composition, primitive recursion and minimization. In minimization, search $0, 1, 2, \ldots$ for the first argument where the tested function returns zero; every earlier test must terminate with a positive result. A counter program implements the initial functions directly. Disjoint scratch counters preserve arguments across composition. For primitive recursion it keeps the recursion index and accumulated result, increments the index, and runs the step program until it reaches the supplied bound. For minimization it runs the tests successively; an undefined test leaves the run there forever, as the definition requires. Induction on these constructions supplies a finite counter program for every partial recursive function.

Conversely encode a counter configuration by nested pairs $\langle m, n \rangle = (2n + 1)2^m$, with a separate finite code for the line. Addition, multiplication, powers, bounded division and bounded search are primitive recursive: their loops have a supplied natural bound and can be written as recursion on that bound. Pair decoding is bounded by the encoded positive number. The next configuration and the halt test are then primitive recursive for any fixed finite program. The configuration after t steps is primitive recursion on t, keeping halted configurations fixed. Minimize the total halt test over t, then read the output counter. This is the program's partial function, and it is undefined exactly when the program never halts.

The language L. Fold two instruction cards shut. One says "put three beans in the output cup and stop"; the other says "keep adding beans forever." A zero input is to choose the first card. If your rule makes you carry out both cards before choosing, you never deliver the three beans. You have to keep the instructions folded until the choice is made. An abstraction does that job in weak call-by-value: its body waits until the abstraction is applied. The Scott numeral definitions and the delayed recursion construction in Appendix 5 give zero, successor, predecessor, case distinction and guarded recursion as terms. A case on a numeral n can be delayed explicitly as

$$n\,(\lambda d.u)\,(\lambda p.\lambda d.(v\,p))\,(\lambda a.a)\,:$$

at zero it evaluates u, and at a successor it evaluates v on the predecessor. Both branches are values until selected, so call-by-value does not evaluate the unused branch. Composition is application; primitive recursion repeats a step on the predecessor; minimization is a guarded search that evaluates each test before moving to its successor. Build the term in the same order as the function: begin with its initial functions, then use the corresponding term operation each time composition, recursion or minimization is used. Each operation consumes the results of the already built pieces exactly as the function definition requires. On numerical inputs, a computation that finishes therefore returns the Scott numeral of its answer. If a required subcomputation never finishes, evaluation waits for it; if minimization never finds a successful test, its search keeps going. In either case there is no resulting value.

In the other direction, encode a finite term as a finite string or as the nested-pair code of its syntax tree. Reading the outer constructor, finding a variable index, renaming bound indices and substituting a finite term are terminating traversals of finite trees. A tape algorithm performs them with an explicit stack of unvisited children. For weak call-by-value evaluation it evaluates an application's function and argument, then substitutes the argument value in the abstraction's body; it never reduces inside an unapplied abstraction. Each operation is a finite tape computation. Induction on a terminating evaluation gives a terminating tape run with its value; reading a terminating run backwards gives the evaluation tree. Scott numerals are decoded by counting successor constructors. So an L computation of a numerical relation is a tape computation of it.

The tape and counter translations, the two counter conventions, binary stacks, FRACTRAN, partial recursive functions and L now lie in one connected family of translations. Composing the translations preserves their input/output relation. None of them keeps time, space, or a ledger of what was paid just because it keeps a partial function. Where I need a ledger kept, I build that in on purpose, as the universal program of Appendix 7 does; time and space I don't claim at all.

6.6 Searching for a finite computation

Give yourself one token and run one instruction of the program you want to test. If it has not stopped, reset its starting arrangement, give yourself two tokens and try again. Then three. A program that stops after seven instructions is found on the seven-token attempt. Seeing six attempts fail says only that it has not stopped within six instructions; it does not tell you it runs forever.

The evaluators used in Appendix 21.2 work by that search. A program and a natural budget determine a finite simulation. Enumerate budgets $0, 1, 2, \ldots$, and stop at the first one that exhibits a halting run, returning its output. If no such run exists, the search continues forever. Encoding a finite program, decoding an instruction, and performing one step use the finite operations above, so this search is itself a partial recursive function and has a finite counter program. The host can execute that program instruction for instruction, using as many registers as the construction requires. Its computation uses only INC, DEC and HALT, so it adds nothing to the ledger and leaves the fact table, channel and flag alone. This supplies the evaluators and program transformations used in the host's s-m-n and fixed-point arguments.

CHAPTER 7

The universal program in blocks

Put the guest's instructions on one sheet and the host's on another, with a finger marking the current line on each. The guest takes one instruction. The host takes as many of its smaller instructions as it needs, then returns to the head of its sheet. Compare the two arrangements there: counts, live facts, commitment, flag and ledger. The host is allowed to do more work between comparisons. It is not allowed to lose a fact, revive a dead one, or charge another unit for its bookkeeping. The lists below let you carry out that comparison instruction by instruction.

Here is the finite list of instructions that is U. A block is a list, not an extra instruction: replace every block name by its list, place the lists consecutively, and resolve each label to its instruction number. The first instruction is at address 1. Within a block, o is its first address and $o + k$ its offset k. Unless a branch says otherwise, execution continues into the next block. The only final instructions are INC, DEC, HALT, CHECK, COMMIT and CERTIFY, with the meanings and prices of Definition 15.3 on the host's registers (Definition D.2).

7.1 Arithmetic blocks

Write $J(a, p)$ for the two-instruction list INC a; DEC a p. It jumps to p and restores the value of a. The following lists specify the other arithmetic blocks. Semicolons concatenate lists; a named block in a list is placed at the next available address. A target such as $o + 3$ refers to the enclosing block's start, not the nested block's start.

Block	Expansion at address o
$Z(x)$	DEC x o
$J_0(x, p)$	DEC x $(o + 3)$; $J(x, p)$; INC x
$M(x, y)$	$J(x, o + 3)$; INC y; DEC x $(o + 2)$
$M_2(x, y, z)$	$J(x, o + 4)$; INC y; INC z; DEC x $(o + 2)$
$C(x, y, t)$	$Z(y)$; $Z(t)$; $M_2(x, y, t)$; $M(t, x)$
$H(a, x, p)$	$Z(a)$; $M(x, a)$; $J(a, o + 8)$; INC x; DEC a $(o + 11)$; $J(a, p)$; DEC a $(o + 7)$

Block	Expansion at address o
$P(a,x,y)$	$Z(a)$; $M_2(x,a,a)$; INC a; $M(a,x)$; $J(a,o+22)$; $M_2(x,a,a)$; $M(a,x)$; DEC y $(o+13)$
$N(a,x,y)$	$J_0(x,0)$; $Z(y)$; $J(a,o+8)$; INC y; $H(a,x,o+7)$

Their lengths, in the order $Z, J_0, M, M_2, C, H, P, N$, are $1,4,4,5,11,12,23,20$. The source and destination of a move are distinct; M_2 may have equal destinations, as in P. In C the three registers are distinct; in H, $a \neq x$; in P and N, all three are distinct.

Each claim about these blocks follows from a decreasing counter. Z decrements to zero. M adds the old source to the destination and empties the source; M_2 does it to both destinations. So C leaves x unchanged, writes its old value to y, and leaves $t = 0$. J_0 tests zero without changing the value: a positive counter is decremented and then restored, while zero takes the jump.

In H, first move the old x into a. Each complete pass removes two from a and adds one to x. So it leaves $a = 0$ and $x = \lfloor x_{\text{old}}/2 \rfloor$; an even old value jumps to p, an odd one falls through. In P, the first four blocks make $x = 2x_{\text{old}} + 1$ and $a = 0$. Each subsequent pass doubles x while decrementing y, so it ends at $x = (2x_{\text{old}} + 1)2^{y_{\text{old}}}$, with $a = y = 0$. In N, zero jumps to address 0, which holds no instruction, so the run stops. A positive number is repeatedly halved until odd, counting the even divisions in y. So N sends $x = \langle m, n \rangle$ to $x = n$, $y = m$, $a = 0$. These invariants prove both the returned values and termination on the stated inputs.

7.2 Selection, comparison and fetching

The selector $D(x, t; p_0, \ldots, p_{k-1})$ has length $3k$. For $i = 0, \ldots, k-1$, place at offset $3i$ the three instructions DEC x $(o + 3i + 3)$; $J(t, p_i)$. If the entering value is $i < k$, it takes target p_i with $x = 0$. If it is at least k, it falls through with x reduced by k. This follows by counting the decrements before the first zero.

The equality block $E(x, y; t_1, t_2, u; p)$ first makes copies $C(x, t_1, u)$ and $C(y, t_2, u)$, occupying offsets 0 to 21. Its remaining list is

$$
\begin{aligned}
22: &\quad \text{DEC } t_1 \ (o+29),\\
23: &\quad J_0(t_2, p),\\
27: &\quad J(u, o+32),\\
29: &\quad \text{DEC } t_2 \ (o+22),\\
30: &\quad J(u, o+32),\\
32: &\quad Z(t_1);\ Z(t_2).
\end{aligned}
$$

The listed offsets count expanded instructions, so the block has length 34. The temporary registers are distinct from each other and from x, y. The two copies count down together. If they reach zero together, the block jumps to p; if one reaches zero first, it clears both and falls through. So it preserves x, y, leaves its temporary counters zero at either exit, and branches exactly on equality.

For fetching, define $F(q, l; w, k, h, a, t; p_0, p_e)$ by the list

$$\begin{aligned} 0:&\ C(q, w, t),\\ 11:&\ C(l, k, t),\\ 22:&\ J_0(k, p_0),\\ 26:&\ \text{DEC } k\ (o + 27),\\ 27:&\ J_0(w, p_e),\\ 31:&\ N(a, w, h),\\ 51:&\ J_0(k, o + 56),\\ 55:&\ \text{DEC } k\ (o + 27). \end{aligned}$$

Its length is 56. Register q holds a list encoded by 0 for empty and $\langle \text{head}, \text{tail} \rangle$ for a nonempty list, and l is a line number. The scratch registers are distinct and separate from q, l. A zero line takes p_0. Otherwise decrementing the line copy once makes $k = l - 1$. Each pass removes the next list head into h and its tail into w. If $k = 0$, that head is the selected instruction and the block exits; otherwise it decrements k and repeats. An exhausted list takes p_e. The original program and line values are restored by the copies. Every call terminates: each complete pass decreases k, or exits.

7.3 Registers and instruction codes

Put $T_i = i + 4$ for $0 \le i < 10$. For bank $c \in \{0, 1\}$ and slot $0 \le k < 16$, put

$$R_c = c, \quad Q_c = 14 + c, \quad V_{c,k} = 16 + 16c + k, \quad S_{c,k} = 48 + 16c + k.$$

Register 2 is the program, register 3 its line, and register 80 is the dead end. V are the mirrors, S the slots, and Q the counts of successful checks in each bank. Property codes are 0 for zero, 1 for even, and $n + 2$ for at least n. The code of an instruction is the pair of its opcode and operand: INC has opcode 0 and operand c; DEC has opcode 1 and operand $\langle c, j \rangle$; HALT has opcode 2 and operand 0; CHECK has opcode 3 and operand $\langle c, p \rangle$; COMMIT has opcode 4 and that same operand form; CERTIFY has opcode 5 and operand 0. A program is their list code. These rules specify the loaded number in register 2.

A bump of slot S and mirror V is INC S; DEC S to the following instruction; $Z(V)$. It has length 3. It leaves the slot's value alone, increases its version by two, and

empties the mirror. Let B_c concatenate the sixteen bumps of bank c, in increasing slot order; its length is 48.

7.4 The fifteen blocks of U

The following table fixes the order, start address and expanded length of every major block. Labels in the definitions below mean these addresses; H without arguments denotes the head at 1 and K the halt block at 98.

Block	Start	Length
Head	1	97
Halt	98	11
INC bank selection	109	8
INC bank 0	117	52
INC bank 1	169	52
DEC bank selection	221	28
DEC bank 0	249	60
DEC bank 1	309	60
CHECK bank selection	369	28
CHECK bank 0	397	1022
CHECK bank 1	1419	1022
COMMIT bank selection	2441	28
COMMIT bank 0	2469	628
COMMIT bank 1	3097	628
CERTIFY	3725	4

The head is $F(2,3;T_0,T_1,T_2,T_3,T_4;K,K);\ Z(T_0);\ N(T_3,T_2,T_5);\ D(T_5,T_4;109,221,98,369,2441,3725);\ J(T_4,K)$. Fetching supplies an instruction code in T_2. Unpacking leaves its operand in T_2 and its opcode in T_5. The six-way selector chooses its handler. The final jump handles an invalid opcode, though a loaded program has none. The halt block is $Z(T_0);\cdots;Z(T_9)$ followed by HALT.

INC's bank selector is $D(T_2,T_4;117,169);J(T_4,K)$. Each other bank selector first runs $N(T_3,T_2,T_6)$, leaving the property code or jump target in T_2 and the bank in T_6, then selects its two bank blocks with $D(T_6,T_4;\text{bank }0,\text{bank }1);J(T_4,K)$.

The INC block for bank c is INC R_c; B_c; INC 3; $J(T_4,H)$. The DEC block, starting at o, is DEC R_c $(o+5)$; INC 3; $Z(T_2)$; $J(T_4,H)$; B_c; $Z(3)$; $M(T_2,3)$; $J(T_4,H)$. A zero counter follows the first return, advancing the guest line without bumping slots. A positive counter jumps over it, bumps the bank and installs the target line.

A CHECK slot block for (c, k) is the following 60 instructions after expansion:

$$C(R_c, T_8, T_4);\ C(T_2, T_9, T_4);\ P(T_3, T_8, T_9);\ M(T_8, S_{c,k});$$
$$\text{CHECK Slot } S_{c,k};\ Z(V_{c,k});\ M(T_2, V_{c,k});$$
$$\text{INC } V_{c,k};\ \text{INC } Q_c;\ \text{INC 3};\ J(T_4, H).$$

The CHECK bank block at o is $C(Q_c, T_7, T_4)$, then a selector on T_7 whose sixteen targets are $o + 62 + 60k$, then CHECK Slot 80 and $J(T_4, K)$, then the sixteen slot blocks. Its length is $11 + 48 + 1 + 2 + 16 \cdot 60 = 1022$. The selected unused slot starts at zero, so the move into it installs exactly the pair of property code and guest value. The host's own table has capacity sixteen: if it is full, its CHECK traps even when this bank's count is below sixteen. So the capacity is shared between the banks, just as it is in the guest. A count of sixteen in one bank already fills the table, so the dead-end check and the full-table trap are the same case.

A COMMIT slot block is COMMIT Slot $S_{c,k}$; $Z(T_2)$; INC 3; $J(T_4, H)$, of length 5. The COMMIT bank block at o first increments T_2, making the wanted mirror value one more than the property code. It then runs, for $k = 0, \ldots, 15$, $E(V_{c,k}, T_2; T_7, T_8, T_4; o + 548 + 5k)$. After those comparisons come COMMIT Slot 80 and $J(T_4, K)$, followed by the sixteen slot blocks. Its length is $1 + 16 \cdot 34 + 1 + 2 + 16 \cdot 5 = 628$. The comparison reads the mirror, leaving the slot untouched. Its first match commits that slot's current fact; absence of a match reaches the dead end and traps. The CERTIFY block is CERTIFY; INC 3; $J(T_4, H)$, of length 4.

These expansions determine every instruction and target. Adding the fifteen lengths gives 3728. There are sixteen ordinary CHECK sites and one dead-end CHECK in each bank, sixteen ordinary COMMIT sites and one dead-end COMMIT in each bank, and one CERTIFY site: $2 \cdot 17 + 2 \cdot 17 + 1 = 69$ paid sites. Every other instruction costs nothing.

7.5 Why the blocks simulate a guest

Run a case that catches more than a wrong sum. Start the guest with four beans in A and run CHECK Even A. Put its fact on a card saying "even, version 0." The host copies the value into an unused slot as the pair (Even, 4), whose code is $(2 \cdot 4 + 1)2^1 = 18$, and checks that slot. Put one unit on each ledger.

Now let the guest add one bean and take it away. Its pile is four again, but its version is 2. The old fact still says version 0. On the host, each of those guest writes bumps the slot by one and back: 18 to 19 to 18. The slot's value ends where it began, while its version rises and its mirror is cleared. Try COMMIT Even A using the old check. Both machines must refuse, even though four is even. Both pay one more unit for that failed attempt. A translation that merely compares the piles at the end would miss the failure this experiment is designed to catch.

Start again and leave out the two bean moves. CHECK, COMMIT, CERTIFY must succeed on both sheets, raising both flags for exactly three units. The longer list of host instructions adds nothing to the ledger for copying or looking up a line. These two runs test the reason for keeping both values and versions. The argument for all runs has to keep the following correspondence at every return to the head.

At the head, registers 0, 1 and 3 give the guest counters and line. Each successful guest CHECK has one host slot fact holding its property and checked value. Its mirror is the property code plus one while the guest fact is current, and zero after the guest counter has been written. The matching host slot fact is current in exactly the first case. Count registers say how many slots have been used in their bank; unused slots and mirrors are zero. Tables have corresponding entries in the same order, and channels name corresponding entries. The ledgers and flags agree. Loading a guest establishes every part of this relation.

Fetching and dispatching terminate and preserve that relation's stored values. For INC or a positive DEC, the guest writes its counter; the selected bank's bumps invalidate exactly its former slot facts and clear their mirrors. A zero DEC writes nothing and bumps nothing. These cases add nothing to either ledger. For CHECK, an unused slot receives the exact pair tested by the guest. Both checkers answer the same, and the tables have the same length. A passing check appends the corresponding fact and updates its mirror and count. A false check or a full table traps both machines, after the same single paid act. For COMMIT, a nonzero mirror matches exactly a current guest fact with the requested property and counter. Its host slot has not been read or changed, so the corresponding COMMIT succeeds. If none matches, both machines trap. Again exactly one unit is paid. For CERTIFY, channels are either both empty or both hold corresponding commitments, so the instruction either traps both or raises both flags, for one unit.

All arithmetic between these paid acts terminates by the block invariants. A normal case returns to the head; a halting or trapped case stops. No arithmetic block has a paid instruction. This proves the one-step correspondence and bounds every intermediate host ledger between the guest's old and new ledgers. Induction proves the run correspondence of Theorems D.2 and D.4. It also proves the reverse halting implication: an infinite guest run entails infinitely many nonempty host stretches, and a running guest instruction cannot produce a stopped host except in its own halting or trapping case.

7.6 Adding PAY

For U_P, use opcode 6 with operand 0 for PAY, append a PAY handler to the six handlers, and append its block after CERTIFY. The block is PAY; INC 3; $J(T_4, H)$, of length 4. The head's selector now has seven entries and is three instructions

longer. Every block after the head then starts three addresses later. Resolve targets to those new addresses; the head itself remains at address 1. The new PAY block starts at 3732. The total length is $3728 + 3 + 4 = 3735$ and there are $69 + 1 = 70$ paid sites. The interpretation of Slot uses the priced host's property language from Appendix D.2; the arithmetic blocks and their invariant do not depend on the particular meaning of a property code. A PAY guest step is one PAY host step surrounded by arithmetic that costs nothing, so the same ledger argument applies.

CHAPTER 8

The CHSH derivation, in full

Appendix 24 tells the story; this appendix does the algebra. Each item gives the statement with its hypotheses and the steps you need to check it. Nothing here says that a matrix is a physical state. Where quantum strategies come in, I say exactly how they stand to the matrices.

8.1 Setting and notation

Start with four piles of tally sheets, one for each pair of settings Alice and Bob can be given. On each round their answers either agree or disagree. Give agreement a score of $+1$ and disagreement a score of -1. In each pile, add the scores and divide by the number of rounds. That pile's average is its *correlator*. It says how strongly the two answers agree for that pair of settings.

For example, three agreements and one disagreement give a total of two over four rounds, hence an average of one half. The four averages make a little square table: Alice's setting chooses the row, Bob's chooses the column. The score I will study adds three cells of that table and subtracts the bottom-right one. Here are the names for those quantities.

Definition 8.1 (Settings, outcomes, correlators)**.** Alice receives a setting $x \in \{0,1\}$ and Bob a setting $y \in \{0,1\}$. Each answers a bit, recoded as an outcome $a, b \in \{+1,-1\}$ (bit 0 becomes $+1$, bit 1 becomes -1). The correlator E_{xy} is the average of the product ab over the rounds with settings (x,y). The CHSH quantity is

$$S = E_{00} + E_{01} + E_{10} - E_{11}.$$

I collect the four correlators into the 2×2 matrix

$$E = \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix},$$

whose rows are indexed by Alice's setting and whose columns by Bob's. The rows of E are (E_{00}, E_{01}) and (E_{10}, E_{11}); the columns are (E_{00}, E_{10}) and (E_{01}, E_{11}).

Definition 8.2 (Counters)**.** There are eight counts: for each setting pair, same_{xy} counts rounds whose two answer bits matched and diff_{xy} counts rounds whose bits differed. A round adds one to exactly one of the eight. Write

$$N_{xy} = \text{same}_{xy} + \text{diff}_{xy}, \qquad D_{xy} = \text{same}_{xy} - \text{diff}_{xy}, \qquad E_{xy} = \frac{D_{xy}}{N_{xy}}.$$

A matched round contributes $ab = +1$ and a differing round $ab = -1$, so D_{xy} is the sum of the products and D_{xy}/N_{xy} is their average. Since $|D_{xy}| \le N_{xy}$, every correlator from counters lies in $[-1, 1]$. When a bucket is empty, its correlator is defined as 0. The score is $E_{00} + E_{01} + E_{10} - E_{11}$.

8.2 The classical bound

Write a fixed plan on a card: Alice's answer to setting zero, her answer to setting one, and Bob's two answers. Each answer is a sign, plus or minus. Before multiplying anything, look at Bob's pair. Either the signs agree or they differ. If they agree, adding them gives plus or minus two and subtracting them gives zero. If they differ, the sum is zero and the difference is plus or minus two. One of those two quantities vanishes; the other cannot exceed two in size. The next calculation uses exactly that split.

Lemma 8.1 (Deterministic strategies)**.** Let $a_0, a_1, b_0, b_1 \in \{+1, -1\}$. Then $a_0b_0 + a_0b_1 + a_1b_0 - a_1b_1$ is 2 or -2. □

Proof. Factor: $a_0b_0 + a_0b_1 + a_1b_0 - a_1b_1 = a_0(b_0 + b_1) + a_1(b_0 - b_1)$. If $b_0 = b_1$ then $b_0 + b_1 = \pm 2$ and $b_0 - b_1 = 0$; if $b_0 \neq b_1$ the other way round. In both cases exactly one of the two brackets is ± 2 and the other is 0, so the expression is ± 2 times a sign, which is $+2$ or -2. There are sixteen sign choices, and every one of them gives exactly ± 2. None gives 0 or ± 4. ■

You can also just list the sixteen sign choices and add each one up. Nothing in the argument needs real numbers: the signs are whole numbers, so the bound holds the same way whether you read the correlators as rationals (what a counter gives) or as reals.

A fixed card is only the first case. You can shuffle among several cards, perhaps using some more often than others. Every card still tells each player an answer using that player's setting alone. The shuffle changes how often each card contributes; it does not change the score that a particular card would produce. Since each card's score lies between minus two and two, their average must stay between those same ends. The following definition writes out that weighted shuffle.

Definition 8.3 (Finite factorizable model)**.** Four real correlators $E_{00}, E_{01}, E_{10}, E_{11}$ are factorizable when there are a natural number $\lambda_{\max}$, weights $p(\lambda)$ and response signs $A(x,\lambda)$, $B(y,\lambda)$ for $\lambda = 0,\dots,\lambda_{\max}$ such that: $0 \le p(\lambda) \le 1$; $\sum_{\lambda=0}^{\lambda_{\max}} p(\lambda) = 1$; every $A(x,\lambda)$ and every $B(y,\lambda)$ is -1 or $+1$; and for all x, y, $E_{xy} = \sum_{\lambda=0}^{\lambda_{\max}} p(\lambda)A(x,\lambda)B(y,\lambda)$. In words: a hidden card λ is drawn with probability $p(\lambda)$, and each player reads an answer off the card using only their own setting. Only the correlators are modelled.

□ **Theorem 8.2** (The classical bound for a finite local model)**.** If E is factorizable, then $-2 \le E_{00} + E_{01} + E_{10} - E_{11} \le 2$.

Proof. Substitute the factorization: $S = \sum_\lambda p(\lambda)S_\lambda$, where S_λ is the deterministic expression of Lemma 8.1 at the signs $A(\cdot,\lambda), B(\cdot,\lambda)$. Each $S_\lambda \in [-2,2]$, the weights ■ are nonnegative and sum to 1, so $-2 = \sum_\lambda p(\lambda)(-2) \le S \le \sum_\lambda p(\lambda)\cdot 2 = 2$.

Remark 8.1 (Scope of the classical bound)**.** Fine's 1982 theorem is an equivalence between several descriptions of local models and the Bell inequalities. The preceding theorem proves one direction: a finite factorizable model obeys the bound. The converse of this single inequality is false: $(1,1,-1,1)$ has $S = 0$, but it breaks another of the sign variants ($E_{00} + E_{01} - E_{10} + E_{11} = 4$), so no local model gives it. The correlation-form equivalence needs each correlator between -1 and 1 and all eight inequalities; with those, Section 25.10 builds the local model. The preceding theorem is stated for finitely many hidden cards ($\lambda \le \lambda_{\max}$), and that loses nothing. Whatever the hidden variable is and however it is drawn, the correlators see it only through its four answer signs, so lumping together the cards that carry the same four signs gives at most sixteen cards and the same four correlators. All the lumping needs is a probability for each of the sixteen sign patterns.

The finite-tally statement concerns one fixed strategy replayed on every recorded round.

Definition 8.4 (A tally consistent with one local plan)**.** For bits $a_0, a_1, b_0, b_1 \in \{0,1\}$ and a counter record w, w is locally consistent with (a_0, a_1, b_0, b_1) when, for each setting pair (x,y): if $a_x = b_y$ then $\mathrm{diff}_{xy} = 0$, otherwise $\mathrm{same}_{xy} = 0$; and every bucket is sampled, $N_{00}, N_{01}, N_{10}, N_{11} > 0$.

Theorem 8.3 (A tally above 2 is not one replayed strategy)**.** Let w be a counter record whose rational score $S(w)$ is greater than 2. Then for all bits a_0, a_1, b_0, b_1, w is not locally consistent with (a_0, a_1, b_0, b_1). Every locally consistent record has $S(w) = 2$ or $S(w) = -2$; the first statement is that fact turned around. □

Proof. Under local consistency each sampled bucket is pure: if $a_x = b_y$ every round in the bucket matched, so $E_{xy} = N_{xy}/N_{xy} = +1$; otherwise every round differed and $E_{xy} = -1$. In signs, $E_{xy} = \alpha_x \beta_y$ with $\alpha_x = (-1)^{a_x}$ and $\beta_y = (-1)^{b_y}$. The score is then the deterministic expression of Lemma 8.1, which is 2 or -2. A score above 2 contradicts this. ■

The theorem rules out one explanation, a single four-bit plan used on every round. It says nothing about a local model that redraws its hidden state every round; Appendix 24 gives a four-round coin-flip tally with score 4. The clause that every bucket is sampled does work here. An empty bucket reads as 0, and the plan that answers 0 to everything, never asked the pair 11, would score $1 + 1 + 1 - 0 = 3$.

8.3 Tsirelson's bound from algebra

Now put the cards aside and look just at four numbers in the square. I want to bound their three-plus-one-minus score from their squared sizes. The useful trick is to write the proposed allowance minus the squared score as a pile of squares. A square cannot be negative. If the difference really is that pile, the score cannot outrun the allowance.

The identity below is the receipt for that trick. The letters a, b, c, d are the four entries read across the first row, then across the second. It holds for any four real numbers. After proving it, I will put in the two bounds on row length that this part of the checker supplies.

Lemma 8.4 (The six-square identity)**.** For all real a, b, c, d, □

$$4(a^2 + b^2 + c^2 + d^2) - (a + b + c - d)^2 = (a - b)^2 + (a - c)^2 + (a + d)^2 + (b - c)^2 + (b + d)^2 + (c + d)^2.$$

Proof. Expand the square on the left: $(a + b + c - d)^2 = a^2 + b^2 + c^2 + d^2 + 2ab + 2ac - 2ad + 2bc - 2bd - 2cd$. So the left side is

$$3(a^2 + b^2 + c^2 + d^2) - 2ab - 2ac + 2ad - 2bc + 2bd + 2cd.$$

On the right, each variable appears in exactly three of the six brackets (a in $a-b$, $a-c$, $a+d$; b in $a-b$, $b-c$, $b+d$; c in $a-c$, $b-c$, $c+d$; d in $a+d$, $b+d$, $c+d$), so the squared terms add to $3(a^2+b^2+c^2+d^2)$. The cross terms are $-2ab$ from $(a-b)^2$, $-2ac$ from $(a-c)^2$, $+2ad$ from $(a+d)^2$, $-2bc$ from $(b-c)^2$, $+2bd$ from $(b+d)^2$ and $+2cd$ from $(c+d)^2$. Both sides agree term by term.

Since the right side is a sum of squares, $S^2 \le 4\sum_{xy} E_{xy}^2$ for any four reals.

□ **Theorem 8.5** (Row-bound form of Tsirelson's bound)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$ with $E_{00}^2 + E_{01}^2 \le 1$ and $E_{10}^2 + E_{11}^2 \le 1$, $S^2 \le 8$. The absolute form $|S| \le \sqrt{8} = 2\sqrt{2}$ follows.

Proof. Adding the two row bounds gives $\sum_{xy} E_{xy}^2 \le 2$. By Lemma 25.3, $S^2 \le 4\sum_{xy} E_{xy}^2 \le 8$. ■

□ **Theorem 8.6** (Tightness)**.** The point $E_{00} = E_{01} = E_{10} = 1/\sqrt{2}$, $E_{11} = -1/\sqrt{2}$ satisfies both row bounds and has $S = \sqrt{8}$.

Proof. Each row has squared length $1/2 + 1/2 = 1$, and $S = 4/\sqrt{2} = 2\sqrt{2} = \sqrt{8}$. ■

Remark 8.2 (Where equality can happen)**.** Equality $S^2 = 8$ under the row bounds forces equality in both steps of the proof above: the six squares must all vanish, which gives $a = b = c = -d$, and the rows must be tight, which gives $2a^2 = 1$. So the point of Theorem 25.5 and its negative are the only places the bound is reached. A tally's correlators are ratios of whole numbers and $2a^2 = 1$ has no such solution, so no tally reaches the bound; the integer check's passing tallies stay strictly below it and come as near as you like. (Take rational a, b with $a^2 + b^2 = 1$, from a Pythagorean triple, and the correlators $(a, b, b, -a)$: they pass the check of Section 25.5 with no slack, and $S = 2(a+b)$, which gets as close to $2\sqrt{2}$ as you like as a/b approaches 1.)

□ **Lemma 8.7** (A rational constant above the bound)**.** $\sqrt{8} < 5657/2000$.

Proof. Both numbers are positive, so it suffices to compare squares. $5657^2 = 32{,}001{,}649$ and $8 \cdot 2000^2 = 32{,}000{,}000$, so $(5657/2000)^2 =$

$32{,}001{,}649/4{,}000{,}000 > 8 = (\sqrt{8})^2$. If $\sqrt{8} \geq 5657/2000$ held, squaring both nonnegative sides would give $8 \geq (5657/2000)^2$, a contradiction. ∎

The constant is the four-decimal round-up of $2\sqrt{2} = 2.8284271\ldots$; the gap is about 7.3×10^{-5}. It lets rational code test "above the Tsirelson bound" without a square root. It plays no part in the integer check of Section 25.5.

8.4 The zero-marginal 5×5 moment matrix

The four averages fill only the Alice–Bob part of a larger table. Give that table five labels: a constant reading 1, Alice's two possible readings, and Bob's two. Each cell records an average product of its row and column readings. A reading multiplied by itself is 1, so the diagonal is all ones. The four averages measured fill the cross-player cells and their reflected partners.

Other cells are still empty. This particular check fills them with zero: each player's average answer against the constant reading, and the cells between that player's two settings. These are choices of the check; the four cross-player averages alone did not supply them. I will later let the two within-player cells vary and ask whether any filling works. For now, write down this one fixed filling and test it.

Definition 8.5 (The pinned level-1 matrix)**.** Take rows and columns I, A_0, A_1, B_0, B_1. Put 1 on the diagonal and the four correlators in the Alice–Bob cells, and set the four one-party entries (each player's own average answer) and the two within-party entries (A_0 against A_1, B_0 against B_1) to zero. The resulting matrix is:

$$M(E) = \begin{array}{c} \\ I \\ A_0 \\ A_1 \\ B_0 \\ B_1 \end{array} \begin{array}{c} \begin{array}{ccccc} I & A_0 & A_1 & B_0 & B_1 \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{pmatrix} \end{array}$$

What do I ask of this table? Choose five numbers, one beside each row and one beside the matching column. For each cell, multiply its entry by its row's number and its column's number, then add all twenty-five results. Do this with any five numbers you like, positive, negative or zero. I require the total never to be negative. That requirement is called *positive semidefiniteness*, abbreviated PSD.

Write the five chosen numbers as a column v. The superscript T turns a column into a row, so ordinary matrix multiplication writes that same weighted total as

$v^{\mathrm{T}}Mv$. The condition is therefore $v^{\mathrm{T}}Mv \geq 0$ for every choice of v. The table is symmetric too: reflect it across its diagonal and the entries agree.

The first row and column have only one nonzero entry, the corner 1. Their contribution is the square of the first chosen number. The remaining four rows and columns can be tested on their own. Grouping them gives

$$M(E) = \begin{pmatrix} 1 & 0 \\ 0 & G \end{pmatrix}, \qquad G = \begin{pmatrix} I_2 & E \\ E^{\mathrm{T}} & I_2 \end{pmatrix}.$$

Here I_2 is the two-by-two identity table: ones on its diagonal and zeros elsewhere. The superscript on E again swaps rows and columns. The weighted total for $M(E)$ is the first number's square plus the weighted total for G. If every total for G is nonnegative, adding a square keeps it nonnegative. If some total for G is negative, choose the first number to be zero and the same negative total appears for $M(E)$. Thus the two tables pass or fail together.

The next operation is completing a square, with several numbers at once. For one number, $u^2 + 2uw + rw^2$ becomes $(u+w)^2 + (r-1)w^2$. The first square cannot hurt, and choosing $u = -w$ makes it vanish. So the whole expression stays nonnegative for every choice exactly when the remaining part does. The matrix formula below does the same job for two groups of numbers.

Here *positive definite* means that the weighted total for P is strictly positive for every nonzero choice of weights. In particular, P has an inverse, written P^{-1}: applying it undoes multiplication by P. The matrix left after completing the square is called the *Schur complement*. Its name is less useful here than the operation that produces it.

□ **Lemma 8.8** (Schur complement, the standard form)**.** Let P be symmetric positive definite. The symmetric block matrix $\begin{pmatrix} P & Q \\ Q^{\mathrm{T}} & R \end{pmatrix}$ is PSD if and only if $R - Q^{\mathrm{T}}P^{-1}Q$ is PSD.

Proof. For u, w of the right sizes,

$$\begin{pmatrix} u \\ w \end{pmatrix}^{\mathrm{T}} \begin{pmatrix} P & Q \\ Q^{\mathrm{T}} & R \end{pmatrix} \begin{pmatrix} u \\ w \end{pmatrix} = (u + P^{-1}Qw)^{\mathrm{T}} P\,(u + P^{-1}Qw) + w^{\mathrm{T}}(R - Q^{\mathrm{T}}P^{-1}Q)\,w.$$

The first term is never negative because P is positive definite, and it is zero at $u = -P^{-1}Qw$. So the form is nonnegative for all (u, w) exactly when the second term is nonnegative for all w. ■

The identity itself holds for any invertible symmetric P. One direction (block PSD gives the complement PSD) needs only that: put $u = -P^{-1}Qw$. The other direction is where P has to be positive definite. For the pinned matrix I don't need the lemma in the abstract: I carry out its one instance with $P = I_2$, $Q = E$, $R = I_2$, where the complement is

$$I_2 - E^{\mathrm{T}}E = \begin{pmatrix} 1 - E_{00}^2 - E_{10}^2 & -(E_{00}E_{01} + E_{10}E_{11}) \\ -(E_{00}E_{01} + E_{10}E_{11}) & 1 - E_{01}^2 - E_{11}^2 \end{pmatrix}.$$

The diagonal of $E^{\mathrm{T}}E$ holds the squared lengths of the columns of E, which is why the conditions below are column conditions.

Definition 8.6 (Column contractive)**.** Write $p = 1 - E_{00}^2 - E_{10}^2$, $q = 1 - E_{01}^2 - E_{11}^2$ and $s = E_{00}E_{01} + E_{10}E_{11}$. The correlators are column contractive when

$$p \geq 0, \qquad q \geq 0, \qquad p\,q - s^2 \geq 0.$$

Lemma 8.9 (The 2×2 test)**.** If $\alpha \geq 0$, $\delta \geq 0$ and $\alpha\delta - \beta^2 \geq 0$, then $\alpha u^2 + 2\beta uw + \delta w^2 \geq 0$ for all real u, w. □

Proof. If $w = 0$ the form is $\alpha u^2 \geq 0$. Otherwise put $t = u/w$, so the form is $w^2(\alpha t^2 + 2\beta t + \delta)$. If $\alpha = 0$, then $-\beta^2 \geq 0$ forces $\beta = 0$ and the bracket is $\delta \geq 0$. If $\alpha > 0$, $\alpha t^2 + 2\beta t + \delta = \alpha(t + \beta/\alpha)^2 + (\alpha\delta - \beta^2)/\alpha$, a sum of two nonnegative terms. ■

Theorem 8.10 (Contractive implies PSD)**.** If $p \geq 0$, $q \geq 0$ and $pq - s^2 \geq 0$, then $M(E)$ is positive semidefinite. □

Proof. Write $v = (v_0, x_0, x_1, y_0, y_1)$. Reading off the entries of $M(E)$,

$$v^{\mathrm{T}}M(E)\,v = v_0^2 + x_0^2 + x_1^2 + y_0^2 + y_1^2 + 2x_0(E_{00}y_0 + E_{01}y_1) + 2x_1(E_{10}y_0 + E_{11}y_1).$$

Complete the squares in x_0 and x_1:

$$\begin{aligned} v^{\mathrm{T}}M(E)\,v = v_0^2 &+ (x_0 + E_{00}y_0 + E_{01}y_1)^2 + (x_1 + E_{10}y_0 + E_{11}y_1)^2 \\ &+ p\,y_0^2 - 2s\,y_0y_1 + q\,y_1^2. \end{aligned}$$

To check the last three terms, expand the two squares: they contribute $(E_{00}y_0 + E_{01}y_1)^2 + (E_{10}y_0 + E_{11}y_1)^2$ beyond the cross terms, which is $(E_{00}^2 + E_{10}^2)y_0^2 + 2s\,y_0y_1 + (E_{01}^2 + E_{11}^2)y_1^2$; subtracting it from $y_0^2 + y_1^2$ leaves $p\,y_0^2 - 2s\,y_0y_1 + q\,y_1^2$.

The first three terms are squares. The last is the 2×2 form with $\alpha = p$, $\beta = -s$, $\delta = q$, nonnegative by Lemma 8.9. ■

□ **Lemma 8.11** (Discriminant)**.** If $a + 2bt + ct^2 \geq 0$ for every real t, then $b^2 \leq ac$.

Proof. If $c < 0$ the quadratic tends to $-\infty$, so $c \geq 0$. If $c = 0$, the line $a + 2bt$ is nonnegative for every t only if $b = 0$, and then $b^2 = 0 = ac$. If $c > 0$, evaluate at $t = -b/c$: $a - b^2/c \geq 0$, so $b^2 \leq ac$. ■

□ **Theorem 8.12** (PSD implies contractive)**.** If $M(E)$ is positive semidefinite, then $p \geq 0$, $q \geq 0$ and $pq - s^2 \geq 0$.

Proof. Feed the quadratic form three test vectors, written in the order (I, A_0, A_1, B_0, B_1). Each is the choice $x = -Ey$ that zeroes both completed squares in the proof of Theorem 8.10, so only $y^{\mathrm{T}}(I_2 - E^{\mathrm{T}}E)y$ is left.

First, $v = (0, -E_{00}, -E_{10}, 1, 0)$ gives $v^{\mathrm{T}}Mv = p$, so $p \geq 0$.

Second, $v = (0, -E_{01}, -E_{11}, 0, 1)$ gives $v^{\mathrm{T}}Mv = q$, so $q \geq 0$.

Third, for each real t, $v(t) = (0, -(E_{00}t + E_{01}), -(E_{10}t + E_{11}), t, 1)$ gives $v(t)^{\mathrm{T}}Mv(t) = p\,t^2 - 2s\,t + q \geq 0$. Apply Lemma 8.11 with $a = q$, $b = -s$, $c = p$: $s^2 \leq pq$. ■

□ **Corollary 8.13** (The equivalence)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$, the symmetric matrix $M(E)$ is positive semidefinite if and only if the correlators are column contractive.

Proof. Theorem 8.10 is one direction and Theorem 8.12 the other. ■

The equivalence above is about the columns. The bound can go through either the rows or the columns: the six-square identity only needs the four squares to add to at most 2, and the two row bounds give that, and so do $p \geq 0$ and $q \geq 0$.

□ **Theorem 8.14** (PSD implies Tsirelson)**.** If the pinned matrix $M(E)$ is symmetric and positive semidefinite at $E_{00}, E_{01}, E_{10}, E_{11}$, then $S^2 \leq 8$ and $|S| \leq \sqrt{8}$.

Proof. Feed the form the vector that is 1 at A_0, $-E_{00}$ at B_0, $-E_{01}$ at B_1 and 0 elsewhere. Reading off the entries, the form is $1 + E_{00}^2 + E_{01}^2 - 2E_{00}^2 - 2E_{01}^2 = 1 - E_{00}^2 - E_{01}^2$ (the B_0B_1 cell is 0), so $E_{00}^2 + E_{01}^2 \le 1$. The vector that is 1 at A_1, $-E_{10}$ at B_0 and $-E_{11}$ at B_1 gives $E_{10}^2 + E_{11}^2 \le 1$ the same way. Then Theorem 25.4 applies. ∎

Here are the three conditions and the score at a few points, all by direct arithmetic. In this table, $a = 3/5$, $b = 4/5$ and $t = 1/\sqrt{2}$.

point $(E_{00}, E_{01}, E_{10}, E_{11})$	S	p	q	$pq - s^2$	**contractive**
$(0, 0, 0, 0)$	0	1	1	1	yes
$(0.6, 0.6, 0.6, -0.6)$	2.4	0.28	0.28	0.0784	yes
$(a, b, b, -a)$	2.8	0	0	0	yes, with no slack
$(t, t, t, -t)$	$2\sqrt{2}$	0	0	0	yes, with no slack
$(1, 1, 1, 1)$, all-ones plan	2	-1	-1		no
$(1, 0, 1, 0)$, a classical mixture	2	-1	1		no
$(1, 1, 1, -1)$, PR box	4	-1	-1		no

Every deterministic plan has all four correlators equal to ± 1, so $p = 1 - 1 - 1 = -1$ and every one of the sixteen fails.

Remark 8.3 (What PSD of the pinned matrix means)**.** $M(E)$ is the level-1 moment matrix with the two within-party entries $\langle A_0A_1 \rangle$ and $\langle B_0B_1 \rangle$ pinned to zero. Its PSD is sufficient for the correlators to admit some PSD completion (Section 25.7), and it is not necessary: the all-ones plan has a completion but fails here. So a check built on $M(E)$ is sound for the completion set and not complete for it, and it rejects some honest classical tallies. That correlators with a PSD completion are exactly those reachable by quantum measurements is Tsirelson's theorem, and both halves are proved here in finite dimension. For real and for complex amplitudes, a strategy's correlators are inner products of unit vectors (with complex amplitudes, use real vectors of twice the length) and its level-1 moment matrix is PSD, so they have a PSD completion. The other half, that every PSD completion comes from a quantum strategy, comes from this construction: the strategy is built on two four-level systems from the completion's Gram vectors (Section 25.8).

8.5 The integer check

The machine has whole-number tallies. The three conditions above were written using averages, which are fractions. You can test the same conditions without ever dividing: first make sure each denominator is positive, then multiply it away. Multiplying by a positive number cannot turn a passing inequality into a failing one or a failing one into a passing one.

For the first column, the condition starts as one minus two squared averages. Multiplying by the product of their squared denominators leaves three whole-number products. I call that result A. The second column gives B the same way. The cross-column term is a sum of two products of averages; multiplying by all four denominators leaves a whole number I call C. The last condition becomes $C^2 \le AB$, because both sides have had the same positive denominator removed. Four positive bucket sizes and these three comparisons make seven checks. Here are the products to calculate.

Definition 8.7 (The seven-integer check)**.** From the counters of Definition 25.2, with all arithmetic in $\mathbb{Z}$, let

$$\begin{aligned} A &= N_{00}^2 N_{10}^2 - D_{00}^2 N_{10}^2 - D_{10}^2 N_{00}^2, \\ B &= N_{01}^2 N_{11}^2 - D_{01}^2 N_{11}^2 - D_{11}^2 N_{01}^2, \\ C &= D_{00} D_{01} N_{10} N_{11} + D_{10} D_{11} N_{00} N_{01}. \end{aligned}$$

The check is the conjunction of seven facts, in this order: $0 < N_{00}$, $0 < N_{01}$, $0 < N_{10}$, $0 < N_{11}$, $0 \le A$, $0 \le B$, and $C \cdot C \le A \cdot B$.

There is no division, no real number, no square root and no S in it.

□ **Lemma 8.15** (The seven facts are the three conditions with denominators cleared)**.** Assume $N_{00}, N_{01}, N_{10}, N_{11} > 0$ and put $E_{xy} = D_{xy}/N_{xy}$. Then, with p, q, s as in Definition 25.6,

$$p = \frac{A}{N_{00}^2 N_{10}^2}, \qquad q = \frac{B}{N_{01}^2 N_{11}^2}, \qquad s = \frac{C}{N_{00} N_{01} N_{10} N_{11}},$$

$$pq - s^2 = \frac{AB - C^2}{(N_{00} N_{01} N_{10} N_{11})^2}.$$

Because every denominator is positive, $0 \le A \iff p \ge 0$, $0 \le B \iff q \ge 0$ and $C^2 \le AB \iff s^2 \le pq$.

Proof. For p: $1 - D_{00}^2/N_{00}^2 - D_{10}^2/N_{10}^2$; multiply by $N_{00}^2 N_{10}^2$ to get $N_{00}^2 N_{10}^2 - D_{00}^2 N_{10}^2 - D_{10}^2 N_{00}^2 = A$. For q the same with columns 01 and 11, multiplied by $N_{01}^2 N_{11}^2$. For s:

$$\frac{D_{00}}{N_{00}}\frac{D_{01}}{N_{01}} + \frac{D_{10}}{N_{10}}\frac{D_{11}}{N_{11}} = \frac{D_{00}D_{01}N_{10}N_{11} + D_{10}D_{11}N_{00}N_{01}}{N_{00}N_{01}N_{10}N_{11}} = \frac{C}{N_{00}N_{01}N_{10}N_{11}}.$$

Then $pq = AB/(N_{00}^2 N_{10}^2 N_{01}^2 N_{11}^2) = AB/(N_{00}N_{01}N_{10}N_{11})^2$ and $s^2 = C^2/(N_{00}N_{01}N_{10}N_{11})^2$; subtract. Multiplying an inequality by a positive number preserves it in both directions. ∎

The check has the following guarantee:

Theorem 8.16 (Soundness of the integer check)**.** If a tally w passes the seven-integer check, then its four correlators $E_{xy} = D_{xy}/N_{xy}$ are column contractive. Consequently its pinned matrix is positive semidefinite. □

Proof. The first four tests say that every bucket contains a round, so each average is its signed total divided by a positive count. The next two tests say that A and B are nonnegative. Dividing each by its positive denominator gives $p \geq 0$ and $q \geq 0$. The last test says $AB - C^2 \geq 0$. Divide that difference by the positive square of the product of all four counts; the result is $pq - s^2 \geq 0$, as the preceding calculation shows. Those are exactly the three column conditions, and Theorem 8.10 shows that they make the matrix pass the nonnegative-total test. ∎

Corollary 8.17 (A passed check implies the Tsirelson bound)**.** If a tally w passes the seven-integer check, the correlators of w satisfy $S^2 \leq 8$. It is the composition of Theorem 25.10, Corollary 25.7 and Theorem 25.8. □

Two of the seven facts carry the bound by themselves. With every pair sampled, $0 \leq A$ and $0 \leq B$ are $p \geq 0$ and $q \geq 0$, so the four squared correlators add to at most 2 and Lemma 25.3 gives $S^2 \leq 8$. The fact $C \cdot C \leq A \cdot B$ is what positive semidefiniteness adds, and the bound doesn't use it. For the form this takes on the small machine, where a raised flag has to stand on a passing check, see Appendix 24.

A worked tally. Take $(\text{same}, \text{diff}) = (4,1),(4,1),(4,1),(1,4)$ for the pairs $00, 01, 10, 11$. Then $N_{xy} = 5$ for all four, $D_{00} = D_{01} = D_{10} = 3$, $D_{11} = -3$, and the correlators are $0.6, 0.6, 0.6, -0.6$ with $S = 2.4$. Compute:

$$A = 25 \cdot 25 - 9 \cdot 25 - 9 \cdot 25 = 625 - 225 - 225 = 175,$$
$$B = 25 \cdot 25 - 9 \cdot 25 - 9 \cdot 25 = 175,$$
$$C = 3 \cdot 3 \cdot 5 \cdot 5 + 3 \cdot (-3) \cdot 5 \cdot 5 = 225 - 225 = 0.$$

All seven facts hold: four positive N, $0 \le 175$ twice, and $0 \le 175 \cdot 175$. Dividing back, $p = 175/625 = 0.28$ and $s = 0$, the second row of the table in Section 25.4.

Two more, for contrast. One round per pair from the all-ones plan, $(1,0)$ four times, gives $N = D = 1$ everywhere and $A = 1 - 1 - 1 = -1 < 0$: the check fails. The tally $(4,1),(9,1),(9,1),(1,4)$ gives correlators $3/5, 4/5, 4/5, -3/5$, with $N = 5, 10, 10, 5$ and $D = 3, 8, 8, -3$: $A = 25 \cdot 100 - 9 \cdot 100 - 64 \cdot 25 = 0$, $B = 100 \cdot 25 - 64 \cdot 25 - 9 \cdot 100 = 0$ and $C = 3 \cdot 8 \cdot 10 \cdot 5 + 8 \cdot (-3) \cdot 5 \cdot 10 = 0$. It passes with no slack, at $S = 2.8$.

8.6 Level 1+AB

The first table had one place for each player's answer. Enlarge it by giving each Alice–Bob product a place too. There are four such products, so the five places become nine. A cell still asks for the average product of its row and column labels. Some of those products reduce to entries I already have; five more numbers are needed for the particular filling below. Call them γ_1 through γ_5. The longer labels say which products they stand for.

The five additional parameters name the moments $\gamma_1 = \langle A_0 A_1 B_0 \rangle$, $\gamma_2 = \langle A_0 A_1 B_1 \rangle$, $\gamma_3 = \langle A_0 B_0 B_1 \rangle$, $\gamma_4 = \langle A_1 B_0 B_1 \rangle$ and $\gamma_5 = \langle A_0 A_1 B_0 B_1 \rangle$. These are labels for the entries; the definition below does not construct operators.

Definition 8.8 (The level-1+AB matrix)**.** The 9×9 real matrix on rows and columns $I, A_0, A_1, B_0, B_1, A_0B_0, A_0B_1, A_1B_0, A_1B_1$, with the following entries:

	I	A_0	A_1	B_0	B_1	A_0B_0	A_0B_1	A_1B_0	A_1B_1
I	1	0	0	0	0	E_{00}	E_{01}	E_{10}	E_{11}
A_0	0	1	0	E_{00}	E_{01}	0	0	γ_1	γ_2
A_1	0	0	1	E_{10}	E_{11}	γ_1	γ_2	0	0
B_0	0	E_{00}	E_{10}	1	0	0	γ_3	0	γ_4
B_1	0	E_{01}	E_{11}	0	1	γ_3	0	γ_4	0
A_0B_0	E_{00}	0	γ_1	0	γ_3	1	0	0	γ_5
A_0B_1	E_{01}	0	γ_2	γ_3	0	0	1	$-\gamma_5$	0
A_1B_0	E_{10}	γ_1	0	0	γ_4	0	$-\gamma_5$	1	0
A_1B_1	E_{11}	γ_2	0	γ_4	0	γ_5	0	0	1

Two constraints are fixed by this layout. First, the four-body cells carry opposite signs: $+\gamma_5$ at (A_0B_0, A_1B_1) and $-\gamma_5$ at (A_0B_1, A_1B_0). In a real moment matrix those two cells are $\langle A_0A_1B_0B_1\rangle$ and $\langle A_0A_1B_1B_0\rangle$, so the tie says $\langle A_0A_1(B_0B_1 + B_1B_0)\rangle = 0$, which for a real matrix is $\langle\{A_0, A_1\} \otimes \{B_0, B_1\}\rangle = 0$. The level-1 pinnings don't imply it. Take a classical strategy: Alice's two answers a_0, a_1 are independent fair signs, and Bob copies them, $b_0 = a_0$ and $b_1 = a_1$. Every pinned cell of its moment matrix is 0 and both four-body cells are 1, so no choice of the nine numbers gives its matrix. So the matrix here is level 1+AB cut down to the slice $\langle\{A_0, A_1\} \otimes \{B_0, B_1\}\rangle = 0$, and that is what "level 1+AB" names in this book. The slice holds whenever A_0 and A_1 anticommute. The soundness theorems below are about this matrix, so they stand as stated. Second, the matrix keeps the level-1 pinning: every cell whose label reduces to a marginal or to $\langle A_0A_1\rangle$ or $\langle B_0B_1\rangle$ is zero, for instance (A_0, A_0B_0), (A_0B_0, A_0B_1) and (A_0B_0, A_1B_0). The displayed matrix is symmetric.

I want to test this larger table with the same operation as before: put a weight beside each row and column and add the weighted cells. The first three weights will each be collected into one completed square. Everything involving the other six will be left over. That leftover is what I call the residual; it is the part whose sign I still need to settle.

Write a vector as $(v_I, v_{A_0}, v_{A_1}, v_{B_0}, v_{B_1}, v_{00}, v_{01}, v_{10}, v_{11})$, where v_{xy} is the coordinate of A_xB_y. Put

$$
\begin{aligned}
L_1 &= E_{00}v_{00} + E_{01}v_{01} + E_{10}v_{10} + E_{11}v_{11},\\
L_2 &= E_{00}v_{B_0} + E_{01}v_{B_1} + \gamma_1 v_{10} + \gamma_2 v_{11},\\
L_3 &= E_{10}v_{B_0} + E_{11}v_{B_1} + \gamma_1 v_{00} + \gamma_2 v_{01}.
\end{aligned}
$$

Definition 8.9 (The residual)**.**

$$\begin{aligned} R = {} & v_{B_0}^2 + v_{B_1}^2 + v_{00}^2 + v_{01}^2 + v_{10}^2 + v_{11}^2 \\ & + 2v_{B_0}(\gamma_3 v_{01} + \gamma_4 v_{11}) + 2v_{B_1}(\gamma_3 v_{00} + \gamma_4 v_{10}) \\ & + 2\gamma_5(v_{00}v_{11} - v_{01}v_{10}) - L_1^2 - L_2^2 - L_3^2. \end{aligned}$$

The condition called column contractivity at level 1+AB is $R \geq 0$ for all six real inputs.

□ **Lemma 8.18** (Three squares plus the residual)**.** For every vector,

$$v^{\mathrm{T}} M_9\, v = (v_I + L_1)^2 + (v_{A_0} + L_2)^2 + (v_{A_1} + L_3)^2 + R.$$

Proof. The coordinates v_I, v_{A_0}, v_{A_1} appear in the form only through their diagonal squares and the cross terms $2v_I L_1$, $2v_{A_0} L_2$, $2v_{A_1} L_3$, read off rows I, A_0, A_1 of the matrix. Completing each square adds $L_1^2 + L_2^2 + L_3^2$, which the residual subtracts. What is left in R is the diagonal of the remaining six coordinates and the cross terms among them, read off the lower right 6×6 block. ■

□ **Theorem 8.19** (PSD at level 1+AB)**.** For all real E_{xy} and $\gamma_1, \ldots, \gamma_5$, the matrix of Definition 25.7 is positive semidefinite if and only if the residual R is nonnegative for every choice of its six real inputs.

Proof. If $R \geq 0$ everywhere, Lemma 8.18 writes the form as three squares plus R. Conversely, given the six residual inputs, choose $v_I = -L_1$, $v_{A_0} = -L_2$, $v_{A_1} = -L_3$; the three squares vanish and PSD gives $R \geq 0$. This is the level-1 argument of Section 25.4 one size up. ■

As a quadratic form in $(v_{B_0}, v_{B_1}, v_{00}, v_{01}, v_{10}, v_{11})$, numbered 1 to 6, the residual has the symmetric coefficient matrix H with these upper entries. Each entry can be read back off R by collecting the coefficient of $v_i v_j$.

$$
\begin{aligned}
H_{11} &= 1 - E_{00}^2 - E_{10}^2, & H_{22} &= 1 - E_{01}^2 - E_{11}^2,\\
H_{12} &= -(E_{00}E_{01} + E_{10}E_{11}), & H_{13} &= -E_{10}\gamma_1,\\
H_{14} &= \gamma_3 - E_{10}\gamma_2, & H_{15} &= -E_{00}\gamma_1,\\
H_{16} &= \gamma_4 - E_{00}\gamma_2, & H_{23} &= \gamma_3 - E_{11}\gamma_1,\\
H_{24} &= -E_{11}\gamma_2, & H_{25} &= \gamma_4 - E_{01}\gamma_1,\\
H_{26} &= -E_{01}\gamma_2, & H_{33} &= 1 - E_{00}^2 - \gamma_1^2,\\
H_{34} &= -(E_{00}E_{01} + \gamma_1\gamma_2), & H_{35} &= -E_{00}E_{10},\\
H_{36} &= \gamma_5 - E_{00}E_{11}, & H_{44} &= 1 - E_{01}^2 - \gamma_2^2,\\
H_{45} &= -\gamma_5 - E_{01}E_{10}, & H_{46} &= -E_{01}E_{11},\\
H_{55} &= 1 - E_{10}^2 - \gamma_1^2, & H_{56} &= -(E_{10}E_{11} + \gamma_1\gamma_2),\\
H_{66} &= 1 - E_{11}^2 - \gamma_2^2.
\end{aligned}
$$

The top left 2×2 block is $I_2 - E^{\mathrm{T}}E$ from Section 25.4.

The full check does not decide $R \geq 0$ directly. It tests the stronger fact that H is positive definite, by two Schur steps and a Sylvester test, all fraction-free.

Lemma 8.20 (Fraction-free Schur step)**.** For a symmetric $n \times n$ matrix h and any vector v, with $S_{ij} = h_{11}h_{ij} - h_{1i}h_{1j}$ for $i, j \geq 2$ (here S is the Schur matrix; the score plays no part in this lemma), □

$$h_{11}\, v^{\mathrm{T}} h\, v = \Big(\sum_j h_{1j} v_j\Big)^2 + \sum_{i,j \geq 2} S_{ij} v_i v_j.$$

So if $h_{11} > 0$ and the $(n-1) \times (n-1)$ matrix S gives a nonnegative form, then h does too. The cascade uses this identity at sizes six and five.

Proof. Expand the square: $h_{11}^2 v_1^2 + 2h_{11}v_1 \sum_{j\geq 2} h_{1j}v_j + (\sum_{j\geq 2} h_{1j}v_j)^2$. The left side is $h_{11}^2 v_1^2 + 2h_{11}v_1 \sum_{j\geq 2} h_{1j}v_j + h_{11} \sum_{i,j\geq 2} h_{ij}v_i v_j$. The difference is $\sum_{i,j\geq 2}(h_{11}h_{ij} - h_{1i}h_{1j})v_i v_j$. Dividing by $h_{11} > 0$ keeps the sign. ■

Lemma 8.21 (Sylvester for 4×4)**.** Let h be symmetric 4×4 with leading principal minors d_1, d_2, d_3, d_4. If d_1, d_2, d_3 are strictly positive and $d_4 \geq 0$, then $v^{\mathrm{T}} h\, v \geq 0$ for all v. □

Proof. Use the fraction-free factorization

$$d_1^2 d_2 d_3 \, v^{\mathrm{T}} h \, v = d_1 d_2 d_3 \, P_1^2 + d_1 d_3 \, Q_1^2 + Q_2^2 + d_1^2 d_2 d_4 \, v_4^2,$$

where, writing $e_{ij} = h_{11} h_{ij} - h_{1i} h_{1j}$,

$$\begin{aligned} P_1 &= \sum_j h_{1j} v_j, \\ Q_1 &= d_2 v_2 + e_{23} v_3 + e_{24} v_4, \\ Q_2 &= d_1 d_3 v_3 + (d_2 e_{34} - e_{23} e_{24}) v_4. \end{aligned}$$

The identity is Lemma 8.20 three times over, each time on the matrix the last one left. With $f_{ij} = e_{22} e_{ij} - e_{2i} e_{2j}$ for $i, j \geq 3$, the lemma gives $d_1 \, v^{\mathrm{T}} h \, v = P_1^2 + \sum_{i,j \geq 2} e_{ij} v_i v_j$, then $e_{22} \sum_{i,j \geq 2} e_{ij} v_i v_j = Q_1^2 + \sum_{i,j \geq 3} f_{ij} v_i v_j$, then $f_{33} \sum_{i,j \geq 3} f_{ij} v_i v_j = Q_2^2 + (f_{33} f_{44} - f_{34}^2) \, v_4^2$. The corners are minors of h, by one fact about determinants: the $k \times k$ leading minor of e is h_{11}^{k-1} times the $(k+1) \times (k+1)$ leading minor of h. To see it, multiply rows 2 to $k+1$ of that block of h by h_{11}, which multiplies its determinant by h_{11}^k, then subtract h_{1i} times the first row from row i, which changes nothing; those rows now read $0, e_{i2}, \ldots, e_{i,k+1}$, so the determinant is h_{11} times the leading minor of e. So $e_{22} = d_2$ and $f_{33} = e_{22} e_{33} - e_{23}^2 = d_1 d_3$, and the same fact applied to e, whose corner is d_2, gives $f_{33} f_{44} - f_{34}^2 = d_2 \det e = d_1^2 d_2 d_4$. Multiply the first equation by $d_1 d_2 d_3$ and substitute the other two. With $d_1, d_2, d_3 > 0$ and $d_4 \geq 0$ the right side is nonnegative and the factor on the left is positive. ∎

The cascade for the full check runs on H. Step one: $H_{11} > 0$ and the 5×5 matrix $S_{ij}^{(6)} = H_{11} H_{ij} - H_{1i} H_{1j}$, $2 \leq i, j \leq 6$, gives a nonnegative form. Step two: $S_{22}^{(6)} > 0$ and the 4×4 matrix $S_{ij}^{(5)} = S_{22}^{(6)} S_{ij}^{(6)} - S_{2i}^{(6)} S_{2j}^{(6)}$, $3 \leq i, j \leq 6$, gives a nonnegative form. Step three: the four leading minors of $S^{(5)}$ are positive. Together these conditions make the residual nonnegative, and so the nine-by-nine matrix positive semidefinite.

Definition 8.10 (What the integer check computes)**.** The supplied moments arrive as bucket pairs: $\gamma_k = N_{g_k} / D_{g_k}$ with $N_{g_k} = \mathrm{same}_{g_k} - \mathrm{diff}_{g_k}$ and $D_{g_k} = \mathrm{same}_{g_k} + \mathrm{diff}_{g_k}$. Watch the letters: for these five buckets N names the numerator and D the denominator: the difference and the count, the reverse of Section 25.1. Let $K = (N_{00} N_{01} N_{10} N_{11} D_{g_1} D_{g_2} D_{g_3} D_{g_4} D_{g_5})^2$. The check computes the 21 integers $K H_{ij}$, the 15 integers of $S^{(6)}$ at scale K^2 and the 10 integers of $S^{(5)}$ at scale K^4, each by $h_{11} h_{ij} - h_{1i} h_{1j}$ on integers. The decider is the conjunction of: $N_{xy} > 0$ for the four pairs; $D_{g_k} > 0$ for $k = 1..5$; $-D_{g_k} < N_{g_k} < D_{g_k}$ for $k = 1..5$; and six strict positivities, namely $K H_{11} > 0$, $K^2 S_{22}^{(6)} > 0$, and the four leading minors of the

integer $S^{(5)}$. The full check combines the seven-integer check of Definition 8.7 with this decider.

Each cleared integer, read as a real, equals the corresponding real quantity times the stated positive power of K. A $k \times k$ leading minor of a matrix scaled by $c > 0$ scales by c^k. So every integer positivity holds exactly when the real one does.

Two parts of the full check are there to fail early, and no proof below needs them. A passing decider makes H positive definite (Remark 8.4), and the top left two-by-two block of H has p and q on its diagonal and $-s$ off it, so $p > 0$, $q > 0$ and $pq > s^2$, and the seven-integer check passes. And with H positive definite, the nine-by-nine form is zero only at zero: it is three squares plus R (Lemma 8.18), so where it is zero the six inputs of R are zero, then so are L_1, L_2 and L_3, and then so are v_I, v_{A_0} and v_{A_1}. So the nine-by-nine matrix is positive definite; with ones on its diagonal, that puts every γ_k strictly between -1 and 1, which is the bucket bound $-D_{g_k} < N_{g_k} < D_{g_k}$.

Theorem 8.22 (The full 1+AB check implies PSD)**.** Let the counts make every N_{xy} and every D_{g_k} positive, and let the decider just defined pass on them. Then the matrix of Definition 25.7 at the four correlators and $\gamma_k = N_{g_k}/D_{g_k}$ is symmetric and positive semidefinite. □

Proof. The bucket conjuncts make every denominator positive, the scaling remark just above carries the six integer positivities over to the real cascade, and Lemma 8.20 twice plus Lemma 8.21 give $R \geq 0$, and so positive semidefiniteness by Theorem 25.12. ■

Remark 8.4 (Interior only)**.** Every test in the cascade is strict, so a passing check certifies that H is positive definite, which is stronger than semidefinite. A point where H is PSD but singular cannot pass, and neither can a moment with $|\gamma_k| = 1$. For example, the tally of Section 25.5 with correlators $(3/5, 4/5, 4/5, -3/5)$ has $H_{11} = 1 - 9/25 - 16/25 = 0$, so the full 1+AB check fails on it for every choice of γ, though the level-1 check of Section 25.5 accepts it. The argument establishes soundness only; no converse is stated.

A worked example of the cascade in exact rational arithmetic. Take the correlators $(0.6, 0.6, 0.6, -0.6)$ and $\gamma = (0, 0, 0, 0, -1/2)$; as buckets, γ_1 to γ_4 are $(1, 1)$, giving $0/2$, and γ_5 is $(1, 3)$, giving $-2/4$, both inside the strict bounds. Then $H_{11} = H_{22} = 7/25$, $H_{12} = 0$, and every entry H_{1j}, H_{2j} with $j \geq 3$ is zero. So

$S^{(6)}_{22} = (7/25)^2 = 49/625$, and $S^{(5)}$ is a positive multiple of the lower right block of H:

$$S^{(5)} = \frac{343}{781250}\begin{pmatrix} 32 & -18 & -18 & -7 \\ -18 & 32 & 7 & 18 \\ -18 & 7 & 32 & 18 \\ -7 & 18 & 18 & 32 \end{pmatrix},$$

where $50H_{33} = 32$, $50H_{34} = 50H_{35} = -18$, $50H_{36} = 50(-1/2 + 0.36) = -7$, $50H_{45} = 50(1/2 - 0.36) = 7$, and $50H_{46} = 50H_{56} = 18$. The leading minors of the integer matrix are 32, 700, 15000 and 140625, all positive. With $H_{11} = 7/25 > 0$ and $S^{(6)}_{22} > 0$, all six tests pass, and with Section 25.5's $A = B = 175$, $C = 0$ the full check passes. At $\gamma_5 = 0$ instead, the same block becomes $I_4 - \mathbf{e}\mathbf{e}^{\mathrm{T}}$ with $\mathbf{e} = (0.6, 0.6, 0.6, -0.6)$, whose third leading minor is negative ($\frac{1}{25^3}$ times -1250), and the check fails.

That last fact is general. The zero-moment check takes no γ buckets; it combines the level-1 check with the integer form of $E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \le 1$ with denominators cleared by $(N_{00}N_{01}N_{10}N_{11})^2$.

□ **Theorem 8.23** (The zero-moment check sits inside the classical bound)**.** If the zero-moment check accepts a tally w, then the four correlators of w satisfy $E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \le 1$ and $|S| \le 2$.

Proof. The check's extra conjunct, divided back by the positive factor $(N_{00}N_{01}N_{10}N_{11})^2$, is $r = E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \le 1$. Then Lemma 25.3 gives $S^2 \le 4r \le 4$. ■

The level-1 half of that check follows from $r \le 1$ on its own: p and q are each at least $1 - r$, and $pq - s^2 = 1 - r + (E_{00}E_{11} - E_{01}E_{10})^2$.

This is exactly the level-1+AB condition at $\gamma = 0$. There the residual splits into the level-1 form in (v_{B_0}, v_{B_1}) and $|\mathbf{v}|^2 - \langle \mathbf{e}, \mathbf{v}\rangle^2$ in $\mathbf{v} = (v_{00}, v_{01}, v_{10}, v_{11})$, where $\mathbf{e} = (E_{00}, E_{01}, E_{10}, E_{11})$. The second piece is nonnegative for every $\mathbf{v}$ exactly when $r \le 1$: if $r \le 1$, Cauchy–Schwarz gives $\langle \mathbf{e}, \mathbf{v}\rangle^2 \le r\,|\mathbf{v}|^2 \le |\mathbf{v}|^2$; if $r > 1$, the choice $\mathbf{v} = \mathbf{e}$ gives $r - r^2 < 0$.

So the zero-moment check certifies a strict part of the classical region: no point above $|S| = 2$, and none of the sixteen deterministic points either, since each has $|\mathbf{e}|^2 = 4$. A point above 2 needs supplied moments. The matrix is positive semidefinite at correlators $(3/5, 3/5, 3/5, -3/5)$, $\gamma = (0, 0, 0, 0, -1/2)$, with $S = 12/5$. The cleanest way to see it is the γ_5 slice, $\gamma_1 = \cdots = \gamma_4 = 0$. There the residual splits into the level-1 form in (v_{B_0}, v_{B_1}) and the form of $J - \mathbf{e}\mathbf{e}^{\mathrm{T}}$ in the four v_{xy}, where J is I_4 with γ_5 at $(00, 11)$ and $-\gamma_5$ at $(01, 10)$. For $-1 < \gamma_5 < 1$, J

is positive definite, and the second form is nonnegative exactly when $\mathbf{e}^{\mathrm{T}} J^{-1}\mathbf{e} \le 1$: apply Lemma 25.6 to the block matrix $\begin{pmatrix} 1 & \mathbf{e}^{\mathrm{T}} \\ \mathbf{e} & J \end{pmatrix}$ once with the corner 1 as P and once, rows reordered, with J as P. Inverting the two 2×2 blocks of J and multiplying out, that is

$$(1-\gamma_5)\big[(E_{00}+E_{11})^2+(E_{01}-E_{10})^2\big] + (1+\gamma_5)\big[(E_{00}-E_{11})^2+(E_{01}+E_{10})^2\big] \le 2(1-\gamma_5^2).$$

At this point the left side is $\frac{3}{2}\cdot 0 + \frac{1}{2}\cdot(\frac{36}{25}+\frac{36}{25}) = \frac{36}{25} = 1.44$ and the right side is $2\cdot\frac{3}{4} = 1.5$, and the level-1 part holds ($p = q = 0.28$, $s = 0$). The slice test is an illustration, not a third check: on the counters $(4,1),(4,1),(4,1),(1,4)$ and the bucket $(1,3)$, the same comparison with the denominators multiplied out says yes, and the full check of Definition 8.10 accepts the same counters with γ_1 to γ_4 given as the buckets $(1,1)$, as the worked example above shows.

Both 1+AB checks, the full check and the zero-moment check, contain the seven-integer check as a conjunct, so a passing 1+AB check also passes the level-1 check and inherits $S^2 \le 8$ from Section 25.5. This follows directly by composing the definitions and the bound.

8.7 The elliptope completion

Go back to the two within-player cells I filled with zero. Leave them blank this time. Keep all four measured cross-player averages fixed, and ask whether there is any way to fill those two blanks that makes the table pass. A failure with zeros no longer settles the question; another filling may work.

Call the first missing number x and the second y, and copy each into its reflected cell so the table stays symmetric. The collection of averages that admit a passing filling has a name, the *elliptope* completion set. Membership means that at least one choice of those two numbers works. It does not require every choice to work, or require the zero choice to work.

Definition 8.11 (Completion and membership)**.** Complete the matrix of Section 25.4 by setting the within-party entries to free reals: x in the (A_0, A_1) cells and y in the (B_0, B_1) cells, marginals still zero. The correlators are elliptope realizable when some x, y make the completed matrix symmetric and PSD.

$$M(E;x,y) = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & x & E_{00} & E_{01} \\ 0 & x & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & y \\ 0 & E_{01} & E_{11} & y & 1 \end{pmatrix}$$

Its quadratic form at $v = (v_0, \ldots, v_4)$ is $\sum_k v_k^2 + 2xv_1v_2 + 2yv_3v_4 + 2E_{00}v_1v_3 + 2E_{01}v_1v_4 + 2E_{10}v_2v_3 + 2E_{11}v_2v_4$. The pinned matrix is the case $x = y = 0$, so PSD of $M(E)$ implies membership.

□ **Theorem 8.24** (Deterministic plans are inside)**.** If $a_0^2 = a_1^2 = b_0^2 = b_1^2 = 1$, then $(a_0b_0, a_0b_1, a_1b_0, a_1b_1)$ is elliptope realizable, with $x = a_0a_1$ and $y = b_0b_1$.

Proof. With these entries the form equals $v_0^2 + (a_0v_1 + a_1v_2 + b_0v_3 + b_1v_4)^2$. Expanding the second square gives $a_0^2v_1^2 + \cdots$ on the diagonal, which is $v_1^2 + \cdots$ by the unit squares, and the cross terms $2a_0a_1v_1v_2 = 2xv_1v_2$, $2a_0b_0v_1v_3 = 2E_{00}v_1v_3$, and so on for all six pairs. A sum of two squares is never negative. ■

The all-ones plan is one instance. The classical mixture $(1, 0, 1, 0)$ has a completion at $x = 1$, $y = 0$, with form $v_0^2 + (v_1 + v_2 + v_3)^2 + v_4^2$.

□ **Theorem 8.25** (Convexity and finite mixtures)**.** For $0 \le \lambda \le 1$, if E and F are elliptope realizable, so is $\lambda E + (1-\lambda)F$. For finitely many members with nonnegative weights summing to at most 1, the weighted sum is realizable. Every finite mixture of deterministic plans is realizable.

Proof. The form is affine in (E, x, y) for each fixed v. Take the completion $(\lambda x_1 + (1-\lambda)x_2, \lambda y_1 + (1-\lambda)y_2)$; the form at the mixture is the same mixture of two nonnegative forms. For several members, weight each one's completed matrix by its assigned share and add them. If the shares total less than one, give the missing share to the zero-correlator point with both missing entries zero; its completed matrix is the identity. The diagonal of the weighted matrix is then one, its cross entries are the desired weighted correlators, and every weighted total is a sum of nonnegative totals with nonnegative weights. It therefore supplies the required completion. The last statement combines this with Theorem 8.24. ■

Theorem 8.26 (Tsirelson's bound holds on the completion set)**.** If E is elliptope realizable then $S^2 \le 8$. □

Proof. For a symmetric PSD matrix the bilinear form obeys Cauchy-Schwarz, $\mathcal{B}(u,w)^2 \le \mathcal{Q}(u)\,\mathcal{Q}(w)$, because $\mathcal{Q}(u+tw) = \mathcal{Q}(u) + 2t\,\mathcal{B}(u,w) + t^2\mathcal{Q}(w) \ge 0$ for all t and Lemma 8.11 applies. Take $u_1 = e_{A_0}$ and $w_+ = e_{B_0} + e_{B_1}$: $\mathcal{B}(u_1, w_+) = E_{00} + E_{01}$, $\mathcal{Q}(u_1) = 1$, $\mathcal{Q}(w_+) = 2 + 2y$. Take $u_2 = e_{A_1}$ and $w_- = e_{B_0} - e_{B_1}$: $\mathcal{B}(u_2, w_-) = E_{10} - E_{11}$, $\mathcal{Q}(w_-) = 2 - 2y$. So $c_1^2 \le 2 + 2y$ and $c_2^2 \le 2 - 2y$ with $c_1 = E_{00} + E_{01}$, $c_2 = E_{10} - E_{11}$, and both budgets are nonnegative. Then $c_1^2 c_2^2 \le (2+2y)(2-2y) = 4 - 4y^2 \le 4$, so $c_1 c_2 \le 2$, and $S^2 = (c_1 + c_2)^2 = c_1^2 + 2c_1c_2 + c_2^2 \le (2+2y) + 4 + (2-2y) = 8$. ■

The completion can make a row of E longer than 1 (the all-ones plan has rows of squared length 2), which is why this proof does not use the row bounds of Section 25.4.

Corollary 8.27 (The PR box is outside)**.** $(1, 1, 1, -1)$ is not elliptope realizable, since its $S^2 = 16 > 8$. □ ■

Theorem 8.28 (Strictly beyond the classical bound)**.** $(3/5, 3/5, 3/5, -3/5)$ is elliptope realizable and has $S = 12/5 > 2$. □

Proof. At $x = y = 0$ the form equals $v_0^2 + (v_1 + \frac{3}{5}v_3 + \frac{3}{5}v_4)^2 + (v_2 + \frac{3}{5}v_3 - \frac{3}{5}v_4)^2 + \frac{7}{25}v_3^2 + \frac{7}{25}v_4^2$. Check: the two middle squares give v_3^2 coefficient $\frac{9}{25} + \frac{9}{25} = \frac{18}{25}$, plus $\frac{7}{25}$ is 1; the v_3v_4 terms are $+\frac{18}{25}$ and $-\frac{18}{25}$ and cancel; the cross terms are $2 \cdot \frac{3}{5}v_1v_3$, $2 \cdot \frac{3}{5}v_1v_4$, $2 \cdot \frac{3}{5}v_2v_3$ and $-2 \cdot \frac{3}{5}v_2v_4$, as the entries require. ■

By Theorem 25.1 no finite factorizable model has $S > 2$, so this point is not a finite mixture of plans, while Theorem 8.25 puts every such mixture inside. The completion set then strictly contains the finite classical set, and the PR box lies outside it. It sits inside the no-signalling set, for the plain reason that every correlator in it is between -1 and 1 (the vector that is 1 at A_x and ∓ 1 at B_y gives $2 \mp 2E_{xy} \ge 0$), and any four such numbers come from the no-signalling box that answers (a, b) at settings (x, y) with probability $(1 + ab\,E_{xy})/4$, each player's own answer alone a fair coin.

The elliptope gate is an integer decider with a soundness proof. Its completion entries arrive as bucket pairs, and an empty bucket is read as $0/1$, so unlike Section 25.5 it does not require every pair to be sampled.

Definition 8.12 (The two branches)**.** The positive-minor branch: with the completion supplied as buckets and all entries scaled to integers by a positive common factor K, the four leading minors of the 4×4 block on A_0, A_1, B_0, B_1 are strictly positive. The factorization branch: with the completion and a rational certificate supplied (six subdiagonal entries over a common positive denominator, four pivots $d_k \geq 0$ over another), the ten entry equations of $M = LDL^{\mathrm{T}}$ hold in cross-multiplied integer arithmetic. The full gate is their disjunction.

□ **Theorem 8.29** (Soundness of the gate)**.** If the full gate returns yes on a counter record, any completion buckets and any certificate, then the four correlators of the record are elliptope realizable. On the PR box tally the gate returns no for every choice of the remaining arguments.

Proof. In both branches it is enough to show the 4×4 block on A_0, A_1, B_0, B_1 gives a nonnegative form, because the I row and column are 1 on the diagonal and 0 elsewhere. The first branch: dividing by the positive factor K (a $k \times k$ minor scales by K^k) gives four positive leading minors of the real block, and Lemma 8.21 applies. The second: cross-multiplying by positive denominators changes no equation, so $M = LDL^{\mathrm{T}}$ holds over the rationals with L unit lower triangular, and the form is $v^{\mathrm{T}}LDL^{\mathrm{T}}v = \sum_k d_k P_k^2$ with $P = L^{\mathrm{T}}v$ and every $d_k \geq 0$. The PR statement is ■ soundness followed by Corollary 8.27.

Examples by direct computation: the all-ones plan passes through the factorization branch at $x = y = 1$ with pivots $(1, 0, 0, 0)$; the classical mixture $(1, 0, 1, 0)$ passes through the same branch at $x = 1$, $y = 0$ with pivots $(1, 0, 0, 1)$; the $12/5$ point passes through the positive-minor branch at $x = y = 0$; and $(3/5, 4/5, 4/5, -3/5)$ passes through the factorization branch at $x = y = 0$ with pivots $(1, 1, 0, 0)$, since $B_0 = \frac{3}{5}A_0 + \frac{4}{5}A_1$ and $B_1 = \frac{4}{5}A_0 - \frac{3}{5}A_1$ reproduce every entry.

That's all the gate is: no property of the small machine is built from it.

8.8 The four-vector construction

Put two equal-length pencils down with their ends together, and draw a line along one of them. Keep that pencil still and turn the other. Use the square corner of a

sheet of paper to bring the moving pencil's tip straight across to the line, and mark where it lands. The mark moves from a full pencil-length forward, through the joined ends when the pencils make a square corner, to a full pencil-length backward. Call those readings 1, 0 and -1. That is the signed shadow I want. Between those positions the reading is the cosine of the angle. A dot product of unit vectors measures that shadow.

Now ask for four directions, two labelled Alice and two Bob, with a prescribed shadow for each Alice–Bob pair. Try this request: Alice's first points exactly along both of Bob's; Alice's second points along Bob's first and exactly against his second. Each request alone is easy. Put them together and your hands cannot do it: Bob's pencils would have to point both the same way and opposite ways. Four numbers in range need not describe four directions that can coexist.

Write those four readings as $x = E_{00}$, $y = E_{01}$, $z = E_{10}$ and $w = E_{11}$. I haven't specified the shadow between Alice's own two directions, or between Bob's own two. Those are the two blanks, t and u, in the following table. Can they be filled so that the whole table comes from directions? The pencils let you ask it in a plane; the construction will allow four coordinates. A completion is a positive-semidefinite matrix

$$G = \begin{pmatrix} 1 & t & x & y \\ t & 1 & z & w \\ x & z & 1 & u \\ y & w & u & 1 \end{pmatrix}.$$

The extra row and column labelled I in Section 25.7 have diagonal 1 and all other entries zero, so they do not change this condition.

Why positivity? Lay arrows end to end, allowing a backward arrow when its weight is negative. Square the length from the start to the finish. Expanding that square adds up the arrows' pairwise shadows, with their weights. It cannot be negative. A proposed shadow table that gives a negative answer for some choice of weights cannot come from arrows. That is the PSD test. A Gram matrix is simply the table of all those pairwise dot products.

The converse has to build the arrows. Pick the first direction as a baseline. Give every other arrow the component along it that its shadow demands, then build the remaining components perpendicular to it. What remains to be fitted is a smaller shadow table. Here is that construction, including its boundary. If its first diagonal entry is zero, positivity of $h_{11}a^2 + 2h_{1j}ab + h_{jj}b^2$ for every a, b forces $h_{1j} = 0$: with $b = 1$, a nonzero h_{1j} would let the linear term go negative without bound. Delete that zero row and column, construct vectors for the smaller matrix, and put back

the zero vector. If $h_{11} > 0$, set

$$K_{ij} = h_{ij} - \frac{h_{1i}h_{1j}}{h_{11}} \quad (i,j > 1).$$

In the quadratic form of h, take the first coordinate to be $-\sum_{j>1} h_{1j}v_j/h_{11}$. The remaining form is $v^{\mathrm{T}}Kv$, so K is PSD. Repeat the construction on this smaller matrix. Each repetition removes a row and column, so it ends at a single nonnegative entry, represented by its square root, or at the empty table. Working back up supplies vectors r_i with $r_i \cdot r_j = K_{ij}$. Then $u_1 = (\sqrt{h_{11}}, 0)$ and $u_i = (h_{1i}/\sqrt{h_{11}}, r_i)$ have Gram matrix h. The dimension is at most the number of rows. Applied to G, this gives four unit vectors a_0, a_1, b_0, b_1 in $\mathbb{R}^4$, with the required cross inner products.

Three dimensions are always enough. The table fixes the cross entries and leaves two entries of G open, $a_0 \cdot a_1$ and $b_0 \cdot b_1$. Hold the first and let the second be t. The values of t that keep G PSD form an interval: if two values do, so does every value between, because the quadratic form at a mixture is the same mixture of the two forms; the set is closed, because a form that is nonnegative at every value of a convergent sequence is nonnegative at its limit; and it is bounded, because a PSD matrix with ones on its diagonal has every entry between -1 and 1. At an end of that interval G is singular: if it were positive definite there, it would stay positive definite under every small enough change of t, on both sides. Take G at an end. A singular PSD G has a nonzero c with $c^{\mathrm{T}}Gc = 0$, and $c^{\mathrm{T}}Gc$ is the squared length of $\sum_i c_i v_i$ for the four vectors v_i, so that sum is zero and the four vectors lie in a space of at most three dimensions. Coordinates in an orthonormal basis of that space keep every inner product, so a_0, a_1, b_0, b_1 can be taken in $\mathbb{R}^3$.

The next question is harder: how do directions become a prescription for a quantum measurement? Start with two signed cards in a row. One move swaps them; another changes the sign of the card in the right-hand place. Start at $(+,+)$. Swap then change the right sign: $(+,-)$. Start again, change the right sign then swap: $(-,+)$. Every sign is reversed. Try unequal numbers on the cards: the two orders still give negatives of each other. "Anticommute" means exactly that relation between the two moves. Do either move twice and everything is back.

Write the swap as X, the sign change as Z, and doing nothing as I:

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

For the larger construction, label four places by the two-bit words 00 through 11 and put a signed number at each. An X in one bit-position swaps each pair of places differing at that bit; a Z reverses the signs at places whose bit there is 1. The two positions give the following three moves, written as real 4×4 matrices:

$$\Gamma_1 = X \otimes I, \quad \Gamma_2 = Z \otimes X, \quad \Gamma_3 = Z \otimes Z.$$

The tensor product has entries $(A \otimes B)_{(i,k),(j,l)} = A_{ij}B_{kl}$. Each Γ_i is symmetric and squares to the identity. For $i < j$, the first position where the two products differ has X against Z, and $XZ = -ZX$; all other positions commute. So $\Gamma_i\Gamma_j = -\Gamma_j\Gamma_i$. For a unit vector a, the symmetric matrix $A(a) = \sum_i a_i\Gamma_i$ then satisfies $A(a)^2 = \sum_i a_i^2 I = I$. Its two-outcome measurement has projectors $(I + A(a))/2$ and $(I - A(a))/2$: they are symmetric, square to themselves, are orthogonal, and sum to the identity.

Take $|\Phi_4\rangle = 4^{-1/2}\sum_{k=1}^{4}|k\rangle \otimes |k\rangle$. Expanding this sum gives

$$\langle\Phi_4|A \otimes B|\Phi_4\rangle = \frac{1}{4}\sum_{i,j} A_{ij}B_{ij} = \frac{1}{4}\operatorname{tr}(A^{\mathrm{T}}B).$$

For $i \neq j$, cyclicity of the trace and anticommutation give $\operatorname{tr}(\Gamma_i\Gamma_j) = -\operatorname{tr}(\Gamma_i\Gamma_j) = 0$; for $i = j$ the trace is 4. So the correlator of $A(a)$ and $A(b)$ is $a \cdot b$. Give Alice $A(a_0), A(a_1)$ and Bob $A(b_0), A(b_1)$. Their strategy realizes x, y, z, w. All three Γ_i have trace zero, so the one-party expectations vanish too. The same real matrices and state are also a complex strategy.

With complex amplitudes two levels on each side are enough. Add $Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$ to X and Z: the three are self-adjoint, square to I and anticommute in pairs, and they have trace zero. Give Alice $a_1X + a_2Y + a_3Z$ for her vector a, and Bob $b_1X - b_2Y + b_3Z$ for his vector b. The expansion above holds for complex entries too, with Φ_2 and a factor $\frac{1}{2}$, and the transpose of Alice's matrix is $a_1X - a_2Y + a_3Z$, since Y's transpose is $-Y$. So the correlator is $\frac{1}{2}\operatorname{tr}\big((a_1X - a_2Y + a_3Z)(b_1X - b_2Y + b_3Z)\big) = a \cdot b$.

What would that mean at a bench? Prepare the stated shared state, choose one of the two specified measurements on each side, and write down the two signs returned. For each pair of settings, count agreements as $+1$ and disagreements as -1. Their average is the correlator. The model predicts the four averages x, y, z, w; a finite tally fluctuates around its expectation. The signed-card moves let you check the measurement algebra by hand. Realizing that state and those measurements in a device, and comparing its tallies with the predictions, is the physical experiment.

Conversely, in a normalized quantum state ψ with self-adjoint measurements $A_x^2 = B_y^2 = I$ on separate systems, put $a_x = (A_x \otimes I)\psi$ and $b_y = (I \otimes B_y)\psi$. They are unit vectors and $\operatorname{Re}\langle a_x, b_y\rangle = E_{xy}$. Replace each complex coordinate by its real and imaginary parts. This preserves those real inner products, and their Gram matrix is a completion. A mixed state gives the same construction by taking the direct sum of the vectors of its pure components, weighted by the square roots of their probabilities. So the completion condition and finite-dimensional quantum realizability are equivalent.

The necessary direction does not require finite-dimensional vectors. If a positive normalized expectation ω on an algebra has self-adjoint A_x, B_y, with $\omega(A_x^2), \omega(B_y^2) \leq 1$, then the real symmetric matrix with entries $\operatorname{Re}\,\omega(X_iX_j)$ is

PSD: its quadratic form is $\omega(C^*C) \geq 0$ for the real linear combination C of the four observables. Increasing each diagonal entry to 1 preserves PSD. When Alice's observables commute with Bob's, its cross entries are precisely the real correlators. This supplies the same G. Including the unit as a fifth observable also gives the level-1 moment matrix with its marginals. The sufficient direction already used only two four-level systems.

8.9 The arcsine boundary, in both directions

Keep Bob's two directions hinged at one end. Alice's first direction has to make a specified angle with each. How far can Bob's hinge open while that is possible? Hold Alice's direction upright and sweep each of Bob's around it at its required angle. Point them towards the same side to close the hinge as far as it will go; point them towards opposite sides to open it as far as it will go. There is a whole interval of openings between those limits. Alice's second direction asks for another interval.

Draw both intervals on one strip marked from 0 to 180 degrees. If they miss, no hinge setting satisfies both. If they overlap, choose an opening in the overlap and construct each Alice direction at that opening. The algebra below permits each its own perpendicular component in four dimensions. The pencils show why the shared opening is the question; they need not fit the entire construction on a tabletop. In the impossible request above, one interval is just 0 degrees and the other just 180. There is nowhere to put the hinge.

Assume $x, y, z, w \in [-1, 1]$, and put $s_x = \arcsin x$, with analogous notation for the other three entries. A unit vector making angles $\alpha, \beta \in [0, \pi]$ with two unit vectors can exist at an angle δ between those two exactly when

$$|\alpha - \beta| \leq \delta \leq \min(\alpha + \beta, 2\pi - \alpha - \beta).$$

To see this, decompose the two vectors along and perpendicular to the first. Their inner product ranges from $\cos\alpha\cos\beta - \sin\alpha\sin\beta$ to $\cos\alpha\cos\beta + \sin\alpha\sin\beta$, by Cauchy–Schwarz; every value in that interval is attained by choosing the angle between their perpendicular components. Taking arccosines gives the displayed interval, including zero perpendicular components.

Apply this to Alice's first vector and Bob's two. Since $\alpha = \pi/2 - s_x$ and $\beta = \pi/2 - s_y$, Bob's angle must lie in

$$I_0 = [\, |s_x - s_y|,\ \pi - |s_x + s_y| \,].$$

Alice's second vector gives $I_1 = [\, |s_z - s_w|,\ \pi - |s_z + s_w| \,]$. Each interval is nonempty because $|a - b| + |a + b| = 2\max(|a|, |b|) \leq \pi$ for $a, b \in [-\pi/2, \pi/2]$. They

intersect exactly when

$$|s_x - s_y| + |s_z + s_w| \le \pi, \qquad |s_z - s_w| + |s_x + s_y| \le \pi.$$

Using $|a| + |b| = \max(|a+b|, |a-b|)$, these are the four inequalities

$$|s_x + s_y + s_z - s_w| \le \pi, \qquad |s_x + s_y - s_z + s_w| \le \pi,$$
$$|s_x - s_y + s_z + s_w| \le \pi, \quad |-s_x + s_y + s_z + s_w| \le \pi.$$

So a completion implies all four.

For the converse, choose any $\delta \in I_0 \cap I_1$ and set $b_0 = e_1$, $b_1 = \cos\delta\, e_1 + \sin\delta\, e_2$. If $0 < \delta < \pi$, put

$$c_0 = \frac{y - x\cos\delta}{\sin\delta}, \qquad c_1 = \frac{w - z\cos\delta}{\sin\delta},$$
$$a_0 = x e_1 + c_0 e_2 + \sqrt{1 - x^2 - c_0^2}\, e_3,$$
$$a_1 = z e_1 + c_1 e_2 + \sqrt{1 - z^2 - c_1^2}\, e_4.$$

The interval conditions say exactly that the two radicands are nonnegative: after multiplication by $\sin^2\delta$, the first is $1 + 2xy\cos\delta - x^2 - y^2 - \cos^2\delta \ge 0$, and likewise for z, w. These four vectors have unit length and the prescribed cross inner products. If $\delta = 0$, the intervals force $x = y$ and $z = w$; take $b_0 = b_1 = e_1$, $a_0 = x e_1 + \sqrt{1-x^2}\, e_3$, $a_1 = z e_1 + \sqrt{1-z^2}\, e_4$. If $\delta = \pi$, they force $y = -x$ and $w = -z$; use the same Alice vectors and $b_1 = -b_0$. Their Gram matrix supplies the completion in every case. Together with the preceding construction, this proves the stated quantum characterization.

8.10 The finite local model, in both directions

Make sixteen cards, each with four boxes labelled A_0, A_1, B_0, B_1, and put every possible arrangement of plus and minus signs on them, one arrangement per card. Draw a card before choosing which setting to ask each player about. Alice reads her chosen box; Bob reads his. Neither needs the other's setting. How often each card is drawn is the only choice left in this model.

Try to get perfect agreement for the first three pairs and perfect disagreement for the fourth. Any card that ever gets drawn would need $A_0 = B_0$, $A_0 = B_1$ and $A_1 = B_0$, which makes $A_1 = B_1$ too. Changing how often you draw the cards cannot fix it. For less extreme requests the question is quantitative: can you assign nonnegative weights to the cards, adding to one, that give all four requested averages? The following construction supplies those weights exactly when all eight CHSH inequalities hold and the correlators are in $[-1, 1]$.

There is a useful way to assemble the deck. First allow only Bob's first question and make a distribution of the three answers A_0, A_1, B_0. Then do the same for his second question. Both smaller decks must draw Alice's answer pairs in the same proportions. The adjustable number t is Alice's agreement-minus-disagreement average across her two boxes. Choose it in both intervals

$$J_0 = [\,-1+|x+z|,\ 1-|x-z|\,], \qquad J_1 = [\,-1+|y+w|,\ 1-|y-w|\,].$$

Each interval is nonempty. They overlap exactly when $|x+z|+|y-w| \le 2$ and $|y+w|+|x-z| \le 2$, which expand to the eight CHSH inequalities. For $a_0, a_1, b \in \{-1,1\}$ set

$$P_0(a_0,a_1,b) = \frac{1 + t a_0 a_1 + x a_0 b + z a_1 b}{8},$$
$$P_1(a_0,a_1,b) = \frac{1 + t a_0 a_1 + y a_0 b + w a_1 b}{8}.$$

When $a_0 = a_1$, nonnegativity of P_0 is $1+t \ge |x+z|$; when $a_0 = -a_1$, it is $1-t \ge |x-z|$. These are precisely $t \in J_0$. The same check for P_1 uses J_1. Summing signs shows that both distributions have mass 1 and the desired pair correlators. Their common Alice marginal is $P_A(a_0,a_1) = (1 + t a_0 a_1)/4$.

To join the two smaller decks, first draw Alice's pair with its common probability. Among the first deck's cards bearing that pair, draw Bob's first answer; among the second deck's cards bearing the same pair, draw his second. Those conditional draws give a four-answer card. A pair that never occurs needs no card. Written as probabilities, this procedure is

$$P(a_0,a_1,b_0,b_1) = \begin{cases} P_0(a_0,a_1,b_0)\,P_1(a_0,a_1,b_1)/P_A(a_0,a_1), & P_A(a_0,a_1) > 0, \\ 0, & P_A(a_0,a_1) = 0. \end{cases}$$

If $P_A = 0$, nonnegativity forces both corresponding triple distributions to be zero. Otherwise summing over either Bob answer cancels the denominator. So P has the two triple distributions as marginals and total mass 1. Sample one of its sixteen answer quadruples and let each party answer with its assigned sign. This is a finite mixture of deterministic local plans with exactly the four required correlators.

The other direction is one line per card. On a single card, $A_0(B_0+B_1) + A_1(B_0 - B_1)$ is ± 2, because one bracket is 0 and the other is ± 2. Each of the other seven CHSH expressions is this one with the boxes relabelled or a sign flipped, so it is ± 2 on every card too, and any weighting of the cards keeps all eight averages between -2 and 2 (Theorem 25.1 is the first of them). So a deck exists exactly when the correlators are in $[-1,1]$ and the eight inequalities hold.

8.11 Scope of the constructions

The constructions above pin down the correlators. What follows is where they stop: they aren't a completeness claim for the integer check, they say nothing about full behaviours with marginals, and they say nothing about data from a lab.

The arcsine boundary. For two settings and two outcomes per side, Tsirelson, Landau (1988) and Masanes (2003) give theorems characterizing the correlators reachable by quantum measurements: $|\arcsin E_{00} + \arcsin E_{01} + \arcsin E_{10} - \arcsin E_{11}| \le \pi$ and its sign variants. The characterization holds as an if and only if with the completion set and, through Tsirelson's representation, with the quantum set. The point $(3/5, 4/5, 4/5, -3/5)$ is on the boundary: $\arcsin \frac{3}{5} + \arcsin \frac{4}{5} = \frac{\pi}{2}$ by the 3-4-5 triangle, so the sum is π; Tsirelson's point is on it too. The necessary direction holds in any dimension, for any expectation on an algebra of observables in which Alice's commute with Bob's.

Equality with the quantum set. That the completion set of Section 25.7 is exactly the set of quantum correlators is Tsirelson's representation theorem, and it is proved in finite dimension: for both real and complex amplitudes, with two four-level systems enough for every point, and two two-level ones with complex amplitudes. That PSD of the level-1 moment matrix is necessary for quantum behaviour is proved for strategies in finite dimension, with real amplitudes and with complex ones, and in any dimension, marginals included, for any expectation on an algebra of observables.

Achievability. Theorem 25.5 is a point of $\mathbb{R}^4$. A state and measurements that reach it are explicit: the two-qubit state $(|00\rangle + |11\rangle)/\sqrt{2}$ with $A_0 = Z$, $A_1 = X$, $B_0 = (Z + X)/\sqrt{2}$ and $B_1 = (Z - X)/\sqrt{2}$ has correlators $1/\sqrt{2}$, $1/\sqrt{2}$, $1/\sqrt{2}$, $-1/\sqrt{2}$ and $S = 2\sqrt{2}$, and every strategy in finite dimension has $S^2 \le 8$, with real amplitudes and with complex ones. Both are arithmetic about the model. How close a real laboratory comes, and what that costs in loopholes closed, is the story told in Appendix 24.

Completeness of the checks. The 1+AB cascade and the elliptope gate are proved sound. The cascade has no proved converse; the gate's factorization branch accepts every rational PSD completion, by the row-peeling of Section 25.8 with each pivot kept. The level-1 integer check is the exception: on the small machine the same seven integer facts hold exactly when every pair was sampled and the pinned matrix is positive semidefinite (Theorem 24.4), so that check is complete for what it tests. The 1+AB cascade is strict and misses boundary points. The two 1+AB checks don't sit inside one another. The full check accepts the $12/5$ tally above, which the zero-moment check refuses, and it refuses the tally with $E_{00} = 1$ and the other three correlators 0, where $H_{11} = 0$, which the zero-moment check accepts.

Finite levels. For full behaviours, marginals included, the NPA levels are relaxations. Chaturvedi (arXiv 2607.14569, 2026) argues, in a preprint, that no finite NPA level characterizes the complete quantum set even in this simplest Bell scenario. I haven't gone through that proof, and nothing here rests on it.

Statistics and physics. Nothing here says how many rounds make a tally informative, and nothing says the counters came from honest play; a program can INC whatever counts it likes.

8.12 Prior work

Tsirelson's 1980 paper gives the bound $2\sqrt{2}$ for quantum correlations, and his theorem says that quantum two-outcome correlators are inner products of real unit vectors. Landau (1988) and Masanes (2003, arXiv quant-ph/0309137) each give a condition that, by their proofs, characterizes the quantum correlators exactly for two settings and two outcomes. Wehner (2006, arXiv quant-ph/0510076) gets Tsirelson bounds, CHSH among them, from a semidefinite program on the 4×4 Gram matrix of those vectors. Navascués, Pironio and Acín (2007, arXiv quant-ph/0607119; 2008, arXiv 0803.4290) built the hierarchy of moment-matrix tests; their 2008 paper applies a Schur complement to the level-1 matrix of this scenario, recovers the arcsine condition, and discusses the 1+AB level. I went back to each of those papers for these points, and that is what I found in them. The results are theirs. Two of them are proved again here, in finite dimension: Tsirelson's representation and the arcsine characterization; the rest of their proofs are not redone. Echenim and Mhalla give a detailed derivation of Tsirelson's bound (arXiv 2306.12535), and exact rational sum-of-squares certificates go back at least to Peyrl and Parrilo (2008). So the moment-matrix route to $2\sqrt{2}$ is standard. What is specific here is narrower: an integer certificate computed from outcome counts, and the argument from "the counts pass" to PSD of the pinned matrix and on to $S^2 \leq 8$. I did not find that chain elsewhere, and I make no claim beyond it. On the host machine, the same chain is earned: a raised flag from a clean start stands on a passing check of the packed counts, and the check passes exactly when the pinned matrix is positive semidefinite and every pair was sampled (Appendix 24, Theorems 24.4 and 24.5). The converse direction of the integer check, integers from PSD, is proved there too, and there too I make no claim beyond the chain.

CHAPTER 9

From “A record and a toll”

Take an ordinary computing machine and put a lamp and tally beside it. Let computing run as before, leaving both alone. A separate mark leaves the computing state fixed, adds one to the tally and turns the lamp on when its named event holds. Forget the lamp and tally and the old computation is still there, step for step. Below I build that box, compare different ways to implement it, and ask exactly which differences its questions require. The old machine may already have prices or a reading of its own; this construction supplies the new lamp and its zero/one account. With effective rules an ordinary computer can store all three parts.

What the next move could tell us

Put two boxes beside each other and read everything the task lets you read. Suppose the answers agree. That is not yet enough to put the boxes in the same pile. Press the same button on each. Perhaps one lamp now comes on and the other stays dark. The difference mattered, even though the first reading missed it.

Keep two states together only when every permitted finite continuation gives the same required answer. Include doing nothing: the current answer counts too. This tells us which differences the task can afford to forget.

Theorem 9.1 (Keep what a later question can separate)**.** For a fixed deterministic machine, permitted instructions and required observation, equality of observations after every finite continuation is the unique coarsest equivalence that preserves the current observation and is stable under every instruction. A deterministic implementation preserving those steps and answers cannot identify two states separated by such a continuation. □

Proof. Agreement with oneself, reversal of an agreement, and joining two agreements all hold one continuation at a time, so this is an equivalence. Doing nothing checks the current observation. To check stability after a move, put that move in front of each continuation. If two states were already grouped by any observation-preserving, stable relation, apply its stability once for every move in a continuation,

then read the answer. Those states agree on every continuation and belong to our equivalence. Our grouping therefore contains every permitted grouping. Finally, if an implementation put two separated states in one full state, the same continuation from that state would have to produce two different answers. Determinism gives only one. ■

Change the questions and the piles can change. A box that toggles between two positions needs two distinguishable states if I ask its position. If my only required answer is always “ready”, and every later answer is also “ready”, those positions can share one description. The task tells me what has to survive.

The picture, the lamp and the tally

Take an ordinary computing box. Put a lamp beside it and a tally beneath it. A computing button moves the box by its old rule, costs zero, and leaves the lamp and tally alone. A mark button leaves the box where it is, adds one to the tally, and turns the lamp on if its named event holds there. Once on, the lamp stays on. Start with the lamp down and the tally at zero.

Those instructions tell us what to do without telling us how to build the box. One builder may keep every button press on a tape. Another may keep an extra turn counter. I require each builder to show me the same three things: the computing picture, the lamp and the exact tally. Its computing moves must follow the base rule through that picture. Its marks must leave the picture fixed and latch exactly the named event. Its prices are zero for computing and one for marking; its reading obeys the toll and permanence; its tally increases by exactly the charge on every move.

□ **Theorem 9.2** (Every such box has the same three-part description)**.** Fix those base rules, mark events and accounting requirements. Every implementation has exactly one map to the product of base state, Boolean reading and natural tally that preserves the three readings. That map preserves every instruction and every finite run. From a start with lamp down and tally zero, it maps the reachable implementation states onto all reachable states of the product lift with the matching base start.

Proof. Read the picture, lamp and tally, and write them down in that order. There is no other choice for a map required to preserve those three readings. Press a computing button. The picture follows the base rule. The lamp stays put by the toll and permanence, and the tally stays put because the charge is zero. Press a mark instead. The picture stays put, the lamp takes its old value or the event’s yes, and

the tally rises by one. These are exactly the two rules of the product lift. Repeat for every move in a finite run.

The clean starts agree in all three readings. Any reachable product state has some finite list of buttons reaching it. Press that same list on the implementation. The step argument makes its three readings the desired product state. Every reachable product state therefore has a reachable implementation state above it. ∎

Here are the three stages. First write a list of buttons. Then run it on the builder's box. Then read the picture, lamp and tally. You get the same result as running the list directly on the three-part description. Written as arrows, after we have done it:

$$\text{button lists} \longrightarrow \text{implementation states} \longrightarrow \text{picture, lamp, tally.}$$

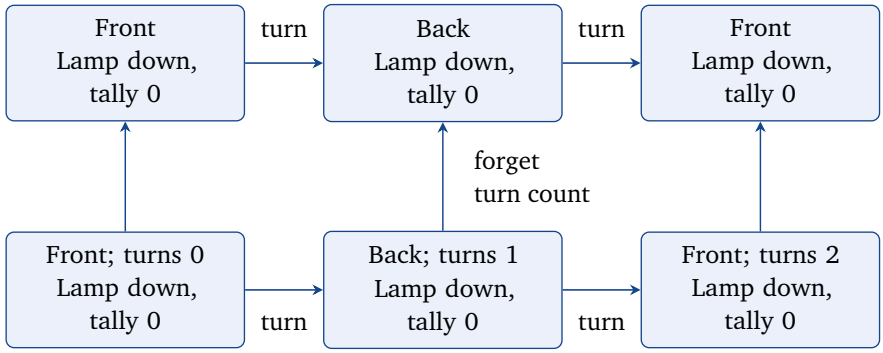
The first arrow has only one possible meaning once the start and buttons are fixed. The empty list gives the start; appending a button takes one more step. This is the free history construction. The second arrow can forget history. Its direction matters.

Turn a box halfway around with a free button. Turn it again. It is back where it started, with the same lamp and tally. Now give another builder's box a counter that goes up on every turn, without charging for it. After two turns its counter is two larger. It meets all the requirements above: the extra counter is separate from the tally of charges.

Proposition 9.3 (Forgetting the turn count goes one way)**.** Even on reachable states, the three-part lift need not have a step-preserving map into every implementation satisfying its contract. □

Proof. In this example two turns return every product state to itself. A map preserving turns would have to send it to an implementation state that also returns after two turns. No such state exists there: its extra counter has increased by two. Forgetting that counter works in the other direction and preserves both turning and marking. ∎

There is an exact way to tell when the reverse map would work. Whenever two button lists reach the same source state, they must also reach the same target state. Otherwise the map would have to send that one source to two targets. Supply one access list reaching each source state, and this condition is sufficient: run its access list on the target. Any other list reaching that source gives the same target by the condition. The empty list preserves the start, appending a button preserves a step, and reachability leaves no other possible map. The turn counter fails exactly this test: the empty list and two turns agree in the source and disagree in the target.



The top row returns to the same full state. The bottom row does not. Forgetting the turn count respects each turn; recovering it from the top row cannot.

□ **Theorem 9.4** (The smallest description that answers these questions)**.** Under the preceding contract, two implementation states give the same three readings after every common finite continuation exactly when their current three readings agree. If every two distinct reachable implementation states can be separated by such a continuation, projection is a bijection onto the reachable product states. It preserves starts and steps and is the unique start- and step-preserving map there.

Proof. If the current readings differ, read them now. If they agree, the two product states are equal. Equal states obeying the same rule give equal next states; repeat for every move. Projection preserves those moves, so the implementation’s readings agree throughout too.

Now require every distinct reachable pair to be separable. Two states projecting to the same triple could never be separated, so they must be one state. Projection is therefore one-to-one; the previous theorem makes it onto the reachable product states. Any start- and step-preserving map is fixed on a reachable state by a list reaching it: map the start, then follow the list. It must agree with projection there. ■

The same reasoning explains uniqueness more generally. Take two reachable machines with the same answers to every button list from their starts, and suppose each keeps no two states that all future answers fail to separate. Supply an access list for each state of the first. Run that list on the second. If two lists reach the same first state, append any continuation: their answers agree on the first machine, hence on the second. Separation in the second makes their endpoints equal. The map is well-defined and preserves steps and outputs. Reachability makes it onto; separation in the first makes it one-to-one. Following an access list forces uniqueness.

For our box, that smallest description keeps the tally because I asked for the tally. Stop asking for it and repeated failed marks may become indistinguishable. Ask for

every past button and the extra tape starts to matter. With a clean start and an event that never holds, no reachable state has its lamp up. Counting unreachable lamp-up states would answer a different question. “Smallest” here means no two retained states are indistinguishable by the required future readings, including on an infinite machine.

This way of grouping states is the familiar behavioral equivalence behind minimal automata. The initial history construction and final observation construction are treated in Rutten’s *The Method of Coalgebra*, Chapters 2 and 4 and Section 8.6.[1] Here the box tells us exactly which observations and operations the lift preserves. Choosing the event, the window and the accounting is where the application enters. Once they are fixed, the argument above determines which implementation differences can be forgotten.

What can choose the question?

Take a dial with two faces and one button. Each press turns it to the other face. For this question, supply no starting face, no lamp and no names with an outside meaning. Draw its two arrows. Now exchange the names on the two faces. The arrows describe exactly the same machine.

Suppose those arrows alone are meant to tell us which face deserves yes. A rule that does not depend on our choice of names must give the exchanged face the same reading. So both faces get the same answer. A nonconstant reading cannot be selected that way. Give one face a role—the loaded start, a completed job, a passed check—and we have supplied more to work with. The role has to be stated and defended. Renaming the arrows did not supply it.

The argument is two lines: the exchange takes the first face to the second; respecting that exchange makes their readings equal. Those are all the faces. It does not make every machine’s reading constant. It rules out a nonconstant reading selected from the transition structure alone on every machine. [2]

That fixes what the uniqueness claim can say. The box, lamp and tally have a unique observable realization under the contract already proved. The arrows by themselves do not choose that contract. Adding the word “intrinsic” cannot do the choosing for us.

[1] Jan Rutten, *The Method of Coalgebra* (2019), `https://www.cs.cornell.edu/courses/cs4810/2025sp/Handouts/Rutten.pdf`.

[2] Checked in `coq/audit/IntrinsicBoundary.v`: `toggle_symmetry_forces_constant_reading` and `no_nonconstant_renaming_invariant_toggle_reading`.

Does the picture keep enough of the device?
Put a lamp on a box and a hidden switch inside it. The button copies the hidden switch to the lamp. Start with the lamp dark in both boxes, and put the hidden switch down in one, up in the other.

Lamp before	Hidden switch	Lamp after the button
Dark	down	dark
Dark	up	lit

A description that keeps only the lamp has one starting state, dark, and is asked for two different next answers to the same button. It cannot do it. Keep the hidden switch too and the problem disappears. This is a reason the additional distinction is necessary for this task: the next press can expose it. It is not a reason every task needs that switch. [3]

There is a complete test for this kind of description. First write down the underlying configurations, the permitted buttons, every permitted successor, the picture being retained, and the answers the task requires. This is the operational model of the device. Then check two things. Configurations with the same picture must give the same required answer now. And the same button, from any configurations with that picture, must leave the same next picture, for every permitted successor.

Both conditions are necessary. An answer decoder has only the picture to read. A deterministic next-picture rule has only the picture and button. Neither can return two different results on one input.

They are sufficient too, with explicit choices. Supply a representative configuration for each picture and a permitted successor for each button at each configuration. Decode a picture by reading its representative. Compute its next picture by pressing the button on that representative and taking the chosen successor. The two conditions ensure that every other configuration and every other permitted successor agrees. Repeat that argument at each step: all finite button lists preserve the required answers. On a finite listed model this can be checked against its complete table. For arbitrary state types the choices and their effectiveness are separate data; the proof does not manufacture a search procedure.

This test allows several successors of an underlying configuration.[4] If two of them give different required answers, no exact deterministic encoding of that same configuration and button can represent both. [5] A probability distribution over outcomes, or a finer state including a chosen input or seed, is a different contract and must be described as one.

[3] `hidden_bit_blocks_visible_only_dynamics`, `coq/audit/ObservationContract.v`.

[4] `operational_adequacy_iff`, `coq/audit/ObservationContract.v`.

[5] `branching_answers_exclude_exact_deterministic_encoding`, `coq/audit/ObservationContract.v`.

The test starts with a model. Establishing that a real apparatus follows that model still needs evidence about the apparatus: preparation, controls, readout and the range over which the description is claimed. Passing the mathematical test does not supply those measurements.

How long did we watch the lamp?
Watch two boxes for three ticks. Both start dark and light on the first tick. Both stay lit on the second and third. One stays lit forever. The other has a counter inside that turns it off on the fourth tick.

Tick	Enduring lamp	Delayed reset
0	dark	dark
1	lit	lit
2	lit	lit
3	lit	lit
4	lit	dark

The first three ticks do not tell the boxes apart. Pick any longer finite watch and I can move the reset one tick beyond it. Both boxes can pay one unit per tick and satisfy the basic toll. Only the first has permanence. The permanent interface excludes the second by its rule, not by pretending that the finite observation already excluded it.

The construction works for every finite horizon.[6] It is not a prediction that every real record fails. It shows why a finite run alone cannot establish an unrestricted forever claim. A proof from an adequate mechanism, or an explicitly bounded reliability claim supported by measurements, has work to do beyond reporting that the lamp stayed lit while we watched.

[6] `finite_observations_do_not_establish_permanence`, `coq/audit/IntrinsicBoundary.v`.

CHAPTER 10

From “Why the toll isn’t arbitrary”

There is also a limited statement about how long a noisy record lasts, using a different quantity: total entropy production during relaxation. Take a device with no outside hand: a random process hopping among finitely many states at rates that never change, with a reverse for every hop and local detailed balance, able to get from any state to any other. Require at least one up-state and one down-state. Left to run, it settles into a long-run spread π. Measure each up or down stretch from its entrance, with entrances sampled in the stationary process, and call the mean lengths T_{up} and T_{down}. Start the device in a known no-state s. The total entropy it produces over relaxation is at least $\ln(1 + T_{\text{up}}/T_{\text{down}})$, in units of k_B.

First compare the starting spread with the spread the device settles into. A state that is common at the start but rare in the settled spread is a large mismatch. For each state, take the ratio of those two chances, take its natural logarithm, and average using the starting chances. With p the starting spread, that number is

$$D(p\|\pi) = \sum_{x:p(x)>0} p(x) \ln \frac{p(x)}{\pi(x)}.$$

It is called relative entropy. Here every $\pi(x)$ is positive, so the ratios are available. The number is nonnegative: apply $\ln y \le y - 1$ to $\pi(x)/p(x)$, multiply by $p(x)$, and add. The right side is at most zero, since the included π chances total at most one and the p chances total exactly one. Negating that comparison gives $D(p\|\pi) \ge 0$.

For a known start s, all the starting chance is at s. There is only one term: $D = \ln(1/\pi(s)) = -\ln \pi(s)$. A state with settled chance one tenth therefore starts with mismatch $\ln 10$. Since s is a no-state, its settled chance cannot exceed the total chance of all no-states. Its mismatch is at least $-\ln \pi(\text{no})$.

The physical step in the argument is the fixed-rate entropy-production split of Massimiliano Esposito and Christian Van den Broeck (2010). In that setting the total entropy produced is at least the decrease in this relative entropy. Once the spread has settled to π, the relative entropy left is zero. So the initial mismatch supplies the lower bound for the whole relaxation.

Now read the same number from the record of up and down stretches. In a long settled run, each entry into YES is followed by an exit; apart from the two ends of the record, there are equally many up-stretches and down-stretches. Total time up is the number of up-stretches times their mean length, and likewise for time down. Dividing gives

$$\frac{\pi(\text{up})}{\pi(\text{no})} = \frac{T_{\text{up}}}{T_{\text{down}}}.$$

The two chances add to one. If the ratio on the right is r, the no-share is $1/(1+r)$, and its negative logarithm is $\ln(1+r)$. That gives the stated bound. The settling and mean-length arguments below justify passing from a long record to these limiting quantities.

For a large up-to-down ratio, doubling that ratio raises this lower bound by about $k_B \ln 2$. Making both stretches longer by the same factor leaves it unchanged. Across a family of devices whose ratio tends to infinity, the bound diverges; no single finite irreducible device in this class keeps its flag up forever. This is a relaxation bound, not a heat charge on each flip. It uses neither a thermodynamic reverse-path construction nor time-symmetric control, so it does not identify the quantities in those other results. It also leaves the known-input case of the deterministic entropy-loss bound where it was: this noisy evolution and its total entropy production are a different question. The entropy is added up over one whole relaxation, from the known no-state s until the device has settled, and local detailed balance is what ties it to heat exchanged with the reservoirs. Total entropy production includes the change in the device's entropy too; it is not heat alone. It also isn't the permanence of the finite-state theorems on a noisy device: on this device the flag comes back down, and T_{up} and T_{down} are average stretches.

The core of it for a chain that ticks, worked out, runs as follows in the case where the long-run spread π gives every state a positive chance and satisfies detailed balance: in the long run as much weight flows from x to y in one step as from y to x, $\pi(x)P(x,y) = \pi(y)P(y,x)$, where $P(x,y)$ is the chance of stepping from x to y. Count the entropy produced in one step as the change in the device's own entropy plus what goes out to the surroundings, which for a hop from x to y is $\ln(P(x,y)/P(y,x))$. Detailed balance turns that ratio into $\pi(y)/\pi(x)$, and the sum collapses: the entropy produced in one step is exactly the drop in relative entropy to π. It's never negative, because a step that leaves π where it is can't make any other spread harder to tell from π (that's the data-processing inequality, a standard fact of information theory). Add the steps up and the sum telescopes. A known start s has relative entropy $-\ln \pi(s)$, so the entropy produced in the first T steps is $-\ln \pi(s)$ minus the relative entropy left at step T: it never exceeds $-\ln \pi(s)$, and it reaches it in the limit exactly when that remainder goes to zero. Next, $-\ln \pi(s) \geq -\ln \pi(\text{no})$,

because s is one of the no-states and so carries at most their total chance. The flow from the no-states to the yes-states equals the flow back, because in the long run the weight on the yes-states doesn’t change, so what flows in equals what flows out. Last, take each mean stretch as its mass over that flow; then $T_{\text{up}}/T_{\text{down}} = \pi(\text{up})/\pi(\text{no})$, and since the two chances add to one, $-\ln \pi(\text{no}) = \ln(1 + T_{\text{up}}/T_{\text{down}})$.

Try the stretches with a bag of ten counters: nine marked YES and one NO. Draw, record the answer, replace the counter and mix again. Each draw gives YES with chance $9/10$, whatever the last answer was. A YES stretch ends on the next draw with chance $1/10$, so its mean length is ten draws: the mean L obeys $L = 1 + (9/10)L$. A NO stretch similarly averages $10/9$ draws. Their ratio is nine, just like the ratio of YES to NO counters. In a long record, count all the YES entries and divide by the number of YES stretches; that is the measured mean stretch length. Divide both counts by the total number of draws and you have mass divided by flow.

For this rule the long-run flow from NO to YES is $(1/10)(9/10) = 9/100$ per draw, and the reverse flow is the same. A start known to be NO has relative entropy $-\ln(1/10) = \ln 10$ against the long-run spread; after one draw the spread is already $1/10, 9/10$, so none of that relative entropy remains. This bag shows the probability accounting. The entropy-production interpretation still needs the stated physical model; shaking counters in a bag does not measure that heat.

Two things need establishing for a general device: that it actually settles, and that “mean stretch length” means what it sounds like. I’ll say what each says, and why it holds.

Settling comes first. A chain doesn’t have to settle. It can get stuck in a corner, or cycle round forever. What rules that out here is a condition named for Wolfgang Doeblin, a German-born French mathematician, son of the novelist Alfred Döblin, who took his doctorate at the Sorbonne in 1938 for work on chains like this one and died in June 1940, at twenty-five. In words: from wherever the device is, its next step lands on each state with at least some fixed fraction δ of that state’s long-run chance. No state is ever out of reach in one step, and none is favoured so heavily that the others starve. Under that condition each step shrinks the distance between where the device is and where it’s going by the factor $1 - \delta$ (the distance here is the total of the differences in chance, state by state). To see why, split each step in two: with chance δ, forget where the device was and draw a fresh state from π; otherwise, move by whatever is left over, and the condition is exactly what makes the leftover a genuine set of chances. The fresh draw puts the device’s spread and π in the same place, so only the other share, $1 - \delta$, can keep them apart, and no step pushes two spreads further apart than they were. So the distance dies off geometrically and the spread converges to π at every state. The payoff for the bound is that the entropy produced from the known start stays under $-\ln \pi(s)$, and it also gets there: after T steps it falls short by at most $(4/\pi_{\min})(1 - \delta)^{2T}$, where $\pi_{\min}$ is the smallest long-run

chance of any state. The shortfall is the relative entropy left, and $\ln y \le y - 1$ once more puts that under the sum of $(p(x) - \pi(x))^2/\pi(x)$, which is at most the square of the distance over $\pi_{\min}$; the distance starts at no more than 2 and shrinks by $1 - \delta$ a step. So $-\ln \pi(s)$ is the exact total entropy production of the whole relaxation, in units of k_B, and the bound $-\ln \pi(s)$ is reached.

For rates satisfying detailed balance with π, the same equality holds when the device hops at random moments, with no clock ticking. Write out how fast the relative entropy to π changes and pair each hop with its reverse. Detailed balance makes each pair's share minus (the flow one way minus the flow back) times the logarithm of their ratio, and that is exactly the pair's share of the rate entropy is produced; a difference times the logarithm of the same ratio is never negative. So at every instant the relative entropy falls at exactly the rate entropy is being produced, and that rate is never negative; add the rate up between two times and you get exactly the drop in relative entropy between them. For settling, Doeblin's condition becomes a condition on rates: the rate of hopping from any state x to any state y is at least c times $\pi(y)$, for some fixed $c > 0$. Split the rates in two again: a part that, at rate c, forgets where the device was and redraws it from π, and the rest, which still satisfies detailed balance. The production rate adds over the two parts, because with detailed balance it's the rates times quantities that don't depend on the rates. The rest produces nothing negative, and the redraw part alone produces at least c times the relative entropy still left. So relative entropy falls at least as fast as e^{-ct}. One wrinkle needs its own line. A known start puts all its weight on one state and none anywhere else, and the logarithms in the relative entropy go bad at zero. The relative entropy is still continuous at the starting instant, because each of its terms has the form $u \ln(u/\pi)$, and $u \ln u$ goes to zero as u does. So adding up the production from the start out to infinity gives exactly $-\ln \pi(s)$.

I should say what Doeblin's condition costs. It asks for every hop to be possible directly, in one step or at a positive rate. A device whose states reach each other only through other states can still settle, and the standard theory of these chains says when: always, for a device that hops at random moments and can get from any state to any other; and for a chain that ticks, as long as it also doesn't march round a cycle. Those devices meet the condition for some fixed stretch of time or number of steps instead of one, and the same argument runs on blocks of that length, because the relative entropy never goes up in between.

Then come the stretches. The bound used the fact that the long-run chances of up and down stand in the ratio of the mean stretch lengths, and I took the mean length of a stretch with the flag up to be the long-run weight of the up-states divided by the flow into them. For a chain that ticks, that's right for the following reason. Start each stretch the way the flow starts them, count the steps until the chain leaves the up-states, and the average is exactly $\pi(\text{up})$ over the flow. Look at the long-run

weight that has been up at each of the last $k+1$ ticks. It splits in two: the part whose stretch began exactly k ticks ago, which is the flow times the chance that a fresh stretch lasts more than k ticks, and the part that was up one tick earlier still. Add those splits from $k = 0$ up to some K and the sum telescopes: $\pi(\text{up})$ is the flow times the sum of those chances, plus the weight that has been up for more than K ticks. The sum of the chances that a stretch lasts more than k is its mean length. Doeblin's condition makes the leftover die: from any up-state the next step lands in the down-states with chance at least $\delta\,\pi(\text{down})$, so the weight that has stayed up for K ticks is at most $(1-\delta\,\pi(\text{down}))^K$.

With the stretches measured in seconds, when the device hops at random moments, it comes out the same, by a different route. Ask, for each up-state, how long on average the device takes to leave the up-states from there. Call that its leaving time. The leaving times are finite: from every up-state the device hops into the down-states at a total rate of at least $c\,\pi(\text{down})$, so the chance a stretch is still running at time t falls at least as fast as $e^{-c\,\pi(\text{down})\,t}$, and no leaving time is more than one over $c\,\pi(\text{down})$. And they obey one plain rule, which you get by asking what happens in the first instant: from any up-state, the rate out of it times its leaving time, minus the rates into the other up-states times their leaving times, comes to exactly one (per second times seconds is a pure number). Average the leaving times over the way the flow starts stretches, multiply by the flow, and the rule plus the balance of the steady spread (as much weight flows into each state as out of it) rearrange the sum into exactly $\pi(\text{up})$, so in seconds too the mean stretch is its mass over the flow. Both of the bound's premises are proved now, in steps and in seconds, under Doeblin's condition.

CHAPTER 11

From “Structural entitlement”

11.1 The count, exactly

I’ll set out the words and then the count, for any system with Section 7’s counter. A *list* is a finite list of candidates, repeats allowed, and $|\Omega|$ is the length of the list Ω, repeats counted. $\lceil \log_2 x \rceil$ is the base-two logarithm of x rounded up: the smallest $k \in \mathbb{N}$ with $2^k \geq x$. At $x = 0$ it is 0, since $2^0 \geq 0$. And $a \dot{-} b$ is subtraction on whole numbers that stops at zero, so $3 \dot{-} 5 = 0$.

Definition 11.1 (Decision tree)**.** A *decision tree* is a leaf, or a branch with two subtrees. Its depth is 0 for a leaf and one more than the deeper subtree for a branch; its leaf count $L(T)$ is 1 for a leaf and the sum of the two counts for a branch. Nothing is written on a tree: no question, no answer, no candidate.

Lemma 11.1 (A tree is no wider than two to its depth)**.** For every decision tree T, $L(T) \leq 2^{\text{depth}(T)}$, and so $\lceil \log_2 L(T) \rceil \leq \text{depth}(T)$. □

Proof. Start at the root. At the first level there can be at most two places, at the second at most four, and each further level at most doubles the places available. To count leaves that stop early as well, extend each such leaf with branches until all tips reach the tree’s greatest depth. This only adds leaves. At depth d the filled-out tree has 2^d tips, so the original tree has at most that many leaves. For depth zero there is just the root, which is one leaf.

Thus d bits are enough to number all the leaves. The least number of bits that will do, $\lceil \log_2 L(T) \rceil$, cannot be greater than d. ■

Nothing about any machine is in the definition or the lemma. They’re about trees, a leaf or a branch, and nothing else.

□ **Theorem 11.2** (Paid narrowing, counted)**.** Take a system with a counter μ and a price list c that obey conservation (Theorem 7.2): each step raises μ by exactly its charge. Let Ω and Ω' be lists with $|\Omega'| > 0$, let T be a decision tree with $|\Omega| \le L(T) \cdot |\Omega'|$ (the tree *covers* the narrowing), and let t be a run from s ending at s' in which at least $\mathrm{depth}(T)$ steps are charged at least one each. Then

$$|\Omega| \;\le\; 2^{\mu(s')-\mu(s)} \cdot |\Omega'|, \qquad \text{so} \qquad \lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil \;\le\; \mu(s') - \mu(s).$$

Proof. Count the bill first. There are at least as many paid steps as levels in the tree. Each paid step adds at least one and every other charge is nonnegative. The total bill is therefore at least the tree’s depth. Conservation says that bill is exactly the counter’s rise, $\mu(s') - \mu(s)$.

Now count the room the tree supplies. It has at most two to its depth leaves. The covering premise allows at most $|\Omega'|$ prior entries per leaf, so

$$|\Omega| \le L(T)|\Omega'| \le 2^{\mathrm{depth}(T)}|\Omega'| \le 2^{\mu(s')-\mu(s)}|\Omega'|.$$

That is the first bound.

For the bit version, let $k = \lceil \log_2 |\Omega'| \rceil$. The posterior has at most 2^k entries. Insert that count in the first bound: the prior has at most $2^{\mu(s')-\mu(s)+k}$ entries. So numbering its entries needs no more than the counter’s rise plus k bits. Subtracting the posterior’s k bits gives the claimed bound; if the subtraction would go negative, the stipulated subtraction stops at zero. An empty prior already has bit count zero and satisfies the same conclusion. ■

Two arguments are stuck together in that proof, and they come apart. The tree half (the covering and the lemma) bounds the index bits by the sum of the charges of the run, on any price list at all. Conservation is only what turns that sum into the counter’s rise. So the form in terms of cost, with the sum of the charges in place of the counter, holds in any Thiele machine, and it’s part (3) of Theorem 9.1 below. The shape of the bound shows in what’s missing from the hypotheses: anything tying the lists to the run.

There are three honest pieces of small print, all of them visible in the statement. Both logarithms round up separately, and the subtraction stops at zero, so when the prior and the posterior round up to the same number of index bits (four candidates narrowed to three, say) the rounded bound says nothing, though the first form of the theorem charges the run one unit: one leaf can’t cover four candidates with three. The lists are plain lists, repeats allowed, so “index bits” is a difference of rounded list lengths with repeats counted; a list can get shorter by dropping a duplicate without losing a single candidate, and “a candidate was ruled out” and “the list got shorter”

are two different claims. And the difference isn't the number of questions anyone asked.

The covering condition, $|\Omega| \leq L(T) \cdot |\Omega'|$, is the nearest thing the count has to an anti-stopped-clock clause, and I have to be exact about how near. Picture each survivor standing for a little crowd of the prior's candidates, a crowd no bigger than the tree has leaves. The condition says the crowds can hold the whole prior between them: share the prior's candidates out among them and none overflows. That limits how much can be deleted per leaf you paid for, but it doesn't say why anything was deleted. So the condition rules out deleting too much for the tree you've paid for, and it doesn't rule out deleting for no reason. A run can satisfy every hypothesis of the count and have ruled nothing out on purpose; the count prices the size of the drop and says nothing about the honesty of it. And the covering can't just be dropped. On the small machine of Part III (Section 15), whose record moves CHECK, COMMIT and CERTIFY cost one each, take the sixteen starts with A at 0 through 15, and the run CHECK "$A \geq 15$," COMMIT, CERTIFY from the start with A at 15. One yes-or-no question with a big constant in it narrows sixteen candidates to one, four index bits, and the run has three paid steps, a ledger of three. A tree no deeper than three has at most eight leaves, and eight times one survivor is less than sixteen, so the covering fails, and without it the bound would say $4 \leq 3$. More candidates only widen the gap: 2^n starts and the constant $2^n - 1$ give n index bits for the same cost of three (Section 16.4, Proposition 16.17).

A side trip is worth taking here, because it looks like it breaks the count and doesn't. Two information theorists, Rudolf Ahlswede and Gunter Dueck (1989), noticed that asking "is it this one?" is a different job from asking "which one is it?" If each receiver only wants to know whether the message is theirs, a noisy channel used n times can serve a number of receivers that grows like two raised to two to the nC, with C the channel's capacity, where naming the message only gets you two to the nC. The task is called identification, and on its face it says checking can be exponentially cheaper than naming. But their tests are allowed to say yes wrongly once in a while. Make the yes sound, a test that never says yes to the wrong message, which is the only kind of check this book lets raise a record, and the double exponential collapses. The argument is two lines of pigeonhole. Each receiver's test, to be any use, says yes on some output its own message can produce, and soundness means no other message can produce that output; so those outputs are all different, one per receiver, and there can't be more receivers than outputs, $|Y|^n$ for an output alphabet Y. That bound is single exponential, the same order as naming. So for a check that never lies, the card count is the right scale after all, and the cheap trick needs a check that can be fooled.

11.2 When the run does the narrowing

Theorem 9.1 takes the lists in. In the version below, the run makes the posterior, and the count stops being an item on the list and becomes a consequence. That version needs one thing a bare Thiele machine doesn't have: which steps are tests, and what each asks. So a reading names them. For each instruction it gives either nothing or a yes-or-no question about the state the run is in when it gets there, and a test step *passes* when its question says yes there. The candidates are states the run could have started from. A passed test is asked of a candidate by running the same moves from the candidate up to that step and asking the same question there. The *posterior* is the candidates that pass every test the run passed, asked that way, in the order the run asked them.

□ **Theorem 11.3** (Structural entitlement, with the narrowing made by the run)**.** Take a Thiele machine and a reading that names its test steps, every test step charged at least one. Take a state s that reads no, a run t from s that ends in a state that reads yes, and a list Ω of candidate starts that contains s. Let Ω' be the posterior t makes from Ω, and let b_i and a_i be the numbers of candidates standing just before and just after the ith test t passed. Then (1) Ω' sits inside Ω and contains s; (2) every candidate of Ω that isn't in Ω' fails, asked that way, a test t passed; (3) $|\Omega| \cdot \prod_i a_i = |\Omega'| \cdot \prod_i b_i$; (4) if every test t passed keeps at least a 2^{-k} share of the candidates standing before it, $b_i \leq 2^k a_i$, then $\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil$ is at most k times the number of tests t passed, that number is at most the number of steps charged at least one, and the index bits are at most k times the sum of the charges; and (5) some step of t took the reading from no to yes and was charged at least one.

Proof. (1) Start with the prior list. Each passed test only removes candidates; it never inserts one. The actual start s passes every test used to filter the list, because these are precisely the tests the actual run passed. So s remains throughout, and the final list is a nonempty part of the original.

(2) Follow any candidate that is absent at the end. It was present at first, so there is a first test at which it disappears. The filter removes it only when its version of that test says no while the actual run's says yes. That test is the required witness.

(3) Write the list lengths on the bench: q_0 before any passed test, q_1 after the first, through q_n after the last. Here $q_0 = |\Omega|$ and $q_n = |\Omega'|$. Every before-count b_i is q_{i-1} and every after-count a_i is q_i. The left product in the statement is therefore $q_0 q_1 \cdots q_n$; the right product has the same factors with q_n placed first. They are equal, including when there are no tests and both products are empty products, equal to one.

(4) At the first test the share premise gives $q_0 \le 2^k q_1$. At the second it gives $q_1 \le 2^k q_2$. Substitute one into the other, and continue. After n tests, $q_0 \le 2^{kn} q_n$. If j bits number the final candidates, then $q_n \le 2^j$, so $kn + j$ bits suffice to number the original ones. The rounded bit drop is at most kn. Each of the n passed tests is a separate paid step, and each adds at least one to the nonnegative bill. Thus n is at most the number of paid steps and at most the total charge, giving the two cost comparisons.

(5) Read the actual run from its initial no until its first yes. The preceding reading was no, so that step raised it. The toll charges that step at least one. ∎

Part (3) is the exact count: take base-two logarithms and $\log_2 |\Omega| - \log_2 |\Omega'|$ is the sum, over the tests the run passed, of $\log_2(b_i/a_i)$, the bits each one took. Part (2) answers the complaint I made about the covering, that it doesn't say why anything was deleted: here everything deleted was deleted by a test the run passed and paid for. Two premises stay, the prior list and the share, a bound k fixed before the run on what one test can take. Every run meets some bound, its own biggest test's, so what the share adds is fixing it in advance, and that is needed. On the small machine one CHECK of "$A \ge 15$," asked of sixteen candidates, keeps one, a 2^{-4} share, and narrows by 4 index bits on a run of 3 record moves (Proposition 16.17); with $k = 1$ part (4) would say $4 \le 3$. The same statement on every certified Thiele-complete machine, where the tests are the CHECK moves and the charges are counted by the ledger, is Theorem 16.16.

CHAPTER 12

From “Records over any base: the latch”

12.1 Bases from outside, and a relation I have to be honest about

The theorem is stated for any base, so let me put it on three models of computation that other people built: a small Turing machine (any program); a register machine of the kind the computer scientists Stephen Cook (of the certificates) and Robert Reckhow described in 1973, with unit-cost indirect addressing (any program); and a lambda calculus (call-by-value, meaning an argument is worked out before it’s plugged in, and made deterministic by letting a term with no step left to take step to itself). On each, the latch form holds, by the one generic theorem applied to that base (the bases are defined below). So a register machine with a base-driven record is a register machine plus a cup, and so is a lambda calculus. (There’s one more instance, the machine I build later, where a particular flag is shown to be a base-driven record and the same theorem makes it a latch, in Section 15.)

I’ll give the three bases explicitly. A base needs a state, a next-state function, a set of starting states and a halting predicate; the latch theorem asks for nothing else about its instruction vocabulary.

Definition 12.1 (Three concrete bases)**.** The **tape base** has a finite Boolean list t, a head index h, a program counter q and a natural-number cost field. Reading beyond the list returns false. Writing at h extends the list with false cells if needed, then changes that cell. Moving right increments h and extends the list through the new head position. Moving left decrements a positive h; at $h = 0$, it prepends a false cell and keeps the head at zero. A fixed finite program contains these instructions:

Instruction	Next state
Write a bit	Perform the write and advance q by one.
Move left or right	Perform the move and advance q by one.

Instruction	Next state
Branch to j	Set $q := j$ if the current cell is true; otherwise advance q by one.
Halt	Leave the entire state unchanged.
ClaimTapeIsZero with a label ℓ	Advance q by one and do nothing else.

An out-of-range program fetch is Halt. Every state is allowed as a starting state, and halting means the fetched instruction is Halt. The cost field is preserved by every instruction in this base. In particular, its ClaimTapeIsZero instruction neither erases the tape nor charges the cost field; neither the name nor the label supplies any semantics.

The **random-access base** has a register function $X : \mathbb{N} \to \mathbb{N}$ and a program counter q. For a fixed finite program its instructions are:

Instruction	Next state
Constant (i, c)	Set $X(i) := c$.
Add (i, j, k)	Set $X(i) := X(j) + X(k)$.
Subtract (i, j, k)	Set $X(i) := \max(0, X(j) - X(k))$.
Indirect load (i, j)	Set $X(i) := X(X(j))$.
Indirect store (i, j)	Set $X(X(i)) := X(j)$.
Positive branch (j, a)	Set $q := a$ if $X(j) > 0$; otherwise advance q by one.
Jump to a	Set $q := a$.
Halt	Leave the entire state unchanged.

Each register-writing instruction advances q by one; its operands are evaluated in the old state, and all other registers are preserved. A fetch outside the program also leaves the state unchanged. Starting states have $q = 0$ and arbitrary register contents. A state is halted when it fetches Halt or is outside the program. Unit cost here counts instructions. It has nothing to do with the record's ledger.

For the indirect instructions, draw boxes 1, 2 and 3 and put 2, 7 and 0 in them. Indirect load $(3, 1)$ first reads the 2 in box 1, uses it as an address, and copies box 2's 7 into box 3. Now put 9 in box 3 and execute indirect store $(1, 3)$: the 2 in box 1 tells it where to put the 9, so box 2 changes and box 1 stays 2. An address is being read from a box; it is not necessarily the box's own number.

The **lambda base** uses the terms and weak call-by-value rule defined in Definition 14.1. To compute its next state, first try to reduce the function part of an application. If that part cannot step and is an abstraction, either substitute an

argument that is already an abstraction or take a step in the argument. If neither applies, the term cannot step. Variables and abstractions cannot step by themselves, and there is no reduction under an abstraction. The base leaves a term unchanged when it cannot step and calls precisely those terms halted. Every term is allowed as a starting state, including open terms.

So all three are bases in the sense of Definition 11.1, and the latch theorem applies to each. It says nothing else about them, nothing about what they can compute or how fast.

There's also a relation defined between bases, *weak base equivalence*: two bases are equivalent when some correspondence covers their start states both ways, related states agree on an observation and on halting, and one step on one side is matched by some number of steps, possibly zero, on the other. That "some number of steps" is the *stuttering* idea Leslie Lamport (the same Lamport as Section 7's message counters) put forward in 1983: a fine-grained machine and a coarse one shouldn't count as different because one takes three ticks to do what the other does in one. The relation carries no cost; a machine that multiplies huge numbers in one step and a Turing machine differ in price. The relation never looks at price, so on what it does look at they can match. It's proved to be an equivalence relation, and the latch form is preserved across it.

Draw a coarse machine $0 \to 1$, with 1 halted, and a finer one $a \to b \to c$, with c halted. Let a and b both show 0 and c show 1; halted states stay where they are. Relate 0 to both a and b, and 1 to c. The fine move $a \to b$ changes nothing visible, so the coarse machine matches it by taking no step. Its move $b \to c$ matches $0 \to 1$. Starting from a, the coarse move is matched by two fine steps. "Possibly zero" allows this; the relation is about matching observations and halting, not matching the number of ticks or their cost.

There is a limit to that statement. The "preserved across it" proof throws away its equivalence hypothesis; the latch holds on every base regardless, so the preservation is true for a boring reason. And nowhere in this book are *two* of the bases actually related by this relation. The relation is defined, it's an equivalence, and it is not used to connect the Turing machine to the register machine to the lambda calculus, or any of them to the machine of Section 15. It would be easy to read that list as if it did. The honest sentence is the one that's proved: each base, separately, carries a latch.

Definition 12.2 (Weak base equivalence)**.** Bases B_1 and B_2, with observations o_1 and o_2 into one common set, are *weakly equivalent* when some relation R relates every starting state of each to some starting state of the other, and any related x,

y have $o_1(x) = o_2(y)$, the same halting, some $n \geq 0$ with $R(\text{next}_{B_1}(x), \text{next}^n_{B_2}(y))$, and some $n \geq 0$ with $R(\text{next}^n_{B_1}(x), \text{next}_{B_2}(y))$.

Theorem 12.1 (Outside bases, and the relation)**.** (1) Weak base equivalence is reflexive, symmetric and transitive. (2) If two bases are weakly equivalent, Observation 11.1 holds over one exactly when it holds over the other. (3) Observation 11.1 holds over each of the three bases in Definition 12.1, for every fixed program where the base has one. □

Proof. (1) A base matches itself by pairing each state with itself and matching one step with one step. Reverse any matching between two bases and it gives a matching in the other direction.

For three bases, pair a state of the first with a state of the third whenever some middle state matches both. A start on either outside has a middle start beside it, and that middle start has a start on the far side. Observations and halting agree across the first pair and then across the second, so they agree across the outside pair too.

Follow one step on the first base. Its match supplies a finite sequence of middle steps. Match the first of those steps on the third base, then the second from where that match ended, and continue. Joining the finite sequences gives a match for the original step. If the middle sequence is empty, take no step on the third base. Do the same in the other direction for a step on the third base. This checks both stepping requirements and proves transitivity.

(2) The latch argument uses only a base step and the stated record rules. It therefore holds over both bases, whether or not they are matched. (3) Each of the three displayed constructions supplies that step, its starts and its halting reading. The same latch argument applies separately to each. ■

12.2 What it takes to be complete

The latch left two choices open, the price list and the event. A machine I can hold up against everything else has to make both, and a name for “this machine computes, and its record keeps something” has to ask for more than the toll does. The toll alone is a low bar. A machine that charges one for every step it takes is a clock bolted to a computer, and it obeys the toll for any record at all: every raise costs at least one because every move does. And a machine whose record never goes up obeys it too, because a toll on a raise that never happens is never broken. The clock-priced machine may keep useful records; its prices simply do not distinguish computing

from recording. The constant reading can keep no new yes/no distinction. The toll permits both.

I'll give the names now, all of them, once, with the reason for each, and they work the way Turing's do. *The Thiele Machine*, with capitals, is the model: a computer with a reading that can go from no to yes, natural instruction prices and the toll. The reading may also be constant or revocable. An internal ledger is extra structure. *A Thiele machine*, lower case, is any particular system that fits the model, the way one rule table is a Turing machine. That bar is low on purpose, and a clock clears it. *Thiele-complete*, defined next, is the bar worth meeting, the way Turing-complete is: a machine that computes anything, with its record kept apart from its computing and able to tell starts apart. Part III adds one stricter bar, *certified Thiele-complete*, for a record that, from a clean start, goes up only on a claim that was checked, committed and certified. And Appendix D names a *universal Thiele machine*: one machine that runs the others, the way Turing's universal machine runs every Turing machine. Those are all of them: the model, the systems that obey it, the ones that compute and keep, the ones whose keeping is earned, and the one that runs them.

The bar comes in four clauses, and nothing in it mentions what the record is about.

Definition 12.3 (Thiele-complete)**.** A machine with the five parts is *Thiele-complete* when it can be read through an *interface* that hands over three things: a universal base, a split of its moves into *computing moves* and *marks*, and a *ledger*, a whole number for each state. A *universal base* is a way of reading each state as a configuration of a machine that runs every program (I use Minsky's two counters, Section 15.2), a class of *live* states, a computing move for each of that machine's instructions that does to the configuration exactly what the instruction does from every live state and keeps it live, and, for each start, a live loaded state showing that start. The interface has to meet four clauses. (A) *Computing can't reach the record, and the record stays.* No computing move changes the record, and a record that's up stays up under every move. (B) *Starts are blank.* Every loaded state has the record down. (C) *Computing costs nothing, marks cost one.* Computing moves cost 0, every other move costs exactly 1, and each move adds its cost to the ledger. (D) *The record tells starts apart.* Some run, the same list of moves, ends with the record up from one loaded state and down from another.

Read the clauses as four plain sentences, with the universal base underneath them: the machine computes anything a Turing machine can. (A) The computing never touches what's kept, and what's kept stays. (B) Nothing is kept before the run starts. (C) The ledger counts the marks, and only the marks. (D) What's kept

depends on where you started: there's a run whose record says something about its start.

(D) is the clause where distinction comes in, and it's a low bar: a mark that reads one bit of the input meets it. What it excludes is a record that ignores the start. A record that goes up on every run from every start keeps a before and an after, and nothing about where the run began. (D) doesn't ask that every raise tell starts apart; it asks that the machine can. A machine may have a mark that raises the record from any state at all; it pays one for it, like every raise, and (D) is still met by some other run. Which runs a machine wants its record to tell apart is its own choice, and Part III makes one.

The name echoes Turing-complete, but this is a specification a machine meets, not a class closed under simulation: a machine is held up against it through an interface, and changing the unit of price drops a machine out.

Theorem 12.2 (What every Thiele-complete machine has)**.** Let a machine be Thiele-complete through an interface. (1) It obeys the toll, and the move that raises its record is a mark, never a computing move, and costs exactly one. (2) Along any run, the ledger rises by exactly the number of marks in it. (3) From any state with the record down, every run that ends with the record up raises the ledger by at least 1. (4) Some run from some loaded state raises the record. (5) Every move costs 0 or 1, and some move costs 0. (6) Its base runs every two-counter program step for step from every live state, and halting from a loaded state matches halting from the configuration it shows. □

Proof. (1) If a computing move raised the record it would change it, against (A); so the raising move is a mark, and a mark costs one by (C). (2) is (C) added up along the run. (3) Somewhere along the run the record goes from down to up, and that move is a mark, by (1), so it adds one. (4) is (D), and by (B) the record started down. (5) Every move is a computing move or a mark, and the universal base supplies computing moves. (6) is the universal base, one instruction at a time, from a loaded state that is live and shows its start. ■

The floor is one, and one is reached: the lift below gives a machine whose record goes up after a single mark.

Observation 12.3 (A clock is a Thiele machine and isn't Thiele-complete)**.** *Take the two-counter machine, let every move cost 1, and take as the record any yes/no question about its configuration; or a flag that switches on, and stays on, the first time a chosen test of the configuration says yes. It obeys the toll and has a two-counter machine inside it, and it isn't Thiele-complete. No machine whose every move costs at least 1 is.*

Proof. A Thiele-complete machine has a move priced 0, by Theorem 12.2(5), and this one has none. ■

A two-counter base priced 0 with a record that's down at every state obeys the toll and meets (A), (B) and (C), and isn't Thiele-complete either: (D) asks for a run that raises the record.

□ **Theorem 12.4** (Each of the four clauses is needed)**.** For each clause there's a machine that meets the other three through some interface and is Thiele-complete through none. Two of them are built on the two counters with two marks, one that raises the record when A is 0 and one that always raises it, and a ledger beside them that each move adds its cost to. (A) fails: that machine, and one more move, priced 0, that puts the record back down. (B) fails: the two counters, every move priced 0, with a record that no move ever changes, up at the starts where A is 0 and down elsewhere. (C) fails: the two counters with two marks, every move priced 1. (D) fails: the two counters, every move priced 0, with a record that's always down.

Proof. Each machine meets the three by construction. The first fails (A) through every interface, because a move lowers the record, so the record doesn't stay. The second: every one of its moves costs 0, so (C) makes every move a computing move under any interface; then by (A) no run changes the record, and with every start blank by (B) no run raises it, so (D) fails. The third has no move priced 0, so it fails by Observation 12.3. The fourth never raises its record. ■

□ **Theorem 12.5** (The record can't be read off the window)**.** Let a machine be Thiele-complete through an interface. (1) Every configuration is shown, at no cost, with the record down, by one computing move from some loaded state. (2) Some loaded state has two runs that end on the same window, one with the record up and one with it down, and their ledgers differ by at least 1. (3) So no rule that reads only the window says, at the end of every run from every loaded state, whether the record is up, and none says what the ledger reads.

Proof. (1) From the loaded state $(1, x+1, y)$, the computing move "DEC A to line p" shows (p, x, y), costs nothing by (C), and leaves the record down by (A) and (B). (2) Take the run (D) gives, from the loaded state where it raises the record. Its end shows some configuration. From the same loaded state, computing moves alone reach any configuration at all: count each counter down to zero, count it up to the value wanted, and jump to the line wanted. That run ends on the same window,

and it's all computing, so by (A) and (B) its record is still down and by (C) its ledger hasn't moved, while the first run's ledger went up by at least one, at its raise. (3) A rule that reads the window gives one answer for one window, and (2) has one window with two answers. ■

Part III proves the same blindness for its certified machines (Observation 16.12), with a gap of three, because a certified raise costs three. Here the gap is one, and one is reached.

Theorem 12.6 (Every universal base becomes Thiele-complete)**.** Take any machine with a universal base and any events on its states, and add a record, a ledger and, for each event, a mark that costs one and raises the record exactly when the event holds; computing moves cost nothing and leave the record alone, and every move adds its cost to the ledger. If some run of computing moves makes some event hold from one loaded state and not from another, the result is Thiele-complete, its computing is the original machine's step for step, and its record goes up after a single mark from any state where the event already holds. □

Proof. (A): computing moves are the original's and don't touch the record, and a mark only adds to it. (B): the record starts down. (C): by construction. (D): run the moves that make the event hold from one loaded state and not the other, then the event's mark. Step for step: a computing move acts on the original part and nothing else. And a mark from a state where its event holds puts the record up. ■

One mark on "A is at least 1" always works: the starts with A at 0 and at 1 already differ on it, so that mark alone, run from each, tells them apart. So Thiele-complete isn't a property only a special machine has. Any model with a universal base becomes Thiele-complete by adding marks, and needs no claims and no checker to do it; a model that simulates the two counters some other way does so once its moves are grouped into ones that carry out a counter instruction each (Theorem 22.6). Two kinds of base don't lift, whatever the events: one whose moves don't depend on the state it's in, and one whose moves lead from each state to only finitely many next states. A universal base needs, from one state, a move at no cost to every line of a program, and that's infinitely many places to go.

Some Thiele-complete machine runs every Thiele machine a program can describe. Appendix D defines a *universal Thiele machine* and builds hosts that run any machine a program can describe, in order, carrying its reading as a latch and matching its ledger up to a surcharge of at most three, with every computing step done by counter moves on the host and every rise of the host's record placed by its program. None of that needs certification: the presented host's own program, read

on a host with no claims, no checker and no table of facts, and one paid mark that raises the record when a register is at least 1, meets the definition too.

The rest of the book holds machines to this bar. Part III gives one example that clears it, and then asks for the stricter certified bar: that from a clean start its record go up only on a claim that was checked, by a checker proved to mean it, and then committed, and that some claim’s check can come out either way. Asking that is a choice of what the record answers to, the second of the latch’s two open choices, made one way, with clean starts and three priced acts added (Theorem 16.3).

CHAPTER 13

From “The shadow, exactly”

13.1 How much gets destroyed, on average

The halving price is a worst case, because it prices the largest landing pile a move can have. The next question is the average one: if I don’t know which state the machine was in, cups and all, but I have some spread of chances over the states, how much do I lose, on average, about which state it was, once I know where it landed? Shannon’s conditional entropy measures exactly that (Section 3 built the entropy from scratch, so I won’t do it again here), and I’ll write it in bits.

Theorem 13.1 (The halving price in bits bounds the average loss)**.** Take any spread of non-negative weights over the states, uniform or not, with finite total weight, on a machine of any size. (1) If every landing pile has at most N states, with $N \geq 1$, the weighted conditional entropy of the state given its landing spot is at most the total weight times $\log_2 N$ bits. (2) So on a machine with the halving price, that weighted information loss is at most its cost in bits times the total weight, even when the spread’s own entropy is infinite. For a probability distribution, the total weight is one and the average loss is at most the cost. (3) If a landing pile has 2^c states, the uniform probability distribution on it loses exactly c bits and attains that bound. □

Proof. Look at one landing spot. Suppose the sources that land there carry total weight $w > 0$. Divide each source’s weight by w. Those shares now add to one: they describe the uncertainty about the source once you know this landing spot. There are at most N possible sources. The logarithm argument in Section 3 says the uncertainty is greatest when those possibilities have equal shares, and is then at most $\log_2 N$ bits. Multiply by w to put this pile back into the whole spread. Its contribution is at most $w \log_2 N$. A pile with zero weight contributes nothing.

Add the contributions. The pile weights add to the original total weight, so the same $\log_2 N$ multiplies that total. Under the halving price, $N = 2^{\text{cost}}$, whose base-two logarithm is exactly the cost.

Infinitely many piles cause no subtraction of one infinite entropy from another. I am adding the nonnegative uncertainties inside the piles. Every finite collection contributes at most its weight times the bound, and its weight is at most the total. Taking the largest value approached by those finite sums keeps the same bound.

Finally put all probability on one pile of 2^c sources, equally shared. Before landing there are 2^c equally possible origins; afterwards the landing spot distinguishes none of them. The uncertainty left about the origin is $\log_2(2^c) = c$ bits. That attains the bound. ■

Where does it stop? My first guess was one place, a landing pile with infinitely many states, and that the average would blow up there right along with the worst case. That’s half right, and the half that’s wrong surprised me. No finite charge satisfying the halving price can price a move that sends infinitely many states to one spot. The weaker merge price could still charge one, since it does not measure the pile’s size. But the average depends on the spread. Put weight $\frac{1}{2}$ on the first state, $\frac{1}{4}$ on the next, $\frac{1}{8}$ on the next, and so on, and the information lost by crushing the pile never reaches 2 bits, however many of its states you count. Under that spread, a two-bit information budget covers the average, though no finite budget covers the worst case. So I went looking for a spread that does blow up, and it took a nastier one: blocks of states, block k holding 2^{2^k} states and a share $\frac{1}{(k+1)(k+2)}$ of the weight, which adds up to one. Under that spread the loss grows past every bound, and no finite information budget covers that average loss.

You can generate the first spread by tossing a fair coin until the first head. Zero tails names state 0, one tail names state 1, and so on. State j has chance $2^{-(j+1)}$, and identifying that outcome takes $j+1$ toss-results, so its entropy term is $(j+1)/2^{j+1}$. Add these by counting each possible toss position: the first is reached with chance 1, the second with chance $1/2$, the third with chance $1/4$, and so on. The total is $1 + 1/2 + 1/4 + \cdots = 2$; every finite partial sum of the state terms is smaller. Resetting all those outcomes to one box can lose only those two bits on average, despite having infinitely many possible starts.

For the block spread, split each block’s weight equally among its states. The block weights sum to one because $1/((k+1)(k+2)) = 1/(k+1) - 1/(k+2)$. Within block k, there are 2^{2^k} equally likely choices, so just identifying a state within that block contributes $2^k/((k+1)(k+2))$ to the average bits. Those contributions grow without bound as k grows, and forgetting which block it was only adds to the loss. And no finite worst-case price can handle an infinite pile: for any proposed cost c, select $2^c + 1$ different states in it. They all land together, violating the halving rule’s maximum of 2^c.

Theorem 13.2 (An infinite pile)**.** (1) A move that sends infinitely many states to one spot isn't priced by the halving price at any finite cost. (2) Under the halving spread $2^{-(j+1)}$, crushing the pile loses exactly 2 bits, and counted over its first n states, less than 2 for every n. (3) Under the block spread, the information lost grows past every bound, so no finite bit budget covers it. □

Proof. The three computations above. ■

And heat? Part I already said where the line is, and it's the same line here. Landauer's principle, that crushing c bits' worth of possibilities gives off at least $c\,kT\ln 2$ of heat on average, is a premise from physics, and a contested one (Section 3). Nothing in this section proves anything about heat. What the counting does give is that, under that premise, on a machine with the halving price, a move's cost priced at $kT\ln 2$ per unit, times the weight of the piles, is always enough to cover the minimum the premise would demand, for every spread of finite total weight. If the premise falls, that sentence falls with it, and every theorem above it stands, because none of them used it.

CHAPTER 14

From “The diagonal”

Two forms of the wall live in this section, and they’re honest in different ways.

The first is the *nat family*, a tiny substrate built so that program two is the Contrarian for a given candidate decider d, and for every d, d is wrong about program two. It’s a *family*, one instance per d, and its recursion theorem holds by construction. It’s the index cards made formal and nothing more.

□ **Theorem 14.1** (The nat family)**.** Fix any Boolean function d on $\mathbb{N}$. Let the substrate N_d have programs and states $\mathbb{N}$; program 0 returns 1, program 1 returns 0, program 2 returns 0 if $d(2)$ is true and 1 if it is false, whatever the input state, and every other program gives nothing; each program is encoded as itself; μ is constantly 0, so any step relation will do; $\mathrm{Rep}(f)$ holds when $f(p) = 1$ for every p with $d(p)$ true and $f(p) = 0$ for every other p; and $A(p)$ holds when program p returns 1 on input 0. Then it is not the case that $d(p) = \text{true} \iff A(p)$ for every p.

Proof. The recursion theorem of N_d holds with the fixed witness 2: for a representable f, $f(2)$ is 1 or 0 according to $d(2)$, and program 2 returns exactly what that program returns. The witnesses for A are 0 and 1, and A is extensional because it reads only what a program returns. The flip of d is representable by the definition of Rep, so Theorem 13.1 applies to d itself. Directly: $A(2)$ holds exactly when $d(2)$ is false. ■

Now push on it once more, because the nat family shows the outside decider in three lines, and you can do them yourself. Stand outside N_d, knowing d. Write down a new decider e: say yes on program 0, say “not $d(2)$” on program 2, and say no on everything else. Check it against A. Program 0 returns 1, so $A(0)$ holds and e says yes. Program 1 returns 0, so $A(1)$ fails and e says no. Program 2 returns 1 exactly when $d(2)$ is false, and that’s what e says. Every other program gives nothing, so A fails and e says no. So e is right about every program. Does that break the wall? It doesn’t, and the reason is the part that matters: in N_d a flip is representable only if it says 1 exactly where d says yes, so the flip of e is representable only if e and d agree

everywhere, and they disagree at program 2. The correct decider exists; its flip just isn't one of the substrate's own moves. (It's three lines and a pencil, and you just did it.) One and the same property is undecidable from inside and decided cleanly from outside, at the same time. That's not a paradox, because it's two different people asking.

The second form is the one where the recursion theorem is *proved* from the reduction rules rather than assumed. It's on *L*, the weak call-by-value lambda calculus that Yannick Forster and Gert Smolka presented in 2017 as a model of computation, and which a group at Saarland has since used as the seed of a whole library of undecidable problems. Here L is built from its reduction rules. The recursion theorem is proved, Rice's theorem follows, halting follows, and the diagonal is reproved in the shape of the substrate theorem, for any property of programs that depends only on what they compute and that holds of one program and fails of another. The statement mentions no shortcut. So on L the result is the diagonal for one fixed model, and I don't plug L into the substrate theorem, for this reason: a substrate's run map has to answer, with a state or with nothing, and for L that map is the halting function itself, which no term of L computes. Using it would take something from outside L, and L is in this book as the model where nothing is taken from outside. So on L I run the argument again from scratch instead of applying the general one. Both are true; they're different sentences.

I'll say what that string of words means, because L is the strangest thing in this book that computes, and the smallest. The lambda calculus is Alonzo Church's, from the 1930s, and it's a programming language with one idea in it: functions. A term is one of three things: a variable; a function, written λ followed by its body, the body saying what to do with whatever you hand it; or one term applied to another, written side by side. Running a program is one move, over and over: when a function meets an argument, put the argument into the body where the variable was. The lambda calculus has no numbers, no memory, no instructions, and in 1937 Turing published a proof that it computes exactly what his machines compute. "Weak call-by-value" is two house rules about the order of moves: an argument is run down to a finished function (a *value*) before it's put in, and nothing runs inside a function that hasn't been handed anything yet. Variables are written as numbers counting how many λs back they were bound (a trick of Nicolaas de Bruijn; it means two variables can never get mixed up by sharing a name), and a term is *closed* when every variable in it is bound inside it, so it leans on nothing outside. Last, to hand a program to a program, L needs a way to write any term as data, the way a program's source is a string you can pass around. The one used here is the *Scott encoding*, named for Dana Scott, a student of Church's who went on to build mathematical models of the lambda calculus itself. As far as I can trace it, the encoding first appeared in lecture notes of his from Berkeley in 1962 that he never published, as a way to write data;

Torben Mogensen carried it over to lambda terms themselves in 1992, and that’s the form used here. All you need about it is that every term t gets one *code*, written $\ulcorner t \urcorner$, and the code is itself a closed value. Two tools get built on top of that in the proof: a *quoting* term Q that takes the code of a term to the code of that code (a program that prints its own source needs exactly this move), and a *fixed-point combinator*, a term that lets a function call itself, which is how Q works its way through a code of any size.

Do two of these on paper before I give you the general rule. Write $I = \lambda x.x$: its instruction is “return whatever you were handed.” Hand I its own card and you get $I\,I \to I$, and it stops. Now write $D = \lambda x.x\,x$: “make two copies of what you were handed and apply the first to the second.” Hand it its own card. Putting D in for both copies of x gives $D\,D \to D\,D$. You’re back at exactly the same instruction and argument, so the rule never finishes. Both arguments were already functions, so neither example sneaks any work under a function that hasn’t been handed anything.

The variable numbers are what keep the copying from getting confused. In $\lambda x.\lambda y.x$, the last x points past the nearest λ (the one for y) to the one before it, so its number is 1; in $\lambda x.\lambda y.y$ the last variable has number 0. Rename x and y however you like and neither recipe changes. The codes below are a way of putting a whole recipe on a card that another recipe can read.

Definition 14.1 (L, as built here)**.** A term of L is a variable (a de Bruijn number), an application $s\,t$, or an abstraction λs; the values are the abstractions. One step is weak call-by-value, left to right: $(\lambda b)\,v$ steps to b with v substituted when v is a value, a step inside the left part of an application is a step of the application, and so is a step inside the right part once the left part is a value. A term s *evaluates to* v when s reaches v in some number of steps and v is a value; $p \equiv q$ when p and q evaluate to the same values. Every term t has a code $\ulcorner t \urcorner$, its Scott encoding, which is a closed value. A map f on terms is *L-computable* when some closed value s has $s\ulcorner p \urcorner \equiv f(p)$ for every p. A term D *decides* a property P of terms when, for every p, $D\ulcorner p \urcorner$ reaches $\lambda\lambda.1$ (true) if $P(p)$ holds and $\lambda\lambda.0$ (false) if it fails. P is *extensional* when $p \equiv q$ and $P(p)$ give $P(q)$.

□ **Theorem 14.2** (Recursion, Rice and halting for L)**.** (1) For every closed value s there is a closed term t that reaches $s\ulcorner t \urcorner$; so every L-computable f has a term p with $p \equiv f(p)$. (2) Let P be extensional, with $P(y)$ and not $P(n)$ for closed terms y and n. Then no closed term decides P. (3) No closed term decides halting, the property of evaluating to some value.

Proof. (1) First build a program that can assemble the code of the very application I am about to run. The quoting program Q takes a term's code and returns the code of that code. Appendix 5 gives the idea of its construction. The program mkapp takes two codes and joins them into the code of an application.

Make A do this when handed a code x: quote x, put that quoted code beside x using mkapp, and hand the result to s. Written out,

$$A = \lambda x.\, s\,(\text{mkapp}\ x\ (Q\,x)).$$

Now run A on its own code, and call that whole application t: $t = A\ulcorner A\urcorner$. The first step puts $\ulcorner A\urcorner$ where x stood. Quoting it supplies the code of $\ulcorner A\urcorner$; joining the two codes supplies the code of $A\ulcorner A\urcorner$. That is the code of t itself. The computation therefore reaches $s\ulcorner t\urcorner$, as promised. All the pieces were closed, so t is closed too. If s computes f, the resulting behaviour is exactly $f(t)$.

(2) Suppose a closed program D decides P. Build another program that asks D about the code it receives, then runs the no-example n after yes and the yes-example y after no. In this language you must keep the two branches wrapped as functions until the answer chooses one; otherwise call-by-value would try to run them before choosing. The following term does that:

$$\lambda x.\, D\,x\,(\lambda_.\, n)\,(\lambda_.\, y)\,(\lambda z.z).$$

The final identity function is the harmless argument that opens the selected wrapper. Thus a true answer runs n, and a false answer runs y.

Apply part (1) to this program, obtaining its self-matching t. If $P(t)$ holds, the judge says true and t behaves like n, where P fails. If $P(t)$ fails, the judge says false and t behaves like y, where P holds. Either answer contradicts the requirement that alike behaviour has the same verdict. No such D exists.

(3) Halting depends only on which values a term reaches. The identity $\lambda z.z$ is already a value. The term $(\lambda x.\, x\,x)(\lambda x.\, x\,x)$ takes a step back to itself each time, and is never a value. These supply the yes-example and no-example for part (2), so halting has no closed decider either. ∎

The construction in that proof isn't mine, and the credit belongs right here. Forster and Smolka's L development of 2017 has the Scott encoding of terms, a closed quoting term with exactly this specification, the code of a term to the code of its code, and a fixed point built the same way: their t is $C\ulcorner C\urcorner$ with $C = \lambda x.\, s\,(A\,x\,(Q\,x))$, so their C is my A, and their A, which builds the code of an application, is my mkapp. They say they follow the proof in the Dutch logician Henk Barendregt's textbook on the lambda calculus (1984), where statement (1) is the full lambda calculus's second fixed-point theorem, and part (2) is what they call

Scott's theorem. The idea of the term construction is in Appendix 5; the route is theirs, and Barendregt's before them.

So I've given the wall on paper, on a family built to fit it, and on one outside model. The machine built in Part III (Section 15) fills in this same shape: on its host machine, the one the universal program of Appendix D.1 runs on, which has a counter for every whole number, programs are numbers, Kleene's recursion theorem is a theorem, and so is Rice's, and the fixed point can be taken to agree on what the program computes and also on the ledger and the flag at the end of every run where they stop (Appendix 21.2).

The table below lines up the picture all of that fills in against Turing's, one column per axis.

Time axis	**Structural axis**
counts: steps	counts: μ
asks: does it halt?	asks: does it admit the shortcut?
the wall: no Turing machine decides halting	the wall: no decider whose flip is representable, under a recursion theorem

Read the table as two uses of one diagonal, each under its own premises, and as saying nothing more than that. The structural wall never uses μ; the μ entry only says what that axis counts. The nat result is a family indexed by the candidate decider, and L's results are about one fixed model and properties that mention no shortcut. Neither hands you a second fixed undecidable problem, and neither separates Turing degrees: a property can sit behind the wall from inside and be decided cleanly from outside, as the pencil argument on N_d showed. And "admits the shortcut" in the right-hand column is the theorem's sense, any behavioural property with a yes and a no. "Has a cheaper way to the same answer" is a second question, and the column has no wall for it.

CHAPTER 15

From “What established means”

15.1 The guy who got closest

Somebody had that clue before me, and on that clue he’s the closest neighbour I’ve found, so he gets the most space and the hardest push.

Bennett again, the one from Part I’s counting of states and the demon, wrote a paper in 1988 that starts where I just ended up: information isn’t the same as value. His example is an almanac of where the moon and planets will be every day for a hundred years. It has no more information than the equations it was calculated from, but it saves you doing the calculating. So the value is the work. He called the amount of work a thing carries its *depth*.

Was this my idea, already done, by a guy at IBM? The only way to find out was to find where it parts from mine, because if I couldn’t find that place I’d just be renaming his idea. And it’s one word: *plausibly*. Depth is the amount of work a thing was *plausibly* made with. Bennett says straight out that you can’t know how a message actually came to be; for all you know it came from flipping coins. Depth is read off the finished thing, the way a fossil’s age is read off the fossil.

Bring the two computers back. The sorted list in the first one has some depth. The identical list in the second has exactly the same depth, because depth looks at the list and nothing else. A photocopy of a deep thing is just as deep. Depth tells you what work a plausible way of making the thing would take. It’s not a record that *this* machine did it, and it’s not a receipt for what doing it cost.

So Bennett’s standing right next to the line, on the other side of it, and as far as I can tell he got his own question right. He asked how much work a thing embodies. I’m asking who has the right to lean on it and what they paid to get there. His answer lives in the object. Mine, if I’ve got one, has to live in the machine.

15.2 The record, the checker and the host

Take the flag and ledger off the small machine’s drawing. The core follows the same steps: neither field tells it what to do next. Put them back and you can read whether certification occurred and what the instruction schedule charged. That is the passive

record’s job. The latch argument in Section 11 gives its form over any fixed base when the record is driven by that base and never comes down.

The checker has another job. CHECK of “$A \geq m$” tests the counter in one instruction; a program using only the two counter moves would have to work through their operations. A step comparison therefore starts with the instructions each machine is given. Every checker built in this book is a program. The abstract certified interface also admits a supplied checker, so its computational power includes whatever that checker can do: supply a halting oracle and CHECK can answer halting questions. The effective implementations here use effective checks.

Now put a guest program inside the host. A guest that can write any value into its own “checked” variable can write YES without checking. The host I build gives the guest three entry points instead: CHECK, COMMIT and CERTIFY. Its step rule controls those entry points for every allowed guest program. The fact table and channel determine whether the next act is permitted; the flag and ledger record the result. An ordinary universal machine can implement these rules. The work is in preserving them through every guest instruction.

The universal program in Appendix D.1 does that work with the small machine’s six instruction kinds on a host with more counters. It fetches a guest instruction, decodes it, and carries it out. At matching points the guest and host ledgers agree. The host halts exactly when the guest halts, keeps the guest’s answer in its counters, and raises its own flag exactly when the guest raises its flag, through its own earned chain. Its guests are the small machine’s programs from clean starts, with Minsky’s encoding supplying their universal computation. Appendix 21.1 gives a simpler picture with a guest-step operation supplied directly; the instruction-by-instruction construction shows how to build it.

Add a one-unit PAY instruction and the host can also follow a computably presented Thiele machine: its selected moves, transitions, prices and total yes-or-no reading are all given by programs. Appendix D.2 constructs that interpreter. Starting with the guest’s reading at no, the host follows its ledger with at most two extra units at the raise. Starting with the guest already at yes, the clean host must establish its own flag immediately and spends three. The same clean-start three-act count explains the surcharge when the guest certifies with a whole ledger below three.

CHAPTER 16

From “The earned-chain counter example”

16.1 Nothing gets forged

The theorem and its proof are in the book; this is what stood beside them.

The closest earlier result is in capability hardware, and it’s much bigger. The computer scientists Jack Dennis and Earl Van Horn, at MIT, introduced *capabilities* in 1966: tokens a process has to hold to touch an object, which it can pass on and can’t forge. CHERI, a capability extension of real instruction sets built at the University of Cambridge and SRI International, rests on that idea, and in 2020 Kyndylan Nienhuis and twelve coauthors proved, for the whole CHERI-MIPS instruction set, that arbitrary code given some capabilities can’t increase them as it runs (up to any change of protection domain). Their result says nothing gets forged across a full instruction set. Theorem 15.14 is the same shape, one instruction at a time, for six instruction kinds and a table of checked facts.

The actual counter covers every added fact

Put the table beside the bill. Every passing CHECK writes one more entry and adds one to the ledger. A failed CHECK writes no entry. COMMIT and CERTIFY can add to the bill without adding a table entry. Walk down the run one instruction at a time: the table can never grow faster than the bill.

Theorem 16.1 (The native bill bounds the table)**.** For every clean run, whether driven by a finite instruction word or by the stored-program step for a finite number of ticks, □

$$\mathrm{len}(\mathrm{facts}) \le \mu.$$

More generally, table growth is exactly the number of passing CHECKs, and is at most the increase in the ledger from any starting state.

Proof. A passing CHECK adds one entry and one unit to the bill. A false CHECK, a full-table CHECK or a trapped CHECK adds no entry but still adds one unit. COMMIT and CERTIFY add one unit and no entry. INC, DEC and HALT add neither. No instruction removes an entry.

Place a mark beside each passing CHECK in the run. The final table has exactly one more entry for every mark. Each mark also accounts for one unit on the bill; unmarked instructions can add further units, never take them away. So table growth is exactly the number of marks and at most the ledger’s increase. This works from any starting state.

At a clean start the table is empty. The initial ledger is a nonnegative whole number, so the table length is at most the final ledger. At the specified loaded starts the ledger begins at zero as well. For a stored program, every running tick executes one of the six instructions just counted, and every stopped tick leaves both table and bill alone. The same count therefore applies to any finite number of ticks. ■

One passing CHECK from a clean start gives one entry and a bill of one. Repeat a passing check and its duplicate entry counts as another line in the table. The tally follows what was written there. COMMIT and CERTIFY can then make the bill larger while the number of lines stays put.

16.2 A full table traps

The table has room for sixteen facts and evicts none of them, so there are two halves to prove: nothing ever leaves, and nothing ever overflows.

□ **Theorem 16.2** (Bounded table, no eviction)**.** Every instruction keeps every fact already in the table. If the table holds 16 or more facts, CHECK $p\ c$ puts the trap up and leaves the table as it was. If the table holds at most 16 facts, it still holds at most 16 after any instruction.

Proof. With a raised trap no instruction changes the table. With the trap down, only CHECK can change it, and a successful CHECK adds one fact without removing an entry. Success requires a previous length strictly below 16, so the new length is at most 16. At length 16 or greater that branch is unavailable: CHECK raises the trap and preserves the table. Every other instruction preserves it as well. ■

Why trap? Why not make room? Because “making room” means deciding which fact dies, and a fact that can die is a fact whose absence you have to explain later. A table that only grows keeps every checked fact available to read back, and the shared-memory result above leans on exactly that, since its oldest fact can never

move. Provenance doesn't: a table that dropped its oldest fact to make room would still hold only facts a CHECK wrote. What the choice gives up is real, though, and it's in the subsection near the end of this chapter on what the machine doesn't do.

16.3 What it takes to be certified

Section 12.2 set the bar worth meeting, Thiele-complete, and it says nothing about what the record is about. A clock fails it, and so does a machine whose record never goes up. The small machine clears it (Theorems 16.11 and 16.3(2) below), and it does more: from a clean start, its flag goes up only on top of a check of a claim, by a checker proved to mean what the claim says, then a commitment to the same claim with nothing changed underneath, then a certificate. A Thiele-complete machine can raise its record with one mark that goes up from any state at all, a receipt for nothing (Section 14). A certified one can't. The second definition is the first one with three things added, and it's written so that any machine can be held up against it. You name a language of claims first, then hand over a reading of the machine in that language, a list of things, and then the reading has to pass four clauses.

Definition 16.1 (Certified Thiele-complete)**.** A *machine* has states, moves, a step, a cost for each move and a yes/no reading of each state, the *record*. A *claim language* for it is fixed first: a type of *claims* with an exact equality test; a *meaning* for each claim, a statement about a state; for each claim, a relation "the thing this claim is about is unchanged" between two states; and a proof that a meaning that holds keeps holding while the thing it's about is unchanged. An *interface* over that language hands over the rest: a *universal base*, meaning a two-counter configuration shown by each state (its *window*), a class of *live* states, a move for each two-counter instruction, and a loaded state for each start $(1, a, b)$; a *checker* that says yes or no to a claim at a state, with a proof that a yes means the claim's meaning, as the language gives it, holds of that state, with a CHECK of a claim passing at a state only when the checker says yes to it there; a set of *clean* states; a *ledger*, a whole number for each state; and a *kind* for each move: a base move, CHECK of a claim, COMMIT of a claim, or CERTIFY. The machine is *certified Thiele-complete over* the language when some interface over it meets four clauses, and *certified Thiele-complete* when some interface, with claims, meanings and "unchanged" of its own, meets them. (a) *Universal base.* Each two-counter instruction's move is a base move; from every live state it acts on the window exactly as the instruction acts on a configuration, and the state stays live; each loaded state shows $(1, a, b)$, is live and is clean; no base move changes the record; and a record that's up stays up under every move. (b) *Earned record.* Clean states have the record down. Every run from a clean state that ends with the record up contains a CHECK of some claim c that passes, then a COMMIT

of c with the thing c is about unchanged at every state in between, then a CERTIFY, with the record down just before that CERTIFY and up just after it. A passing check means its claim's meaning holds of the state it checked, and a meaning that holds keeps holding while the thing it's about is unchanged. (c) *Exact prices.* Base moves cost 0, each CHECK, COMMIT and CERTIFY costs exactly 1, and each move adds its cost to the ledger. (d) *Non-vacuity.* Some claim c has a CHECK, a COMMIT and a CERTIFY such that those three moves alone, from a loaded state, raise the record exactly when c's meaning holds there; and c holds at some loaded state and fails at another. A Thiele machine *has a universal base* when some interface gives it one and nothing more is asked: each two-counter instruction's move acts on the window exactly as the instruction does from every live state and keeps it live, and each start $(1, a, b)$ has a live loaded state showing it. Nothing is asked about the record, the prices or the checker.

An interface over a language becomes an interface of the plain kind by taking the language's claims, meanings and "unchanged" as its own, and the four clauses are asked of that.

□ **Theorem 16.3** (Certified is Thiele-complete plus certification)**.** (1) A machine is certified Thiele-complete through an interface exactly when it is Thiele-complete through the same interface read without its claims (its base moves as the computing moves, CHECK, COMMIT and CERTIFY as the marks, the same universal base and ledger), and three more things hold: its loaded states are clean; clause (b) holds, so clean states have the record down, every run from a clean state that ends with the record up contains the earned chain, a passing check means its claim holds, and a claim that holds keeps holding while what it's about is unchanged; and clause (d) holds, a claim whose three moves raise the record from a loaded state exactly when it holds there, true at one loaded state and false at another. (2) Every certified Thiele-complete machine is Thiele-complete. (3) Not the other way: the two counters with two marks, one that raises the record when A is 0 and one that always raises it, and a ledger beside them that each move adds its cost to, is Thiele-complete, and certified Thiele-complete through no interface.

So certification is exactly three things on top of the bar: clean starts, an earned record (the chain, a checker proved sound, claims that stay true while what they're about is unchanged), and a claim that could have come out either way. Everything Section 12.2 proved holds of every certified machine. What this section adds is what those three things buy, from a clean start: a floor of three instead of one, a raise that is always a CERTIFY, and no receipt without a check; and, from the loaded states, a check that can fail.

The language comes first for a reason. If the interface could name the meanings, it could take "the checker says yes" as the meaning of every claim, and checker soundness would hold for nothing. In the language-first form the meanings are written down before any checker is handed over, so "a checker proved to mean what it says" has the content it has on the small machine: what the claim says is the language's, and the proof is that a yes from the checker makes it true. The theorems that build certified Thiele-complete machines ask the checker for one thing more, that it say yes exactly when the claim holds, because clause (d) needs a check that can come out yes. The plain form stays because the theorems that follow hold for any interface at all, so they hold over every language too.

Read the four clauses as four plain sentences. (a) It computes anything a Turing machine can, under Minsky's 2^n encoding, done outside the machine, and the computing can't reach the record: the base moves are the two-counter machine, they leave the record exactly where it was, and a record that goes up stays up. (b) From a clean start, the record goes up only on top of the small machine's chain: a check that passed, a commitment to the same claim with nothing changed underneath, then a certificate. (c) Computing costs nothing on the ledger and each of the three acts costs one unit, so the ledger counts the acts. The steps in between add nothing to it. (The unit belongs to the ledger; it isn't a unit of heat.) (d) Somewhere there's a claim whose check could have said no. A machine doesn't qualify over a language whose every claim is "at least 0", and neither does one with nothing to certify.

And look where the checker's honesty sits. In (b), "a passing check means its claim holds" is a condition on the interface, proved once, outside every run, exactly where it sat for the small machine (Lemma 15.2), and over a claim language it's proved against meanings the interface didn't write. The definition doesn't ask the machine to vouch for its own checker. The nearest reasons not to ask are Gödel's second theorem, about a system proving its own consistency, and the diagonal of Section 13, about deciding a property of every program. Neither forbids a machine checking a proof about a checker, so keeping that proof outside is a choice. Nor does it ask the checker to be something a machine could run. Clause (b) wants a yes or no for every claim at every state, sound for the claim's meaning, and nothing in it says that answer has to be computable. Every instance in this book has a checker that's an actual program, so anything those machines certify could be listed by a program that runs them. For example, take a function that answers whether an arbitrary program runs forever. No program computes it (that's Turing's halting theorem). Hand the definition a checker built on it and it's still met, and the machine certifies claims no program could ever confirm. Nothing in this book builds one. So "a certified Thiele-complete machine certifies exactly the claims a program could list" is true only with "and its step and its checker are computable" added, and the definition doesn't require either.

The same scope goes on Section 14's "it isn't more computing power." The record (the flag and the ledger) is never read by the step that computes the next core, so erasing it changes no run, and it makes nothing faster. The checker is part of the machine under the record. With an oracle checker that machine decides what the oracle decides, since whether a CHECK traps is there for anyone to see; with a checker that's a program, it does what programs do. And a checker that's a program still makes CHECK a test at unit cost.

□ **Theorem 16.4** (CHECK is a unit-cost test)**.** From any start (a, b), the single move CHECK Ge(m) A costs 1, and the trap latch is down after it exactly when $a \geq m$.

The two counters alone look at A only through the zero test of DEC, and started one pebble below a threshold or right at it, that test can't come out differently until the pile has been counted down to zero, so they need about as many steps as the threshold for what CHECK does in one. Theorem 16.21 gives the precise lower bound and the matching ladder. So the record makes nothing faster, and CHECK, a primitive test, can.

Three kinds of exact. The word exact does three jobs in this book, and each job keeps one name. *Rise-only pricing* charges at least one unit on every step that raises the reading and nothing on any other step; its tightest form, one unit per raise and nothing else, is the *exact schedule*. *Instruction-exact pricing* charges every instruction a fixed listed price. On this machine that's one unit each for CHECK, COMMIT and CERTIFY and nothing for the rest, and the definition of certified Thiele-complete calls it *exact prices*, not a toll, because it charges acts that raise nothing. CHECK and COMMIT raise no flag, so the small machine is instruction-exact without being rise-only, and no rule is broken by that. What the three units pay for isn't one thing three times. A passing CHECK adds a fact to a table that holds sixteen, so it takes a fresh state and fills one slot of a bounded store; a passing COMMIT overwrites the channel and a passing CERTIFY raises the flag, each sending two states with the same base to one (Theorem 3.19). And each of the three, when it fails, sends the failed state and its twin with the trap already up to one state. So a passing CHECK pays by taking a fresh state, and every one of the three units is also a price on a merge. *Simulation-exact accounting* is a host whose ledger equals its guest's ledger at every point where the two runs match, as the universal program U's does (Appendix D.1). A machine can meet any one of the three and fail the others.

One word needs pinning before the theorems, using those three names. In clause (c), *exact prices* means instruction-exact pricing: each of the three acts, CHECK and COMMIT included, costs exactly one, so the ledger counts the moves filed as record moves. Those are the acts that make a claim, and also a check that fails, a COMMIT

of a fact already committed, and a CERTIFY with the flag already up: each is filed as a record move and costs one. Back in Section 7, one result is about rise-only pricing in its tightest form, the *exact schedule*: one unit on the step that raises the flag and nothing on any other step. Measured against the flag alone, the small machine isn't rise-only, since CHECK and COMMIT raise no flag and cost one each; measured the way (c) asks, it is instruction-exact. Neither is wrong. They answer different questions: does the ledger count the acts, or does it charge the flag and only the flag? Simulation-exact accounting, the third name, comes up with the universal program U in Appendix D.1.

Theorem 16.5 (What every certified Thiele-complete machine has)**.** Let a machine be certified Thiele-complete through an interface. (1) It obeys the toll and has a universal base; its base runs every two-counter program step for step from every live state, and halting from a loaded state matches halting from the configuration it shows. (2) Along any run, the ledger rises by exactly the number of CHECK, COMMIT and CERTIFY moves in it. (3) A run from a clean state that ends with the record up contains at least three of those moves and raises the ledger by at least 3, and the three moves of clause (d), from a loaded state where its claim holds, raise it by exactly 3. (4) Some CHECK fails at some loaded state, on a claim that is false there, and some run from some loaded state raises the record. (5) On a run from a clean state, the move that raises the record is a CERTIFY. (6) Every move costs 0 or 1, and some move costs 0. □

Theorem 16.6 (The language first, and what a certificate then means)**.** (1) A machine that is certified Thiele-complete over some claim language is certified Thiele-complete, so every part of Theorem 16.5 holds of it. (2) If a machine is certified Thiele-complete through an interface whose claims have an exact equality test, it is certified Thiele-complete over the language made of that interface's claims, meanings and "unchanged," with clause (b)'s proof that a meaning keeps holding as the language's own. (3) If a machine is certified Thiele-complete over a claim language L, some interface over L meets the four clauses and has this property: every run from one of its clean states that ends with the record up contains a CHECK of a claim c of L that passed, then a COMMIT of c, and L's meaning of c holds at the state the CHECK saw and at the state the COMMIT saw. □

Part (2) says where the strength of the language-first form lives: in the naming. Any reading can be turned into a language when its claims come with an exact equality test, so for those "certified Thiele-complete over some language" asks no

more than certified Thiele-complete does. "Certified Thiele-complete over L," for an L written down without looking at the checker, says that what the machine certifies is what L says. Every instance below is stated that way.

Each item in the proof names the clause it comes from, and that's all the definition gives. A certified Thiele-complete machine needn't have a fact table, a CHECK that traps when it fails, a COMMIT that looks anything up, or claims about counters. The definition asks for an earned chain on the runs that raise the record and leaves the rest to the machine. So the small machine's table provenance, its trap, and its channel are facts about that machine, and they don't carry over to every machine that meets the definition. Through a plain interface its claims can be any labels the interface names; over a claim language they are the language's. Either way its checker can be an oracle, as the halting-function checker above shows.

The second half of (6) in Theorem 16.5, that some move costs 0, is the one that rules out the clock: a machine that charges every move. It doesn't rule out a program that makes the ledger tick. On the host of Appendix D.1, a program that checks one fact once and then commits it again before every instruction runs any two-counter program with the flag down and pays one unit a step, plus one for the check: the checked fact is about a counter the program never writes, so its edition never changes, and every COMMIT finds it.

Observation 16.7 (A clock is a Thiele machine and isn't certified Thiele-complete)**.** *Take the two-counter machine, let every move cost 1, and take as the record any yes/no question about its configuration whatsoever; or take a flag that switches on, and stays on, the first time any chosen test of the configuration says yes. Either way the machine obeys the toll and has a two-counter machine running inside it, and it is not certified Thiele-complete. More generally, no machine whose every move costs at least 1 is certified Thiele-complete.*

(It also follows from Observation 12.3 and Theorem 16.3(2).) The proof is short, and it's short because clause (c) does exactly the job it was written for: it says computing costs nothing, and a clock charges for computing. The other clauses do work too, and that's worth checking, because a definition with a clause nothing needs is a definition hiding something.

□ **Theorem 16.8** (Each clause does work)**.** (1) Take a two-counter base priced 0 with a ledger beside it, and one extra move TICK that costs 1 and raises a flag. It obeys the toll, and read with TICK as CERTIFY it meets the universal base clause and the exact prices clause; it is not certified Thiele-complete, and more generally no machine with a move that puts the record up from every state is. (2) Take a two-counter base priced 0 and a record that is down at every state, so it never rises.

It obeys the toll and meets clauses (a), (b) and (c); it is not certified Thiele-complete, and more generally no machine whose record never rises is.

The paid latch, the TICK machine of (1), is the receipt for nothing from Section 14, wearing a universal base: honest about the price, silent about what was bought. The definition throws it out.

The silent machine of (2) meets three clauses and fails only (d). The paid latch fails (d) as well as (b), so it doesn't show (b) doing work alone, and nothing above says whether a clock meets (a), (b) and (d). The sharper check, the usual evidence that a definition isn't written around one machine, is a machine for each clause that meets the other three and fails it.

Theorem 16.9 (The four clauses are independent)**.** For each clause of Definition 16.1 there is a machine with an interface that meets the other three clauses and fails that one. Each of the four machines obeys the toll and has a two-counter machine running inside it, and none of them is certified Thiele-complete through any interface at all. (a) fails for this chapter's machine with one more move, DROP, priced 0 and read as a base move, which lowers the flag and trips the trap latch. (b) fails for this chapter's machine with one more move, FORGE, costing one and read as a CERTIFY, which raises the flag on any state whose trap is down, with no commitment behind it. (c) fails for this chapter's machine with every move costing one. (d) fails for the silent machine of Theorem 16.8(2). □

Three of the four are this chapter's machine with one thing changed, and in each case the one clause it fails is the only thing keeping it out. Clause (c) also fixes something it has no business fixing: the size of the unit. Double every price on the small machine and it leaves the class, because its CHECK now costs two and (c) asks for one. A definition that a change of currency breaks would be a strange one, so the next theorem states (c) with the unit left open.

Theorem 16.10 (Any unit)**.** Call a machine *certified Thiele-complete at unit c*, for a whole number $c \geq 1$, when some interface meets clauses (a), (b) and (d) and this form of (c): base moves cost 0, each CHECK, COMMIT and CERTIFY costs exactly c, and each move adds its cost to the ledger. (1) At unit 1 that is certified Thiele-complete. (2) Multiply every price of a certified Thiele-complete machine by $c \geq 1$ and the machine is certified Thiele-complete at unit c; divide every price of a machine certified Thiele-complete at unit c by c and the machine, with the same states, moves, step and record, is certified Thiele-complete, with the ledger read as $\lfloor \ell / c \rfloor$. (3) At every unit some move costs 0, so the clock of Observation 16.7, □

with any reading or any latch as its record, is certified Thiele-complete at no unit, and neither is any machine whose every move costs at least 1. (4) At unit c, a run from a clean state that ends with the record up contains a CHECK, a COMMIT and a CERTIFY and raises the ledger by at least $3c$; the claim it stands on held when it was checked and still held when it was committed; and the move that raises the record is a CERTIFY. (5) The small machine with every price doubled is certified Thiele-complete at unit 2 and isn't certified Thiele-complete.

So the class doesn't care what a unit is worth. It cares that computing costs nothing and that the three acts cost the same. The clock fails at every unit for the reason it failed at one: it charges for computing. I keep unit 1 in the definition because the small machine is built that way and every count in this chapter is in its units.

Now to the other direction, because a definition nobody meets is as useless as one everybody meets.

□ **Theorem 16.11** (The small machine, the generic one and the sorted one are certified Thiele-complete)**.** (1) This chapter's machine is certified Thiele-complete, read with INC, DEC and HALT as base moves, CHECK $p\ c$ and COMMIT $p\ c$ as record moves on the claim “p holds of c,” CERTIFY as CERTIFY, the checker as eval on the value of c, the window as (line, A, B), live states as those with the trap down, start(a, b) as the loaded state for $(1, a, b)$, “unchanged” as “same value and same version of c,” clean states as in Definition 15.2 and the ledger as μ. (2) So is the same machine over any property language with an exact equality test and a checker proved to say yes exactly when the claim holds, as long as some property is true of one number and false of another. (3) In particular the machine whose property is “this counter, read as a list, is sorted” is certified Thiele-complete. (4) Every certified Thiele-complete machine is a Thiele machine (Definition 2.2) and obeys the universal floor; for this chapter's machine, that system agrees step for step and cost for cost with the one of Theorem 15.7, and for the generic machine with the generic machine's own. (5) Each of the three is certified Thiele-complete over its own claim language, fixed before the checker. For this chapter's machine the language's claims are pairs of a property and a counter, equal when both parts are, the meaning of “p holds of c” is the meaning holds of Definition 15.1 applied to c's value, written without the checker, and “unchanged” is “same value and same version of c.” For the generic machine the language is built from the properties, their equality test and what they mean, and the checker isn't part of it; the sorted machine is that case.

Appendix 22 carries the same fixed-language form to other bases. The claims keep their stated meanings; the new interfaces supply the checkers.

There's one thing every machine that meets the definition can't help doing. It has Section 8's shape again, only this time it isn't about one machine but about all of them.

Observation 16.12 (Every certified Thiele-complete machine hides its record and its ledger from its window)**.** *Let a machine be certified Thiele-complete through an interface, and let its window be the two-counter configuration its universal base shows. (1) For every state s there's a loaded state and one base move priced 0 from it that lands on a state with s's window, with the record down and the ledger unchanged. (2) There's a loaded state and two runs from it that end with the same window, one with the record up and one with it down; and likewise two runs from one loaded state that end with the same window and ledgers at least 3 apart. (3) So no function of the window gives the record at the end of every run from every loaded state, and no function of the window gives the ledger. (4) The same holds of the Thiele machine built from the machine.*

The observer in that theorem sees one thing, the final window. An observer who sees the window before and after each step, or the whole trace of windows, or who is also handed the program and the start, is a different observer, and the collision says nothing about them; given the program and the start, the run is fixed and so is its record. The theorem's two runs also start from the same loaded state, so they're reachable witnesses, which is stronger than two arbitrary states that happen to look alike. And "no function of the window gives the record" is all it says. It doesn't say the window tells you nothing about it.

Notice how little that took, and say it out loud so nobody mistakes it for more. It's the fibre fact from Section 8 plus one observation: a universal base can print any window at no cost, and that is the point. The window here is everything a Turing machine would call the machine, the line and the counters. On a certified Thiele-complete machine, that's never enough to tell you what was established or what it cost, and it can't be, because computing costs nothing and can't touch the record. A richer window, one that shows the flag, is a different window, and nothing above says it fails.

16.4 Parts I and II, run on the example machine

Parts I and II proved their counts and their collisions for any system with the named pieces, and they left a promise that the machine of this part would be put through them, and this section is where it's kept. The eight-state machine of Section 3 gets run by the small machine, price for price. Structural entitlement, the count of what narrowing a list of rivals costs, is proved for every certified Thiele-complete machine, with the small machine as a worked member, a larger search on the host that has a counter for every number, and a run that shows the count's one awkward hypothesis, the covering, can't be dropped. And the time a check saves gets an exact exchange

rate, for the part of the machine that has no record. (The no-bare-verifier result is in Section 8, next to the collision it comes from.)

The eight-state machine, run by the small machine. Look at the small machine through a window that shows two things: its line number read modulo four, which I'll call its *slot* (the eight-state machine's word from Section 3, unrelated to the slot registers of the universal program U in Appendix D.1), and its flag. Call a state *live* here, a narrower use of the word than in the definition of certified Thiele-complete, when its trap latch is down, the fact "$A \geq 0$" about A's current version is in its table, and its channel holds a commitment, which is what CERTIFY needs in order to run.

Definition 16.2 (The eight-state machine, compiled)**.** Stamp is the single instruction CERTIFY. Next-slot is the single instruction INC B. Jump to slot a, for a one of $0, 1, 2, 3$, is the four instructions COMMIT $(A \geq 0)$ A; COMMIT $(A \geq 0)$ A; INC B; DEC B a. The *setup* is the two instructions CHECK $(A \geq 0)$ A; COMMIT $(A \geq 0)$ A.

The two COMMITs in a jump change nothing but the line and the channel, and each costs one, which is where the jump's price of two comes from. The INC makes B positive, so the DEC takes one off and jumps to line a at no cost, and B is back where it was.

□ **Theorem 16.13** (The small machine runs the eight-state machine)**.** (1) For every live state s and every move m of the eight-state machine, running m's instructions from s ends in a live state whose window is the eight-state machine's step of s's window by m, and the ledger has risen by exactly m's charge: one for stamp, two for jump, zero for next-slot. (2) The same holds for any list of moves: the window after the compiled list is the eight-state machine's run of the list, the state is live, and the ledger has risen by the sum of the charges. (3) So a run of compiled moves from a live state that takes the flag from down to up has raised the ledger by at least one. (4) From any start (a, b), the setup ends in a live state with window (slot 3, flag down) and the ledger at 2; after it, any compiled list runs as on the eight-state machine from (slot 3, flag down), with the ledger at 2 plus the sum of the charges; and if the flag is up at the end, the whole run, setup included, contains the earned chain of Theorem 15.11.

There are three reservations, and I'll say them now. The setup's claim is true of every value, so its CHECK can't fail; it's there because the small machine won't CERTIFY without a checked, committed claim, and not as a test. And the jump's price of two is two paid COMMITs, a choice a program makes that the machine allows and doesn't force. The window is a projection, too: the small machine's full state keeps far more than eight states' worth.

Structural entitlement, for every certified Thiele-complete machine. Theorem 9.1 was about any Thiele machine, and what the run paid went in as a hypothesis. For a certified Thiele-complete machine (Definition 16.1) what the run pays is known. Call the CHECK, COMMIT and CERTIFY moves of a run its *record moves*; each costs exactly one, so the ledger rises by the number of record moves, and clause (b) says what a raised record must contain.

Theorem 16.14 (Structural entitlement represented)**.** Let a machine be certified Thiele-complete through an interface. Take a clean start s_0 and a run t from it; a type of rivals and a type of observations with an exact equality test; an observation obs of each rival; a decoder d from runs to observations; lists Ω, Ω' of rivals and a decision tree T. Suppose (M1) Ω' strictly narrows Ω, as in Theorem 9.1, and (M2) some dropped rival shows an observation different from every survivor's; (M3) the run ends with the record up and the decoded run $d(t)$ is shown by some survivor; (M4) $\mathrm{depth}(T)$ is at most the number of record moves in t; (M5) $|\Omega'| > 0$; and (M6) the covering: $|\Omega| \le L(T) \cdot |\Omega'|$. Then (1) the membership test of Ω' is strictly stronger than that of Ω; (2) t contains the earned chain of clause (b), with the meaning of the claim true at the CHECK and at the COMMIT; (3) $\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil$ is at most the number of record moves of t; (4) the number of record moves of t is exactly the ledger's rise; and (5) the ledger rose by at least 3. □

It's a checklist, as I said in Section 9, and the small print is the same kind. (M3)'s second half feeds no conclusion: only the record's being up is used. (M1) is used only for the inclusion; its gap feeds nothing, since (M2) already hands over a dropped rival. And (M5) is used but can go: (M2) puts a rival in the prior, and the covering then forces a survivor. What the machine adds to Theorem 9.1 is conclusions (2), (4) and (5): the payment is the machine's own three acts, each at exactly one, and a raised record carries the earned chain.

There are two more, one where the tree comes from the run's own questions and one about what the record adds.

Theorem 16.15 (A floor with no supplied tree)**.** Take a machine and an interface that meets the exact-prices clause (c) of Definition 16.1, so each record act costs exactly one. Call the *questions* of a run the claims of its CHECK moves, and say two rival states *answer alike* when every one of those claims gets the same verdict at both (each verdict taken at the rival itself, as it starts; the states the run passed through don't enter). If, for a list Ω of rival states and a number $m > 0$, the rivals of Ω that answer alike with any one rival, itself included, are never more than m, then □

$\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 m \rceil$ is at most the number of CHECK moves of the run, which is at most its number of record moves, which is the ledger's rise.

The tree has gone, and the run's own questions stand in its place. There are limits, and they're the ones in the statement. The verdicts are taken at the rivals as they start, and the states the run had reached when it asked don't come into it. A branch that costs nothing, DEC on zero, can still narrow a list without adding anything to the ledger. So the theorem prices a narrowing that the paid checks explain, and no more.

The next statement takes the verdicts where the run took them. A rival is a state the run could have started from; a CHECK the run passed is asked of a rival by running the moves before it from the rival and checking the same claim at the state that reaches. Nothing is supplied but the prior.

□ **Theorem 16.16** (Structural entitlement, with the narrowing made by the run)**.** Let a machine be certified Thiele-complete through an interface. Take a clean start s_0, a run t from it that ends with the record up, and a list Ω of rival starts that contains s_0. Let the posterior Ω' be the rivals that pass every CHECK t passed, each asked as above, in the order t made them, and let b_i and a_i be the numbers of rivals standing just before and just after the ith. Then (1) Ω' sits inside Ω and contains s_0; (2) every rival of Ω that isn't in Ω' fails, asked that way, a CHECK that t made and passed; (3) $|\Omega| \cdot \prod_i a_i = |\Omega'| \cdot \prod_i b_i$; (4) if every CHECK t passed keeps at least a 2^{-k} share of the rivals standing before it, $b_i \le 2^k a_i$, then $\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil$ is at most k times the number of CHECKs t passed, that number is at most the record moves of t, the record moves are the ledger's rise, and so the index bits are at most k times the ledger's rise; (5) t contains the earned chain and the ledger rose by at least 3; and (6) with no share assumed, the index bits are at most $\lceil \log_2(|\Omega|/|\Omega'|) \rceil$, which is $\lceil \sum_i \log_2(b_i/a_i) \rceil$ and at most the sum over the CHECKs t passed of $\lceil \log_2(b_i/a_i) \rceil$.

Part (3) is the exact form of “index bits are the sum of what each test took”: take base-two logarithms and $\log_2 |\Omega| - \log_2 |\Omega'|$ is the sum of $\log_2(b_i/a_i)$. Parts (4) and (6) are what survives rounding to whole bits, and (6) rounds once, at the end, so it can be a bit lower for every test than rounding test by test. Rivals $(a, 0)$ for $a = 0, \ldots, 26$ with CHECKs $A \ge 9$, $A \ge 15$, $A \ge 19$ stand at 27, 18, 12 and 8: the index bits are $5 - 3 = 2$, rounding once gives 2, and rounding each test gives 3. Here the run does the narrowing, every candidate dropped was dropped by a check the run paid for, and what stays a premise is the prior and the share, a bound fixed in advance; part (6) shows what the run pays without one.

Proposition 16.17 (One CHECK can take four bits)**.** Take the sixteen starts $(a, 0)$ with $a = 0, \ldots, 15$, and the run CHECK $(A \geq 15)$ A; COMMIT $(A \geq 15)$ A; CERTIFY from $(15, 0)$. It raises the record. Its one passing CHECK keeps one rival of sixteen, so the posterior is $(15, 0)$ alone and the index bits are 4, while the run has 3 record moves. So index bits at most the record moves, the bound of Theorem 16.16(4) read with $k = 1$, fails here. □

The share here is 2^{-4}, and with $k = 4$ part (4) reads $4 \leq 4$. It's the example of Section 9 that showed the covering can't be dropped, seen from the run's side: one test can take many bits, and the bound of part (4) covers the bits per test times the tests.

Theorem 16.18 (What the record adds)**.** Let a machine be certified Thiele-complete through an interface. (1) From a clean start, the record is up at the end of a run exactly when the run contains the earned chain, and a run with the record up raised the ledger by at least 3. (2) Conclusions (1), (3) and (4) of Theorem 16.14 hold without a clean start and without the record being up, for a machine that only meets the exact-prices clause (c): the hypotheses (M1), (M2), (M4), (M5) and (M6), and the decoded run being shown by some survivor, are all it needs. □

So the record doesn't make narrowing cost more. It adds the earned chain and a floor of 3, and that's all it adds: a run meeting the same counting and payment hypotheses is still bound by that index-bit floor even if it never raises the record, and the floor of 3 no longer applies to it.

Weights, fibres and partitions. Three more pieces of the same count. First, suppose the rivals aren't equally many: give each a whole-number weight, a multiplicity, and call the sum of the weights of a list its *mass*. The count survives with masses in place of lengths. If the prior's mass is at most $L(T)$ times the posterior's, the rounded index bits of the two masses differ by at most depth(T); so for any Thiele machine a run that paid at least that many charged steps has a cost at least the drop, and for a certified Thiele-complete machine the representation theorem goes through with masses and the same five conclusions. On such a machine the ledger rises by the same number from every start, the number of record moves, so the weighted sum of ledger rises is the prior's mass times the record moves, and the drop in rounded bits times that mass is at most the weighted sum. Divide by the total mass and you'd have an average rise; I leave the weights as whole numbers, cross-multiplied, so no probability comes into it. Second, a way to meet the covering (M6) that you can see. Give each survivor a little crowd of rivals, its *fibre*, no bigger than the tree has leaves,

and let the fibres' sizes add up to at least the prior. Then $|\Omega|$ is at most the sum of the fibres, which is at most $L(T)$ times the number of survivors, and that's (M6). Third, a plainer way to get fibres. Pick a *representative* observation of each rival, and say the rivals are *partitioned* by it when every rival looks like some survivor under it and no observation class, counted inside the prior, has more than $L(T)$ members. Take each survivor's class as its fibre: every rival is in the class of some survivor, so the fibres add up to at least the prior, and (M6) holds. The converse is false: take four rivals that all look alike, two survivors and a tree with two leaves, and give each survivor two of the four as its fibre. That covers, but the one class has four members, more than two.

A worked member on the small machine. Eight rivals: the starts (a, b) with $a \in \{0, 1, 2, 3\}$ and $b \in \{0, 1\}$, counter A at a and counter B at b. The run is CHECK (Even) A; CHECK ($A \geq 2$) A; COMMIT ($A \geq 2$) A; CERTIFY, from $(2, 0)$. The program raises the flag from a rival exactly when a is even and at least 2, so exactly from $(2, 0)$ and $(2, 1)$, and the posterior is those two. The decoder reads one "passed" for each CHECK in the run's receipts; the tree is the complete tree of depth 2; and the covering comes from fibres by a representative observation, "B is 0," which isn't constant: each survivor's fibre is the four rivals that share its B.

□ **Proposition 16.19** (The eight-start member)**.** For every a and b, the run raises the flag from (a, b) exactly when a is even and at least 2. All hypotheses of Theorem 16.14 hold, with eight rivals narrowed to two: the test of the posterior is strictly stronger; the index bits are $3 - 1 = 2$; the run has 4 record moves and the ledger rises by exactly 4; and the earned chain is in the run. The run's own questions (Theorem 16.15) are its 2 CHECKs, so the floor is met with equality. With the two CHECKs alone, the record stays down, the ledger rises by 2, and the bound $2 \leq 2$ still holds.

It's a real narrowing, and not the cups of Section 9 sitting under it in a costume. The posterior is exactly the set of rivals the run certifies, so here a proof ties the lists to the run, and nobody supplies them. Read with Theorem 16.16, the run's two passing CHECKs, replayed from the eight rivals, leave four and then two, the same two, so the posterior is the one the run makes; each keeps half, so $k = 1$, and the index bits, 2, are at most $1 \cdot 2$ and at most the ledger's rise of 4.

A search through n bits. The same shape comes back at a size you can't enumerate, on the machine of Definition D.2, the host in Appendix D.1 with a counter for every number, with the small machine's property language of Zero, Even and At-least-n, read at registers, in place of Slot. That machine is certified Thiele-complete too, by the argument of Theorem D.6 with these three properties in place of Slot. Hide

$n = k + m$ bits, one in each of registers 0 to $n - 1$. The program asks k questions, one per bit of the first k: CHECK "at least 1" of register i for a 1, CHECK "is 0" for a 0. Then it COMMITs the first claim and CERTIFYs. The prior is every n-bit value; the posterior is the 2^m values that begin with the answers; the tree is the complete tree of depth k; and a survivor's fibre is the 2^k values that agree with it on the m bits nobody asked about.

Theorem 16.20 (A search through n bits)**.** For $1 \leq k \leq 16$ and any m: from a world holding a value v of at least k bits, the program certifies exactly when v begins with its answers, so the posterior is exactly the set of rivals it certifies from; the test of the posterior is strictly stronger than the prior's; the earned chain is in the run; the prior has 2^{k+m} members and the posterior 2^m, so the index bits are exactly k; the run has exactly $k + 2$ record moves and the ledger rises by exactly $k + 2$; and the run's own questions are k CHECKs, so Theorem 16.15 is met with equality. With the COMMIT and the CERTIFY left off, the questions alone narrow 2^{k+m} values to 2^m, the record stays down, and the ledger rises by exactly k. And no run from a clean start can execute more than 16 CHECKs before its first successful CERTIFY: a failed CHECK would trap, and the table holds only 16 passing ones. Thus this program with 17 questions followed by its first COMMIT and CERTIFY certifies from no start. A trace that certified earlier can keep its flag up even after later failed instructions. □

The sixteen belongs to the machine, and the program has no say in it: it's the same ceiling as the item "Sixteen checks per run, ever" in the list of what the machine doesn't do, near the end of this chapter, and it's why this search never asks more than sixteen questions however many bits are hidden. Unasked bits can be any number. The lists, the tree and the fibres are supplied here, as in the representation theorem, and tied to the run only for this member. The covering still prices only the size of the drop, and its reason goes unpriced.

The covering can't be dropped. It's a hypothesis, and you'd hope it's a nuisance you could remove. It isn't, and the small machine shows it. Fix $n \geq 4$, take the 2^n starts $(a, 0)$ with a below 2^n as the rivals, and run the three-move chain on the claim "$A \geq 2^n - 1$": CHECK, COMMIT, CERTIFY. It certifies exactly from $A = 2^n - 1$, so the posterior is one start, n index bits are gone, and the ledger rose by 3. So for $n \geq 4$ the claim "the index-bit drop is at most the ledger's rise" is false for this run, and no tree of depth at most 3 covers the narrowing: (M6) would need $2^n \leq 8 \cdot 1$. One property with a big constant in it carries n bits in a single yes-or-no question; what the bound prices is a narrowing whose classes are balanced, a tree with enough leaves. A narrowing of any other shape is outside it.

The time a check saves. The question is whether counter A is at least m. There are two ways to find out. The *unpaid* way uses base moves only, INC and DEC, which add nothing to the ledger. But a base move reads A one pebble at a time: a DEC takes a pebble off A or finds it empty. After n moves, a run that started with A at least n has never seen A empty, so it can't have behaved differently for different such A except by a constant shift in A itself. The *paid* way is one CHECK ($A \geq m$): one move, a ledger of one, and the line it ends on says which way it went. Certifying the answer takes the whole chain, CHECK, COMMIT and CERTIFY.

□ **Theorem 16.21** (The time tax). (1) For every program of two-counter instructions and every $n < m$, the line number and B after n moves (or where the run stopped, if it stopped sooner) are the same from $A = m - 1$ as from $A = m$, with the same B. So no such program, read off its line number and B after fewer than m moves, tells whether $A \geq m$ at every start; in particular none that stops within fewer than m moves does. (2) A program of m DEC/INC pairs (a ladder: pair i is DEC A on line $2i - 1$, jumping on success to the next pair at line $2i + 1$, and INC B on line $2i$, where a DEC that finds A empty falls through) decides it when observed after m moves: after m moves its line is $2m + 1$ exactly when $A \geq m$; and it costs 0. (3) Executed as a three-move trace from start(a, b), the paid chain raises the flag exactly when $a \geq m$ and has a ledger of exactly 3, whatever m is. As a stored program it pays 3 on success and stops after its first failed CHECK with a ledger of 1. (4) From any loaded start start(a, b), a single CHECK ($A \geq m$) decides it in one move from the line number, with a ledger of exactly 1, whatever m is. (5) So if a unit on the ledger is worth λ moves, the paid route to deciding costs $1 + \lambda$ and the unpaid route costs at least m, and the paid route is cheaper exactly when $1 + \lambda < m$; the paid route to certifying costs $3 + 3\lambda$.

When $3 + 3\lambda < m$, the paid chain both answers and certifies within a combined budget smaller than the unpaid lower bound for answering alone. That compares different guarantees; the unpaid route never certifies. For a bare answer on both sides, the comparison is $1 + \lambda < m$. The exchange rate is sharper than a comparison of two programs I happened to write would give: the unpaid side is a floor under every program of INC and DEC there is, so no choice of program comes into it, and the paid side doesn't move with m at all. The bound has limits. It's for programs that make no record move, INC and DEC with HALT or falling off the end as stopping. It doesn't cover every certified Thiele-complete machine, because the definition doesn't forbid a base move that changes a counter by more than one pebble, and such a machine is a different machine, one the bound doesn't speak for. And the two routes put their answers in different places, both bare decisions in the line number, with

the paid chain also putting a historical record in the flag; this compares two ways to answer one question, not two certifications. Zero cost isn't zero heat, either: every move on either route is an event, and the moves are what λ prices.

CHAPTER 17

From “Five small models”

The fragment is a three-instruction memory-bounds language with the policy “every address is below the limit”: the checker accepts exactly when the condition holds, and a certificate implies the condition. One example sits on top of that, a computation you can do in your head: a program that reads address two under a limit of two is rejected. The fragment isn’t the paper’s real case study, and nothing connects it to anything else in this book. And it has a limit: a program that writes garbage to address zero passes if zero is below the limit. The proof is of the stated policy and nothing else.

And what was the paper’s real case study, the one my fragment isn’t? It was network packet filters, the little programs an operating system runs on incoming network traffic to decide which program gets to see which packet. They run inside the kernel, the most trusted part of the system, and they have to be fast, so they’re exactly the code you’d most like to take from a stranger and least like to trust. In the 1996 paper that first set out the idea, Necula and Peter Lee wrote several filters by hand in the assembly language of a DEC Alpha, shipped each with its proof, and reported a one-time cost of 1 to 3 milliseconds to check the proof, after which the filter ran with no checks at all. Those milliseconds put a number on “checking is cheap by design”. What the paper reports changes nothing about my fragment, which still isn’t that case study, and it doesn’t add a price: the milliseconds measure the checker’s work in that experiment. They are not a per-crossing toll theorem, whether or not an implementation saves the result.

Definition 17.1 (The bounds fragment)**.** Instructions are READ a, WRITE a v, and HALT. For a memory limit L, the policy allows READ a and WRITE a v when $a < L$, and always allows HALT. The *verification condition* of a program is that the policy allows every instruction in it. A *certificate* is a list of per-instruction policy facts, one for each instruction in order. The *checker* tests the policy on each instruction and returns the conjunction.

Theorem 17.1 (The checker is the verification condition)**.** For every limit L and program P: the checker accepts P exactly when the verification condition holds; any certificate for P gives the verification condition; and with $L = 2$ the checker rejects READ 2 followed by HALT. □

Proof. Lay the instructions in a row. For each READ or WRITE, compare its address with L; put yes beneath it exactly when the address is smaller. Put yes beneath HALT without asking about an address. The checker accepts exactly when every mark in that row is yes. The verification condition asks for exactly those same per-instruction facts. An empty row has no failing instruction, and extending the row preserves acceptance precisely when the new instruction also passes. This covers programs of every finite length.

A certificate supplies one policy fact at each place in the same row. Reading them in order gives every fact the verification condition asks for, so it implies that condition too.

With limit two, the first instruction READ 2 asks whether $2 < 2$. It does not. The later HALT is allowed, but cannot undo that failed comparison, so the checker rejects the two-instruction program. ■

17.1 One example, three levels

Before I lay all five side by side, let me walk one of them up and down the three levels from the start of the section, because every claim in here lives at exactly one level and it's easy to let a claim slide up a floor without noticing. I'll take gas, since it's the one place all three line up. Think of a restaurant's real recipe, my photocopy of one page of it, and a cookbook chapter on what any recipe has to contain. I can check my photocopy line by line. That tells you nothing about what the kitchen cooks tonight.

Start with the real specification. Ethereum's execution specification says, in a document that runs real money, what a storage write costs, what clearing it refunds, and how the refund is capped at the end of the transaction. None of what I proved is at this level. What the network actually charges is whatever the clients people run actually compute, and the evidence for that is the clients and the chain. This book has none of it.

Next comes the transcription. I copied one mechanism out of that document, one slot in one transaction, gas and refund arithmetic only, at a named commit (Definition 17.2, Theorem 17.3). What the result tells you: *if* the storage rule is what I copied, then leaving a fresh slot set costs at least twenty thousand, and

set-then-clear nets twenty-three hundred at the counter. What it can’t tell you: anything about the refund cap (not copied, which is why what a real set-then-clear pays came out so different from the counter earlier), the stack, calls, other slots, or whether any client implements the rule as written. The boundary of the claim is the edge of the copy.

Last comes the wrapper. The gas schedule of Definition 17.3 isn’t Ethereum. It’s any system with steps, a cost per step and a yes/no commitment flag, plus one thing *stipulated*: a charge predicate, with the premise that a charged step costs at least one and an uncharged one costs zero. That premise is the whole of what the wrapper adds, and it’s added by hand; no specification supplies it. Given it, the theorem says every run costs exactly its number of commits, never letting a commit go unbilled, never billing one twice and never billing a non-commit, exactly when the charge predicate is the commit predicate at unit price. What that tells you is what exact pricing would have to look like, in any system. It doesn’t say Ethereum meets it, and Ethereum doesn’t: real gas bills computation that commits nothing, which the transcription shows in numbers. So the wrapper prices a definition, the transcription prices a copied rule, and neither prices the chain.

The other four sit on the same three levels, and only one more wrapper is spelled out in this section. Casper’s transcription is the accountable-safety proof above; its wrapper is a small gadget with the stake at risk handed in. A finalize step that risks nothing turns the record on at no charge, which breaks Section 2’s toll, and one that risks at least one unit has a floor. It records exposure to loss (no collateral is destroyed in it), has no Casper checkpoints in it at all, and is priced by stipulation, the same way the gas wrapper is. Attestation, transparency and proof-carrying code have transcriptions that prove what their verifiers do and no price theorem, because no specification of theirs puts a price on the verdict. Their wrappers, which I don’t spell out here, are Section 8’s verifier argument told in each system’s vocabulary, with a colliding pair behind each one, and that argument is all they carry here.

The book’s rule is that every claim gets a way to break it, so I’ll say how to knock down the four claims with the most weight on them. For the gas transcription: a list of stores from a fresh slot that ends nonzero with net charge under 22100. For the gas wrapper: a schedule in which every run costs exactly its number of commits, whose charge predicate still differs from the commit predicate at some step. For the finality gadget: a finalizing trace from unfinalized to finalized whose charged stake is zero while the gadget claims a floor. For the attestation fragment: a verifier reading only the quote’s fields that decides a runtime flag the quote ignores.

CHAPTER 18

From “The pointer thesis”

18.1 What’s proved

This section has definitions, two toys, a theorem about independence, a survey, and two refutations of things that would have been stronger.

The definitions are the ones above, plus “unique pointer among rivals”: the event proliferates and none of a listed set of rival events does. Uniqueness is *relative to the list*, necessarily, since any event that holds on exactly the same states as a proliferating event proliferates whenever it does: the same fact under another name, so someone has to say which names count as rivals. And the first criterion is stated as a schema: for every ecosystem in some class, its metered event is the unique pointer among its rivals. The class and the rivals are inputs. Nothing about them is proved. A second set of definitions, after this one, tightens “proliferates” to “copied and relied on” and drops the rival list.

The formal object is small enough to hold in one hand.

Definition 18.1 (Ecosystem, records, proliferates, pointer)**.** An *ecosystem* is a type of states, a number N of observers, and for each observer i a Boolean fragment F_i on states. Observer i *records* an event E (a property of states) when, for every state s, $E(s)$ holds exactly when $F_i(s)$ is true. E *proliferates* when every observer $i < N$ records it. E is the *unique pointer among* a list of rival events when E proliferates and no rival in the list does. Given a class C of ecosystems, and for each member a designated metered event and a list of rivals, the *pointer criterion holds* for C when every member’s metered event is the unique pointer among its rivals.

Nothing in the definition makes two fragments read different parts of the state, and “metered” is only a label: the definition has no metering in it.

One bit per observer can’t even state the objection a replicated system raises. Every full node of a blockchain holds the whole shared state, so every fact about it, “gas used is at least one” as much as “final,” sits on every node. The tightened definitions give each observer whatever it holds, an action it takes on what it holds, and a flag for whether it produced the record.

Definition 18.2 (Copied and relied on, independent carriers, the pointer)**.** An *ecosystem with reliance* is a type of states, a number N of observers, and for each observer i a view V_i (a function from states to some type of things an observer can hold), an action a_i (a Boolean function of the view) and a producer flag. Fix a set R of reachable states. An event E is *copied* on R when for each $i < N$ some Boolean function d of the view has $E(s)$ exactly when $d(V_i(s))$ is true, for every s in R. Observer i *relies on* E when $a_i(V_i(s))$ true gives $E(s)$ for every s in R, and $a_i(V_i(s))$ is true for some s in R. E is *relied on by parties who didn't produce it* when some observer $i < N$ is not a producer and every such observer relies on E; it is *copied and relied on* when it is both. E is the *pointer* on R when it is copied and relied on, and every event that is copied and relied on holds at every state of R where E holds. The carriers are *independent* under a family of relations P_i (states that agree on observer i's own part of the world) when each view gives equal values on states related by its P_i, and for any two observers $i \neq j$ below N and any states s, t there is a state u with $P_i(u, s)$ and $P_j(u, t)$: the two parts can be set separately.

Two pointers on the same R agree on every state of R, so the pointer is unique with no list of rivals. And the definition of independence has a consequence I didn't put in on purpose.

Put one yes/no card in front of each of two readers. If the cards can be set separately, all four arrangements are allowed: no/no, no/yes, yes/no and yes/yes. Ask each reader to recover one shared event using only their own card. Suppose that event is false at no/no and true at yes/yes. At no/yes, the first reader sees exactly what they saw in the false case, while the second sees exactly what they saw in the true case. Their answers conflict. That mixed arrangement is exactly what the independence condition hands the proof.

Now impose a copying procedure that only leaves no/no or yes/yes. On those reachable arrangements each reader can report the shared bit correctly. Separate cards are things that *could* carry a copy; what makes them copies is that the arrangements you actually reach have the two agreeing. The theorem below is about recovering the event exactly on *all* arrangements, the mixed ones included, which the copying never leaves behind.

□ **Theorem 18.1** (Independent copies on every state: the event is equivalent at any two states)**.** In any ecosystem with reliance whose carriers are independent and which has at least two observers, an event copied on the set of all states holds at one state exactly when it holds at any other, so it holds at every state or at none.

Proof. Let observers 0 and 1 decide the event by d_0 and d_1. Given states s and t, take u with observer 0's part from s and observer 1's from t. Then the event at s is d_0 of observer 0's view, which is the same at u, so it equals the event at u; and the event at u equals the event at t the same way through observer 1. ■

So separate parts of the world can each carry an exact copy of a real record only on the states the world actually reaches, where the copying made them agree, and that's why the definition asks for copies on a set R of reachable states. Zurek's copies are the same: the light carries the dial's reading because the dial put it there.

Theorem 18.2 (A replicated header: copying doesn't separate, reliance does)**.** Let the states be a block header (a finalized bit, a gas-used count) and three nodes' copies of it, with R the states where every copy equals the header; node i's view is its own copy, its part of the world is that copy, node 0 is the producer, and every node acts exactly when its copy says finalized. Then the carriers are independent; "finalized" and "gas used is at least one" are both copied on R; "finalized" is the pointer on R; and "gas used is at least one" is not relied on by parties who didn't produce it. □

Proof. Independence: to give node i the copy from s and node j the copy from t, overwrite node j's copy in s. On R every copy is the header, so each node reads either fact off its copy. Node 1 acts only where its copy, and so the header, says finalized, and acts at the finalized header with no gas used; that state shows gas used fails node 1's reliance. Any event relied on by node 1 holds wherever node 1 acts, which on R is wherever the header is finalized, so "finalized" is the strongest. ■

It's a sanity check with maps I wrote, and it shows only that the tightened definition can tell the two apart where plain copying can't. Which of the two it picks, the actions pick: let every node act when its copy says gas was used, and gas used is the pointer. When what the other parties act on is copied, that is the pointer, so here the argument lives in the actions, and I wrote those too.

The toy has three observers on a replicated ledger, where certification proliferates and "work done" doesn't. It's an illustration, and that's all it is; it isn't evidence.

Theorem 18.3 (The toy)**.** Let the toy have states (a certified bit, a work count), three observers, and every fragment equal to the certified bit. Then "certified" is the unique pointer among the one rival "work at least 1". □

Proof. Every fragment is the certified bit, so every observer records “certified” by definition. For the rival, take the state with the bit false and work 1: the event holds and observer 0’s fragment is false, so observer 0 does not record it. ■

The survey covers twelve events in one theorem: for each of the five systems in Section 17, the designated event (finalized, committed, attested, logged, checked) proliferates in the observer map I wrote for that system, and a rival (proposer work, a scratch counter, measurement noise, submission latency, prover steps) doesn’t. The five together are one statement, an instance of the schema above. The survey adds a symmetric message-authentication code, which does *not* proliferate, and a digital signature, which does, for twelve rows in all. The five maps all use the same construction: a state has a designated Boolean event and a separate nonnegative rival count, three observers each return the event bit, and the rival asks whether its count is at least one. The bit and count can vary independently. The signature map likewise gives all three observers the authentic-signature bit. In the MAC map, observers 0 and 1 receive the authentic-tag bit and observer 2 always returns false. These maps, not a protocol running on a network, are the entire tests here. And there’s a thirteenth, which I think is the most important one: swap which bit every observer copies, and the criterion picks the *other* event. The criterion selects whatever the observer map copies. The map is where the argument lives, and I wrote the maps. So the five “confirmations” are five instances of one construction with the fields renamed. I’m saying that twice on purpose, so it can’t slip past once.

□ **Theorem 18.4** (The survey, and the swap)**.** (1) In each of the five observer models (proof-of-stake, gas, attestation, transparency, proof-carrying), the designated event proliferates and the designated rival does not; in the symmetric-MAC model the authentic-tag event does not proliferate; in the signature model the authentic-signature event does. (2) Let the states be pairs of bits (first, second), with three observers whose fragments all equal the second bit. Then “second is true” is the unique pointer among the one rival “first is true”.

Proof. Each confirmation holds by definition: each observer’s fragment is true exactly when the event holds. Each refutation exhibits one state where the event holds and some observer’s bit is down. In (2) that state is (true, false). ■

Two stronger claims are both refuted, and I’d ask you not to run them together, because they fail for different reasons.

The first stronger claim is that *every forgery-resistant commitment proliferates*. If that were true, certification would be the pointer because it’s the commitment. It’s refuted in three observer models (deniable authentication, where only the

designated verifier's bit tracks the event; a symmetric MAC, where the key holders share a secret and nobody else can check; an object capability, an unforgeable token that grants access to whoever holds it) and confirmed in two (a public log; a digital signature). Build the first with three observers: give one a copy of the event's bit and the other two a constant false. The one records it; it doesn't proliferate. The proved half is the map computation. Whether my maps are *fair* to the real designs is argued in prose, and you should argue back.

Theorem 18.5 (One blind observer is enough)**.** (1) In any ecosystem, if some observer $i < N$ has $F_i(s_0)$ false at a state s_0 where E holds, then E does not proliferate. (2) Let the deniable model have states (authentic, verifier-accepts), three observers, observer 0's fragment the authentic bit and observers 1 and 2 constantly false. Then observer 0 records "authentic", and "authentic" does not proliferate. The symmetric-MAC and object-capability models also fail to proliferate their events; the public-log and signature models proliferate theirs. □

Proof. (1) Recording would require $E(s_0) \iff F_i(s_0)$, and the left side is true while the right is false. (2) Observer 0's fragment is true exactly when the event holds. Apply (1) with observer 1 and the state (true, true). The other two refutations apply (1) to a blind observer of their own model (the symmetric-MAC map gives observer 2 a constant false; the object-capability map gives the token holder the bit and the other two constant false). The two confirmations hold by definition: the public-log map gives all three observers the logged bit, and the signature map gives all three the authentic-signature bit. ■

The theorem is about the observer maps, and only them. That a third party in deniable authentication really cannot be convinced is the modelling claim, and the model has no transcript, key or verifier in it to check that against.

The second stronger claim is that *agreement plus accuracy plus locality imply permanence*. That is, if all observers agree, each sees the event exactly, and each updates by the same local rule with no coordinator, then the recorded event can't be undone. The smallest thing imaginable refutes it: one bit, two observers who both see it, and the rule "flip." All three conditions hold and true becomes false in one step. Do it on paper; it takes a minute and it's humbling. What *does* give permanence is durability, "an observer's true view stays true," added as a premise, and then it follows in a few lines. So durability is the premise doing the work, and durability is Section 3's permanence wearing a different coat. Agreement doesn't buy it. Do the paper version again with one change, the rule "stay true once true" in place of "flip," and the bit can't fall, because the rule won't let it. The observers agreeing has nothing to do with it. (The small machine of Section 15 has a reading that's durable

in exactly this sense: its flag never comes down, by its step rule (Theorem 15.6).) None of this rules out a narrower theorem with real assumptions about the network, the adversary or durability; it rules out these two strong forms.

Definition 18.3 (Observation game)**.** A *game* has states, a number N of observers, a Boolean event e, a Boolean view V_i for each observer, and a step map. For $i, j < N$ it has *consensus* when $V_i = V_j$ on every state; *authenticity* when $V_i(s) = e(s)$ for every state and every $i < N$; *coordinator-free update* when one function ℓ on Booleans has $V_i(\text{step}(s)) = \ell(V_i(s))$ for every state and every $i < N$; *durable views* when $V_i(s)$ true gives $V_i(\text{step}(s))$ true for every $i < N$; and a *permanent event* when $e(s)$ true gives $e(\text{step}(s))$ true.

□ **Theorem 18.6** (What buys permanence)**.** (1) It is false that every game with $N > 0$, consensus, authenticity and coordinator-free update has a permanent event. (2) Every game with $N > 0$, authenticity and durable views has a permanent event.

Proof. (1) The toggle game: states are Booleans, two observers, event and both views the identity, step “flip”. Consensus and authenticity hold trivially, the local rule is “flip”, and the state true steps to false. (2) If $e(s)$ is true, observer 0’s view is true by authenticity, stays true by durability, and authenticity turns it back into ■ $e(\text{step}(s))$.

18.2 What isn’t proved, and must stay that way on the page

What isn’t proved is that certification *in deployed systems* is singled out by being copied and relied on across *independent* carriers. The criterion defines independence; it has no precise definition of “deployed” or “faithfully model,” and no cost law connecting being copied and relied on to price. Every correspondence between a mathematical observer map and a real system is a sentence I wrote. The thesis is a thesis, and anyone who says the formal object proves it is reading more than is there.

18.3 What the thesis asks of the examples

There are three concrete comparisons to make. First, look at the record a deployed discipline prices. Do parties who did not produce it copy and rely on it? A priced record that nobody else relies on would contradict the thesis’s second half.

Second, compare certification with rival events. If an event not implied by certification is copied and relied on to the same standard, certification is not the

unique pointer. Multiplication counts or head-turn counts would qualify if they met that same test in the system being described.

Third, apply the independence condition of Definition 18.2 to a faithful model of the ecosystem and ask whether certification itself meets the test. The symmetric-MAC example shows why the carriers and their access matter. The two stronger forms already refuted above help locate the distinction this thesis needs.

These comparisons supply the evidence the worldly claim asks for. The mathematical criterion supplies a definite question once the model and its observer maps are chosen.

CHAPTER 19

From “The counter against physics”

19.1 The calibration is a unit conversion

Multiply the counter by an energy per unit of cost that you choose, and you get a theorem: a state with one unit, at the scale kT ln 2 per unit, carries kT ln 2 of energy, for every k and T. Read that for exactly what it is: a unit conversion, true by definition, with the scale as an unforced input. Nothing in it picks the scale. The theorem would be just as true with the scale set to a dollar a unit. What *would* connect units of cost to joules is the premise in Section 3, Landauer’s principle, which is a physical premise. And even as a scale, $k_B T \ln 2$ a unit is the floor for one particular squeeze, a halving. Corollary 3.13’s floor for a flip that brings k states in beside m already reading yes, starting from the even spread on those $m+k$ states, is the larger of $k_B T \ln \frac{m+k}{m}$ and $\frac{2k}{m+k}\, k_B T \ln 2$, and for one newcomer among many yes-states that’s far below a halving, about $1.4\, k_B T / m$. Price every unit at $k_B T \ln 2$ and you can overstate the floor of a single narrow flip by a factor that grows with m.

Remember the constant from Boltzmann’s grave in Section 3, the exchange rate between a count and heat? It has a newer chapter, and it makes the point sharper than I can. The k_B in $k_B T \ln 2$ turns a temperature into an energy, and for most of its history it was a measured number, with error bars, pinned down by careful experiments. Since 20 May 2019 it isn’t. That day the international system of units was redefined so that Boltzmann’s constant is exactly 1.380649×10^{-23} joules per kelvin, by definition, and the kelvin is whatever makes that true. So the conversion from temperature to energy has been a convention since then, in the strictest sense: its numerical value in those units is fixed. Realising the kelvin in a laboratory still requires measurements, with uncertainty. What’s still physics is everything around it, whether wiping a bit at a temperature really sheds that much heat, which is what the Lyon bead was measuring. The calibration theorem is the same kind of move as that redefinition. It fixes a conversion, and the physics, if there is any, is somewhere else.

Definition 19.1 (Energy at a scale)**.** For a real number c (joules per unit) and a state s, the energy of s at scale c is $\mu(s) \cdot c$. The *Landauer calibration* at k_B and T is the statement that every state with $\mu(s) = 1$ has energy $k_B T \ln 2$ at scale $k_B T \ln 2$.

Theorem 19.1 (The calibration is a unit conversion)**.** (1) For all real k_B and T, the Landauer calibration at k_B and T holds. (2) For any counter, there are two real-valued functions of the state that give 1 and 2 on every state with one unit, so the counter alone fixes no energy value. □

Proof. (1) The scale says what one ledger unit is worth. At a ledger reading of one, multiplying by that scale returns the scale itself: $1 \cdot k_B T \ln 2 = k_B T \ln 2$. No condition on the signs of k_B or T is needed for that multiplication.

(2) Choose a scale of one and a scale of two. The same ledger reading of one is assigned energy one by the first choice and energy two by the second. The ledger has not changed; only the conversion I supplied has. Thus that conversion was not determined by the counter. ■

The physics is the premise of Section 3, restated here once in the form the next theorems test. In words first: a *dissipation* hands each instruction a whole number of heat units, each unit worth kT ln 2. The floor premise says an instruction that takes at least one state where some yes-or-no property is false to a state where it's true, never sends a true one back to false, and sends two different states to one, dissipates at least one unit. That's a different statement from Landauer's principle as Premise 3.5 has it. That one depends on the spread over states, $Q \geq k_B T \ln 2 \cdot \Delta H$, and gives no floor for a known state (there the distinction was absent from the chosen point distribution; that alone says nothing about a prior preparation bill). This one gives every such instruction a floor of one whole unit, whatever the spread. To get the second from the first you'd need more, at the least a spread with a full bit of uncertainty removed at every such step, and the known-state theorem and the narrow squeeze of Section 3 show that isn't automatic. So I give it its own name, *structural dissipation pricing*. The calibration says no instruction dissipates more units than the counter charges it. One word about that word: "dissipation" here means heat handed to the surroundings, the quantity Section 3 read off Sagawa as owed even when no entropy is produced, the heat that pays for where the forgotten bit goes. It is not what Section 1 calls dissipating, which is entropy produced, and it has nothing to do with waste, or with doing the job badly. There are two premises here, nothing is proved about either, and the only question is whether they can hold together:

Premise 19.2 (Structural dissipation pricing)**.** A *dissipation* is a function δ from instructions to naturals, counted in units of $k_B T \ln 2$. The *floor*, a hypothesis separate from Premise 3.5 of Section 3: every instruction that collapses a yes-or-no property has $\delta(i) \geq 1$, where *i collapses P* when some state with P false is sent to a state with P true, every state with P true is sent to one with P true, and some two different states are sent to the same state. The *calibration*: $\delta(i) \leq \text{cost}(i)$ for every instruction. Neither is proved anywhere; the theorems below ask whether they can hold together.

The last clause of the floor is there because the first two don't make a step forget anything on a machine with infinitely many states. INC A sends a state with $A = 0$ to one with $A = 1$ and keeps every positive A positive, so it raises "$A \geq 1$" one way, and it is one-to-one. Any one-to-one map that misses some state does the same to the property "is in my image". Its input can be read back off its output, so it loses no entropy, and Landauer's premise as this book takes it then puts no heat floor under it at all. It pays in fresh states instead: INC moves every value of A up one, Hilbert's front desk, and only a counter with no top can keep doing it. A thermal implementation can dissipate under the model-specific conditions above; the premise only sets no floor under it. A floor read without the last clause would charge a whole unit. On a finite machine the clause asks nothing extra. A step that sends one false state into the true ones and keeps every true state true puts one more state into the true set than the set holds, so two of them land on the same state. And on any machine the property comes along for the ride: an instruction that sends two different states a and b to one collapses the property "is reached from a in one or more steps", since one of a and b lies outside it and lands in it. So the floor is a price on merging, one unit for every instruction that merges, read as heat.

19.2 The calorimeter protocol, as a specification

What follows is how one *would* measure this, and it's here because it makes the boundary exact. Take a two-state register with energies zero and a gap, a population p in the excited state, and the simplest possible rate equation for how p changes in one time step (a rate for climbing into the excited state, a rate for falling out of it, and the length of the step). For the canonical reset (population one half to zero, with the rates chosen so the equation closes), the heat delivered to the bath is exactly half the gap. Choose the gap to be twice kT ln 2 and that heat is exactly kT ln 2.

Read that twice, because the temptation is to read it as a derivation of Landauer's number and it's the opposite. The gap was *chosen* to make the heat come out as Landauer's number. The same populations and rates with half that gap give half the Landauer value, and there's a theorem exhibiting identical population dynamics

with different heat. The rate equation fixes the *shape*; the energy scale is a selected Hamiltonian (the physicist's name for the rule that says what energy each state has), and selecting it is not deriving it.

The canonical reset is a bookkeeping toy, and its rates show it. A register that trades heat with a bath at temperature T has rates in *detailed balance*: the rate up over the rate down is $e^{-\Delta/k_BT}$, which at the gap $2k_BT\ln 2$ is $1/4$. The canonical rates are 0 and 1, and the factor $e^{-\Delta/k_BT}$ is never 0, so no gap and no temperature put them in that ratio. With rates that are in it, a relaxation at a fixed gap moves toward the equilibrium population $1/(1+e^{\Delta/k_BT})$ and, with steps short enough not to overshoot, never goes below it. At the gap $2k_BT\ln 2$ that population is $1/5$. The toy has no work term either; nothing in it pays for the squeeze. Part (4) of the theorem below says what follows from that: at the gap $k_BT\ln 2$ the toy books less heat than Landauer's bound asks for the same squeeze from one half to zero, so the toy has no second law.

Definition 19.3 (The two-state protocol)**.** For a gap Δ, the register's mean energy at excited population p is $\Delta\cdot p$. One step of the rate equation is

$$p' = p + \delta t\,\big(k_{01}(1-p) - k_{10}\,p\big),$$

and the heat to the bath over a step from p to p' is defined as $\Delta p - \Delta p'$. The canonical reset takes $p=\frac{1}{2}$, $\delta t = 1$, $k_{01}=0$, $k_{10}=1$, ending at $p'=0$, and counts one logical event.

Theorem 19.2 (The heat is fixed by the chosen gap)**.** (1) The canonical numbers satisfy the rate equation. (2) For every Δ, the canonical reset's heat is $\Delta/2$. (3) For all k_B, T, the gap $\Delta = 2k_BT\ln 2$ gives heat $k_BT\ln 2$. (4) If $k_B>0$ and $T>0$, the heat is below $k_BT\ln 2$ exactly when the gap is below $2k_BT\ln 2$; at the gap $k_BT\ln 2$ it is half of $k_BT\ln 2$. (5) There are two different gaps for which the same canonical populations satisfy the rate equation and the heats differ.

Proof. (1) Half the population starts excited. The upward rate is zero, so none enters the excited state; the downward rate is one for a step of length one, so the entire excited half leaves. Substitution gives $\frac{1}{2}+1\cdot(0-\frac{1}{2})=0$.

(2) The register began with mean energy $\Delta/2$ and ended with zero. The defined heat is their difference, $\Delta/2$.

(3) Choose $\Delta = 2k_BT\ln 2$. Half of that gap is $k_BT\ln 2$.

(4) Asking whether $\Delta/2 < k_BT\ln 2$ is the same as multiplying both sides by two and asking whether $\Delta < 2k_BT\ln 2$. At the particular gap $k_BT\ln 2$, part (2) gives half that value. With positive k_B, T it is strictly smaller because $\ln 2$ is positive.

□

(5) Take gaps one and two. The population equation in part (1) contains neither gap, so the same populations solve it in both cases. Their heats are one half and one respectively. The rate equation alone did not select between them. ■

A reset that pays its way is driven. Raise the gap a little, let the register settle with rates in detailed balance, raise it again, and so on, starting from a gap of zero, where the two states are alike and the population is one half. Raising the gap while the population holds is work done on the register; settling at the new gap is heat to the bath. That's the textbook way to wipe a bit slowly, and here it is with its numbers worked out.

Definition 19.4 (The driven reset)**.** Write kT for k_BT. At gap Δ the equilibrium excited population is $\pi(\Delta) = 1/(1+e^{\Delta/kT})$ and the register's free energy is $F(\Delta) = -kT\ln(1+e^{-\Delta/kT})$. Rates satisfy *detailed balance* at gap Δ when $k_{01} = k_{10}\,e^{-\Delta/kT}$. A *driven reset* along a schedule of gaps $g_0, g_1, \ldots, g_n$ takes, at step m, the gap from g_m to g_{m+1} with the population held at $\pi(g_m)$, which is work $\pi(g_m)(g_{m+1}-g_m)$, and then lets the register settle to $\pi(g_{m+1})$, which is heat $g_{m+1}\big(\pi(g_m)-\pi(g_{m+1})\big)$ to the bath. The reset's work and heat are the sums over its steps. The uniform schedule from 0 to D in N steps has $g_m = Dm/N$.

□ **Theorem 19.3** (A driven reset pays $kT\ln 2$ in work)**.** (1) With rates in detailed balance at gap Δ, one step of the rate equation takes p to $\pi(\Delta) + \big(1-\delta t\,(k_{01}+k_{10})\big)\big(p-\pi(\Delta)\big)$; when $\delta t\,(k_{01}+k_{10}) = 1$ this discrete update lands on $\pi(\Delta)$ from any p. (2) For any schedule, work minus heat is $g_n\pi(g_n) - g_0\pi(g_0)$, the change in the register's mean energy. (3) If $kT > 0$ and the schedule never falls, the work is at least $F(g_n) - F(g_0)$; if also every step raises the gap by at most h, the work is at most $F(g_n) - F(g_0) + h\big(\pi(g_0)-\pi(g_n)\big)$. (4) If $kT > 0$, $D \geq 0$ and $N \geq 1$, the uniform reset from 0 to D in N steps does work between $kT\ln 2 - kTe^{-D/kT}$ and $kT\ln 2 + D/(2N)$, and leaves an excited population of at most $e^{-D/kT}$.

Proof. (1) Detailed balance gives $k_{01} = (k_{01}+k_{10})\pi(\Delta)$. In the rate equation, the upward contribution minus the downward one is $k_{01} - (k_{01}+k_{10})p$. Substitute the balance equation and this becomes $-(k_{01}+k_{10})(p-\pi(\Delta))$. The update therefore subtracts the fraction $\delta t(k_{01}+k_{10})$ of the distance from equilibrium. What remains is exactly

$$p' = \pi(\Delta) + \big(1-\delta t(k_{01}+k_{10})\big)\big(p-\pi(\Delta)\big).$$

When that fraction is one, none of the distance remains: $p' = \pi(\Delta)$.

(2) Follow one raise from gap a to gap b. Its work is $\pi(a)(b-a)$ and its heat is $b(\pi(a)-\pi(b))$. Subtract the heat from the work. The two occurrences of $b\pi(a)$ cancel, leaving $b\pi(b)-a\pi(a)$, the final mean energy minus the initial mean energy. At the next raise, that final energy becomes the next initial energy. Adding over the schedule cancels every intermediate energy and leaves only $g_n\pi(g_n)-g_0\pi(g_0)$.

(3) Differentiating the displayed free energy gives slope $F'(\Delta)=\pi(\Delta)$. With $kT>0$, this slope decreases as the gap rises. Over $a\le b$, the increase of F therefore lies between the widths times the two endpoint slopes:

$$\pi(b)(b-a)\le F(b)-F(a)\le \pi(a)(b-a).$$

The work uses the right-hand quantity, the slope before the raise. It is at least the free-energy increase. Its excess is at most $(b-a)(\pi(a)-\pi(b))$. If the raise is at most h, that excess is at most $h(\pi(a)-\pi(b))$.

Add those bounds over the schedule. Intermediate free energies cancel, and intermediate populations cancel in the excess bound. I obtain work between $F(g_n)-F(g_0)$ and that amount plus $h(\pi(g_0)-\pi(g_n))$.

(4) For the uniform schedule, $h=D/N$, $\pi(0)=1/2$, and

$$F(D)-F(0)=kT\ln 2-kT\ln(1+e^{-D/kT}).$$

For $x\ge 0$, $0\le \ln(1+x)\le x$: it starts at zero and its slope $1/(1+x)$ is between zero and one. The free-energy difference is thus at least $kT\ln 2-kTe^{-D/kT}$ and at most $kT\ln 2$. The excess from (3) is at most $(D/N)(1/2)$, giving the work bounds. Finally the denominator of $\pi(D)=1/(1+e^{D/kT})$ is at least $e^{D/kT}$, so the remaining excited share is at most $e^{-D/kT}$. ∎

Try one finite step in units where $kT=1$: start with gap zero and half the registers excited, then raise the gap to $\ln 2$. The work is $\frac{1}{2}\ln 2$. After settling, the excited share is $1/3$, so the bath receives $(\frac{1}{2}-\frac{1}{3})\ln 2=\frac{1}{6}\ln 2$ and the registers retain $\frac{1}{3}\ln 2$ more energy. Work equals those two amounts added together. A third are still excited: one such step has not wiped the bit.

The word “step” matters here. Part (1) is a discrete update of the population. Its multiplier can be zero. Continuous relaxation at fixed positive rates instead multiplies the distance from equilibrium by $e^{-(k_{01}+k_{10})t}$, which is positive at every finite time. Exact settling in the schedule is an idealisation; the algebra does not prove that a continuous bath completes it in one finite waiting period.

Here nobody picked the gap to make the number come out. Take D large and N much larger than D: the work closes in on $kT\ln 2$ from both sides, and the population left in the excited state goes to zero. Perfect erasure is the limit, not the result at any finite gap. To reuse the original symmetric memory, the gap must also return to zero while transitions are suppressed. Lowering it at the final fixed

population recovers $D\pi(D)$ of work and leaves that population unchanged. This term tends to zero in the same limit. The reset of the population, restoration of the trap and time needed for settling are separate parts of the physical protocol. Detailed balance supplies the thermal premise; the displayed sums supply the work and heat accounting for the idealised schedule.

What ties it to a machine is that the protocol counts one logical event, by definition, and one is what the small machine’s CERTIFY charges, the last of the three paid acts behind a raised flag (Section 15). The tie is two numbers that happen to match, the protocol’s one by definition and the machine’s the least the record-layer premise lets CERTIFY cost, and nothing more. No theorem maps the protocol’s two register states onto any state of the machine. And “heat to the bath” here means the register’s mean energy before minus after, which quietly assumes the bath gets exactly what the register loses; nothing here establishes that. So a device test would have to measure the gap, the populations, the rates, and the bath heat independently; separately establish that the device’s rates are in detailed balance (that the up and down rates stand in the ratio the temperature dictates), which the driven reset assumes and the canonical one breaks; and separately establish that the device’s physical reset *is* the machine’s certify event. The driven reset is an executable experimental specification. The canonical reset is a bookkeeping toy. Neither is a measured device, and neither is an unconditional conversion of units of cost to joules, and anyone who tells you otherwise has skipped the conditions.

19.3 What this section is

This section holds a calibration that’s a unit conversion; a pair of premises, consistent on the steps the toll is about and inconsistent on the whole instruction set; a history that makes any step injective by paying with a fresh state at every step; and a measurement protocol that’s a specification with a selected scale. The physics in this book is in Part I: Landauer’s principle as a named premise with its critics named (Section 3), and conditional finite-time dissipation bounds, cited, and proved for one model (Section 1). Here there’s accounting, and I’ve tried to call every line by its right name. That’s the end of the example machine. What’s left, after one suspicion I can’t prove, is how you’d break any of this, and where it stops.

CHAPTER 20

From “The claims and their conditions”

20.1 The diagonal and the worked models

The diagonal argument supplies the contradiction inside a substrate that can represent the candidate’s flip and has the stated recursion property. Its fixed point makes the proposed decision disagree with itself. An outside calculation of the natural-number example does not supply an inside program for that substrate. Section 13 and the appendices state the input conventions separately for the natural-number family, L, the counter programs and the host.

The protocol examples in Section 17 isolate particular mechanisms: finality, a charge on commitment, an attestation window, an inclusion proof and a bounds check. Their statements follow the operations in those fragments. The pointer thesis then asks why certification is the event to choose. It proposes a connection to records copied and relied on by independent parties. The counting and window arguments apply to any record meeting their premises; the thesis supplies the proposed reason for this choice.

The game checks in Appendix 24 give another worked construction. Four fixed bits yield the classical bound by the displayed sign calculation. The matrix arguments relate contraction, positive semidefiniteness and the score bound. The integer checker implements its specified test on tallies with every pair sampled. Its earned chain connects a raised flag to that test. The pinned-matrix test and the completion test ask different questions: the appendix’s all-ones and PR-box examples make the difference visible.

Those connections are what the book has built: a count, a schedule, an observation and an earned record, with instructions carrying them through actual runs.

20.2 Bell-test vocabulary

A correlation between two outcomes is the average of their product; with outcomes plus or minus one it lies between minus one and one, plus one meaning always match, minus one always differ. A correlator is one such average in a labelled setting, E00 for Alice's setting 0 and Bob's 0. The marginal of a joint distribution is the distribution of one side alone with the other averaged out; zero-marginal means each side's outcome is plus or minus one equally often on average. A local hidden-variable model is one where both outcomes are fixed by a shared random variable agreed before the game plus each side's own setting; Bell's theorem says no such model produces certain correlations quantum mechanics does. No-signalling means neither side's marginal depends on the other's setting. Quantum-realisable means compatible with some quantum state and measurements; the NPA hierarchy gives a ladder of necessary tests, each a positive-semidefiniteness condition on a moment matrix. For the four correlators of this game alone, the first rung is already exact; for the full table, each side's own odds included, Chaturvedi's 2026 preprint argues that no finite rung is, even in this simplest case. The PR box is the no-signalling correlation that wins the game every time, stronger than quantum mechanics allows. Cauchy-Schwarz is the inequality that a dot product is bounded by the product of the lengths. The Tsirelson bound is two root two, the largest game score quantum mechanics allows; the classical bound is two and the PR bound is four. (If you go looking for Boris Tsirelson's 1980 paper, look under Cirel'son, the spelling the journal printed, or you may not find it.) The elliptope is the set of correlators that can be filled out into a positive-semidefinite matrix with ones down the diagonal.

20.3 Matrices

A matrix is positive semidefinite (PSD) when $v^{\mathrm{T}}Mv \geq 0$ for every vector v; the game check is all about one matrix being PSD. The determinant of a square matrix is a single number that is zero exactly when the matrix squashes some direction flat. For a two-by-two matrix with rows (a, b) and (c, d) it's $ad - bc$. For a bigger one, expand along a row: for each entry, delete its row and column, take the determinant of what's left, multiply by the entry, alternate the signs, and add up; the recursion bottoms out at two by two. A principal submatrix is what you get by choosing some of the indices and keeping only those rows and the same columns; its determinant is a principal minor. A symmetric matrix is PSD exactly when every principal minor is at least zero, every one of them, and the step from the game check to Tsirelson's bound uses two of the three-by-three ones (Appendix 25.4). The Schur complement turns a question about a block matrix into a question about a smaller one. For a symmetric block matrix with corner block P, off-diagonal block Q and other corner

R, it's $R - Q^{\mathrm{T}}P^{-1}Q$, and when P is positive definite the whole matrix is PSD exactly when that complement is (Lemma 25.6). Positive definite matters; invertible isn't enough. Take $P = -1$ and every other entry zero: the complement is zero, which is PSD, and the whole matrix isn't.

20.4 Thiele-complete and certified Thiele-complete, defined so a clock fails

Known: Turing completeness, and the observation every theorist makes in four seconds, that a toll alone is cheap: a machine that charges for every step pays any toll. Enforcement mechanisms have been compared and sorted before, and they aren't one notion seen several ways: James Morris (1973) separated access limitation from authentication in programming languages, and Kevin Hamlen, Greg Morrisett and Fred Schneider (2006) classified which policies execution monitors, program rewriters and static analysis can each enforce, and they differ; Wai Wong (1995), on a neighbouring track, recorded and checked HOL proofs so a theorem could carry its derivation. Mine: a definition of Thiele-complete with four clauses that say nothing about what the record is about (computing that runs every program, costs nothing and can't touch the record, while every other move costs one; a record that stays; blank starts; and some run whose record tells two starts apart), with the proofs that a clock and a silent machine fail it, that each clause is needed, that every universal base becomes one by adding marks, and that every machine meeting it hides its record and ledger from its own window (Section 12.2); and a definition of certified Thiele-complete with four clauses (a universal base that can't touch the record and a record that stays up once it's up, an earned record, exact prices, nothing for the base and one for each of the three record acts, and some claim whose check can fail), the proofs that every clock-style machine and a paid latch and a silent machine fail it while the small machine, the same machine over any property language with an exact equality test, a checker proved to say yes exactly when the property holds and a property that holds of one number and fails of another, and the universal program's host meet it, that the four clauses are independent, each failed by some machine with a two-counter machine inside that meets the other three (Theorem 16.9), and the theorem that every machine meeting it hides its record and ledger from its own window (Section 15), with the theorem that it is exactly Thiele-complete with clean starts, the earned chain and a claim that could have failed added (Theorem 16.3). Kind: definitions with short consequences. The window theorem is the fibre fact of item 6 plus one observation, that a universal base prints any window at no cost on the ledger. I'm not claiming the definition is the only reasonable one, or that LCF kernels, reference monitors and proof-carrying code are all certified Thiele-complete machines in disguise; Hamlen, Morrisett and

Schneider's sorting says such mechanisms genuinely differ, and the book doesn't have a theorem showing the definition holds up across such differences.

20.5 The game check

Known: that positive semidefiniteness of a correlation matrix gives the Tsirelson bound is Tsirelson's (1980) and Stephanie Wehner's semidefinite proof (2006), from a paper on Tsirelson bounds for generalized CHSH inequalities; the moment-matrix hierarchy, the Schur-complement analysis of the level-one matrix of this scenario and the level $1 + AB$ are Navascués, Pironio and Acín's (2007, 2008), with both four-body cells unconstrained where my matrix ties them; that every point with a PSD completion is the correlator table of a quantum strategy, built from anticommuting observables in a maximally entangled state, is Tsirelson's too (1980); the arcsine characterization is Landau's (1988) and Masanes's (2003); Echenim and Mhalla (2023) give a further detailed derivation of the CHSH and Tsirelson bounds. Rational sum-of-squares certificates are Helfried Peyrl and Pablo A. Parrilo's (2008), from a paper on computing sum-of-squares decompositions with rational coefficients. Mine: a decision procedure in integer arithmetic on outcome counts; a proved chain from "the counts pass" to "the pinned level-one matrix is positive semidefinite" to "$S^2 \leq 8$"; explicit constructions of the representation in finite dimension, on two four-level systems with real amplitudes and two two-level ones with complex, and a proof of the arcsine characterization; proofs of the contraction equivalence by explicit test vectors; and the level-$1 + AB$ cascade (on the slice $\langle \{A_0, A_1\} \otimes \{B_0, B_1\} \rangle = 0$) and an elliptope gate in whole numbers, with a soundness proof (Appendix 24 and its derivation, Appendix 25). Kind: an integer procedure connecting counts to these known bounds, with known theorems (Tsirelson's representation, the arcsine characterization, the equivalence of contraction and PSD) proved again. I found no earlier chain that takes counts as input where I looked. The same check earned on the small machine's host, a raised flag standing on it, is built and proved (Theorems 24.4 and 24.5); the idea isn't new.

CHAPTER 21

From “The universal machines”

21.1 A simpler host, with the guest step built in

There’s a much simpler host than U, and I keep it in the book because it’s easy to read and it shows the copy rule in one line. Its universality is by construction: it has a move that runs one guest step by calling the small machine’s own step. It isn’t a universal Thiele machine in the sense of Definition D.1, and the reasons are exact. Its interpreter loop U_0 never halts: it keeps returning to its first line even after the guest stops. The host itself does have an OWN HALT instruction, which another program can use. The loop also keeps charging one per pass, even when the guest has stopped or its next step costs nothing, so no fixed surcharge bounds it. And it isn’t a register host, since its guest step does the computing. A host that fixed the first two and kept the built-in step would still fail the last (the argument after the definition). U is the universal Thiele machine of this book. Read it as a picture of what U does the long way, and as a host that starts from any guest state at all, clean or not.

It carries a guest inside its own state: the guest’s program as data, the guest’s counters, its table, its ledger, its flag. The host has one extra move, run one guest step. In that one move it looks at the guest’s flag, takes the step, looks again, and if the flag went from down to up, the host’s own copy of that flag goes up in the same move, and the host pays for it.

Definition 21.1 (The host)**.** A *guest* is a program P of the small machine together with a state g of it (Definition 15.2). A *host state* h has six parts: its own core $\mathrm{core}(h)$, its ledger $\mu(h)$, its own flag $\mathrm{own}(h)$, the guest program $P(h)$ held as a list of instructions, the guest state $g(h)$, and the *mirror* $m(h)$, a yes/no copy of the guest’s flag. The host has two kinds of move. OWN i runs the instruction i on the host’s own core, costs what the small machine charges for i, and raises own exactly when that machine’s flag would rise. GSTEP b costs b. If the host’s core is trapped it does nothing else. Otherwise it moves the host to its next line and, if the guest’s next instruction costs at most b, takes one program step of the guest, which leaves a stopped guest where it is; in the same move the mirror becomes $m(h) \vee (\neg\mathrm{cert}(g(h)) \wedge \mathrm{cert}(g'))$,

where g' is the new guest state. The host's reading is $\mathrm{own}(h) \vee m(h)$. Its stored program is $U_0 = [\text{GSTEP 1; OWN (INC } A\text{); OWN (DEC } A\ 1)]$: one guest step, then two moves that bring the host back to its first line. It *loads* a program P from (x, y) with $g(h) = \mathrm{start}(x, y)$ and the mirror and its own flag down.

Every instruction of the small machine costs at most one, so GSTEP 1 runs every guest instruction there is. Why charge the budget b? Why not the cost of whatever instruction got fetched? Because charging the budget keeps the price a function of the move alone, which is what the definition of Section 2 asks for: you read the price of a host move off the move, the way you read the small machine's prices off its table, without peeking inside the guest. What that costs the host is easy to see. A host that steps a guest instruction priced 0, an INC say, still pays one for the move.

So the host's ledger is closer to a clock than to a receipt for its guests: one unit per guest step, whatever the step was. It never falls behind the guest's own ledger (part (3) of Theorem 21.3 below), and it matches that ledger only when every guest step happens to cost one. Could a cleverer host have simulation-exact accounting, a ledger equal to its guest's? Not with this one fixed-price move: GSTEP 1 is the same move whether the guest's next instruction is an INC priced 0 or a paid CHECK, so one price can't match both. Two moves can: one that costs nothing, takes the guest's step when that step costs nothing and otherwise jumps, and one that costs one and takes it. A loop that tries the first before the second keeps the host's ledger equal to the guest's at every pass. That host still has its guest step built in. U gets the match with ordinary instructions: it pays only on the record moves it runs itself, so its ledger equals its guest's (Theorem D.4).

And there's one thing to say about the definition before the theorems, so you know which of them carry weight. GSTEP runs the guest's instruction by calling the small machine's own step on the guest's state: the guest's step is a primitive of the host. So "the host runs any guest" (Theorem 21.1) is true by construction. The host was built to do exactly that, and the proof follows the three moves around the loop. The theorems with something in them come after it. From a loaded host, or any state where the two agree, the mirror always equals the guest's flag (Observation 21.2). Nothing raises the mirror unless the guest itself passes a CERTIFY inside a guest step the host paid for (Theorem 21.4). No OWN move, in any host program at all, touches the guest's state or the mirror (the second sentence of that same theorem), and no host move of either kind rewrites the guest's program. And the mirror is tied to the guest's earned chain: after any host run from a loaded host, if the mirror is up, the guest's own run from its start contains a passing CHECK, then a COMMIT of the same claim, then a CERTIFY, since the mirror is the guest's flag and the guest started clean (Theorem 15.11). So this host runs any small-machine program by construction and keeps the guest's record by theorem, as a copy of the guest's flag.

The mirroring program U_0 does not earn the host's own flag; a different program could do so with the host's OWN CHECK, COMMIT and CERTIFY. U does both the long way: it runs the guest with its own instructions, and it earns its own flag.

Theorem 21.1 (The host runs any guest)**.** (1) If the host's core is not trapped, then after n moves GSTEP 1 the guest state is the guest program's own run of n steps from $g(h)$, the guest program is unchanged, and the host is still not trapped. (2) If the host's core is at line 1 and not trapped, then $3n$ steps of the program U_0 leave the guest exactly where n steps of its own program put it. (3) For every two-counter program M and inputs a, b: M halts from $(1, a, b)$ exactly when the host program made of OWN moves of the compiled M halts, from a host whose core starts at (a, b), whatever guest that host carries. □

Observation 21.2 (The copy is the record)**.** *Say a host state h* agrees *when $m(h) =$ cert$(g(h))$. Every host move takes an agreeing state to an agreeing state, so every host trace and every stored host program does too, and a freshly loaded host agrees. In particular, if h agrees and is not trapped, the mirror after n moves* GSTEP 1 *is the guest's flag after n steps of its own program.*

Theorem 21.3 (The host's step collects the guest's toll)**.** (1) If h agrees, is not trapped, and the guest step that budget b allows takes the guest's flag from down to up, then the mirror is down before GSTEP b and up after it, the host's ledger rose by exactly b, and $b \geq 1$. (2) In the run of (1) of Theorem 21.1, from a host that agrees, if the guest's flag goes up between its steps n and $n+1$, the mirror goes up between the host's moves n and $n+1$, and that move costs 1. (3) For every host trace t, $\mu(g(\text{run}(t,h))) + \mu(h) \leq \mu(\text{run}(t,h)) + \mu(g(h))$: the host's ledger grows at least as fast as the guest's. □

Theorem 21.4 (No unpaid host certification)**.** If the mirror is down at h and up after a host move i, then i is GSTEP b with $b \geq 1$, the host was not trapped, the guest's next instruction was CERTIFY, and the guest's flag went from down to up in that move. A trace made only of OWN moves leaves the mirror and the guest state exactly where they were. And the host's own flag rises only by an OWN CERTIFY that passes on the host's core. □

□ **Theorem 21.5** (The host is a Thiele machine)**.** For every host state and move, if the host's reading goes from no to yes, the move costs at least 1; the same holds for the mirror alone and for the own flag alone. So the host read through own $\vee$ m, the host read through m alone, and the guest class are Thiele machines (Definition 2.2), and any host trace that raises the host's reading costs at least 1.

Two statements have been proved here. The designated guest-step runs follow the guest computation. Separately, every permitted host instruction preserves agreement with a genuine guest run, so arbitrary host instruction sequences from a loaded start cannot fabricate the guest's mirror. The second statement uses the full host instruction set. It does not follow merely from the first, or from charging for a raise. The combined reading own-or-mirror remains a different observation: the host may establish its own record while the guest's record stays down.

Remark 21.1 (Two guests, run)**.** Load the guest CHECK *is zero* A; COMMIT *is zero* A; CERTIFY; HALT with both counters at 0. After two moves GSTEP 1 the mirror is down; after three it is up, the guest's flag is up, the host's own flag is still down, and the host has paid 3. Under the program U_0 the mirror is down at host step 6 and up at step 7, with 3 paid. The guest CERTIFY; CERTIFY traps on its first step: after five moves GSTEP 1 the mirror is down, the guest is trapped, and the host has paid 5 for nothing.

That copy can't be faked. The proof walks every host move there is, and the copy only rises in a guest step where the guest itself ran CERTIFY and passed. A host program that never touches the guest leaves the copy exactly where it was. From a loaded host the copy always equals the guest's flag, and the host's ledger always grows at least as fast as the guest's, and a three-instruction loop runs any small-machine program one guest step per pass. Here the mirror, the crossing test and the charge sit in the host's step rule, so Theorems 21.3 and 21.4 hold for every sequence of host moves, including ones that never run a guest. The fixed program U supplies simulation, an exact ledger at matching points and an earned flag using ordinary instructions. Its guest simulation theorem concerns that program from its loaded starts. The host's native earned-chain rule covers other programs too, but does not make every such program simulate the same guest.

21.2 The diagonal, on this machine's programs

Section 13 left the diagonal standing on any substrate, on a little family built to fit it, and on L, handled on its own terms rather than as a substrate. It also said what a

machine has to supply to fill in the same shape, two things: programs that can be handed to programs as numbers, and a recursion theorem for the transformations it can carry out. This machine's universal program reads a whole guest program out of one register, with a numbering of programs that goes there and back (Definition D.3), and halting is undecidable here in the reduction sense of Theorem 15.5. What's left are the two classical results that turn a universal machine into a place where the diagonal bites: Rice's theorem and Kleene's recursion theorem. They come with a wrinkle I should put first, because it decides which machine they're about.

The wrinkle is the one from Section 15, at the end of the subsection on what two counters compute. The small machine takes its inputs straight into two counters, and Schroeppel's memo gives a proof that two plain counters can't even compute 2^n from n. Rice's reduction has to keep a program's input safe while a copy of a halting instance runs. Kleene's construction has to run an evaluator while keeping its input. Two counters have nowhere to keep it. So the two theorems land where they're true and where I could prove them. The host machine of Definition D.2, the one with a counter for every number that U runs on, is a fine programming system: its programs are numbers, an input sits in register 1, and there's room to spare. On it both theorems hold in full. On the two-counter machine itself the story splits, and I'll tell it at the end of the subsection: Rice's theorem holds on every input, by a trick for keeping the input that I'll show you; Kleene's holds when inputs are written as powers of two, the way Minsky packed them; and on plain inputs Kleene's is false, and that's a theorem too.

The host machine's programs are numbered in the same style as U numbers a guest's. Write $\langle m, n\rangle$ for $(2n+1)\cdot 2^m$. INC r is $\langle 0, r\rangle$, DEC r j is $\langle 1, \langle r, j\rangle\rangle$, HALT is $\langle 2, 0\rangle$, CHECK r is $\langle 3, r\rangle$, COMMIT r is $\langle 4, r\rangle$, CERTIFY is $\langle 5, 0\rangle$, and a program is the list code: the empty program is 0, and an instruction x followed by a program t is $\langle x, \mathrm{code}(t)\rangle$. An instruction number that is none of these codes reads as HALT. Then every number decodes to some program, since a positive number is $\langle x, n\rangle$ with n smaller than it, and decoding a program's code gives the program back. When I put one program after another, the later one's jumps move down by the length of what comes before it, and a jump to a line where it has no instruction, line 0 included, goes to the line just after it, so a stop inside it hands on to whatever follows. Say a host program *outputs* y on input x when, started with x in register 1 and every other register 0, with a clean table, it halts with y in register 0.

Rice comes first, because it's the shorter road. Henry Rice, whom you met in Section 8, proved in his 1951 doctoral thesis, and published in 1953, that no program decides any property of what programs *do* unless every program has it or none does. Properties of how a program is written aren't covered. The road from halting is a single detour. Take a property of what programs do, a program y that has it, and a program n that doesn't. Given an instance of two-counter halting, a

Minsky program H with starting counters a and b, build a host program that first loads a and b into two fresh registers (not register 1, and named by neither y nor n), then runs a copy of H moved onto those registers, then empties both, then runs y. If H halts on (a, b) the built program behaves like y and has the property. If H runs forever the built program never stops on any input, and it behaves like the program that loops, which lacks the property (if the looping program has it instead, swap the roles of y and n). Everything the built program does before y is INC and DEC, which add nothing to the ledger and touch no fact, channel, trap latch or flag. So a decider for the property would be a decider for halting, and by Theorem 15.5 that would let a program list every single-tape Turing machine that doesn't halt. Two words carry the weight, and they're the ones to check in the statement: what “behaves the same” sees, and where the program starts.

Definition 21.2 (Behaving the same, on the host)**.** A host program *ends* in a state when it halts there, from the start with x in register 1 and the rest clean; a trap counts as stopping. Two host programs *behave the same* when for every input x, any state one ends in has a state the other ends in that agrees with it on every register's value, the fact table, the channel, the trap latch, the ledger and the flag, and the same the other way round. The line number and the versions aren't compared. A property of host programs is *extensional* when behaving the same carries it from one program to the other.

□ **Theorem 21.6** (Rice's theorem on the host machine)**.** Let Π be an extensional property of host programs that some program has and some program lacks. Then Π is undecidable in the reduction sense of Theorem 15.5.

A property that depends only on the partial function a program computes, whether it stops and what it leaves in register 0, is extensional, so it is covered. That case also follows from the recursion theorem below.

On the small machine itself the same road works from the clean start, and only from there. Say two two-counter programs *behave the same from the clean start* when, whatever state one halts in from counters $(0, 0)$, the other halts in a state with the same counters, trap latch, ledger and flag, and the same the other way round. The fact table and the channel aren't compared, because the prefix moves the versions of A and B by fixed amounts and the facts, the channel's included, carry versions. There is no input to keep, since the program starts at $(0, 0)$, and H's instance is loaded straight into A and B.

Theorem 21.7 (Rice's theorem for the small machine, from the clean start)**.** Let Π be a property of two-counter programs that's carried from a program to any program that behaves the same from the clean start, held by some program and lacked by some other. Then Π is undecidable in the reduction sense of Theorem 15.5. In particular "halts from the clean start" is undecidable. □

That theorem covers the clean start only. Every input is harder, because the standard reduction has to keep the input safe while the halting instance runs, and two counters have nowhere obvious to keep it. The way through is at the end of the subsection.

Kleene's recursion theorem is the deeper one, and it's a construction, which is Section 13's point about Kleene's little gem (Moschovakis's phrase): the content is the building, and the contradiction in Section 13 was just the last line. It says: for every transformation F of programs that some program can carry out on numbers, there's a program p that computes the same thing as $F(p)$. The shape of the building is the old one. From the program that carries out F, build an *evaluator* that reads a number, treats it as a program's code, specialises that program to its own code, runs F's program on the result, and runs whatever program comes back. The specialising step is what logicians call s-m-n: fix one input of a program and get a new program that needs one input fewer. Then the witness is the evaluator specialised to its own code. Run on any input, it builds its own code, hands it to F's program, gets the code of $F(p)$, and runs that.

Each piece on the host machine comes next, with its exact premise.

Theorem 21.8 (s-m-n, and a universal program for host programs)**.** (1) For a number c and a host program V, let $\text{spec}(c, V)$ be c copies of INC 2 followed by V with its jumps moved to follow them. Then for all x, y, $\text{spec}(c, V)$ outputs y on input x if and only if V, started with x in register 1 and c in register 2, halts with y in register 0. (2) There is a host program W made only of INC, DEC and HALT such that, for all c, x, y, W started with x in register 1 and c in register 2 halts with y in register 0 exactly when the host program numbered c outputs y on input x. □

One thing to be exact about: U is not that universal program. U runs two-counter guests, and a two-counter guest given its input plainly can't run the evaluator. With inputs written as powers of two one can, as Theorem 21.14 near the end of this subsection shows. W runs host programs, and it can, because it's a host program on a machine with registers.

□ **Theorem 21.9** (Kleene's recursion theorem on the host machine)**.** (1) For every host program T there is a host program e such that for all x, y: e outputs y on input x if and only if, for some d, T outputs d on the number of e, and the program numbered d outputs y on x. (2) So for every transformation F of host programs that some host program carries out on numbers (on the number of p it outputs the number of $F(p)$, for every p), there is a host program e such that for all x, y, e outputs y on x if and only if $F(e)$ does. (3) The same on numbers: for every function f on numbers, defined everywhere, that some host program computes, some number e has the program numbered e and the program numbered $f(e)$ output the same on every input.

That fixed point is of the *function*: e and $F(e)$ stop on the same inputs and leave the same number in register 0. Their ledgers, flags and fact tables can differ: e is all INC and DEC, so whatever $F(e)$ pays, e only simulates. I'd like a fixed point that agrees on the record too, and the next two theorems say how much of that can be had. The ledger and the flag can be had, but agreement on every register can't.

□ **Theorem 21.10** (The recursion theorem, up to the record)**.** Let F be a transformation of host programs that some host program T carries out on numbers. Then there is a host program e such that, for every input x, any state e ends in has a state $F(e)$ ends in that agrees with it on register 0, the trap latch, the ledger, the flag, the number of facts in the table, and whether the channel is empty, and the same the other way round. So e stops on x exactly when $F(e)$ does, and the ledgers are equal exactly, with no surcharge. T may itself check, commit and certify.

Look only at the function computed and you get the usual fixed-point statement back. There are two limits, and the second one is a theorem. First, e pays after it has computed everything. It is not the way U runs a guest, mirroring every move: e pays the counts, CHECKs, then COMMITs, then the CERTIFY, whatever order $F(e)$ used, so the order of the record's events isn't preserved, and for a program $F(e)$ that never stops, e never stops and pays nothing, though $F(e)$ may pay without bound. The relation says nothing about states in the middle of a run. Second, no fixed point agrees on every register, or on what the facts name.

□ **Theorem 21.11** (No fixed point of full behaviour)**.** There is a transformation F of host programs, carried out by a host program, such that no host program e behaves the same as $F(e)$ in the sense of Definition 21.2.

A variant of F that also CHECKs register r and commits to it gives $F(e)$ a fact and a commitment about r that e can't have, so no fixed point agrees on the fact table alone, or on the channel alone, or on the versions of the registers. What survives is what Theorem 21.10 states, plus any register named in advance, or any finite set of them: before it pays, the evaluator reads that register's final value off the simulated run of $F(e)$ and writes it into its own copy of the register, keeping its scratch work and its two payment registers elsewhere. The register F above writes is chosen by looking at e, and that's the one no fixed point can follow.

And with the recursion theorem proved, the wall of Section 13 goes up on this machine without assuming that theorem as an extra premise. Call a property of host programs *functional* when it depends only on the partial function a program computes.

Theorem 21.12 (The diagonal on the host machine)**.** Let Π be a functional property of host programs that some program has and some program lacks. (1) Read "f is representable" as "some host program carries f out on numbers." Then no function d from programs to Booleans whose flip is representable is right about Π on every program: this is the argument of Theorem 13.1, reproved for host programs, with its recursion premise a theorem here. (2) No host program decides Π outright: there is no program D that, on the number of every program, halts with a number in register 0 that's 1 exactly when the program has Π. □

Rice and Kleene are the ingredients the diagonal of Section 13 asks a substrate for, and the wall they put up doesn't care about the record: the base does the computing and the flag watches. A program's ledger and flag are among what "behaves the same" sees on the host, so a nontrivial property of those is undecidable too, in the reduction sense of Theorem 21.6, and the fixed point is up to the record, as far as the theorems above say and no further.

Now the two-counter machine itself, which I promised at the start of the subsection: Minsky's two bins of pebbles and nothing else. Rice comes first. Say a two-counter program *behaves the same* as another on a set X of inputs when, on every input n in X, started with n pebbles in bin A and none in bin B, one stops exactly when the other does, and then they agree on both bins, the trap latch, the ledger, the flag, and on which property and which counter each recorded fact and the channel are about. The versions are left out: they say which write a fact is about, and the trick below moves them.

The trick is the whole problem. Rice's reduction has to keep the program's input safe while a copy of a halting instance H runs, and there's no third bin to keep it in. My first guess was bin B, and that dies at once, because H needs both bins. The

second guess is Minsky's own packing: keep several numbers in one bin as powers of different primes, H's two counters as the exponents of 2 and 3, and the input as the exponent of 5. That dies more quietly. To put the input there, the program would have to turn n pebbles into 5^n, an exponential, and Schroeppel's memo, from Section 15, gives a proof that two plain counters can't even build 2^n from n. I had no way to build 5^n.

What works is to stop keeping the input beside the work. Turn n pebbles into $6n+5$, which is only adding, and then multiply in H's counters as powers of 2 and 3. H runs by halving the pile and splitting it in thirds and building it back up, and the $6n+5$ rides along in the middle of every one of those piles. It's odd, and counted off in threes it leaves two over, so no halving or thirding ever reaches into it.

□ **Theorem 21.13** (Rice's theorem for the two-counter machine, on any inputs)**.** For any set X of inputs, a property of two-counter programs that's carried from a program to any program that behaves the same on X is undecidable in the reduction sense of Theorem 15.5, unless every program has it or none does, and then it's decidable. X can be every input, the powers of two, or $\{0\}$.

So $6n+5$ is the place to keep the input that two counters seemed not to have. It isn't beside the work but inside it, in a factor the work never touches. I like that one more than I can justify: the input hides in the leftover that every halving and thirding walks straight past.

Kleene needs more than keeping an input. It needs a program that runs a program. So write a number x as 2^x pebbles and read a result as the exponent of 2, the way Minsky did. With inputs written that way, there's one two-counter program, call it U_2, that, started on $2^x \cdot 3^e$, stops with 2^y exactly when the program numbered e gives y on x; specialising a program to a number is exact; and the recursion theorem holds. (U_2 is a different program from the universal host program U, which runs on the machine with a register for every number.)

□ **Theorem 21.14** (Kleene for the two-counter machine, inputs written as powers of two)**.** (1) For every transformation F of programs that some program carries out on program numbers, there's a program e that computes the same function as $F(e)$, with inputs and outputs written as powers of two. (2) The fixed point is of the function only: there's such an F for which no e behaves the same as $F(e)$ once the properties and counters of the recorded facts count.

The counterexample in (2) is short. Number the programs the way U numbers its guests (Appendix 7), so a program's number is above the code of every property

it names. Let $F(p)$ be the one-line program that checks that bin A holds at least the number of p plus one. A program can only record facts about properties it names, so no e can record the fact $F(e)$ records, and from a large enough input $F(e)$ does record it.

Even with Schroeppel in hand, my first guess about Kleene was that the powers of two were a convenience, and that plain inputs would only take more work. They don't take more work: with plain inputs, Kleene's theorem is false. A transformation is *carried out* if some program, started on 2^n where n is the number of a program p, stops with 2^m where m is the number of $F(p)$.

Theorem 21.15 (No plain recursion on two counters)**.** (1) No program of n lines stops with $c \cdot x$ pebbles in bin A, started on x pebbles in bin A and none in bin B, for every x, where c is the least prime above $N = 16(n+1)(4n+1)^{16}$. (2) So there's a carried-out transformation F such that no program e computes the same plain function as $F(e)$. □

Nothing in that proof used more than this: two bins that move one pebble at a time, finitely many controls, a next step fixed by the control and the bins, and bins that, above some threshold, are read only through their parity. Any machine like that has the same obstruction, and the same computable transformation F with no program e equivalent to $F(e)$ on plain inputs.

I went looking for Schroeppel's own memo and found a scan of it. As I read it, his route is close in spirit: he trades a two-counter machine that starts with one counter empty for a machine with one register that can only add, multiply and divide by fixed numbers and take one away, each of those built out of one of the program's loops, and that's the stages here under another name. I haven't followed his proof line by line, so I compare the two at that level and no further; the argument this book stands on is the one above. The c it uses is astronomically bigger than anything a real multiplier needs, and finding the best bound is a question I leave alone.

So on two bins, Rice holds on every input; Kleene holds with inputs written as powers of two, for the function, and fails for the facts; and with plain inputs Kleene fails outright. The last one is the answer I didn't see coming: the powers of two weren't a convenience, and on two bins they're the difference between a recursion theorem and none.

That failure has a setting computability theorists know. The mathematician Hartley Rogers, in his 1967 textbook, calls a numbering of the computable functions *acceptable* when every other computable numbering of them can be translated into it by a computable map, and in every acceptable numbering Kleene's theorem holds. Read on plain inputs, the two-bin programs number something smaller than all

the computable functions, since none of them computes 2^n from n (Schroeppel’s theorem; compare Theorem 21.15), so they fall outside Rogers’s class, and his route to the theorem isn’t open there. The computer scientists James Royer and John Case (1994) studied that territory: programming systems for classes of functions smaller than all the computable ones, which can’t be acceptable, and what can still be programmed and proved in them. And the logician and puzzle writer Raymond Smullyan’s two books on recursion theory and self-reference (1993, 1994) take the fixed-point theorems apart axiomatically, so you can see which property of a numbering each one leans on. The plain-input failure is a small instance on the map they drew.

21.3 The universal machine’s guest interface

I’ll say what the two universal programs cover plainly, a sentence at a time. The stated class of guests of U is the small machine’s programs, every one of them, from every clean start start(x, y). The whole-run theorems don’t cover a guest handed facts in its table, a commitment in its channel or a raised flag at the start; the one-step simulation, by itself, works from any related pair. It’s the same kind of class Turing’s universal machine has: a fixed instruction set, and a finite program held as data, here packed into one number the way Turing packed his into a description number. Each guest keeps its computation, its halting, its answer, its record and its prices, exactly: the host halts when it halts, its flag goes up when the guest’s would, on a chain of its own, and its ledger is the guest’s.

The guests of U_P are every Thiele machine whose driver, step, prices and reading are computed by programs, with the reading decided on every state. Each keeps its computation and its halting; its reading becomes the host’s flag as a latch, raised on an earned chain whose claim is the guest’s own reading; and its prices become the host’s ledger, exactly, except for a surcharge at the move where the reading first says yes: at most two when the reading starts at no, exactly three when it starts at yes. The surcharge is zero when that move costs three or more, and it can’t always be avoided: when the guest certifies with a ledger below three, no program on that host, run from a clean start, matches its ledger exactly (Theorems D.7 and D.8). Picture a clerk who’ll stamp anybody’s receipt. He copies it onto his own forms, bills at your rates to within two when your reading starts at no and to within three when it starts at yes, and the stamp at the bottom is one he earned by checking your own reading, not one he copied.

What’s outside the class is outside the theorems. A machine whose reading no program decides (an oracle checker, say, which the definition of certified Thiele-complete doesn’t rule out) doesn’t come in. Nor does a machine whose next move is nobody’s computation. The two universal programs have different start conditions:

U's whole-run result starts its guest clean; U_P starts its guest where the presentation says, even if that state already has a yes reading. The host starts clean and earns a claim about that reading. It does not reconstruct or authenticate the guest's earlier history.

And insisting on no built-in step is the point, and a plain interpreter shows why. A plain interpreter on an ordinary computer runs guests too. If its guest record is freely writable data, its meaning depends on the interpreter following the intended update rule. An ordinary computer can also protect that data with a monitor or another enforced interface. Here the rule is explicit in the host's instruction semantics: U and U_P are fixed interpreters, and nothing in them raises the host's flag except the host's own CERTIFY, and the host's step won't let that pass without a COMMIT, or the COMMIT without a CHECK on a slot nobody has touched. The guest's record isn't data the host updates. It's a record the host earns again, under the same rule, on its own registers.

CHAPTER 22

Every model

After the smallest honest machine (Section 15) and the universal programs of the previous appendix, there was a worry I couldn't put down. Everything in them was built on Minsky's two counters. The definition of certified Thiele-complete names a two-counter window in clause (a). The universal program runs two-counter guests. So is all of this a fact about counters? Pick a different machine to start from, a Turing machine with its tape, a register machine, Alonzo Church's lambda calculus, and does the record still go on the same way, or did the definition depend on my example?

I chased it with a picture: a box with a panel bolted to its front. The box computes, with levers for moves and a window showing what clause (a) asks to see, two numbers and a line number. The panel is the example machine's earned layer: version tallies that click when the box writes, a rack for facts, the flag, and the three buttons CHECK, COMMIT and CERTIFY at a cost of one each. The worry is whether that panel only bolts onto the one box I built.

So this appendix goes both ways. First the panel goes onto boxes I didn't build, and I say exactly what a box has to supply for it to fit. Then other boxes, panel and all, get run inside mine. If the record is really about the record and not about pebbles, both trips should work, and the conditions on the way should say what has to survive the move.

22.1 From any base up

Start with the direction I care about most: the panel onto a box I didn't build. The box can be any machine at all that has a universal base in the sense of clause (a), and in the picture that means two things: it shows a two-counter window, and for each two-counter instruction it has a lever that acts on the window as that instruction does. What else is inside, what its states look like, I don't get to know.

My first guess was to keep whatever the box came with. Plenty of machines have a meter of their own, or a light on the front that means something's done, so leave those where they are, bolt the panel on beside them, and see what happens. It can't work, and the definition says why. If a lever can cost anything, computing adds to

the ledger, and certified Thiele-complete asks that it add nothing, which is a rule of the price list, not a claim about heat. If a lever can switch on the box's own light, a record went up with no CHECK behind it. Certified Thiele-complete rules out both. So the meter and the light come off, along with everything else the box already had: its costs, its record, all of it, except the states, the levers and what each lever does to the state.

Then bolt on a panel cut down from the small machine's, and nothing else: one version number for the whole box that every lever raises, which is the tally, a table of facts, a channel, the flag, and three new moves, CHECK, COMMIT and CERTIFY, each costing one, with every lever costing nothing. The three act as on the small machine, with the tally in place of a counter's version: CHECK files its claim with the tally when the checker says yes and the table has room, COMMIT puts the fact in the channel when the table holds its claim at the current tally, and CERTIFY raises the flag when the channel holds a fact. Call that the *lift* of the machine. A lever moves the box only while the trap is down. The trap works as on the small machine: a CHECK, COMMIT or CERTIFY that fails sets it, nothing takes it down, and once it's up every move, a lever included, changes nothing but the ledger.

Theorem 22.1 (Every universal base lifts)**.** (1) The lift of any box with a universal base, over a language of claims about the box's state ("unchanged" meaning the box's state is the same), with an exact equality test and a checker proved to say yes exactly when the claim holds, is certified Thiele-complete, as long as the claims its CHECK can ask include one that's true at one loaded state and false at another, and the fact table has room for at least one fact. (2) The claims "this register is at least n" and "the program is at line n," read off the window, always include such a claim, so with those claims every universal base lifts. (3) Conversely, every certified Thiele-complete machine has an interface meeting the four clauses under which its base moves, taken alone, form a machine with a universal base. (4) The panel leaves the box alone: from a state whose trap is down, a run of lever pulls in the lift moves the box exactly as the box's own run does, and the trap stays down. □

Proof. (1) The four clauses, one at a time. (a) The lift keeps the box's states, levers and step and adds the tally, the table, the channel, the flag, the trap and the ledger. With the trap down, a lever moves the box's part of the state as the box's step does, raises the tally by one, and touches nothing else. So from a live state, which is a state whose box part is live and whose trap is down, the window moves as the instruction does and the state stays live; no lever changes the flag; the loaded states are the box's loaded states with an empty panel, so they're clean; and the flag, once up, is lowered by nothing. (b) Start with the panel empty. The only move that

raises the flag is CERTIFY, and CERTIFY traps unless the channel holds a fact; the channel is filled only by a COMMIT, which traps unless the table holds its claim at the current tally; and the table is filled only by a CHECK that passed, which files the claim with the tally it was checked at. So in front of the raise there's a passing CHECK of some claim c, then a COMMIT of c at the same tally, then the CERTIFY. The trap is down from the CHECK to the COMMIT, since the COMMIT passed and nothing takes the trap down, and with the trap down the tally rises with every lever pull, so an unchanged tally means no lever was pulled in between, so the box's state is unchanged, and that's what "the thing c is about is unchanged" says in the lift's language, whose claims are about the box's state. A passing CHECK means the checker said yes, and the checker was proved to mean what it says. (c) is the price list: levers at a cost of zero, the three buttons at one each, the ledger the running sum. (d) Take the claim that's true at one loaded state and false at another. From the first, CHECK passes and files it, the table having room, COMMIT finds it at the same tally, and CERTIFY raises the flag. From the second the CHECK traps, and a trapped machine raises nothing. (2) "A is at least 1" is true at the loaded state showing $(1, 1, 0)$ and false at the one showing $(1, 0, 0)$. A window claim is a register and a number, or a line number, so two claims can be tested equal, and its checker reads the window, which is its meaning, so the checker is sound by definition. (3) Take an interface the machine is certified Thiele-complete with and throw away every move whose kind isn't base. What's left has the same states, the base moves as levers, and the interface's window, live states and loaded states, and clause (a) says of exactly these that the window follows every instruction from every live state, that the state stays live, and that each loaded state shows $(1, a, b)$ and is live. That's a universal base. (4) A lever pull with the trap down changes the box's part by the box's step and the tally by one, and nothing else; the trap goes up only on a CHECK that fails, a COMMIT without its fact, or a CERTIFY with the channel empty, never on a lever. So along a run of lever pulls the box's part follows the box's own run, pull for pull, and the trap stays down. ■

Of the checker's exactness, (1) uses one half whole and the other half once: (b) needs the checker to say yes only when the claim holds, and (d) needs it to say yes on the one claim at the one loaded state where that claim is true. The theorem asks for both halves whole, the way every theorem about the small machine does, so one checker serves all of them.

Clause (b), the small machine's proof with the bins swapped out for whatever the box has, is the part I'd braced for, and it turned out to be nothing. The chain CHECK, COMMIT, CERTIFY never looked at anything but the tally and the claim. The levers changed only the tally on the panel, making old facts stale; they never

inserted a fact, made a commitment or raised the flag. Neither side of the proof ever needed to know it was looking at pebbles.

So the earned layer isn't something two counters do. It sits on top of any box with a universal base, and it never needed to know which one. I'd half expected to be rewriting the proof once per model.

22.2 What never lifts

A theorem that says the panel fits every box is one I don't trust until I've watched it fail to fit something. So I went looking for boxes it won't turn into a certified Thiele-complete machine, and the first one I tried was the one I was surest would work.

It was a Turing machine, with one fixed table, and each lever does one row of the table. That looked like the obvious base to me, and it isn't one at all. Clause (a) asks for a lever for every two-counter instruction, and there are infinitely many two-counter instructions, since a jump can go to any line. A fixed table has finitely many rows, so it has finitely many levers, and it runs out of levers long before it runs out of instructions. The first two items below are that failure in two forms, and the others fail for other reasons, and each is proved.

Theorem 22.2 (What doesn't lift)**.** (1) A box with only finitely many levers has no universal base, and no box with a universal base is finitely branching, meaning that from every state all its levers together lead to only finitely many different states. (2) The lift of a finitely branching box is never certified Thiele-complete, whatever claims and fact table it gets. (3) Neither is the lift of a box whose step ignores the state it's in, where each lever sends every state to the same place. (4) On a record axis where every place is at or below every other, there is no certified Thiele-complete machine at all. (5) With a checker that says yes exactly when the claim holds, the lift's own non-vacuity clause holds exactly when the fact table has room and some claim is true at one loaded state and false at another. □

Proof. (1) Take a universal base and its loaded state showing $(1, 1, 0)$. For each j the lever for DEC $A\ j$ must take it, in one pull, to a state showing $(j, 0, 0)$: A was above 0, so one comes off and the line jumps to j. Different j, different window, different state. So one state has infinitely many different successors, which is what not finitely branching means; and since one lever lands in one place from one state, the levers for the DEC $A\ j$ are all different, so there are infinitely many of them. (2) In the lift the moves that cost nothing are the levers, since each button costs one, and from any state the levers together lead to finitely many states: with the trap

down, the box's finitely many successors, each with the tally raised by one, and with it up, the state itself. (The buttons can lead to infinitely many, one CHECK for each claim, but each costs one.) If the lift were certified Thiele-complete through some interface, that interface's base moves would cost nothing by clause (c), so they'd all be levers, and by Theorem 22.1(3) they'd form a universal base on their own. By (1) a universal base has a state from which its moves reach infinitely many states, and levers of a finitely branching box can't. (3) Suppose each lever lands in one place whatever the state. The move an interface gives an instruction is a base move, at a cost of zero by clause (c), while each button costs one, so it's a lever. Pull the lever for INC A twice from a loaded state showing $(1, 0, 0)$, and pull the lever for INC B and then the lever for INC A from the same state. Both runs end with the lever for INC A, so both end with the box in the one place that lever lands, with the tally two higher and the panel untouched: the same state of the lift. But the window would have to show $(3, 2, 0)$ after the first run and $(3, 1, 1)$ after the second, and a window is a function of the state. The same two runs refute the box on its own. (4) On a record axis, raising the record means leaving the down-set of where it stood. If every place is at or below every other, every down-set is the whole axis and no move ever leaves one; but clause (d) asks for three moves that raise the record from some loaded state. (5) If the table has no room, every CHECK traps and no CERTIFY ever fires, so (d) fails. If no claim is true at one loaded state and false at another, (d)'s second half has no witness. And when both hold, (d) was proved in (1). ■

So my first test case failed, and at first I read item (1) as saying a Turing machine can't be a base at all. It says something narrower. A Turing machine *with one fixed table* can't be one. A classic model comes in the other way round, with one lever for each program that carries out one two-counter instruction, run to its end. Pull it and the box runs that program from start to stop in a single move. That's how the register machine below comes in, and it's the honest reading of "a Turing machine is a base" too. The levers are the programs, not the rows.

That still leaves a worry about clause (a) itself. It asks for one lever per two-counter instruction, which is a fact about how the two-counter machine is presented, and a box that does each instruction in several pulls looks shut out for a reason that has nothing to do with computing. It isn't shut out, and the fix is to let the pulls be grouped. Call a box a *base by simulation* when it has a window reading a two-counter configuration off each state, through any encoding at all, a class of live states, a finite list of its own levers for each two-counter instruction, and a live loaded state showing $(1, a, b)$ for each start, such that from every live state the list acts on the window exactly as the instruction acts on a configuration and the state stays live. The *grouped box* has the same states, and its levers are finite lists of the box's levers, a list pulled in order.

Theorem 22.3 (The one-lever presentation loses nothing)**.** (1) Every box with a universal base is a base by simulation. (2) Every base by simulation is a universal base of its grouped box, and a run of the grouped box is the box's run along the joined lists. (3) So for every base by simulation, the lift of its grouped box, with the window claims and a fact table with room for at least one fact, is certified Thiele-complete. (4) The box with five levers, add one to A and go to the next line, the same for B, add one to a jump target (leaving the window alone), and for each of A and B "if it's above 0, take one off and jump to the target, otherwise go to the next line, and put the target back to 0 either way," is a base by simulation, has no universal base, and its grouped box lifts to a certified Thiele-complete machine. □

Proof. (1) Each list has one lever. (2) The grouped box's step is the box's run along the list, so the simulation clause of a base by simulation is clause (a) for the grouped box. (3) is (2) with Theorem 22.1. (4) INC r is one pull; DEC r j is j pulls of the target lever, then the decrement lever for r, from a live state, where live means the target is 0; the decrement lever puts the target back to 0, so the state stays live. The box has five levers, so item (1) of Theorem 22.2 says it has no universal base. ■

So clause (a) can be read by simulation, and nothing is lost: a box that carries out each two-counter instruction with one fixed finite list of its own moves, through any window, is a universal base the moment its moves are grouped, and it lifts. Item (1) of Theorem 22.2 is about the presentation, and the five-lever box is the proof that a finite lever set is not what fails. What this doesn't give is a Turing machine's rule table running a tape: the five-lever box is a register box, and the next subsection is still how Turing machines come in.

Item (3) is a box with no memory. Pull a lever and it lands wherever that lever always lands, whatever happened before. So a window on it can't tell INC A followed by INC A from INC B followed by INC A, and a box that can't tell those apart can't keep two counters in its window.

Item (4) is the trouble with bare preorders from Section 4, where the cups stopped telling you the place, showing up one level higher. If every place on the axis is at or below every other, the flag has no up to go to. No step ever turns a cup over, nothing ever has to be paid for, and the earned chain has nothing to earn.

22.3 Computation is the slice

The lift says the panel goes onto any box with a universal base. Turn it round and it says something about the box: the box is what you see of the lift when you stop

looking at the panel. That's where ordinary computation sits in this book, so I want it said exactly, with a name.

Definition 22.1 (Universal-base slice)**.** Take a machine with a ledger, a whole number for each state, and a loaded state for each start, and take a box with a universal base. A *universal-base slice*, called a slice in this appendix, is different from the global sliced interface of Definition 2.3. It requires a universal base, a working domain and reachable hidden witnesses; it does not require all non-computing moves to fix the view. It is a map from the machine's states to the box's states, a move of the machine for each lever of the box, called its *computing move*, and a class of the machine's states, called *working*, such that: (i) each loaded state is working, and the map sends it to the box's loaded state for the same start; (ii) from a working state, a computing move lands on a working state, and the map sends where it lands to where the box's lever takes the map's image; (iii) from some loaded state there are two runs whose ends the map sends to one box state, with the record up at one end and down at the other, and two runs whose ends it sends to one box state, with different ledgers.

When there's a slice, I'll say the box is a slice of the machine. The first two clauses say the slice runs as the box runs. Start with a working state and pull one computing lever. Clause (ii) says that its picture moves exactly as the box moves, and that the full state is working again. That last part lets you apply the same rule to the next lever, then the next. A whole run therefore follows the box's run, move for move. Clause (i) puts the two at matching starts, so they finish at matching pictures too. The third clause says it can't see what was kept or what it cost: a function of the box state that gave the record at the end of every run from every loaded state would have to give both answers at the one box state the clause names, and the same goes for the ledger. Nothing else is hidden in the definition. Read the box's window through the map and take as live the working states whose image is live, and the first two clauses make the computing moves for the box's instruction levers a universal base of the machine. So a machine has a slice onto some box exactly when it has a universal base whose window meets the third clause, and then the window itself is a slice onto the two-counter box, whose states are configurations and whose levers are the instructions. The third clause does work. A Thiele-complete machine's computing moves, taken alone, are a box, and the map that forgets nothing meets the first two clauses onto it and fails the third, because the record and the ledger are functions of the state; and the window of the first clock of Observation 12.3, whose record is a question about its counters, fails it too.

Theorem 22.4 (Computation is the slice)**.** (1) Take the lift of a box with a universal base, as in Theorem 22.1(1). Forgetting the panel, which keeps the box's part of each state and drops the tally, the table, the channel, the trap, the ledger and the flag, is a slice of the lift onto the box, with the levers as its computing moves and the states whose trap is down as its working states. Beyond that, CHECK, COMMIT and CERTIFY never move the box's part, from any state; from any box state with an empty panel, the lever runs are sent, under forgetting, to exactly the box's own runs from that state, lever list for lever list; and from a loaded state where the claim of Theorem 22.1(1) is true, the run of no moves and the run CHECK, COMMIT, CERTIFY of that claim end on one box state with the flag down and up and ledgers exactly 3 apart, CERTIFY three times ends there too with the flag down and the same ledger as the second run, and CERTIFY d times, for any d, ends there with the flag down and a ledger d above the first run's. (2) With no room in the fact table, the lift of any box, with any claims, is certified Thiele-complete through no interface at all. (3) Let a machine be Thiele-complete through an interface; every certified Thiele-complete machine is (Theorem 16.3). Its computing moves are exactly its moves that cost nothing, so they're the same through every interface it's Thiele-complete through; taken alone, with the interface's window, live states and loaded states, they form a machine with a universal base; and the window is a slice of the machine onto the two-counter box, with the instructions' moves as computing moves and the live states as working. (4) Let a machine be run as one rule: from each state, every move leads to the same next state. It has no universal base, no base by simulation and no universal-base slice onto any box. It obeys the toll exactly when every move costs at least one or no state reading no steps to one reading yes, so it's a Thiele machine only as a clock or with a reading that never rises.

Proof. (1) The lift's loaded states are the box's loaded states with an empty panel and the trap down, so they're working, and forgetting sends each to the box's loaded state for its start. From a state with the trap down a lever moves the box's part as the box's step does and leaves the trap down (Theorem 22.1(4)), which is the second clause. Start with any box state and an empty panel. The first pull matches the box's move and keeps the trap down; the next pull therefore starts under the same conditions. Repeat through the word and every pull matches. A CHECK, COMMIT or CERTIFY that passes changes only the table, the channel, the flag and the ledger; one that fails sets the trap and adds to the ledger; and with the trap up every move changes nothing but the ledger. None of them touches the box's part. Now take the loaded state where the claim is true. The run of no moves ends there with the flag down and the ledger at 0. On the other run the CHECK passes and files the claim,

□

the table having room, the COMMIT finds it at the same tally, and the CERTIFY raises the flag: three moves at one each, none of them moving the box, so the same box state, the flag up and the ledger at 3. CERTIFY from a clean state finds the channel empty and traps, and every later move changes only the ledger, so CERTIFY d times ends at the same box state with the flag down and the ledger at d, which at $d = 3$ is the chain's ledger. So the third clause holds, for the flag and for the ledger, and no function of the box state and the ledger together gives the flag. (2) Take any interface and the three moves its clause (d) names. Each costs one by clause (c), so each is a button. Take a loaded state from which they raise the flag; it's clean by (a), so its flag is down by (b). If its trap is up, nothing passes and nothing rises. If its channel holds a fact, CERTIFY alone raises the flag from it: a run of one move from a clean state that ends with the flag up, where (b) asks for a CHECK and a COMMIT before the raise. If its channel is empty, some COMMIT has to pass inside the three moves, before the CERTIFY that raises the flag. Until then no CHECK has passed, because a CHECK passes only with room in the table, and no button moves the tally, so that COMMIT found its fact in the loaded state's own table at the loaded state's own tally; that COMMIT and a CERTIFY, run from the loaded state, then raise the flag in two moves, against (b) again. So the three moves raise the flag from no loaded state, and clause (d) fails. (3) By (C), computing moves cost 0 and every other move costs 1, so the computing moves are the moves that cost 0, whichever interface the machine is read through. Throw the rest away and what's left has the same states, the interface's window, live states and loaded states, and a move for each instruction that acts on the window as the instruction does from every live state and keeps it live, with each loaded state live and showing $(1, a, b)$: a universal base, as in Theorem 22.1(3). The same facts are the first two clauses of a slice onto the two-counter box, and the third is Theorem 12.5(2): two runs from one loaded state end on one window, one with the record up and one with it down, and their ledgers differ. (4) From each state all the moves lead to one state, so the machine is finitely branching, and by Theorem 22.2(1) it has no universal base. A slice would give it one, as the remark after Definition 22.1 says. A base by simulation fails too. A list of p moves does p steps of the rule, so two lists run one after the other end in the same place in either order. From the loaded state showing $(1, 0, 0)$, INC A and then DEC A to line 5 shows $(2, 1, 0)$ and then $(5, 0, 0)$, while DEC A to line 5 and then INC A shows $(2, 0, 0)$ and then $(3, 1, 0)$, and one state can't show both. For the toll: if a state reading no steps to one reading yes, every move takes that step, so every move costs at least one, every step does, and the machine is a clock (Observation 12.3); if none does, the reading never rises and the toll asks nothing. ■

Not every part of that carries the same weight, and I'd rather say which before somebody else does. Some of it holds by construction. That the slice runs as the

box runs, and that CHECK, COMMIT and CERTIFY leave it alone, are how the lift was built, and the blindness in (1) follows from the second once the chain raises the flag: the chain and the run of no moves end on one box state because no button moves the box. In (3), the window meeting the first two clauses of a slice is the definition of Thiele-complete read out, and so is the fact that the moves that cost nothing are the same through every interface. Three parts could have failed. That the lift is certified at all is the earned chain of Theorem 22.1, clause (b). That the window of every Thiele-complete machine can't see what was kept or what it cost is Theorem 12.5, and it could have gone the other way: the first clock's window sees its record. That a table with no room certifies through no interface is (2). And (4), that a machine run as one rule has no universal-base slice and is a Thiele machine only as a clock or with a reading that never rises, is short, and its second half rests on one choice in Definition 2.2: the toll puts the price on the move, the same from every state. With a price that could read the state, a machine run as one rule could charge exactly its raises.

What "computation" means in the theorem's name is a box's levers acting on its states. A model that comes with a move for each two-counter instruction is covered as it stands, and one with a fixed list of its own moves for each is covered once its moves are grouped (Theorem 22.3). A model run as one rule, a Turing machine stepping by its one table, is covered by (4) only: it has no universal-base slice, and it cannot supply the appendix's universal-base target. Its ordinary transition rule still lifts to the global sliced machine of Definition 2.3: put the lamp and tally beside it. The fixed-list obstruction concerns its instruction interface. A Turing machine read the other way, with a lever for each program that carries out one two-counter instruction, as in the previous subsection, is a box like any other. Nothing in the theorem uses the equivalence of the seven models below (Theorem 22.6(1)), which is other people's work; that equivalence is what says a box with a universal base computes whatever the other models compute.

A machine run as one rule can still run programs, and its program can keep a flag. Whether that flag is a record the machine keeps or a bit it only simulates can be said exactly too, once the program is part of the state.

Observation 22.5 (A record or a simulated bit)**.** *Take a machine whose moves are operations on configurations, with a reading of the configurations, and put the program in the state: a state is a program, which picks an operation for each configuration, together with a configuration, and one step applies to the configuration the operation the program picks there and keeps the program. Take every program. Then the reading never comes down under any operation from any configuration exactly when, read on these states, it never comes down from any state.*

Proof. If no operation brings the reading down anywhere, no step does, since each step applies an operation. If an operation brings it down from some configuration, the program that picks that operation at every configuration is one of the programs, and the step from the state made of that program and that configuration brings it down. ∎

So whether a cell is a record or a simulated bit is a fact about the one rule that steps program and configuration together, and it takes the whole set of states to see it. A certification cell that some program can clear, like the one a universal machine keeps as plain data on its tape (Section 15.11), is a simulated bit, though on a run whose program never clears it the cell goes up and stays, as a record would.

Why the live-state clause stays in the slice

Go back to the blown fuse. A free lever turns the live machine's gears and leaves the trapped machine still. Hide the difference between those states and one camera picture would have to prescribe both outcomes. Theorem 15.1 gives the exact consequence: hide every native CHECK globally, require autonomous free moves, and every projected free move becomes identity.

Keep the camera promise on live states instead. The computing moves act there and keep the machine live. A failed panel check leaves that domain. That is the boundary in the slice above. It lets the computational view move while the native failure rule keeps its power to stop the run. Restricting the promise to reachable states would not do this: append a failed CHECK to a run and the trapped state is reachable too. It is the live boundary, which that check crosses, that makes the difference.

The certificate and ledger remain passive beside the core. The trap is inside the core, where the gears read it. Add the publication file and you can keep which claims were certified, with the native lamp as its nonempty reading. The fuse still works as before. The file records what was published; the live-state clause says where the computing promise holds. Both boundaries belong in the description of the machine. Theorem 15.9 proves that the lamp reports whether the publication file is empty. Theorem 15.18 counts how many different files can sit above the same full native state.

22.4 The same thing, in seven models

Now the other direction, the one the worry was really about. Put boxes with different insides in a row: one with a tape, one rewriting lambda terms, one with bins. Is a panel riding on one of them the same as a panel riding on another? The honest way to ask is whether the boxes agree on what they compute.

My first idea was to wire the boxes to each other myself, two at a time, with a relation that says one box's run keeps pace with another's, but that's the wrong tool. Section 11's weak base equivalence is about runs keeping pace, and what I need here is coarser: do the boxes compute the same relations on numbers? For that the work is already done, and it isn't mine. The seven models are the weak call-by-value lambda calculus L, Turing machines, two Minsky instruction conventions, binary stack machines, Conway's FRACTRAN, and the μ-recursive functions, and that they all compute the same things is Minsky's, Shepherdson and Sturgis's, and Forster, Larchey-Wendling and their coauthors' work. The translations, with how each one writes its inputs and reads its outputs, are theirs (Appendix 6).

Theorem 22.6 (The base doesn't matter)**.** (1) A relation on numbers that's computable in any one of those seven models is computable in all seven. (2) The box that keeps a whole two-counter configuration in one number, with one lever for each instruction, has a universal base, its step is computable in all seven models, and its lift, with a table of sixteen facts, is certified Thiele-complete. (3) A register machine of the kind Cook and Reckhow described, where each two-counter instruction compiles to at most five of its own instructions, has a universal base too, and its lift is certified Thiele-complete. (4) Any box on numbers with a universal base lifts to a certified Thiele-complete machine, and its step, computable in one model, is computable in all seven. □

Proof. (1) Compose the translations (Appendix 6). (2) Encode a configuration as $\langle \ell, \langle a, b \rangle \rangle$. Decoding, performing one INC or DEC, and encoding the result are the total finite arithmetic operations constructed there. So the window follows each instruction exactly; loading is the encoding of $(1, a, b)$. (3) Let RAM registers 0, 1 and 2 hold the simulated line, A and B, and keep register 3 equal to 1. To increment counter register r, add register 3 to r and then to register 0. To decrement it with target j, use these five lines, numbered 0 to 4: if $r > 0$ jump to 3; add register 3 to register 0; jump to 5; subtract register 3 from r; set register 0 to j. At line 5 the finite block has ended. A base move restarts a block at its first line, runs it for its length, and keeps the resulting registers; an ended block stays put. The zero and positive cases give exactly the required window, and neither changes register 3. Loading puts $(1, a, b, 1)$ in registers 0 to 3. (4) Apply the lift theorem to that universal base and apply (1) to its step function. ■

So the answer to the worry is that the record doesn't care. A box with a universal base, described in any of the models people have used to say what computing is, takes the same panel, becomes the same kind of machine, and pays the same price, one each, for the same three acts.

And the trip goes back down. So far the panel went onto other boxes. Now put another box, panel and all, inside mine. Appendix D.2 proved that the fixed program U_P on the small machine's host runs every computably presented Thiele machine. In terms of the seven models, that reads: if the four relations that describe a Thiele machine (its driver, which picks the next move or says stop, its step, its cost and its reading, on codes) are computed in any one of the seven by programs that answer on every input, then the machine is computably presented, and U_P runs it, halting when it halts, with its flag, and with a surcharge of at most two when its reading starts at no.

So a Thiele machine described in lambda calculus runs, under one fixed program, on the small machine's host, which gives model independence in both directions: up, by lifting any universal base, and down, by running anything presentable on one fixed program. When I saw both arrows close I stopped worrying that the definition depended on my example. A definition that did wouldn't survive being carried both ways.

22.5 Why two counters

If the box doesn't matter, why did I build on two bins? I wanted the smallest box that holds up, and the obvious smaller one has a single bin. Nothing earlier in this book says one bin is too few. So before I'd let myself say two is the fewest, I tried one.

Watch what a program on one bin can see. While the bin isn't empty, it can't tell how full it is: a stretch of lines that never finds the bin empty does exactly the same thing from a fuller bin, with the extra pebbles riding along. So the only news a one-bin program ever gets is the bin running dry. Between two of those moments it's just walking its own lines, and among more moments than there are lines, two are on the same line, which is the pigeonhole again. Put those together and a one-bin program either stops within a count you can write down in advance, or never stops at all.

□ **Theorem 22.7** (One counter is too few)**.** (1) A program P of INC, DEC and HALT instructions on one bin, started with n pebbles in it, halts exactly when it has stopped within $k(n+k+1)$ steps, where k is the number of lines in P. So whether it halts is decidable. (2) Halting for the book's two-counter machine is undecidable. (3) So no computable translation takes each two-counter program and start to a one-counter program and start that halts exactly when the original does.

Proof. (1) If $k = 0$, the program has already stopped. Otherwise suppose it is still running after $T = k(n + k + 1)$ steps, so the $T + 1$ configurations at times $0, 1, \ldots, T$ are all live. If the counter has stayed at most $n + k$ up to time T, those live configurations have only $k(n + k + 1) = T$ possibilities: k lines and $n + k + 1$ counter values. There are $T + 1$ of them, so one repeats, and the deterministic run loops forever.

If instead the counter reached $n + k + 1$ by time T, take the first time it did, and look back to the last time before that when it was at most n. In between it stayed above n, so above 0. It moves at most one per step, so on the way up it arrived at each of the $k + 1$ levels $n + 1, \ldots, n + k + 1$; mark the first arrival at each. There are $k + 1$ arrivals and k lines, so two arrivals have the same line ℓ, at levels $a < b$. Every DEC in the stretch between them found the counter positive. Start that same stretch at level b: every counter value is larger by $b - a$, so still positive, every DEC goes the same way, and the run comes back to line ℓ at level $2b - a$. Repeat forever. Either way a run still live at time T never stops, so a run that stops does so within T steps, and running P for T steps decides it. (2) The tape simulation, prime packing and halting diagonal (Appendix 6), carried over to the book's instruction set line by line. (3) Such a translation, followed by the test in (1), would decide two-counter halting, against (2). ∎

So two counters are the fewest in exactly this sense: one counter can't compute everything, because its halting can be decided, and two, handed their inputs packed, can. I like where the answer came from. The pigeonhole that left no spare chair in the closed room is the same one that says why the box has two bins.

22.6 The lift and its instruction interface

The lift gives each box with a universal base an earned panel. In the other direction, every certified Thiele-complete machine supplies a universal base through its own base moves. The obstruction results also identify two kinds of base that never lift. Together these describe the coverage proved here. Two further classification questions concern bases outside those classes and the possibility of certification through an interface other than the lift's own.

The tape-to-counter construction (Appendix 6) connects bases through what they compute. The lift then supplies the record operations. Its tally is one edition number for the whole box, whereas the small machine keeps an edition per counter. That coarser tally is enough for the lift's arguments. To run the lifted machine inside the presented-machine interpreter, supply the four computability relations for its chosen presentation.

The construction lets you follow the box and the panel separately: the base provides its computing moves, the panel earns its record, and the live projection recovers the box's working run.

CHAPTER 23

Running it

Everything so far is definitions and proofs about them. Does any of it run? This appendix is me finding out. I built the small machine twice, once straight out of the definitions and once by hand from the same description, ran both, and compared the whole state after every step. Then I ran the universal programs on guests small enough to finish, and real programs through the part of the compiler that executes.

One rule holds for the whole appendix: the runs test the builds, and the proofs carry the theorems. A run shows what a build did on the inputs it was given, and no theorem in the book leans on one: every theorem in it stands on its argument, on the page.

There are two counts to keep apart as you read. The ledger counts what the price list charges. The number of interpreter steps counts the work this particular build does to carry the run out. One guest step can take millions of host steps and add one to the ledger.

Stage	What the next stage must preserve
Source orders	Variable values and completed procedure calls
Assignments and jumps	The same branches, loops and final answer
Counter blocks	Values at block boundaries; spare counters restored
Host instructions	The same result, with the opposite DEC convention translated
Packed two-counter guest	The result as an exponent in the packed counter
Fixed interpreter U_P	Guest counters at matching points; decode the exponent to read the answer

This is a path through the compiler, not a claim that one step at each stage takes the same time. The source compiler inserts no certification chain. Running the executable uses physical hardware and energy. These runs did not measure heat per host step, and the ledger is not a heat measurement. The times only tell you which runs are practical on this build. The table shows what ran and what didn't.

What	Status
The small machine	run: the executable versions agree on every tested step
The host machines, with and without PAY	run, compared the same way
U and U_P on tiny guests	run, against a separate guest implementation
U on a guest that certifies	proved and counted, not run
U_P on a guest that certifies	proved, not run
Source programs through the compiler	run as far as the host program
The two-counter guest and U_P running it	proved, not run
The presented machine carried by U_P	proved, not run

□ **Theorem 23.1** (The compiler is correct)**.** For every source program with no recursion, and every input: the program computes y exactly when the runner (the bounded evaluator of the compiled host program) halts with y for some amount of fuel; exactly when the compiled host program halts with y; exactly when the two-counter guest halts with y; and exactly when U_P running that guest halts with y, with its first counter at 0, its ledger at 0 and its flag down. So a terminating compiled program delivers its answer and certifies nothing. To certify a claim about that answer, a target program must add a CHECK, a COMMIT and a CERTIFY; this compiler does not insert that chain. The runner also takes exactly as many steps as the host program does, with the rest of its fuel left over, and a run that doesn't halt has used all its fuel.

CHAPTER 24

The CHSH check

The example machine checks whether a number is zero, even, or at least something. That's a fine way to show the chain, and a modest thing to certify. This appendix builds a check worth earning from zero, and then shows the one thing the small machine's host does with it, which is smaller and more exact than "it checks for quantum behaviour," a sentence that's tempting and wrong. In brief, so you know where this is going: the check never computes the famous number and never writes down the famous bound. It tests seven facts about integers in eight counts. If they pass, a certain five-by-five table has a property called positive semidefinite, and that property forces the famous number under the famous bound. The wall is a consequence of the check, and it never goes into it. Everything else is explaining those sentences.

Why put it in the book? Because it pulls four things apart that the small examples keep bunched together: the counts, the relation the counts satisfy, the act of checking that relation, and the record kept of the act. Writing a tally down costs nothing on this machine. The checker decides a real condition about a matrix, using nothing but integer arithmetic. The step ties the passing check to this edition of the tally, and the earned chain costs three, the same three it costs for "A is zero." How deep the condition is and what the three acts cost are different numbers, and here you can see them apart.

And the tally on its own carries no history of where it came from. A tally somebody made up passes the same arithmetic as one that was played. What's claimed here is about the numbers and what the machine does with them, and the guarantee paragraph below says exactly how far that goes.

24.1 The game

There are two players, Alice and Bob, in separate rooms. A referee flips two independent fair coins, independent also of the players' plan, and hands Alice a question x, zero or one, and Bob a question y, zero or one. Alice answers a, zero or one; Bob answers b. No talking during the game. They may agree on a plan beforehand, as detailed as they like. They win a round when their answers *differ*

exactly when both questions were one. In words: if either question is zero, match; if both are one, differ.

Try to win every time. A plan is a list: what Alice says to x equals zero, what she says to x equals one, and the same two for Bob, four bits in all. Now write the four requirements the rounds impose: a0 equals b0 (both asked zero); a0 equals b1; a1 equals b0; a1 differs from b1. Chain the first three: a0 equals b0 equals a1, and a0 equals b1, so b0 equals b1. But the fourth says a1 differs from b1, and a1 equals b0 equals b1, a contradiction, so one of the four must break. So any plan loses at least one of the four question pairs, and if the referee asks all four equally often, the best a plan can do is three out of four, seventy-five percent. Do it the long way too: there are sixteen plans (four bits), and if you write them all out and count wins you'll find eight win three rounds and eight win one, and none wins four (the broken requirements always come in odd numbers, which you can see by adding up the four equations and keeping only whether the totals come out even or odd, with "equal" as 0 and "differ" as 1: each of the four answers appears twice on the left, so the left sides add to an even number, and the right sides add to one). Sharing a coin before the game to pick a plan at random doesn't help; a mixture of numbers no larger than three-quarters is no larger than three-quarters. Arthur Fine, a philosopher of physics at the University of Washington, much of whose work is on the physicist Albert Einstein and the quantum theory, proved in 1982 that the whole "pre-agreed answers, no influence between rooms" story is *exactly* the mixtures of those sixteen lists, one scoreboard for all four questions at once; the sixteen are the whole classical story. That one I don't have to take on his word: the form of his theorem that speaks of the four averages is proved in this book, and the end of this appendix says where.

Alice's question	Bob's question	Both answer 0	Required relation	Result
0	0	equal	equal	win
0	1	equal	equal	win
1	0	equal	equal	win
1	1	equal	different	lose

Changing the four planned answers can move the loss to another question pair. It cannot remove every loss: the first three equality requirements already force the fourth equality.

Now the number itself: recode answers zero and one as plus one and minus one. For each question pair, let E be the average, over many rounds, of Alice's answer times Bob's. The CHSH quantity is S equals E00 plus E01 plus E10 minus E11. For one fixed plan each win adds one to S and each loss subtracts one, so S is wins minus losses: a three-win plan has S equal to two, a one-win plan has minus two. With equal question frequencies, the win rate is one half plus S over eight. So

"seventy-five percent" and "S at most two" are the same fact, and the second is the one that got the name.

The ceiling is easy to reach and costs nothing to write down. Tally one round per question pair with both players answering zero every time: after the recoding every answer is plus one, so all four correlators are one. That's the plan that answers zero to everything, which matches on every pair and so wins the three pairs that ask for a match, $S = 1 + 1 + 1 - 1 = 2$. On the small machine's host a tally like that is a few INCs on counters, and INC costs nothing. Nothing in it says the rounds were played; anybody can tally anything at no charge on the ledger. The other side, that no finite mixture of plans beats two, is Theorem 25.1 in Appendix 25, one direction of Fine's result. The full equivalence, in the form that speaks of the four averages, appears in the last subsection of this appendix.

24.2 Where it came from, and why anyone cared

So who started this, and why would anybody care whether two rooms can win a coin game? The names come in a line, and the line didn't mean much to me until I saw that each of them moved the same thing a notch: from an argument about what's real, which you can only agree with or not, toward a pile of counts a referee can judge. Each one rests on a different piece of the game above.

The trail starts in 1935, with Einstein, by then at the Institute for Advanced Study in Princeton, and two younger colleagues there, Boris Podolsky and Nathan Rosen, asking whether quantum mechanics describes everything that's real about a pair of particles. Their argument combines certain prediction with a no-disturbance premise: if measuring one particle lets you predict the other's result without disturbing it, their criterion treats that result as an element of reality. They argue that the quantum description leaves something out. What they rest on is the plan in each player's pocket, a pre-agreed list of answers, the plan above. And the first thing worth checking is whether the paper even says what Einstein meant, and by his own account it doesn't quite. Podolsky did the writing, and that June Einstein wrote to Erwin Schrödinger, the Austrian physicist whose 1926 wave equation is at the heart of quantum mechanics, that the essential thing had been "smothered by formalism." The person who lays that letter out is Arthur Fine, the same Fine whose theorem closed the sixteen plans above, in his encyclopedia entry on the argument. So Fine stands at both ends of the line: he checked the founding paper against what its author meant, and he proved exactly what the plans can do. Even the founding paper is a lead to check, not a verdict.

For thirty years that was philosophy. In 1964 John Bell turned it into arithmetic: if each particle carries pre-agreed answers and nothing travels between the rooms, certain averages can't exceed a fixed number, and quantum theory predicts they do.

What that rests on took me a few reads. For this game the assumptions can be said without that shorthand: the questions are independent of the shared preparation, and each room produces its answer using its own question and that preparation, with any local randomness included. Even stochastic response rules under those conditions are mixtures of the sixteen plans. Giving up predetermined answers alone therefore does not evade the bound. What fails is this local explanation under the stated independence condition. And where does a paper like that get published? In a brand-new journal called *Physics Physique Fizika*, which charged authors no page fees and in fact paid them by the word, ten cents a word, and which folded in 1968 after four volumes, or three. I went looking for that count, and the sources don't agree on it: four is the number the American Physical Society's archive of the journal gives, and Iulia Georgescu, writing about the journal in *Physics World* in 2026, says three. She's also where the ten cents comes from. As Andrew Whitaker tells it in his biography of Bell, the journal gave out no free copies, so Bell spent his payment buying copies of his own paper to mail to other physicists. A result nobody else holds a copy of hasn't left the room yet, and Bell paid to send his across. In 1969 four physicists, John Clauser, then a graduate student at Columbia, Michael Horne and Abner Shimony at Boston University, and Richard Holt at Harvard, rewrote it as the four-average form above, S at most two, which could be tried in a lab with imperfect equipment. It's called the CHSH inequality, after their initials. That notch rests on what a real lab can deliver. Shimony was in philosophy and in physics at Boston University both, which is the whole line in one man.

Then the obvious next question: did anybody actually try it? Stuart Freedman, a graduate student at Berkeley, and Clauser, by then a postdoc there too, tried it in 1972. The physicist Alain Aspect's group at the Institut d'Optique in Orsay, with two colleagues there, Jean Dalibard and Gérard Roger, did it in 1982 with the measurement settings switched while the photons were in flight. Their switches changed the analyzer faster than a light signal could cross between the stations. They operated at fixed, incommensurate frequencies, however, rather than choosing fresh independent random settings on every trial. Rapid switching and unpredictable questions are separate requirements; this experiment did not close every loophole. Before that, in 1975, Aspect went to see Bell at CERN to tell him what he planned, and by Aspect's own account Bell heard him out in silence and then asked, first thing, whether he had a permanent position. Aspect did. Bell's reason, as Aspect tells it, was that most physicists would take the experiment for a waste of time, or take him for a crackpot. I read that as the toll in its oldest form: a check that could have come out the other way costs somebody something to run, and Bell wanted to know who'd pay. And in 2015 three groups of experimental physicists (Bas Hensen's in Delft, with electrons 1.3 kilometres apart; Marissa Giustina's in Vienna; Lynden Shalm's at NIST, the National Institute of Standards and Technology, in Boulder) closed the

two main escape routes at once. In 2022 Clauser, Aspect and the Austrian physicist Anton Zeilinger shared the Nobel Prize "for experiments with entangled photons, establishing the violation of Bell inequalities and pioneering quantum information science." The 2015 tests came out of that line of work, and Zeilinger was one of the authors of the Vienna one. So in under fifty years the subject went from a question you needed a permanent job to risk to a Nobel. The lab number sits above two. A remaining escape, that the referee's coins were secretly correlated with the particles, is a philosophical one and I'm not going to pretend it's closed.

Now put this book's check on the same line, because that's where my position gives. It sits past the far end, at the counts. Those fifty years went into making the counts worth judging, rooms really apart, questions really late, escape routes really shut, and my check starts after all of it. It's handed eight counts and can't tell a lab's tally from one somebody wrote down at no charge. So the check is a referee of counts, and everything that made these counts worth refereeing happened upstream of it.

What quantum play does: if the players share an entangled pair and choose what to measure from their question, the best they can do is S equals two times the square root of two, about 2.83, a win rate of about eighty-five percent. Boris Tsirelson, a Soviet mathematician who later taught at Tel Aviv University, in a 1980 paper called "Quantum generalizations of Bell's inequality," proved that that's a *ceiling*: quantum correlations violate Bell, but only up to there. I prove that ceiling below, and show the state that reaches it, with real amplitudes and with complex ones. The bound holds in arbitrary dimension, including infinite dimension; the construction attaining it needs sufficient room, supplied by the two-qubit example below. The first thing proved is the algebra.

Then there's the wall above that, which is the one I find most instructive. In 1994 the physicists Sandu Popescu and Daniel Rohrlich asked: what if you only demand that nothing can be *signalled*, that Alice's answers on their own have the same odds whatever Bob is asked, and the other way round? Then you can write down a "box" (the PR box, after their initials) that wins *every* round (its answers differ exactly when both questions are one, by fiat, each answer alone a fair coin), S equals four, and nothing about it lets either room send a message. So no-signalling on its own doesn't explain the quantum ceiling; the PR box meets that condition and beats 2.83. (A box isn't a physical theory, and nobody is claiming one that meets everything relativity asks.) The order is: two for plans, 2.83 for quantum, four for no-signalling.

And the box has the same kind of history as everything else in this book: somebody got there first. Popescu and Rohrlich's initials stuck, but the big review of the field by Nicolas Brunner and four coauthors credits boxes like it to Peter Rastall, whose 1985 paper on locality and Bell's theorem appeared in *Foundations of Physics*,

and, separately, to Leonid Khalfin, a physicist at the Steklov Institute's Leningrad branch, and Tsirelson himself, both in 1985, nine years before. Then the question that really matters: if relativity isn't what stops things at 2.83, what is? People have tried candidates. In 2005 the computer scientist Wim van Dam, who works on the theory of quantum computing, argued, in a paper on the implausible consequences of such boxes, that with enough PR boxes, Bob could work out any yes-or-no function of both their inputs after Alice sends him a single bit, which would make the whole question of how much two parties need to talk collapse, and that's a strong hint the boxes are too good to be real. In 2009 Marcin Pawłowski, a physicist at the University of Gdańsk, and five coauthors proposed a principle they called information causality: Bob can't get more information about Alice's data than the bits she actually sends him. They argue that ordinary and quantum physics both obey it, and that any theory that beats 2.83 breaks it. How much of the rest of the quantum boundary it reaches is something I haven't checked. Which principle fixes the whole quantum boundary, if any one does, this book won't answer. The check below is a fence at the middle step.

24.3 Five sentences that are easy to say by accident

Each one comes below with what I can stand behind beside it.

"Faster than light." The correlations alone cannot carry a chosen message between the rooms: each room's marginal probabilities do not depend on the other question. That operational statement does not settle every account of what happens between measurements. "Quantum means random." The state and the chosen measurements determine the outcome probabilities and expected score; a particular outcome can be certain or random. "A Bell violation proves nonlocality." A statistically supported violation rejects the local response model under the setting-independence and experimental assumptions. Calling it a rejection of "realism" does not identify those assumptions.

"A tally above two proves nonlocality." No: S is a *population* average, and one finite tally can land above two by luck. Four rounds, one per question pair, with coin-flip answers, reach S equals four one time in sixteen. That gap between a population bound and a finite sample is the problem Yanbao Zhang, Scott Glancy and Emanuel Knill, at NIST in Boulder, work through for real Bell tests (Physical Review A, 2011 and 2013), and the book doesn't solve it. The way they solve it is worth knowing, because it has the shape of this book's record. Turn each round into a small bet that no local plan can expect to win, even a plan that remembers every earlier round, multiply the bets as the rounds come in, and the French mathematician Jean Ville's inequality, from his 1939 thesis, one of the works that started the theory of martingales, says the chance the product ever climbs past one over δ is at most δ. So

a flag can go up for good the moment the product crosses, with its error stated up front. It needs one thing the small machine doesn't have: settings drawn underneath the program, where the program can't choose them. And how much can one round tell you? Wim van Dam, Richard Gill, a mathematical statistician who later held the chair in that subject at Leiden, and Peter Grünwald (2005) measured exactly that, in bits of evidence against every local plan. Of the two-party proofs they compare, CHSH comes out the strongest, "much stronger" than the others in their words, at about 0.046 of a bit per round, a few hundredths, so a convincing record takes hundreds of rounds, and four won't do. About a finite tally, this book proves only the narrow thing, that a tally above two, with every question pair asked at least once, is not explained by *one fixed plan replayed every round*, and that narrow thing is what the sentence has to stay. The bound for a finite local model is a separate theorem, and it says nothing about one tally. And "the PSD test proves the data is quantum." (PSD is short for positive semidefinite, the property of the five-by-five table from the opening of this appendix; the next subsection builds both.) It doesn't, and the direction matters. Passing implies the five-by-five table, pinned in a way the next subsection explains, is PSD, and that implies S squared is at most eight; both are proved. The correlators of a PSD table like that are the correlators of some quantum strategy, by Tsirelson's representation theorem, which is proved here: the strategy is built on two systems of four levels each, or two levels each with complex amplitudes. The exact edge of the quantum set for this game was found by Lawrence Landau, whose 1969 Berkeley thesis was on the quantum theory of local observables, and Lluís Masanes, then at the University of Barcelona (next subsection), and that's proved here too. Failing says nothing about whether the play was quantum: the check is complete for the pinned matrix, the quantum set is wider than that, and quantum behaviour can fail it. Every one of the sixteen plans fails it, and I'll show you why. What it guarantees is in one paragraph, once the check is on the host machine.

24.4 The five-by-five, and what positive semidefinite means

The mathematics the check rests on is one idea: a table of numbers *could be a table of dot products*. Take a two-by-two symmetric table with a and c on the diagonal and b off it. It's positive semidefinite (PSD) when a and c are at least zero and a times c minus b squared is at least zero. Check two by hand: ones on the diagonal and one half off, the determinant is three quarters, passes; ones on the diagonal and 1.2 off, the determinant is negative, fails. The reading: PSD means the table could be the dot products of some arrows, and 1.2 is too big a dot product for two arrows of length one. Everything else is that idea, generalised to bigger tables.

Now the table. Rows and columns labelled I, A0, A1, B0, B1 (the identity, Alice's two questions, Bob's two). Diagonal all ones. The I row and column zero off the diagonal (that's "zero marginals": the table takes each answer alone to average to zero, a fair coin; the counts record no marginals and the check doesn't test this). A0 with A1 zero, B0 with B1 zero (no round asks Alice both questions, so no tally holds that average, and the table sets it to zero). Those zeros are choices. They pin the table to one slice, and I'll come back to what they cost. And the four Alice-Bob cells hold the four correlators E, which leaves a five-by-five with four free numbers in it. It splits into the one in the corner and a four-by-four block made of an identity and the two-by-two grid E of correlators, and PSD of that block is the same as saying *E shrinks every vector* (E is a contraction), which is three inequalities: one minus E00 squared minus E10 squared at least zero; one minus E01 squared minus E11 squared at least zero; and their product at least the square of (E00 E01 plus E10 E11).

Run it. All four correlators 0.6 except E11 at minus 0.6, S equals 2.4: the two diagonal terms are 0.28 and 0.28, the cross term zero, passes; above two, below 2.83. All four at one over root two (E11 negative): diagonals zero, cross term zero, passes with no slack, S equals two root two exactly, the 2.83; nudge any entry's size up and a diagonal goes negative. The all-ones plan (one, one, one, one): first diagonal is one minus one minus one, negative, *fails*, though S equals two and it's a perfectly legal classical play. The PR box (one, one, one, minus one): fails the same way. The check is a fence at the middle step, and a fence that keeps out every one of the sixteen plans: each has all four correlators at plus or minus one, so the first diagonal is always one minus one minus one. Of the classical plays, only diluted mixtures of plans get through.

That point, one over root two in size in every entry, is four real numbers, and no tally of rounds ever lands on it. A tally's correlators are ratios of whole numbers, and the point needs each of them to be one over root two, which would make root two a fraction, and you killed that fraction with a pencil in Section 6. That point and its mirror image, every sign flipped, are the only places the bound is reached (Remark 25.1 in the appendix). So a passing tally always has S squared strictly below eight. It can get as close as you like: the (17, 3) tally in the five examples below has S squared equal to 7.84, and tallies of the same shape with a fraction nearer one over root two get nearer still. Two root two is the least bound the check can promise, and no tally the check accepts ever reaches it. Hold on to that corner; it comes back.

Picture what quantum play can reach as a field, and every test as a fence. In 2007 Miguel Navascués, Stefano Pironio and Antonio Acín, in a paper called "Bounding the Set of Quantum Correlations" (NPA, after their initials), came at the edge from outside, resting on necessary conditions. In their terms a table like this one is the

first rung of a ladder: bigger tables of expected products, each PSD, each a necessary condition for quantum behaviour when its entries are left free, each rung tighter, and (their 2008 paper) converging to the quantum set (strictly, the set from commuting operators, which can differ from the tensor-product set in general scenarios). Each rung is a fence with the whole field inside it. The check's table is the first rung with the marginals and the two within-party entries pinned to zero. Pinning is a stronger demand than the rung makes, so passing puts you inside the first rung, and a quantum point can still fail. My fence cuts across the field. Lawrence Landau in 1988 and Lluís Masanes in 2003 came at the edge itself, for this two-questions-each case only. I'd have drawn a flat wall at 2.83. The exact boundary they found is a curved surface (the arc-sine form) touching 2.83 at one point. It's proved here: a table has a PSD completion exactly when its entries lie between minus one and one and their arcsines pass four inequalities, so those are exactly the quantum tables.

So the obvious next question: if every rung is tighter than the last, does some rung finally land exactly on the quantum set and stop? My first guess was yes, at least here. A 2026 preprint by Anubhav Chaturvedi, a quantum-information theorist at the University of Gdańsk, says no: no *finite* rung of the ladder captures the whole quantum set even in this simplest case. And for the four correlators alone, a 2008 paper by the same Navascués, Pironio and Acín works out that the first rung already lands on Landau and Masanes's curved edge. That looked like a contradiction until I saw what each one fences. Landau and Masanes draw the edge for the four correlators alone. Chaturvedi's is the full behaviours, each room's own odds included. So there's the corner again from the other side: tallies never reach it, and, if his proof holds, rungs never close on the field. I wanted that to be one fact, but it's two. Mine is root two again, about which points a count can hit. His is about the fences. His result backs, from outside, the caution this book keeps about the ladder, that no finite rung may close on the whole quantum set. It says nothing about my own check, though. My check's caution, that a quantum point can fail it, comes from the pinning, and his paper never pins anything.

24.5 The check, in integers

There are eight counts: for each of the four question pairs, one counting rounds where the answers matched and one where they differed. A round is one bump to one count; nothing about the round is tested, and no marginals are recorded. Then the check reads the eight and tests seven integer facts. For each pair, let N be same plus different and D be same minus different (so E is D over N). The seven: the four N are strictly positive (every pair was sampled; a pair with no rounds is rejected outright, and never read as a correlator of zero); A, which is N00 squared N10 squared minus D00 squared N10 squared minus D10 squared N00 squared, is

at least zero; B, the same with the other column, is at least zero; and with C equal to D00 D01 N10 N11 plus D10 D11 N00 N01, C squared is at most A times B. Those are the three contraction inequalities with the denominators cleared, and there's no division in them, no reals, no square root and no S. Try it: counts (same, different) of (4, 1), (4, 1), (4, 1), (1, 4) give E of 0.6, 0.6, 0.6, minus 0.6; N is five and D is three (minus three for the last); A is 175, B is 175, C is zero; passes. These counts are the first example above, the S equals 2.4 one, written in integers.

Question pair	same	different	total	difference
00	4	1	5	3
01	4	1	5	3
10	4	1	5	3
11	1	4	5	−3

First check the four totals: all are positive. Each column's remaining allowance is $625 - 225 - 225 = 175$. The cross term is $225 - 225 = 0$. Thus the final comparison is $0 \leq 175^2$, and the check passes. These steps verify the arithmetic; they supply no evidence that rounds were played.

The theorem chain, in words: a passed check gives PSD of the five-by-five built from the counts; contraction and PSD are equivalent, both directions; and PSD forces S squared at most eight. The names are in the statements below. The underlying algebra is one identity: four times the sum of the four correlators squared, minus S squared, is a sum of six squares, so if the row sums of squares are at most one, S squared is at most eight, and the point one over root two reaches it. So Tsirelson's bound comes out as *algebra*, with no Hilbert space (the mathematical home of quantum states) in it.

The chain goes below as three statements. The derivations are in Appendix 25, and the statements say what they prove.

Definition 24.1 (Correlators from the counts; the pinned matrix)**.** For eight counts and a question pair xy, let $E_{xy} = (\text{same}_{xy} - \text{diff}_{xy})/(\text{same}_{xy} + \text{diff}_{xy})$, read as 0 when both are 0. For reals E_{xy}, $M(E)$ is the symmetric five-by-five matrix on I, A_0, A_1, B_0, B_1 with ones on the diagonal, E_{xy} in the cells (A_x, B_y) and (B_y, A_x), and zero everywhere else (Definition 25.5). M is *PSD* when $v^{\mathrm{T}} M v \geq 0$ for every real vector v, and $S(E) = E_{00} + E_{01} + E_{10} - E_{11}$.

□ **Theorem 24.1** (The integer check implies the pinned matrix is PSD)**.** If eight counts pass the seven integer facts, then $M(E)$ for their correlators is symmetric and PSD.

Proof. The four positivity facts make every denominator positive, so each E_{xy} is D_{xy}/N_{xy}, and dividing the other three facts by positive squares turns them into the three contraction inequalities (Lemma 25.9). Contraction is equivalent to PSD of $M(E)$ by a Schur complement on the four-by-four block (Corollary 25.7). ∎

In the appendix this is Theorem 25.10, and Corollary 25.7 gives the equivalence of contraction and PSD in both directions.

Theorem 24.2 (PSD forces the Tsirelson bound)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$: if $M(E)$ is symmetric and PSD, then $S(E)^2 \le 8$. □

Proof. Call the four entries a, b, c, d, read across the rows. PSD means that no choice of five weights makes the matrix's weighted total negative. Choose weights $(0, 1, 0, -a, -b)$ in the order (I, A_0, A_1, B_0, B_1). The diagonal contributes $1 + a^2 + b^2$, and the two cross terms contribute $-2a^2 - 2b^2$. The total is $1 - a^2 - b^2$, so $a^2 + b^2 \le 1$. Weights $(0, 0, 1, -c, -d)$ give $c^2 + d^2 \le 1$. All four squares therefore add to at most two.

Now compare four times that sum with the squared score. Their difference is

$$4(a^2+b^2+c^2+d^2) - (a+b+c-d)^2 = (a-b)^2 + (a-c)^2 + (a+d)^2 + (b-c)^2 + (b+d)^2 + (c+d)^2.$$

To check it, each squared letter occurs three times on either expanded side; the cross terms are $-2ab, -2ac, +2ad, -2bc, +2bd, +2cd$ on both. The difference cannot be negative because it is a sum of squares. Thus the squared score is at most four times two, which is eight. ∎

This is Theorem 25.8 in the appendix.

Corollary 24.3 (A passed check means the bound)**.** If eight counts pass the seven integer facts, their correlators satisfy $S^2 \le 8$. In fact $S^2 < 8$, and passing tallies come as close to 8 as you like. □

Proof. Theorem 24.1 makes the pinned matrix of these four correlators PSD, and Theorem 25.8 in the appendix, which holds of every such matrix, gives $S^2 \le 8$. The score is a ratio of whole numbers, and no such ratio squares to 8, so the bound is strict. The tallies whose four correlators are $n/q, n/q, n/q, -n/q$ with $2n^2 \le q^2$ pass, and their scores $4n/q$ come as near $2\sqrt{2}$ as you like. ∎

Remark 24.1 (Sound, and incomplete)**.** The pinned matrix is stricter than the quantum question it is usually read against. PSD of $M(E)$ gives the correlators a PSD completion, since the zeros are one choice of the two free entries. The converse fails. The all-ones plan has a PSD completion, and every deterministic plan is a quantum strategy; yet its minor on rows A_0, B_0, B_1 is $1 - 1 - 1 = -1$, so $M(E)$ is not PSD there, and a tally of one matching round per pair gives $A = 1 - 1 - 1 = -1$ in the integer check, so the check fails. The second failure is a named theorem on the machine: the tally of one matching round per pair is refused forever (Proposition 24.6). Every deterministic plan fails the same way. The tally of one round per pair fails the check for each of the sixteen plans, and so does any tally of one plan replayed over any number of rounds (a pair left unsampled fails the positivity facts outright), because no deterministic plan's pinned matrix is PSD. So a pass proves the bound, and a failure proves only that the tally's pinned matrix isn't PSD or a pair went unsampled; it says nothing about whether the play was classical or quantum. The tally from the start of this appendix, written down at no cost, is exactly such a tally: it reaches $S = 2$ and the check throws it out.

None of that algebra is new, and I don't want it to read as if it were. Mnacho Echenim and Mehdi Mhalla, computer scientists at Grenoble, give a detailed treatment of the CHSH and Tsirelson bounds (2023). Navascués, Pironio and Acín already analyse this very matrix by Schur complement and discuss the level above it. What I think is mine here is smaller: an integer decision procedure on the counts, and a proved link from "the check passed" to PSD to the bound.

The picture under the check is older still. Tsirelson showed, in a longer paper of 1987 after his 1980 note, that the correlations two separated quantum parties can reach are exactly the tables of inner products of unit vectors, one vector for each question on each side. With the marginals pinned to zero the pinned matrix is that picture, and by its Schur complement it is PSD exactly when the two-by-two table of correlators, read as a matrix, has operator norm at most one. The physicists Reinhard Werner and Michael Wolf's survey of Bell inequalities (2001) sets out that correlation-matrix view for two-party inequalities in general.

24.6 The check, earned on the host machine

Now put it on the machine. A check on the small machine is a property: a checker, which is a little program that says yes or no, a meaning, which is a statement, and a proof that the two agree. The version of the small machine with its property language left open (Section 15, Theorem 16.11) takes any property language with an exact equality test whose properties come with those three things, and for every one of them a flag raised from a clean start stands on a passing CHECK of one

claim, a later COMMIT of the same claim with its counter untouched in between, and a later CERTIFY, the three acts costing one each. For the game, pack the eight counts into one counter, as a list, the way the sorted-list machine packs a list into one number (Section 15). Let the checker be the seven integer facts read off the unpacked counts. And let the meaning be what the facts are for: every pair was sampled, and the pinned five-by-five is positive semidefinite. Then the proof the generic machine asks for, that the checker says yes exactly when the meaning holds, is the chain above with the denominators cleared in both directions, and a flag raised on that machine from a clean start means: these counts, at this edition of their counter, passed the check, somebody committed to that with nothing changed underneath, and the correlators they give satisfy $S^2 \leq 8$. The theorems below state that exactly, and they're stronger than the chain above in one way I want to say once and plainly. The earlier theorems said that a passed check makes the pinned matrix PSD. The check is also complete for it: it passes *if and only if* every pair was sampled and the pinned matrix is PSD, because clearing the denominators works in both directions. The statement is over the machine of Definition D.2, the small machine with a counter for every number, with PCHSH in place of its one property, Slot. A CHECK reads one counter, which is why the eight counts have to travel in one.

Definition 24.2 (The packed tally, and the property PCHSH)**.** Read a whole number as a list in the list code (0 is the empty list, and $2^x(2y+1)$ is x followed by the list y), and take as its *tally* the first eight entries, in the order $\text{same}_{00}, \text{diff}_{00}, \text{same}_{01}, \text{diff}_{01}, \text{same}_{10}, \text{diff}_{10}, \text{same}_{11}, \text{diff}_{11}$, a missing entry reading 0. Every number is some tally. The property PCHSH has the checker "the tally passes the seven integer facts" and the meaning "every question pair was sampled, and the pinned matrix $M(E)$ of the tally's correlators is symmetric and PSD."

Theorem 24.4 (The check is exact)**.** (1) The seven integer facts hold of a tally if and only if every pair was sampled and $M(E)$ is symmetric and PSD. (2) For every whole number v, the checker of PCHSH says yes at v if and only if the meaning holds of v's tally. (3) Reading the number that packs a tally gives that tally back. (4) If the facts hold, the score of the tally satisfies $S^2 \leq 8$ and $|S| \leq 2\sqrt{2}$; in fact $S^2 < 8$ and $|S| < 2\sqrt{2}$, and passing tallies come as close to 8 as you like. □

Proof. (1) The forward half is Theorem 24.1. For the backward half, PSD gives the three contraction inequalities (Corollary 25.7); with every N positive, multiplying them through by the squares of the denominators gives $A \geq 0$, $B \geq 0$ and $C^2 \leq AB$, which is Lemma 25.9 read the other way. (2) is (1) with the packing, since every

number is a tally. (3) is the code and its decoder being inverse. (4) is (1) forward, then Theorem 24.2; the strict form and the closeness are Corollary 24.3. ■

□ **Theorem 24.5** (A raised flag stands on the check)**.** On the machine above, let τ be a trace from a clean start s_0 that ends with the flag up. Then, for some counter c,

$$\tau \;=\; pre \cdot \text{CHECK PCHSH } c \cdot mid_1 \cdot \text{COMMIT PCHSH } c \cdot mid_2 \cdot \text{CERTIFY} \cdot post,$$

and with v the value of c after pre: the tally of v passes the seven integer facts; every pair was sampled and the pinned matrix is symmetric and PSD; its score satisfies $S^2 \le 8$ and $|S| \le 2\sqrt{2}$, and in fact $S^2 < 8$, though passing tallies come as close to 8 as you like; and c still held v when the COMMIT ran. For any stored program run from a fresh start (counters set, every version 0, line 1, the table and channel empty, the trap and the flag down, the ledger at 0) that raises the flag, the trace it executed contains a CHECK PCHSH c that passed, and the tally c held when it ran passes the seven integer facts, has every pair sampled and the pinned matrix symmetric and PSD, and has $S^2 \le 8$ and $|S| \le 2\sqrt{2}$. Conversely, from a fresh start in which a counter c holds the packing of a tally t, the three instructions CHECK PCHSH c, COMMIT PCHSH c, CERTIFY raise the flag if and only if t passes the check, and then the ledger reads exactly 3; if t fails, the trap goes up at the CHECK and no number of further steps raises the flag.

Proof. A raised flag from a clean start has the earned chain (Theorem 15.11, which holds for this machine as for the small one): a passing CHECK, then a COMMIT of the same claim with the counter's edition unchanged, then a CERTIFY. A passing CHECK of PCHSH at c means the checker says yes at c's value, which is the meaning by Theorem 24.4(2), and the bound is (4) of the same theorem. An unchanged edition means an unchanged value. For the converse, the CHECK passes exactly when the tally passes; if it does, COMMIT and CERTIFY find their facts and the three acts cost three; if it fails, the trap latch goes up, and no later move can raise the flag. A stored program stops at the trap; an explicitly supplied instruction trace may still incur its defined charges (as in Theorem 15.20). ■

So the one thing the machine does with the check, as this appendix's opening said, is that a raised flag can stand on it. Five tallies, each a theorem with its counts written out, show the check isn't vacuous and where its edges are.

Proposition 24.6 (Five tallies)**.** Counts are given as (same, diff) for the pairs 00, 01, 10, 11. (4,1), (4,1), (4,1), (1,4) has $S = 12/5$: the check passes, the flag rises, the ledger reads 3. (17,3), (17,3), (17,3), (3,17) has $S = 14/5 = 2.8$, just under $2\sqrt{2}$, with $S^2 = 7.84$: it passes, the ledger reads 3. (9,1), (9,1), (9,1), (1,9) has $S = 16/5$ with $S^2 > 8$: refused forever, whatever follows. (1,0), (1,0), (1,0), (0,1), the PR box, has $S = 4$: refused forever. (1,0), (1,0), (1,0), (1,0), the all-ones classical plan, has $S = 2$: refused forever. □

Proof. A pair's correlator is $(\text{same} - \text{diff})/(\text{same} + \text{diff})$, and S adds the first three and subtracts the fourth. $(4, 1)$ gives $3/5$ and $(1, 4)$ gives $-3/5$, so $S = 12/5$; the seven facts are the worked tally after Corollary 25.11, with $A = B = 175$ and $C = 0$, so the check passes, and Theorem 24.5 gives the flag and the ledger at 3. $(17, 3)$ gives $7/10$ and $(3, 17)$ gives $-7/10$, so $S = 14/5$ and $S^2 = 196/25 = 7.84$; with $N = 20$ in every pair and $D = 14, 14, 14, -14$, $A = B = 400 \cdot 400 - 196 \cdot 400 - 196 \cdot 400 = 3200$ and $C = 14 \cdot 14 \cdot 400 - 14 \cdot 14 \cdot 400 = 0$, so the seven facts hold and the same theorem applies. $(9, 1)$ gives $4/5$, so $S = 16/5$ and $S^2 = 256/25 > 8$; the PR box gives $1, 1, 1, -1$ and $S = 4$; both fail the check, because a tally that passes has $S^2 \leq 8$ (Corollary 25.11), and a failed check traps at the CHECK, after which no step raises the flag (Theorem 24.5). The all-ones plan gives $N = D = 1$ in every pair, $S = 2$, and $A = 1 - 1 - 1 < 0$, the worked tally's second computation, so it fails the check and is refused the same way. ■

The last one is the check being stricter than the bound, as a named theorem: a perfectly legal classical play with $S = 2$ is turned away.

What the theorem doesn't say stays what it was. The tally is *placed*. Nobody plays it on the machine: the theorem starts from a counter that already holds the packing, and no instruction plays a round, so a program can put any code in a counter at no cost. There are no marginals. The pinned matrix is stricter than the quantum set, so failing says nothing about whether the play was quantum, and there is no margin for sampling noise at Tsirelson's point.

Writing a tally costs nothing. Each count is a counter, a round is an INC on one of them, and INC costs nothing, so a pile of trials costs nothing to write down (what it spends is fresh states, in a counter that never runs out of numbers); the packing into one counter is a program's business, and in the theorem the packed counter is simply there at the start. What costs is the claim about the pile. On the ledger, statistics cost nothing; the claim about them is what costs.

So what the check guarantees fits in one paragraph. If it passes, then every question pair was sampled, the four ratios from the counts satisfy three polynomial inequalities, so the pinned five-by-five is PSD, so S squared is at most eight. It

does *not* guarantee that the tally came from honest play (a program can INC whatever counts it likes at no cost, including four rounds that put S at four). It does not guarantee any marginal is zero (no marginals are recorded and none are tested). It does not put the players' state behind the counts. What it does give is a quantum strategy with the same correlators: a passing tally's correlators have a PSD completion, and every point with one is the correlator table of a strategy on two four-level systems. And it fails the tally of every single plan, honest classical play included. It has no margin for sampling noise either. Honest play at Tsirelson's point aims at correlators of plus or minus one over root two, and there every pair of squared correlators the check adds up comes to exactly one, the edge of what it accepts. So an honest tally aimed at that point sits right on the line, and ordinary luck over a finite number of rounds pushes it over most of the time. I simulated it, drawing each round's answers at Tsirelson's correlators: the check fails 92 percent of the time at 10 rounds per question pair, 87 percent at 100, and between 85.4 and 85.9 percent from 1,000 to 100,000. In those simulated trials, fair coins passed every time from 100 rounds per pair on, and failed about 2.5 percent of the time at 10. I saw that in the trials, and it's no guarantee about any one tally. The limit has a closed form. At Tsirelson's point the correlator matrix, read as a two-by-two, turns arrows without stretching them (it's a mirror), and every one of its entries wobbles by the same amount, independently. To first order the largest singular value of the noisy matrix is one plus the larger eigenvalue of $(G + G^{\mathrm{T}})/2$, where G is the wobble turned back by that mirror, a two-by-two of independent Gaussian entries of equal size, and the check passes only when that eigenvalue is at most zero. Write that symmetric matrix's eigenvalues as $u \pm \sqrt{v^2 + w^2}$, with u, v, w independent and of equal spread (scale them to standard normals). Both are at most zero exactly when $u \le -\sqrt{v^2 + w^2}$. Now $v^2 + w^2$ has the exponential law with mean 2, so the chance is $\frac{1}{2}\,\mathbb{E}\left[1 - e^{-u^2/2}\right] = \frac{1}{2}\left(1 - 1/\sqrt{2}\right) = (2 - \sqrt{2})/4$. So honest play at Tsirelson's point fails with probability $(2 + \sqrt{2})/4 \approx 0.854$ in the long run. The arithmetic is on the page; the first-order step is not proved here, and the simulation is what backs it.

The check can also allow a tolerance. For $\varepsilon = p/q$ it tests the same seven facts in integers with the 1 in the contraction inequalities replaced by $(1 + \varepsilon)^2$. At $\varepsilon = 0$ it is the check above. A pass means the two-by-two correlator matrix has operator norm at most $1 + \varepsilon$, and $S^2 \le 8(1 + \varepsilon)^2$: divide every correlator by $1 + \varepsilon$ and the tolerance check becomes the exact one, so the chain above applies to the shrunken table. A tally of 100 rounds per pair with correlators $0.72, 0.70, 0.72, -0.72$ fails the exact check and passes at $\varepsilon = 1/10$. In the same kind of simulation, with $\varepsilon = 3/\sqrt{n}$ for n rounds per pair, honest Tsirelson play failed it less than once in ten thousand trials at every size from 100 to 100,000 rounds per pair: under once in a hundred thousand at 100, five at 1,000, eight at 100,000, close to the long-run

rate the same first-order picture gives, about nine in a hundred thousand. In that Gaussian calculation the exact expression is $1 - \Phi(6) + e^{-9}\Phi(3\sqrt{2})/\sqrt{2}$, where Φ is the standard normal cumulative probability. Dropping its tiny tail corrections gives $e^{-9}/\sqrt{2}$. This is the asymptotic calculation under independent sampling, not a finite-sample confidence guarantee. What the tolerance gives up shows in the bound: a pass caps S at $2\sqrt{2}\,(1+\varepsilon)$, a little outside Tsirelson's number. The exact check remains the default. With p and q fixed for a machine instance, the tolerance check also runs as CHECK on the same versioned registers: its CHECK, COMMIT, CERTIFY chain certifies exactly when the integer tolerance test passes, costs three on success, and traps permanently on failure. From a clean start, every raised flag has that earned chain behind it, with the same register value at CHECK and COMMIT, and establishes the relaxed bound above: it's Theorem 24.5 again, with the tolerance test as the checker and "every pair sampled and operator norm at most $1+\varepsilon$" as the meaning, the scaling just described being the proof that they agree. The check certifies a *shape* of the tally. What the shape means is a question for whoever supplied the rounds.

And notice which way the fence faces. The most practical use anyone has found for a Bell test, certified randomness, needs the opposite bound. Stefano Pironio and colleagues (2010) argued, and backed with an experiment, that a Bell violation, with settings the devices can't choose, certifies that the outputs are random, and a group of quantum cryptographers, Rotem Arnon-Friedman, Frédéric Dupuis, Omar Fawzi, Renato Renner and Thomas Vidick (2018), made the accounting tight with what they call entropy accumulation. Under those assumptions, a predetermined local output strategy passes a properly calibrated test only within its stated false-acceptance probability, so it's the cleanest case I know, outside this book, of a check that can fail, separating output that was established from output that was printed. That needs S bounded from below, with an error probability. This check bounds S from above and fails every single plan, so it's a fence that keeps out the super-quantum and the fixed plan. As a nonlocality test it's no use. The deployed version exists, too: Gautam Kavuri and colleagues, physicists at NIST and the University of Colorado (2025), run a public randomness beacon whose outputs each trace back through a Bell test to a public record, the nearest working system I know to a certified claim written down for good. And records cut both ways there. Jonathan Barrett, Roger Colbeck and Adrian Kent, working on quantum cryptography (2013), pointed out that a device which keeps its own memory of earlier rounds can leak it through outputs it makes public later, so reusing a device can undo a guarantee the user thought was settled. A permanent record in the wrong box is an attack.

24.7 A rational stand-in for an irrational number

One useful rational bound: 5657 over 2000, which is 2.8285, and two root two is 2.82842... It's the four-decimal round-up, chosen so that rational arithmetic can test "above the Tsirelson bound" without square roots, and it's safe because 5657 squared is 32,001,649 and eight times 2000 squared is 32,000,000, so the fraction sits above root eight by about seven hundred-thousandths. It's used in a few rational theorems. It is *not* used in the integer check, and the real-number theorems use root eight.

24.8 Two more tests, as mathematics

The second rung: a nine-by-nine table on I, the four questions, and the four Alice-Bob *products*, with the correlators and five extra free numbers for three- and four-body entries. Its two four-body entries are tied, one the negative of the other. That tie is a constraint, $\langle \{A_0, A_1\} \otimes \{B_0, B_1\} \rangle = 0$: the zero entries of the first rung don't imply it, and a classical strategy breaks it. So the second rung is level 1+AB on that slice, and that's what I mean by level 1+AB everywhere in this book. Its PSD is again equivalent to a contraction-like condition. The check for it proves something stronger, that the matrix is positive definite, by a cascade of two Schur steps down to a four-by-four that Sylvester's criterion closes. Sylvester's name on a test about matrices fits in a way I enjoy, because the word *matrix* is his. In 1850, in the *Philosophical Magazine*, the English mathematician James Joseph Sylvester used it for an "oblong arrangement of terms" out of which you cut square blocks and take their determinants, and *matrix* was already an old word for the place something grows from. The test does exactly that: it cuts the top-left square blocks out of the big grid, one row deep, two, three, four (the leading minors), and asks each one's determinant to be positive. Was he the first to state the test? The name doesn't settle that; names stick to people for all kinds of reasons, and I'd treat this one as a lead. There's also a fact that's easy to list without noticing what it means: with all five extra numbers at zero, the condition certifies a strict *subset of the classical set* (the ball where the four correlators' squares sum to at most one), forcing S at most two; it can't certify anything above the classical line. To get a point above two you have to supply the extras (S equals 2.4 with one extra at minus one half). Appendix 25 has the algebra.

The two sizes are one test. A single positive-semidefinite predicate takes the size as a parameter, and the five-by-five and the nine-by-nine are it at two sizes.

Definition 24.3 (The size-generic PSD test)**.** A *claim* is a top index n and a matrix M indexed by naturals. It is *realizable* when $M_{ij} = M_{ji}$ for all $i, j \le n$ and $\sum_{i \le n} \sum_{j \le n} v_i M_{ij} v_j \ge 0$ for every real vector v. The CHSH claim of E is top index 4 with the matrix $M(E)$ of Definition 24.1; the level-1+AB claim uses top index 8 and the nine-by-nine matrix.

Corollary 24.7 (The level-1 test is the size-five case)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$: the three contraction inequalities hold if and only if the CHSH claim of E is realizable. The same holds at size nine for the level-1+AB contraction condition and its claim. □

Proof. Top index 4 means the five indices $0, 1, 2, 3, 4$. The double sum visits each of their twenty-five cells once, multiplies it by its row and column weights, and adds the results. That is precisely the weighted-total test for the five-by-five matrix. The completed-square calculation in Corollary 25.7 equates that test with the three contraction inequalities. Top index 8 similarly visits the eighty-one cells of the nine-by-nine matrix. Theorem 25.12 writes its total as three squares plus the residual and proves that its test is exactly the residual condition. The same definition therefore asks the same question at either size. ■

The third thing is the *elliptope*, the set of correlators for which *some* choice of the two missing entries (A0 with A1, B0 with B1) makes the five-by-five PSD. Every one of the sixteen plans is inside it, mixtures stay inside, every point inside obeys $S^2 \le 8$, Tsirelson's point is inside and reaches it, the PR box is out, and it's strictly bigger than the classical set. There's a certificate checker for it with a soundness proof, in two branches: a strict Sylvester check with the completion supplied, and a rational factorisation certificate of the completed matrix, checked in integers. What's proved is that a pass puts the correlators inside the set and that no choice of inputs gets the PR box through, and, by computation, that some named points pass, the all-ones plan and a point that the arc-sine boundary puts exactly on the quantum edge among them. And every point with a rational PSD completion has such a certificate: peel the completed matrix a row at a time, as in Section 25.8, but keep each pivot instead of taking its square root; a zero pivot forces its row to zero, and every number that comes out is rational. The certificate checker is sound and, on rational completions, complete, as mathematics; none of the small machine's properties is the elliptope check.

24.9 The quantum set

Three descriptions of the same set have been running through this appendix: the tables some quantum strategy can produce, the tables whose five-by-five matrix has a PSD completion, and the tables inside Landau and Masanes's curved edge. They are the same set, for strategies on finitely many levels, and the next subsection takes away the "finitely". Each piece is taken in turn below, because the constructions are the part worth seeing.

Quantum tables are inside the completion set. A strategy's correlator is the inner product of two unit vectors: Alice's observable applied to the shared state, and Bob's applied to it. Four unit vectors always have a PSD table of inner products, so every quantum table has a completion, and Tsirelson's bound follows from that: $S^2 \leq 8$ for every strategy, and its level-one moment matrix passes the PSD test with the marginals in. Quantum mechanics uses complex amplitudes, and the complex case goes through by a plain trick: a complex vector of length n is a real vector of length $2n$, its real parts followed by its imaginary parts, and the real part of a complex inner product is the ordinary inner product of those longer real vectors. So the real argument runs unchanged. The two-qubit strategy written out in Appendix 25 reaches $2\sqrt{2}$ in both readings.

The completion set is inside the quantum tables. This direction is Tsirelson's representation, and the construction is his. Take a table with a PSD completion. A PSD matrix is a table of inner products of vectors, always: peel off one row at a time, and a zero on the diagonal forces its whole row to zero. So the completion hands over four unit vectors, a_0, a_1 for Alice and b_0, b_1 for Bob, with $a_x \cdot b_y$ equal to the table's entries, and they fit in three dimensions: slide one of the two entries the table leaves open to the end of the range that keeps the completion PSD, and there the completion is singular, which puts the four vectors in a space of three dimensions. Section 25.8 has the picture behind the factorization and the measurements. Now comes the trick. There are three real four-by-four matrices, built from the two-by-two ones that swap two entries or flip the sign of one, that each square to the identity and anticommute in pairs: multiply any two of them in either order and the answers differ only in sign. Use a vector's three components to mix those three matrices, and you get an observable that squares to the identity, a legitimate measurement. Give Alice and Bob a maximally entangled pair of four-level systems, and the correlator of two such observables comes out as exactly the inner product of their vectors. So the strategy built from the four vectors produces the table, for both real and complex amplitudes. Four levels on each side is always enough, and with complex amplitudes two are.

The curved edge is the same set again. Each correlator is the cosine of an angle, the angle between one of Alice's vectors and one of Bob's. Walk the loop a_0, b_0, a_1, b_1

and back to a_0: the angle across any one leg can't exceed the sum of the other three, because angles on a sphere obey the triangle inequality. Flip one of Alice's vectors to its opposite and walk the loop again, then do the same with her other one, and between them you get the other four inequalities, with 2π in them. Written in arcsines, those eight inequalities are Landau and Masanes's four. Going the other way, any four angles that pass them can be met by actual vectors: choose an angle between Bob's two that meets both pairs of constraints, build each of Alice's from the two angles it has to make with them, and check that the determinant that says such a triangle exists comes out nonnegative. So a table has a PSD completion exactly when its arcsines pass the four inequalities, and those are exactly the quantum tables. The point $(3/5, 4/5, 4/5, -3/5)$ sits on the edge, because $\arcsin\frac{3}{5} + \arcsin\frac{4}{5} = \frac{\pi}{2}$ by the 3-4-5 triangle, and so does Tsirelson's point. The interval construction, including both endpoint cases, is pictured in Section 25.9.

24.10 Any number of levels, and one converse that fails

Nothing here cares about dimension. Quantum mechanics also allows systems with infinitely many levels, and the arguments don't need finitely many. Whatever the dimension, a shared state v gives an expectation on the operators the players measure with: $x \mapsto \mathrm{Re}\,\langle v, x\,v\rangle$. That one line is the whole step. The proofs use only what that expectation always has: you can add, multiply, scale and take adjoints, the product spreads over sums, and the expectation of x^*x is never negative. Each measurement is its own adjoint with the expectation of its square at most one, which covers unsharp measurements too, and Alice's commute with Bob's. That last demand is weaker than putting them on two separate systems, so the result covers the wider, commuting model as well. On that footing Tsirelson's sum of squares goes through, and the CHSH value is at most $2\sqrt{2}$. The same nonnegative form, with each diagonal entry raised to one, completes the correlator table to a PSD matrix, so the four arcsine inequalities hold, and with a unit the full first-level moment matrix, marginals and all, is PSD. The other direction needs no help from infinity: every point inside the arcsine edge is reached on two four-level systems.

Also, the finite-mixture theorem proves only one direction, that a finite local model satisfies the bound, and its converse as stated is false: a table with $S = 0$ that no local model produces. Fine's own theorem, in the form that speaks of correlations, is proved separately as an equivalence: a table of four correlators comes from a local model exactly when each is between -1 and 1 and all eight CHSH inequalities hold. Section 25.10 pictures the finite distribution. And one more thing about tallies, which the guarantee paragraph above already used: a tally loaded at no charge is not thereby classical (the four rounds with S equal to four). The correlators (one, one, one, minus one) are the PR box, which no classical model has as its averages,

though coin-flip play lands on that four-round tally one time in sixteen; the classical vertex this appendix used, which fails the check like every plan, is (one, one, one, one).

That covers the game, the number, the two walls, and the check. The derivations come in full in the next appendix, Appendix 25; what all this costs against physics was Section 19.

CHAPTER 25

The CHSH derivation, in full

Appendix 24 tells the story; this appendix does the algebra. Each item gives the statement with its hypotheses and the steps you need to check it. Nothing here says that a matrix is a physical state. Where quantum strategies come in, I say exactly how they stand to the matrices.

25.1 Setting and notation

Start with four piles of tally sheets, one for each pair of settings Alice and Bob can be given. On each round their answers either agree or disagree. Give agreement a score of $+1$ and disagreement a score of -1. In each pile, add the scores and divide by the number of rounds. That pile's average is its *correlator*. It says how strongly the two answers agree for that pair of settings.

Definition 25.1 (Settings, outcomes, correlators)**.** Alice receives a setting $x \in \{0,1\}$ and Bob a setting $y \in \{0,1\}$. Each answers a bit, recoded as an outcome $a, b \in \{+1,-1\}$ (bit 0 becomes $+1$, bit 1 becomes -1). The correlator E_{xy} is the average of the product ab over the rounds with settings (x,y). The CHSH quantity is

$$S = E_{00} + E_{01} + E_{10} - E_{11}.$$

I collect the four correlators into the 2×2 matrix

$$E = \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix},$$

whose rows are indexed by Alice's setting and whose columns by Bob's. The rows of E are (E_{00}, E_{01}) and (E_{10}, E_{11}); the columns are (E_{00}, E_{10}) and (E_{01}, E_{11}).

Definition 25.2 (Counters)**.** There are eight counts: for each setting pair, same_{xy} counts rounds whose two answer bits matched and diff_{xy} counts rounds whose bits

differed. A round adds one to exactly one of the eight. Write

$$N_{xy} = \text{same}_{xy} + \text{diff}_{xy}, \qquad D_{xy} = \text{same}_{xy} - \text{diff}_{xy}, \qquad E_{xy} = \frac{D_{xy}}{N_{xy}}.$$

A matched round contributes $ab = +1$ and a differing round $ab = -1$, so D_{xy} is the sum of the products and D_{xy}/N_{xy} is their average. Since $|D_{xy}| \le N_{xy}$, every correlator from counters lies in $[-1, 1]$. When a bucket is empty, its correlator is defined as 0. The score is $E_{00} + E_{01} + E_{10} - E_{11}$.

25.2 The classical bound

Write a fixed plan on a card: Alice's answer to setting zero, her answer to setting one, and Bob's two answers. Each answer is a sign, plus or minus. Before multiplying anything, look at Bob's pair. Either the signs agree or they differ. If they agree, adding them gives plus or minus two and subtracting them gives zero. If they differ, the sum is zero and the difference is plus or minus two. One of those two quantities vanishes; the other cannot exceed two in size. The next calculation uses exactly that split.

Definition 25.3 (Finite factorizable model)**.** Four real correlators $E_{00}, E_{01}, E_{10}, E_{11}$ are factorizable when there are a natural number $\lambda_{\max}$, weights $p(\lambda)$ and response signs $A(x,\lambda)$, $B(y,\lambda)$ for $\lambda = 0, \dots, \lambda_{\max}$ such that: $0 \le p(\lambda) \le 1$; $\sum_{\lambda=0}^{\lambda_{\max}} p(\lambda) = 1$; every $A(x,\lambda)$ and every $B(y,\lambda)$ is -1 or $+1$; and for all x, y, $E_{xy} = \sum_{\lambda=0}^{\lambda_{\max}} p(\lambda) A(x,\lambda) B(y,\lambda)$. In words: a hidden card λ is drawn with probability $p(\lambda)$, and each player reads an answer off the card using only their own setting. Only the correlators are modelled.

□ **Theorem 25.1** (The classical bound for a finite local model)**.** If E is factorizable, then $-2 \le E_{00} + E_{01} + E_{10} - E_{11} \le 2$.

Definition 25.4 (A tally consistent with one local plan)**.** For bits $a_0, a_1, b_0, b_1 \in \{0, 1\}$ and a counter record w, w is locally consistent with (a_0, a_1, b_0, b_1) when, for each setting pair (x, y): if $a_x = b_y$ then $\text{diff}_{xy} = 0$, otherwise $\text{same}_{xy} = 0$; and every bucket is sampled, $N_{00}, N_{01}, N_{10}, N_{11} > 0$.

Theorem 25.2 (A tally above 2 is not one replayed strategy)**.** Let w be a counter record whose rational score $S(w)$ is greater than 2. Then for all bits a_0, a_1, b_0, b_1, w is not locally consistent with (a_0, a_1, b_0, b_1). Every locally consistent record has $S(w) = 2$ or $S(w) = -2$; the first statement is that fact turned around. □

25.3 Tsirelson's bound from algebra

Now put the cards aside and look just at four numbers in the square. I want to bound their three-plus-one-minus score from their squared sizes. The useful trick is to write the proposed allowance minus the squared score as a pile of squares. A square cannot be negative. If the difference really is that pile, the score cannot outrun the allowance.

Lemma 25.3 (The six-square identity)**.** For all real a, b, c, d, □

$$4(a^2+b^2+c^2+d^2) - (a+b+c-d)^2 \\ = (a-b)^2 + (a-c)^2 + (a+d)^2 + (b-c)^2 + (b+d)^2 + (c+d)^2.$$

Theorem 25.4 (Row-bound form of Tsirelson's bound)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$ with $E_{00}^2 + E_{01}^2 \le 1$ and $E_{10}^2 + E_{11}^2 \le 1$, $S^2 \le 8$. The absolute form $|S| \le \sqrt{8} = 2\sqrt{2}$ follows. □

Theorem 25.5 (Tightness)**.** The point $E_{00} = E_{01} = E_{10} = 1/\sqrt{2}$, $E_{11} = -1/\sqrt{2}$ satisfies both row bounds and has $S = \sqrt{8}$. □

Remark 25.1 (Where equality can happen)**.** Equality $S^2 = 8$ under the row bounds forces equality in both steps of the proof above: the six squares must all vanish, which gives $a = b = c = -d$, and the rows must be tight, which gives $2a^2 = 1$. So the point of Theorem 25.5 and its negative are the only places the bound is reached. A tally's correlators are ratios of whole numbers and $2a^2 = 1$ has no such solution, so no tally reaches the bound; the integer check's passing tallies stay strictly below it and come as near as you like. (Take rational a, b with $a^2 + b^2 = 1$, from a Pythagorean triple, and the correlators $(a, b, b, -a)$: they pass the check of Section 25.5 with no slack, and $S = 2(a+b)$, which gets as close to $2\sqrt{2}$ as you like as a/b approaches 1.)

25.4 The zero-marginal 5×5 moment matrix

The four averages fill only the Alice–Bob part of a larger table. Give that table five labels: a constant reading 1, Alice's two possible readings, and Bob's two. Each cell records an average product of its row and column readings. A reading multiplied by itself is 1, so the diagonal is all ones. The four averages measured fill the cross-player cells and their reflected partners.

Definition 25.5 (The pinned level-1 matrix)**.** Take rows and columns I, A_0, A_1, B_0, B_1. Put 1 on the diagonal and the four correlators in the Alice–Bob cells, and set the four one-party entries (each player's own average answer) and the two within-party entries (A_0 against A_1, B_0 against B_1) to zero. The resulting matrix is:

$$M(E) \;=\; \begin{array}{c} \\ I \\ A_0 \\ A_1 \\ B_0 \\ B_1 \end{array} \begin{array}{c} \begin{array}{ccccc} I & A_0 & A_1 & B_0 & B_1 \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{pmatrix} \end{array}$$

□ **Lemma 25.6** (Schur complement, the standard form)**.** Let P be symmetric positive definite. The symmetric block matrix $\begin{pmatrix} P & Q \\ Q^{\mathrm{T}} & R \end{pmatrix}$ is PSD if and only if $R - Q^{\mathrm{T}} P^{-1} Q$ is PSD.

Definition 25.6 (Column contractive)**.** Write $p = 1 - E_{00}^2 - E_{10}^2$, $q = 1 - E_{01}^2 - E_{11}^2$ and $s = E_{00}E_{01} + E_{10}E_{11}$. The correlators are column contractive when

$$p \geq 0, \qquad q \geq 0, \qquad p\,q - s^2 \geq 0.$$

□ **Corollary 25.7** (The equivalence)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$, the symmetric matrix $M(E)$ is positive semidefinite if and only if the correlators are column contractive.

Theorem 25.8 (PSD implies Tsirelson)**.** If the pinned matrix $M(E)$ is symmetric and positive semidefinite at $E_{00}, E_{01}, E_{10}, E_{11}$, then $S^2 \le 8$ and $|S| \le \sqrt{8}$. □

25.5 The integer check

The machine has whole-number tallies. The three conditions above were written using averages, which are fractions. You can test the same conditions without ever dividing: first make sure each denominator is positive, then multiply it away. Multiplying by a positive number cannot turn a passing inequality into a failing one or a failing one into a passing one.

Lemma 25.9 (The seven facts are the three conditions with denominators cleared)**.** Assume $N_{00}, N_{01}, N_{10}, N_{11} > 0$ and put $E_{xy} = D_{xy}/N_{xy}$. Then, with p, q, s as in Definition 25.6, □

$$p = \frac{A}{N_{00}^2 N_{10}^2}, \qquad q = \frac{B}{N_{01}^2 N_{11}^2}, \qquad s = \frac{C}{N_{00}N_{01}N_{10}N_{11}},$$

$$pq - s^2 = \frac{AB - C^2}{(N_{00}N_{01}N_{10}N_{11})^2}.$$

Because every denominator is positive, $0 \le A \iff p \ge 0$, $0 \le B \iff q \ge 0$ and $C^2 \le AB \iff s^2 \le pq$.

Theorem 25.10 (Soundness of the integer check)**.** If a tally w passes the seven-integer check, then its four correlators $E_{xy} = D_{xy}/N_{xy}$ are column contractive. Consequently its pinned matrix is positive semidefinite. □

Corollary 25.11 (A passed check implies the Tsirelson bound)**.** If a tally w passes the seven-integer check, the correlators of w satisfy $S^2 \le 8$. It is the composition of Theorem 25.10, Corollary 25.7 and Theorem 25.8. □ ■

Two of the seven facts carry the bound by themselves. With every pair sampled, $0 \le A$ and $0 \le B$ are $p \ge 0$ and $q \ge 0$, so the four squared correlators add to at most 2 and Lemma 25.3 gives $S^2 \le 8$. The fact $C \cdot C \le A \cdot B$ is what positive semidefiniteness adds, and the bound doesn't use it. For the form this takes on the small machine, where a raised flag has to stand on a passing check, see Appendix 24.

25.6 Level 1+AB

The first table had one place for each player's answer. Enlarge it by giving each Alice–Bob product a place too. There are four such products, so the five places become nine. A cell still asks for the average product of its row and column labels. Some of those products reduce to entries I already have; five more numbers are needed for the particular filling below. Call them γ_1 through γ_5. The longer labels say which products they stand for.

Definition 25.7 (The level-1+AB matrix)**.** The 9×9 real matrix on rows and columns $I, A_0, A_1, B_0, B_1, A_0B_0, A_0B_1, A_1B_0, A_1B_1$, with the following entries:

	I	A_0	A_1	B_0	B_1	A_0B_0	A_0B_1	A_1B_0	A_1B_1
I	1	0	0	0	0	E_{00}	E_{01}	E_{10}	E_{11}
A_0	0	1	0	E_{00}	E_{01}	0	0	γ_1	γ_2
A_1	0	0	1	E_{10}	E_{11}	γ_1	γ_2	0	0
B_0	0	E_{00}	E_{10}	1	0	0	γ_3	0	γ_4
B_1	0	E_{01}	E_{11}	0	1	γ_3	0	γ_4	0
A_0B_0	E_{00}	0	γ_1	0	γ_3	1	0	0	γ_5
A_0B_1	E_{01}	0	γ_2	γ_3	0	0	1	$-\gamma_5$	0
A_1B_0	E_{10}	γ_1	0	0	γ_4	0	$-\gamma_5$	1	0
A_1B_1	E_{11}	γ_2	0	γ_4	0	γ_5	0	0	1

Write a vector as $(v_I, v_{A_0}, v_{A_1}, v_{B_0}, v_{B_1}, v_{00}, v_{01}, v_{10}, v_{11})$, where v_{xy} is the coordinate of A_xB_y. Put

$$\begin{aligned} L_1 &= E_{00}v_{00} + E_{01}v_{01} + E_{10}v_{10} + E_{11}v_{11}, \\ L_2 &= E_{00}v_{B_0} + E_{01}v_{B_1} + \gamma_1 v_{10} + \gamma_2 v_{11}, \\ L_3 &= E_{10}v_{B_0} + E_{11}v_{B_1} + \gamma_1 v_{00} + \gamma_2 v_{01}. \end{aligned}$$

Definition 25.8 (The residual)**.**

$$\begin{aligned} R = {} & v_{B_0}^2 + v_{B_1}^2 + v_{00}^2 + v_{01}^2 + v_{10}^2 + v_{11}^2 \\ & + 2v_{B_0}(\gamma_3 v_{01} + \gamma_4 v_{11}) + 2v_{B_1}(\gamma_3 v_{00} + \gamma_4 v_{10}) \\ & + 2\gamma_5(v_{00}v_{11} - v_{01}v_{10}) - L_1^2 - L_2^2 - L_3^2. \end{aligned}$$

The condition called column contractivity at level 1+AB is $R \geq 0$ for all six real inputs.

Theorem 25.12 (PSD at level 1+AB)**.** For all real E_{xy} and $\gamma_1, \dots, \gamma_5$, the matrix of Definition 25.7 is positive semidefinite if and only if the residual R is nonnegative for every choice of its six real inputs. □

25.7 The elliptope completion

Go back to the two within-player cells I filled with zero. Leave them blank this time. Keep all four measured cross-player averages fixed, and ask whether there is any way to fill those two blanks that makes the table pass. A failure with zeros no longer settles the question; another filling may work.

Definition 25.9 (Completion and membership)**.** Complete the matrix of Section 25.4 by setting the within-party entries to free reals: x in the (A_0, A_1) cells and y in the (B_0, B_1) cells, marginals still zero. The correlators are elliptope realizable when some x, y make the completed matrix symmetric and PSD.

$$M(E; x, y) \;=\; \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & x & E_{00} & E_{01} \\ 0 & x & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & y \\ 0 & E_{01} & E_{11} & y & 1 \end{pmatrix}$$

Its quadratic form at $v = (v_0, \dots, v_4)$ is $\sum_k v_k^2 + 2xv_1v_2 + 2yv_3v_4 + 2E_{00}v_1v_3 + 2E_{01}v_1v_4 + 2E_{10}v_2v_3 + 2E_{11}v_2v_4$. The pinned matrix is the case $x = y = 0$, so PSD of $M(E)$ implies membership.

Definition 25.10 (The two branches)**.** The positive-minor branch: with the completion supplied as buckets and all entries scaled to integers by a positive common factor K, the four leading minors of the 4×4 block on A_0, A_1, B_0, B_1 are strictly positive. The factorization branch: with the completion and a rational certificate supplied (six subdiagonal entries over a common positive denominator, four pivots $d_k \geq 0$ over another), the ten entry equations of $M = LDL^{\mathrm{T}}$ hold in cross-multiplied integer arithmetic. The full gate is their disjunction.

□ **Theorem 25.13** (Soundness of the gate)**.** If the full gate returns yes on a counter record, any completion buckets and any certificate, then the four correlators of the record are elliptope realizable. On the PR box tally the gate returns no for every choice of the remaining arguments.

25.8 The four-vector construction

Put two equal-length pencils down with their ends together, and draw a line along one of them. Keep that pencil still and turn the other. Use the square corner of a sheet of paper to bring the moving pencil's tip straight across to the line, and mark where it lands. The mark moves from a full pencil-length forward, through the joined ends when the pencils make a square corner, to a full pencil-length backward. Call those readings 1, 0 and -1. That is the signed shadow I want. Between those positions the reading is the cosine of the angle. A dot product of unit vectors measures that shadow.

25.9 The arcsine boundary, in both directions

Keep Bob's two directions hinged at one end. Alice's first direction has to make a specified angle with each. How far can Bob's hinge open while that is possible? Hold Alice's direction upright and sweep each of Bob's around it at its required angle. Point them towards the same side to close the hinge as far as it will go; point them towards opposite sides to open it as far as it will go. There is a whole interval of openings between those limits. Alice's second direction asks for another interval.

25.10 The finite local model, in both directions

Make sixteen cards, each with four boxes labelled A_0, A_1, B_0, B_1, and put every possible arrangement of plus and minus signs on them, one arrangement per card. Draw a card before choosing which setting to ask each player about. Alice reads her chosen box; Bob reads his. Neither needs the other's setting. How often each card is drawn is the only choice left in this model.

25.11 Scope of the constructions

The constructions above pin down the correlators. What follows is where they stop: they aren't a completeness claim for the integer check, they say nothing about full behaviours with marginals, and they say nothing about data from a lab.

The arcsine boundary. For two settings and two outcomes per side, Tsirelson, Landau (1988) and Masanes (2003) give theorems characterizing the correlators reachable by quantum measurements: $|\arcsin E_{00} + \arcsin E_{01} + \arcsin E_{10} - \arcsin E_{11}| \leq \pi$ and its sign variants. The characterization holds as an if and only if with the completion set and, through Tsirelson's representation, with the quantum set. The point $(3/5, 4/5, 4/5, -3/5)$ is on the boundary: $\arcsin \frac{3}{5} + \arcsin \frac{4}{5} = \frac{\pi}{2}$ by the 3-4-5 triangle, so the sum is π; Tsirelson's point is on it too. The necessary direction holds in any dimension, for any expectation on an algebra of observables in which Alice's commute with Bob's.

Equality with the quantum set. That the completion set of Section 25.7 is exactly the set of quantum correlators is Tsirelson's representation theorem, and it is proved in finite dimension: for both real and complex amplitudes, with two four-level systems enough for every point, and two two-level ones with complex amplitudes. That PSD of the level-1 moment matrix is necessary for quantum behaviour is proved for strategies in finite dimension, with real amplitudes and with complex ones, and in any dimension, marginals included, for any expectation on an algebra of observables.

Achievability. Theorem 25.5 is a point of $\mathbb{R}^4$. A state and measurements that reach it are explicit: the two-qubit state $(|00\rangle + |11\rangle)/\sqrt{2}$ with $A_0 = Z$, $A_1 = X$, $B_0 = (Z + X)/\sqrt{2}$ and $B_1 = (Z - X)/\sqrt{2}$ has correlators $1/\sqrt{2}$, $1/\sqrt{2}$, $1/\sqrt{2}$, $-1/\sqrt{2}$ and $S = 2\sqrt{2}$, and every strategy in finite dimension has $S^2 \leq 8$, with real amplitudes and with complex ones. Both are arithmetic about the model. How close a real laboratory comes, and what that costs in loopholes closed, is the story told in Appendix 24.

Completeness of the checks. The 1+AB cascade and the elliptope gate are proved sound. The cascade has no proved converse; the gate's factorization branch accepts every rational PSD completion, by the row-peeling of Section 25.8 with each pivot kept. The level-1 integer check is the exception: on the small machine the same seven integer facts hold exactly when every pair was sampled and the pinned matrix is positive semidefinite (Theorem 24.4), so that check is complete for what it tests. The 1+AB cascade is strict and misses boundary points. The two 1+AB checks don't sit inside one another. The full check accepts the $12/5$ tally above, which the zero-moment check refuses, and it refuses the tally with $E_{00} = 1$ and the other three correlators 0, where $H_{11} = 0$, which the zero-moment check accepts.

Finite levels. For full behaviours, marginals included, the NPA levels are relaxations. Chaturvedi (arXiv 2607.14569, 2026) argues, in a preprint, that no finite NPA level characterizes the complete quantum set even in this simplest Bell scenario. I haven't gone through that proof, and nothing here rests on it.

Statistics and physics. Nothing here says how many rounds make a tally informative, and nothing says the counters came from honest play; a program can INC whatever counts it likes.

25.12 Prior work

Tsirelson's 1980 paper gives the bound $2\sqrt{2}$ for quantum correlations, and his theorem says that quantum two-outcome correlators are inner products of real unit vectors. Landau (1988) and Masanes (2003, arXiv quant-ph/0309137) each give a condition that, by their proofs, characterizes the quantum correlators exactly for two settings and two outcomes. Wehner (2006, arXiv quant-ph/0510076) gets Tsirelson bounds, CHSH among them, from a semidefinite program on the 4×4 Gram matrix of those vectors. Navascués, Pironio and Acín (2007, arXiv quant-ph/0607119; 2008, arXiv 0803.4290) built the hierarchy of moment-matrix tests; their 2008 paper applies a Schur complement to the level-1 matrix of this scenario, recovers the arcsine condition, and discusses the 1+AB level. I went back to each of those papers for these points, and that is what I found in them. The results are theirs. Two of them are proved again here, in finite dimension: Tsirelson's representation and the arcsine characterization; the rest of their proofs are not redone. Echenim and Mhalla give a detailed derivation of Tsirelson's bound (arXiv 2306.12535), and exact rational sum-of-squares certificates go back at least to Peyrl and Parrilo (2008). So the moment-matrix route to $2\sqrt{2}$ is standard. What is specific here is narrower: an integer certificate computed from outcome counts, and the argument from "the counts pass" to PSD of the pinned matrix and on to $S^2 \leq 8$. I did not find that chain elsewhere, and I make no claim beyond it. On the host machine, the same chain is earned: a raised flag from a clean start stands on a passing check of the packed counts, and the check passes exactly when the pinned matrix is positive semidefinite and every pair was sampled (Appendix 24, Theorems 24.4 and 24.5). The converse direction of the integer check, integers from PSD, is proved there too, and there too I make no claim beyond the chain.

CHAPTER 26

Methodology note

Everything this book claims as its own stands or falls on the arguments written in it; what it borrows is cited to the people who proved it, and the physics enters as a named premise. If a proof on the page is wrong, the result is wrong, and nothing else I could tell you would rescue it.

Language models were my hands, as I said in Section 24: they did symbol-pushing, drafted code and helped work through derivations. I have to answer for what stayed. Start with the definitions and follow how each step leads to the conclusion. If I counted a finite construction, its cases belong on the page. If I claimed something for every run, the argument has to explain why the next step preserves it. A successful example cannot carry that claim, and a name attached to a proof cannot fill a missing step.

The same demand applies to the picture around the mathematics. A definition has to say what I meant, and a model of a device has to fit that device. Those questions belong beside the premises and the argument, where you can examine the fit.

CHAPTER 27

Rows of the claim ledger for the passages above

The book's claim ledger listed these claims, each with the kind of claim it is and where it lived.

Claim	Kind	Home
The small machine runs the eight-state machine through a window, move for move, paying exactly its charges	proved (from live states; the setup's claim can't fail)	Section 16.4, Theorem 16.13
The small machine as a whole is not merge-priced	proved	Section 16.4, Theorem 15.22
A decision tree has at most two to its depth leaves	definition; proved	Section 9, Definition 11.1, Lemma 11.1
Paid narrowing, counted	proved with the total of the charges, which equals the counter's rise by conservation; both are proved (premises: a covering tree, paid steps)	Section 9, Theorem 11.2
Structural entitlement for every certified Thiele-complete machine: stronger test, earned chain, index bits at most the record moves, ledger rise at least 3	proved (premises: a clean start, and M1 to M6)	Section 16.4, Theorem 16.14

Claim	Kind	Home
A floor with no supplied tree, from the run's own CHECKs	proved (answers taken at the rivals as they start)	Section 16.4, Theorem 16.15
Structural entitlement with the narrowing made by the run, over any Thiele machine whose test steps are named: every drop made by a test the run passed, the exact product count, index bits at most k times the cost	proved (premises: test steps charged at least one, a prior containing the start, a 2^{-k} share kept by each test)	Section 9, Theorem 11.3
The same for every certified Thiele-complete machine: index bits at most k times the passing CHECKs, at most k times the ledger's rise; the rounded sum with no share assumed; the share can't be dropped	proved (premises: a clean start, the record up at the end, a prior containing the start, the share for part (4))	Section 16.4, Theorem 16.16, Proposition 16.17
What the record adds to the price bound from a clean start: the earned chain and a floor of 3	proved	Section 16.4, Theorem 16.18
The weighted count and the partition form	proved; the weights are whole numbers, and no probabilities enter	Section 16.4
The eight-start member, and a search through n bits with up to 16 questions	proved; the 16 is the machine's fact table	Section 16.4, Proposition 16.19, Theorem 16.20
The covering hypothesis can't be dropped	proved, by a counterexample on the small machine	Section 16.4

Claim	Kind	Home
The time tax: no program of zero-cost two-counter moves decides, in fewer than m moves, $A \geq m$ from its final line and B, one paid move decides it, and three paid moves certify it	proved, for programs of INC and DEC	Section 16.4, Theorem 16.21
The lists, tree and fibres are read off the run	not claimed, except in the worked members	Section 16.4
The latch on a Turing machine, a register machine, L	definition; proved (and the relation is an equivalence)	Section 11, Definition 12.2, Theorem 12.1
Thiele-complete means a universal base whose moves cost nothing and can't touch the record, every other move costing exactly one, a record that stays up, blank starts, and some run whose record ends up from one start and down from another; nothing in it says what the record is about	definition	Section 12.2, Definition 12.3
Every Thiele-complete machine obeys the toll, raises its record only by a mark costing exactly one, counts its marks on its ledger, pays at least one on any run that raises the record, and runs every two-counter program	proved, for every machine and every interface meeting the definition	Section 12.2, Theorem 12.2

Claim	Kind	Home
A clock is not Thiele-complete, nor is a machine whose record never rises, and each of the four clauses is needed	proved	Section 12.2, Observation 12.3, Theorem 12.4
Every Thiele-complete machine hides its record and its ledger from its window, with a gap of one unit, and one is reached	proved	Section 12.2, Theorem 12.5
Every universal base becomes Thiele-complete by adding marks, with no claims and no checker	proved	Section 12.2, Theorem 12.6
Certified Thiele-complete is exactly Thiele-complete with clean starts, the earned chain and a claim that can fail; the inclusion is strict	proved	Section 16.3, Theorem 16.3
Halving pricing bounds weighted conditional entropy; an infinite landing pile admits no finite halving price	proved; average loss still depends on the distribution	Section 12, Theorems 13.1 and 13.2
The nat family: each candidate decider misses program two in the instance built from it	proved	Section 13, Theorem 14.1
Recursion, Rice and halting for L	definition; proved	Section 13, Definition 14.1, Theorem 14.2
Platform and quote; inclusion check; bounds fragment	definition	Section 17, Definition 17.4, Definition 17.5, Definition 17.1

Claim	Kind	Home
The proof-carrying checker is the verification condition	proved	Section 17, Theorem 17.1
Ecosystem, proliferation, pointer; copied and relied on, independent carriers; observation game	definition	Section 18, Definition 18.1, Definition 18.2, Definition 18.3
Independent copies on every state, with two observers or more: the event is equivalent at any two states	proved	Section 18, Theorem 18.1
In the specified replicated-header toy, gas used and finality are copied, and the chosen action maps rely only on finality	proved for those maps	Section 18, Theorem 18.2
In the toy, “certified” is the unique pointer	proved	Section 18, Theorem 18.3
The survey; the swap shows the verdict follows the observer maps	proved	Section 18, Theorem 18.4
One blind observer stops proliferation	proved	Section 18, Theorem 18.5
Agreement doesn’t buy permanence; durable views that report the event do	proved	Section 18, Theorem 18.6
No stored fact is forged on a clean-start trace	proved on the small machine, under the stated premises	Section 15, Theorem 15.14
Bounded table keeps facts and traps when full	proved on the small machine, under the stated premises	Section 15, Theorem 16.2

Claim	Kind	Home
Certified Thiele-complete means a universal base that costs nothing and can't touch the record, a record that stays up once it's up, an earned record with a checker proved to mean its claims, exact prices of one per check, commit and certificate, and a claim whose check can fail	definition	Section 15, Definition 16.1
A machine certified Thiele-complete over a claim language fixed first is certified Thiele-complete, and from a clean state its raised record stands on a claim whose meaning held when it was checked and when it was committed	proved	Section 15, Theorem 16.6
Every certified Thiele-complete machine obeys the toll, runs every two-counter program, counts its record moves on its ledger, pays at least three on any run from a clean state that ends with the record up, has a check that fails and a run that certifies, and from a clean state raises its record only by a CERTIFY	proved, for every machine and every interface meeting the definition	Section 15, Theorem 16.5

Claim	Kind	Home
A clock (every move charged, any reading or any latch as the record) obeys the toll with a two-counter machine running inside it and is not certified Thiele-complete; nor is any machine whose every move costs something	proved, directly from the rules	Section 15, Observation 16.7
Each clause does work: a paid latch meets the universal base and exact prices clauses and isn't certified Thiele-complete, and a silent machine meets three clauses and fails non-vacuity	proved	Section 15, Theorem 16.8
The four clauses of certified Thiele-complete are independent: for each, a Thiele machine with a two-counter machine inside meets the other three, fails it, and is certified Thiele-complete through no interface	proved	Section 15, Theorem 16.9
Certified Thiele-complete at any unit: with record moves at one fixed price $c \geq 1$ the class is the unit class with its prices multiplied by c, and the clock fails at every unit	proved	Section 15, Theorem 16.10

Claim	Kind	Home
The small machine, the same machine over any property language with an exact equality test, a checker proved to say yes exactly when the property holds and some property true of one number and false of another, and the sorted-list machine are certified Thiele-complete	proved on the small machine and on the generic machine	Section 15, Theorem 16.11
Every certified Thiele-complete machine hides its record and its ledger from its base window	proved, for every machine and every interface meeting the definition, directly from the rules	Section 15, Observation 16.12
The calibration is a unit conversion	definition; proved	Section 19, Definition 19.1, Theorem 19.1
Structural dissipation pricing (a whole-unit floor on collapsing instructions, separate from Landauer's principle); dissipation at most cost	assumed (premise)	Section 19, Premise 19.2
A collapse needs the merge clause: INC A raises "$A \geq 1$" one way, one-to-one and costing nothing; on a finite machine a one-way step always merges	proved	Section 19, Premise 19.2
The heat is fixed by the chosen gap	definition; proved	Section 19, Definition 19.3, Theorem 19.2

Claim	Kind	Home
A driven reset with rates in detailed balance, its gap raised from 0 to D in N equal steps, pays between $kT\ln 2 - kTe^{-D/kT}$ and $kT\ln 2 + D/(2N)$ in work	definition; proved	Section 19, Definition 19.4, Theorem 19.3
The simpler host runs any small-machine program as a guest, and halts with any two-counter program	proved on the small machine, by construction: its guest step runs the guest's own step as a primitive; the guest program is held as a list	Appendix 21.1, Theorem 21.1
From a loaded host, or any state where the two agree, the simpler host's mirror always equals the guest's flag	proved on the small machine, directly from the rules	Appendix 21.1, Observation 21.2
From a host that agrees, the simpler host's step collects the guest's toll; on every trace its ledger covers the guest's	proved on the small machine	Appendix 21.1, Theorem 21.3
The simpler host's mirror rises only in a guest step that passed CERTIFY	proved on the small machine, for every sequence of host moves	Appendix 21.1, Theorem 21.4
The simpler host's own instructions never touch the guest	proved on the small machine	Appendix 21.1, Theorem 21.4
From a loaded host, the simpler host's raised mirror means the guest's own run checked, committed and certified the same claim	proved on the small machine	Appendix 21.1
The simpler host is a Thiele machine	proved on the small machine	Appendix 21.1, Theorem 21.5

Claim	Kind	Home
The simpler host's ledger equals the guest's	not claimed; a guest step with budget b costs b whatever the guest instruction cost, so its ledger only covers the guest's; U's is exact	Appendix 21.1, Theorem 21.3
Rice's theorem on the host machine, for properties that depend on everything a program ends with (registers, table, channel, trap, ledger, flag)	proved; undecidable in the reduction sense of Theorem 15.5	Appendix 21.2, Theorem 21.6
Rice's theorem for two-counter programs, from the clean start	proved; the fact table isn't compared	Appendix 21.2, Theorem 21.7
s-m-n, and a universal host program for host programs	proved, with no premise on register 2	Appendix 21.2, Theorem 21.8
Kleene's recursion theorem on the host machine	proved; the fixed point is of the function	Appendix 21.2, Theorem 21.9
Kleene's recursion theorem up to the record on the host machine: stopping, register 0, trap latch, ledger exactly, flag, number of facts, whether the channel is empty	proved; the host program simulates the program T that carries out the transformation, marks and all, and pays for the whole record in one go once it has computed	Appendix 21.2, Theorem 21.10
A fixed point of full behaviour (every register, what the facts name) on the host machine	refuted for registers, and for the fact table, the channel and the versions	Appendix 21.2, Theorem 21.11
No inside decider with a representable flip, and no host decider, for functional properties of host programs	proved (the recursion premise is a theorem there)	Appendix 21.2, Theorem 21.12

Claim	Kind	Home
Rice's theorem for two-counter programs on every input (stop, counters, trap latch, ledger, flag, and what each fact and the channel are about)	proved	Appendix 21.2, Theorem 21.13
Kleene's recursion theorem for two-counter programs, inputs written as powers of two, for the function	proved; refuted once the recorded facts count	Appendix 21.2, Theorem 21.14
Kleene's recursion theorem for two-counter programs on plain inputs	refuted: a carried-out transformation has no fixed point	Appendix 21.2, Theorem 21.15
Every universal base lifts with an exact checker, a nontrivial claim and room for a fact; finite simulations lift after grouping	proved under the stated interface conditions	Appendix 22, Theorems 22.1 and 22.3
Finite branching, state-independent moves and an indiscriminate record order give the stated obstructions to lifting	proved	Appendix 22, Theorem 22.2

Claim	Kind	Home
Every box with a universal base is a slice of its lift with the window claims and room for a fact: forgetting the panel runs as the box runs while the trap is down, reaches exactly what the box reaches, isn't moved by CHECK, COMMIT or CERTIFY, and gives neither the flag nor the ledger, the chain's ledger exactly 3 above the empty run's; with no room in the table no interface certifies the lift; a Thiele-complete machine's computing moves are its moves that cost nothing, through every interface, and its window is a slice onto two counters; a machine run as one rule has no universal base, no base by simulation and no universal-base slice, and is a Thiele machine only as a clock or with a reading that never rises	proved; that the slice runs as the box runs and that the record moves leave it alone, directly from the rules, and the converse's universal base and step-for-step clauses directly from the interface	Appendix 22, Definition 22.1, Theorem 22.4

Claim	Kind	Home
With the program in the state, a reading is kept by every operation exactly when it never comes down from any state; a cell some program clears is a simulated bit	proved, directly from the rules	Appendix 22, Observation 22.5
The seven models compute the same numerical relations under their encodings; one counter has decidable halting	proved	Appendix 22, Theorems 22.6 and 22.7
The source compiler preserves halting and numerical output through the host, two-counter guest and universal program	proved; final two stages not executed	Appendix 23, Theorem 23.1
Executable implementations agree on the tested whole-state transitions	tested, not a guarantee about untested inputs or physical hardware	Appendix 23
The integer check makes the pinned matrix PSD	definition; proved	Appendix 24, Definition 24.1, Theorem 24.1
PSD forces $S^2 \leq 8$	proved (algebra only)	Appendix 24, Theorem 24.2
A passed check means $S^2 < 8$, and no smaller bound holds	proved: $S^2 < 8$, with passing tallies as close to 8 as desired	Appendix 24, Corollary 24.3
A PSD pinned matrix has a PSD completion	proved	Appendix 24, Remark 24.1
Every deterministic plan fails the check	proved; no such plan's pinned matrix is PSD	Appendix 24, Remark 24.1
The level-1 test is the size-five case	definition; proved	Appendix 24, Definition 24.3, Corollary 24.7

Claim	Kind	Home
The 1+AB matrix ties its four-body cells, the constraint $\langle\{A_0, A_1\} \otimes \{B_0, B_1\}\rangle = 0$; the level-1 pinnings don't imply it, and a classical strategy breaks it	proved	Appendix 24, Appendix 25
A tally above two, with every pair asked, isn't one plan replayed	proved	Appendix 25, Theorem 25.2
A tally above two proves nonlocality; a pass proves the data quantum	not claimed	Appendix 24
A finite local model meets the bound	proved; Fine's theorem in correlation form is proved as an iff, and the converse of the single-bound statement is false	Appendix 25, Theorem 25.1, Section 25.10
The elliptope gate is sound; the PR box fails it	proved	Appendix 25, Theorem 25.13
The arc-sine boundary: a table has a PSD completion, and so is quantum, exactly when its arcsines pass the four inequalities (if and only if)	proved	Appendix 25
Every PSD completion comes from a quantum strategy, and the completion set is exactly the set of quantum correlator tables in finite dimension, real or complex	proved	Appendix 25

Claim	Kind	Home
A two-qubit strategy reaches $2\sqrt{2}$; every strategy in finite dimension, with real or complex amplitudes, has $S^2 \leq 8$ and a PSD level-1 moment matrix; every deterministic plan is such a strategy	proved	Appendix 25
The integer route to the bound is new mathematics	not claimed; the route is NPA's	Appendix 24
The integer check passes exactly when every pair was sampled and the pinned matrix is PSD	proved	Appendix 24, Theorem 24.4
Honest play at Tsirelson's point fails the exact check about 85 percent of the time	simulated, with a first-order limit of $(2+\sqrt{2})/4$; no theorem	Appendix 24
The check with a tolerance ε is the exact check at $\varepsilon = 0$; a pass means operator norm at most $1+\varepsilon$ and $S^2 \leq 8(1+\varepsilon)^2$	proved, including the earned CHECK	Appendix 24
A flag raised on the host machine from a clean start stands on a passing CHECK of the packed counts, a COMMIT and a CERTIFY; the chain run from a fresh start certifies for 3 exactly when the check passes	proved	Appendix 24, Theorem 24.5

Claim	Kind	Home
Five tallies: 12/5 and 14/5 certify; 16/5, the PR box and the all-ones plan are refused forever	proved	Appendix 24, Proposition 24.6
The tally was played, or was honest, or puts a bound on nature	not claimed; it is loaded into a counter at no charge on the ledger	Appendix 24
Structural dissipation pricing	every collapsing instruction dissipates at least one unit, and dissipation is at most cost	Section 19, Premise 19.2
Covering tree, paid steps	a decision tree whose leaves cover the narrowing, and at least as many steps charged at least one as the tree is deep	entitlement, Theorems 11.2, 9.1 and 16.14
Share kept by a test	each passed test keeps at least a 2^{-k} share of the candidates, for a k fixed before the run; every run meets it for k its own biggest test	entitlement made by the run, Theorems 11.3 and 16.16
Lift interface	a claim language with an exact equality test, a checker proved to say yes exactly when the claim holds, a claim true at one loaded state and false at another, room for one fact	the lift, Theorem 22.1
The canonical reset's rates aren't in detailed balance at any gap or temperature; with rates that are, a fixed-gap relaxation moves toward $1/(1+e^{\Delta/k_BT})$, which is $1/5$ at the gap $2k_BT\ln 2$	proved	Section 19

CHAPTER 28

From the plain-text statement

These passages were taken out of `THIELE_MACHINE.txt` and are kept here as they stood.

```
BUILD THE BOX, THEN FORGET WHAT THE TASK DOES NOT ASK FOR

The smallest machine above an ordinary computing box has a lamp beside it
and a tally under it. Computing buttons follow the old rule, cost nothing,
and leave the lamp and tally alone. A mark leaves the computing picture
alone, adds one to the tally, and turns the lamp on if its named event holds.
Once on, it stays on. Begin with the lamp down and the tally at zero.

Run only computing buttons and the old machine runs exactly as before.
Forget the lamp and tally afterward and you recover its computation.
The larger description contains it. An event that holds somewhere gives
a visible difference between before and after marking even when the
computing picture stays the same. An ordinary computer can store the
picture, lamp and tally when their rules are effectively given.

Now let two builders implement those instructions differently. One keeps
every button press on a tape. The other uses a few extra counters. Ask
each for the computing picture, lamp and exact tally. Require computing
to follow the base rule, marks to latch exactly the named event without
changing the picture, and each move to add its zero-or-one charge to the
tally. The reading must obey the toll and permanence.

Read those three things from either implementation. After a computing
move, the picture follows the old rule; the free move cannot change the
permanent lamp; adding zero leaves the tally alone. After a mark, the
picture stays put, the lamp latches the event and the tally rises by one.
Those are precisely the simple box's rules. Repeat this for every button
in a list and the readings agree for the whole run. Any list reaching a
state of the simple box reaches an implementation state with those same
readings. Starting together gives all the same reachable triples.

That makes three stages: write a button list, run it on the builder's
```

box, then read the picture, lamp and tally. The last step forgets whatever extra history the builder kept. There is only one way to do it while preserving those three readings: report their actual values.

Try running that last step backward. Turn the simple box halfway around, then turn it again. The picture is back, and the lamp and tally never changed. Give the other box a separate counter that counts free turns. Its counter is now two larger. Which counter value should the simple box's one state turn into? Preserving the turns would require both. There is no such map. Forgetting the extra counter works; recovering it from the three readings does not.

To see what has to be kept, give two states the same future button list. If a required answer differs, those states cannot share one full state in a faithful implementation. That one state would have to give two answers to the same continuation. Include the empty list: reading now counts as a test. Group states only when every such test agrees. Once grouped, pressing a common button keeps them grouped, because that button can be put in front of every remaining test. Every grouping that preserves answers and steps must respect these tests. This is the coarsest one.

For the picture, lamp and tally, equal current readings already give identical future readings: their next values follow exactly the rules above. Different triples are distinguished by reading them now. Remove states that differ only in history these questions can never reveal, and every reachable triple has exactly one remaining state. Any two implementations with no such redundancy differ only in the names of those states. A button list reaching a state fixes its partner in the other implementation, so even that renaming is forced.

The questions do work here. Ask for the extra turn count and it has to survive too. Stop asking for the tally and repeated failed marks can become indistinguishable. If no mark event ever holds, a clean run never reaches a lamp-up state. The minimum concerns what can be reached and what the chosen questions can distinguish. It does not select the event. I develop this familiar idea from minimal automata for the actual lift, with the observations and accounting kept explicit.

Put a hidden switch behind a dark lamp. Pressing the button copies the switch to the lamp. Both boxes can look dark now and give different next answers. Keep only the lamp and the next step cannot be predicted from that picture. Keep the switch too and it can. The exact test is proved: equal pictures must g pictures under every permitted use of the same button, with explicit representatives and successor choices for the converse construction.

The arrows do not always choose which question matters. Nor does a finite watch prove that a lamp stays lit forever: a reset can sit just past the last tick anyone watched. These boundaries have checked counterexamples. Showing that real equipment follows the chosen model needs evidence about that equipment as well as a proof about the model.

The first two counts concern tasks with their own stated operations and recovery requirements. The third concerns a publication-file extension of the counter example. They count fibres of different projections.

First ask which checks passed on the current page. At working value b, let m(b) named predicates be true. Allow each successful check to be requested independently. Require an exact answer for every name since the last WRITE. One check gives two choices: run it or leave it alone. A second independent check doubles the choices, and each further check doubles them again. With m(b) available checks there are 2^m(b) possible selections. Realize any subset by writing b and running exactly those checks. Two different subsets cannot end at the same state because one named query would need opposite answers. The fibre above b therefore needs at least 2^m(b) states.

That bound is attainable. Use states (b,A), where A is a subset of the true check names at b. WRITE replaces b and clears A. A passing CHECK inserts its name. A failed CHECK and IDLE preserve the state. The empty start is correct; every command preserves the intended history reading. These are all the command cases, so the claim holds for every finite program. Every legal state is reachable by the realizing word already given. For finite B, the exact total is the sum of 2^m(b) over its working values.

Next keep the answers across wiped editions. On each edition, publish or leave it alone. Th two histories, the second four, the third eight. If n independent publication choices must remain readable when the workspace returns to one picture b, there are 2^n distinguishable histories above that picture. For one machine that must do this for every n, no finite fibre suffices: given any proposed size K, choose n with 2^n>K. That is an infinite reachable fibre forced by unlimited, recoverable edition history.

CHAPTER 29

From the README

These paragraphs were taken out of the repository README and are kept here as they stood.

```
The smallest machine standing above an ordinary computing box has a lamp beside it and a tall

Another builder can keep an extra tape or counter. Require the same base steps, exact event l

The construction goes from **button lists, to implementation states, to picture, lamp and tal

Which differences must survive? Give two states the same future button list. If any required

Ask for the extra turn count and more has to survive. Stop asking for the tally and less may

Put a hidden switch behind a dark lamp. The next button press copies the switch to the lamp.
```

CHAPTER 30

From the plain-text statement and the README, second pass

These passages were taken out of `THIELE_MACHINE.txt` and the README when the two were rebuilt as the argument, and are kept here as they stood.

```
WHAT A FREE MOVE CAN CHANGE

Slide a tile to the right without paying. Ask whether it is in the place
it just reached. The answer was no and is now yes. If the toll charged
every question, that slide could not be free. Do this with any changed
full state: ask whether it is the destination. Where states can be tested
for equality, charging every Boolean question would freeze every free
move. The definition has to name the reading its toll belongs to.

Now keep that reading permanent. A free move cannot raise it, because a
raise costs at least one. It cannot lower it, because a yes stays yes.
It must leave it alone. That explains the reading part of the horizontal
rule. The window adds another requirement: the next picture comes from
the present picture and the button pressed. Permanence by itself does
not choose a picture or make a paid move leave that picture alone.

There are two ways to state the same reading requirement for a sliced
machine. Leave it alone on horizontal moves and never lower it on vertical
moves, and it is permanent through all moves. Start instead with permanence,
the toll and free horizontal moves, and the two sentences above force
horizontal preservation. Neither route has to assume its own conclusion.


PUT THE PRICE ON THE BUTTON

Put price cards zero, one and two beside a button. If pressing it can raise
the lamp from some state, zero fails there. One works. Two works too, but
one is the least. If the button never raises the lamp from any state,
zero works. Give each button its least card and you have the least valid
schedule, one instruction at a time. Any other least schedule must have
```

the same cards: neither can be smaller than the other.

For a finite machine, inspect a list covering all its states to decide which card each button needs. On an infinite machine that search need not finish. For the lift there is a direct witness: if each mark's event holds somewhere, put the lamp down over that base state and press the mark. It raises the lamp, so one is necessary. Computing leaves the lamp alone, so zero suffices. A mark whose event never holds needs no positive toll; charging one for every use then comes from the chosen accounting rule.

The price stays on the instruction. It does not disappear when you press the same button with the lamp already on. Every run from no to yes contains a crossing and pays at least one; conversely a single crossing is already a run. The step toll and the floor from every start give the same minimum. That is a minimum in the definition's units. Measuring a device's heat requires a physical account of the device.

The allowed moves and the starting states matter. Correctly translating a program proves a simulation. To preserve a guarantee through every allowed host program, the proof must cover every host move from the stated starts. Paying to raise a record does not, by itself, establish what that record means.

This does not require the full states to form a literal product. It only requires the two readings and the displayed equations. If someone reads the hidden record and chooses a different instruction because of it, the two executions no longer receive the same horizontal word.

For the entry arrow u -> f(u) itself to share its destination with an arrow from H, the exact condition is f(u) in f(H). Surjectivity of f on H is sufficient, as is a single holding state. Neither is necessary for a particular entry to collide.

These are full configurations. A fixed array of bits can have many different configurations without acquiring a new memory cell on every tick.

A different accounting interface allows a source-dependent charge a(s), or c(s,i) when instructions are supplied. This extends the instruction-price definition above; it does not change the meaning of c(i) there. Its toll must be stated on each actual crossing. On no -> yes -> yes, charge one at no and zero at yes. A ledger with values zero and one obeys both updates. If instead every ti positive charge, a fixed full state could not hold its own ledger: it would require the same number to equal itself plus a positive amount.

```
The mathematics specifies the operations that must be preserved. A software
implementation must implement those operations through every allowed path.
Keeping a copy of their names or values alone does not establish that.

These bounds follow the information available and the question being asked.
An ordinary program given the live question can accumulate its answer too.
The earned machine adds the requirement that its permitted operations keep
the accepting record faithful to what actually passed.


Every flat machine is the shadow of a sliced machine, and the book builds the one that stan

## Which differences have to stay?

Slide a tile to the right for free. Ask whether it is in the place it just reached. The ans

Keep that reading permanent. A free move cannot raise it, because a raise pays. It cannot l

Put price cards zero, one and two beside a button. If it can raise the reading from some st

Now put a price rule beside the arrows. The abstract model's toll says that a move raising

Storing a representation is part of implementing this machine. Preserving those rules throu

The machine can be implemented with ordinary computation. What matters is which information
```

CHAPTER 31

Further reading

The works below were read for the book and are not named in its finished text or in this volume. They are kept here with their annotations, so that nothing read for the book is lost.

31.1 Structure, information and knowledge

Hans Albert. Wikipedia. Article. German philosopher (1921–2023), with Popper a leading critical rationalist; Traktat über kritische Vernunft (1968).

A. J. Ayer (1956). The Problem of Knowledge. Macmillan, London, p. 34. The other definition Gettier targets.

Max Black. Wikipedia. Article. The Russian-born British-American philosopher, at Cornell from 1946, and his 1952 paper on two spheres.

Roberto Bonola (1912). Non-Euclidean Geometry: A Critical and Historical Study of Its Development. Open Court; Dover reprint, 1955. The attempts on the parallel postulate from Proclus through the Islamic geometers and Saccheri, and Farkas Bolyai's letter to his son.

G. K. Chesterton (1929). The Thing. The fence passage is quoted in the American Chesterton Society's Lecture 57: The Thing.

Gösta Grahne (1998). Updates and counterfactuals. Journal of Logic and Computation 8(1), pp. 87–117. DOI: 10.1093/logcom/8.1.87. Revision of a static world versus update of a changing one.

Jonathan Jenkins Ichikawa and Matthias Steup (2001, substantive revision 2026). The Analysis of Knowledge. Stanford Encyclopedia of Philosophy. Entry. Dharmottara's mirage and Peter of Mantua's running Plato.

Rosalie Iemhoff (2008, substantive revision 2019). Intuitionism in the Philosophy of Mathematics. Stanford Encyclopedia of Philosophy. Entry. Kreisel's creating-subject axioms CS1–CS3 (1967), CS2 being that the creating subject never forgets what it has proven, and Troelstra's strengthening (1969).

Petrus Mantuanus (d. 1400). Logica. Medieval manuscripts in Oxford libraries, Bodleian Library. Catalogue entry. Peter of Mantua as a logician of the late fourteenth century.

Per Martin-Löf (1996). On the Meanings of the Logical Constants and the Justifications of the Logical Laws. Nordic Journal of Philosophical Logic 1(1), pp. 11–60. Lectures given in Siena in 1983. The passage on proof as an act is on p. 28. Lecture text.

Meta Platforms (2021). Introducing Meta: A Social Technology Company. Company announcement, 28 October 2021. Announcement.

Benjamin Morison (2014, revised 2026). Sextus Empiricus. Stanford Encyclopedia of Philosophy. Entry. The source for his dates, his medicine and his skepticism.

J. J. O'Connor and E. F. Robertson (n.d.). Theaetetus of Athens. MacTutor History of Mathematics. Entry.

J. J. O'Connor and E. F. Robertson (n.d.). Paul Richard Halmos. MacTutor History of Mathematics. Biography.

Benoît Rittaud and Albrecht Heeffer (2014). The Pigeonhole Principle, Two Centuries before Dirichlet. The Mathematical Intelligencer 36(2), pp. 27–29. Jean Leurechon's use of the principle in 1622.

Göran Sundholm (1998). Proofs as Acts and Proofs as Objects: Some questions for Dag Prawitz. Theoria 64(2–3), pp. 187–216. DOI: 10.1111/j.1755-2567.1998.tb00187.x.

Göran Sundholm (2014). Constructive Recursive Functions, Church's Thesis, and Brouwer's Theory of the Creating Subject: Afterthoughts on a Parisian Joint Session. In Constructivity and Computability in Historical and Philosophical Perspective, Logic, Epistemology, and the Unity of Science, Springer, pp. 1–35. DOI: 10.1007/978-94-017-9217-2_1.

Timothy Yenter and Ezio Vailati (2003, revised 2024). Samuel Clarke. Stanford Encyclopedia of Philosophy. Entry. Clarke as the British philosopher and theologian who defended Newton's natural philosophy in his letters with Leibniz.

31.2 Cost, records and thermodynamics

János Aczél, Bruno Forte and C. T. Ng (1974). Why the Shannon and Hartley entropies are "natural." Advances in Applied Probability 6(1), pp. 131–146. DOI: 10.2307/1426210. Ng's 1971 Waterloo doctorate under Aczél: Mathematics Genealogy Project.

Roberto M. Amadio, Nicolas Ayache, François Bobot, Jaap P. Boender, Brian Campbell, Ilias Garnier, Antoine Madet, James McKinna, Dominic P. Mulligan, Mauro Piccolo, Randy Pollack, Yann Régis-Gianas, Claudio Sacerdoti Coen, Ian Stark and Paolo Tranquilli (2014). Certified Complexity (CerCo). Foundational and Practical Aspects of Resource Analysis, FOPARA 2013, LNCS 8552, pp. 1–18. DOI: 10.1007/978-3-319-12466-7_1.

Walter J. Boyne. Wikipedia entry: United States Air Force officer, aviation historian, director of the National Air and Space Museum (acting from 1982, director 1983–1986). Entry.

Léon Brillouin (1951). Maxwell's Demon Cannot Operate: Information and Entropy. I. Journal of Applied Physics 22(3), pp. 334–337. DOI: 10.1063/1.1699951.

Arthur Charguéraud and François Pottier (2019). Verifying the Correctness and Amortized Complexity of a Union-Find Implementation in Separation Logic with Time Credits. Journal of Automated Reasoning 62, pp. 331–365; first published online in 2017. DOI: 10.1007/s10817-017-9431-7.

Gavin E. Crooks (1999). Entropy production fluctuation theorem and the nonequilibrium work relation for free energy differences. Physical Review E 60(3), pp. 2721–2726. DOI: 10.1103/PhysRevE.60.2721. A path's chance against its reverse's is e to the dissipated work.

Joe DeBlasio (2025). Discussion: removal of one-6962-log requirement. Chromium ct-policy mailing list, 17 November 2025. Post.

Jérémie Decouchant, David Kozhaya, Vincent Rahli and Jiangshan Yu (2022). Damysus: streamlined BFT consensus leveraging trusted components. Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22), pp. 1–16. DOI: 10.1145/3492321.3519568.

Encyclopaedia Britannica. Donation of Constantine. Web page. Composed in the eighth century; the consensus view puts it in the 750s or 760s.

Encyclopaedia Britannica. Leo Szilard. Web page. Born in Budapest in 1898; Berlin doctorate 1922 and the Institute of Theoretical Physics there.

Ethereum Foundation (n.d.). The Merge. ethereum.org roadmap page, on the transition of 15 September 2022. Page.

Ethereum Foundation (n.d.). Timeline of all Ethereum forks (Tangerine Whistle, EIP-150, and the Berlin upgrade, block 12,244,000). ethereum.org. Page.

Ethereum Foundation. Devcon editions (Devcon 0, Berlin 2014; Devcon 1, London 2015; Devcon 2, Shanghai 2016). devcon.org. Page.

Ethereum Foundation. Ethereum execution specifications, Prague fork, src/ethereum/forks/prague/vm/instructions/storage.py and vm/gas.py, at commit ecb68f21a7d28db101c769878184f6facb319da1. Repository.

Federal Aviation Administration (2008). Revisions to Cockpit Voice Recorder and Digital Flight Data Recorder Regulations. Final rule, Federal Register 73(46), 7 March 2008, pp. 12542–12573. The two-hour cockpit voice recorder.

Federal Aviation Administration (2026). 25-Hour Cockpit Voice Recorder (CVR) Requirement, New Aircraft Production. Final rule, Federal Register 91(21), 2 February 2026, pp. 4447–4462. Federal Register text. The thirty-minute recorders of 1966, the two-hour rule, and the 25-hour standard of ICAO (2016) and EASA (from 2021).

Richard FitzNigel (c. 1180). Dialogus de Scaccario (The Dialogue of the Exchequer). Text and translation ed. Charles Johnson, Nelson's Medieval Classics, 1950. The widths of the cuts on Exchequer tallies.

Fox-IT (2012). Black Tulip: Report of the investigation into the DigiNotar Certificate Authority breach. Version 1.0. Report.

Tom Furtwangler (2017). The vaccine vial monitor: the world's smartest sticker. PATH, 6 November 2017. Article. The stickers first sold in 1996, for oral polio vaccine.

General Conference on Weights and Measures (2018). Resolution 1 of the 26th meeting: On the revision of the International System of Units (SI), effective 20 May 2019.

C. H. Jenkinson (1910). Early wooden tallies relating to Surrey. Surrey Archaeological Collections 23, pp. 203–207. PDF. Hazel; the payer kept the stock, the Exchequer the foil; abolished by statute in 1826; the burning that started the fire.

Joint Committee for Guides in Metrology (2012). International vocabulary of metrology: basic and general concepts and associated terms (VIM), 3rd edition, JCGM 200:2012, entry 2.41, "metrological traceability." The definition is on p. 29; "does not ensure . . . that there is an absence of mistakes" is Note 5 to 2.41, p. 30. BIPM text.

Mikhail Kuter, Marina Gurskaya and Albert Gurskii (2025). Genoa 1340, Double Entry, and 'Gold Coasting'. De Computis, Revista Española de Historia de la Contabilidad 22(1), pp. 1–33. DOI: 10.26784/issn.1886-1881.22.1.11927.

Klaus-Jörn Lange, Pierre McKenzie and Alain Tapp (2000). Reversible Space Equals Deterministic Space. Journal of Computer and System Sciences 60(2), pp. 354–367. DOI: 10.1006/jcss.1999.1672.

Ben Laurie, Adam Langley and Emilia Kasper (2013). Certificate Transparency. RFC 6962. RFC 6962; DOI: 10.17487/RFC6962.

Ben Laurie, Eran Messeri and Rob Stradling (2021). Certificate Transparency Version 2.0. RFC 9162. RFC 9162; DOI: 10.17487/RFC9162.

A. Liapounoff (1907). Problème général de la stabilité du mouvement, traduit du russe par Édouard Davaux. Annales de la Faculté des Sciences de Toulouse, 2e série, 9, pp. 203–474. Numdam. The French translation the 1992 English one was made from.

Sinisa Matetic, Mansoor Ahmed, Kari Kostiainen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels and Srdjan Capkun (2017). ROTE: rollback protection for trusted execution. Proceedings of the 26th USENIX Security Symposium, pp. 1289–1306. USENIX page.

Clyde Lee Miller (2009; revised 2021). Cusanus, Nicolaus [Nicolas of Cusa]. Stanford Encyclopedia of Philosophy. SEP entry.

Yûto Murashita, Ken Funo and Masahito Ueda (2014). Nonequilibrium equalities in absolutely irreversible processes. Physical Review E 90, 042110. DOI: 10.1103/PhysRevE.90.042110. The bound on average entropy production by $-\ln(1-\lambda)$, with free expansion as the first example.

Satoshi Nakamoto (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. Paper.

Devon O'Brien (2018). Certificate Transparency Enforcement in Google Chrome. Chromium ct-policy mailing list, 6 February 2018. Announcement.

Garson O'Toole (2016). There Ain't No Such Thing as a Free Lunch, TANSTAAFL. Quote Investigator, 27 August 2016. Article.

Pavel Pakshin, Elena Panteley and Natalia Pakshina (2017). The Legacy of A. M. Lyapunov to Systems and Control Theory: 110 years of the Toulouse edition. IFAC World Congress 2017, open-track proposal. Davaux's translation, reviewed and corrected by Lyapunov.

Lídia del Rio, Johan Åberg, Renato Renner, Oscar Dahlsten and Vlatko Vedral (2011). The thermodynamic meaning of negative entropy. Nature 474, pp. 61–63. DOI: 10.1038/nature10123; arXiv: 1009.1630.

Jürgen Schnakenberg (1976). Network theory of microscopic and macroscopic behavior of master equation systems. Reviews of Modern Physics 48, pp. 571–585. DOI: 10.1103/RevModPhys.48.571. The entropy a Markov chain hands its surroundings, as the net rate across each pair times the log of the forward-to-backward rate ratio, which at a steady state is the whole entropy production; the biased walk is its smallest case.

Securities and Exchange Commission (2022). Electronic Recordkeeping Requirements for Broker-Dealers, Security-Based Swap Dealers, and Major Security-Based Swap Participants. Release No. 34-96034, 12 October 2022. Final rule. Adds an audit-trail alternative to the write-once requirement of Rule 17a-4(f) in place since 1997.

Caroline Shenton (2012). The Day Parliament Burned Down. Oxford University Press. The tallies burned in the furnaces under the House of Lords, 16 October 1834.

Michael Stock, Pauline Barat, Richard S. Davis, Alain Picard and Martin J. T. Milton (2015). Calibration campaign against the international prototype of the kilogram in anticipation of the redefinition of the kilogram part I: comparison of the international prototype with its official copies. Metrologia 52(2), pp. 310–316. DOI: 10.1088/0026-1394/52/2/310.

Wei Tang (2019). EIP-2200: Structured Definitions for Net Gas Metering. Ethereum Improvement Proposals. eips.ethereum.org/EIPS/eip-2200.

This Day in Aviation (n.d.). 30 October 1935: Boeing Model 299 crash at Wright Field. Page.

Kent Weare (2016). Ethereum Security Alert Issued, Ethereum Foundation Responds with “From Shanghai, With Love”. InfoQ, 19 September 2016. Article.

World Health Organization (2011). PQS performance specification: Vaccine Vial Monitor. WHO/PQS/E06/IN05.2.

31.3 Computation, self-reference and checking

Keith Adams and Ole Agesen (2006). A Comparison of Software and Hardware Techniques for x86 Virtualization. Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII), pp. 2–13. DOI: 10.1145/1168857.1168860. Binary translation of instructions such as POPF, which on x86 runs in user mode without trapping. Intel’s VT-x and AMD-V in 2005 and 2006: Wikipedia, x86 virtualization.

American Standards Association (1963). American Standard Code for Information Interchange, ASA X3.4-1963.

Henk Barendregt. Wikipedia. Article. A Dutch logician known for his work in lambda calculus and type theory.

Gerd Behrmann, Ansgar Fehnker, Thomas Hune, Kim Larsen, Paul Pettersson, Judi Romijn and Frits Vaandrager (2001). Minimum-cost reachability for priced timed

automata. Hybrid Systems: Computation and Control (HSCC 2001), Lecture Notes in Computer Science 2034, pp. 147–161.

N. G. de Bruijn (1972). Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem. Indagationes Mathematicae (Proceedings) 75(5), pp. 381–392. DOI: 10.1016/1385-7258(72)90034-0. Variables written as numbers that count the binders back, now called de Bruijn indices.

Stanley Burris and Marcel Jackson (2010, substantive revision 2026). George Boole. Stanford Encyclopedia of Philosophy. Entry.

Stanley Burris and Javier Legris (2009, substantive revision 2021). The Algebra of Logic Tradition. Stanford Encyclopedia of Philosophy. Entry.

G. K. Chesterton. Wikipedia. Article. An English writer, referred to as the “prince of paradox”.

Donald W. Davies (2004). Corrections to Turing’s Universal Computing Machine. In B. Jack Copeland (ed.), The Essential Turing, pp. 103–124. Oxford University Press. Davies at the National Physical Laboratory from 1947 and the word *packet*: Wikipedia, Donald Davies.

Charles Dickens (1843). A Christmas Carol. Chapman and Hall, London. Stave Three, the game of Yes and No.

William Henry Eccles and Frank Wilfred Jordan (1920). Improvements in ionic relays. British patent GB148582A, filed 21 June 1918, published 5 August 1920. Patent text. The 1918 date in the text is the filing date. The circuit as the flip-flop, the basis for electronic memory: History of Information, entry; and Wikipedia, Flip-flop (electronics), on the Eccles–Jordan trigger circuit as the first electronic latch, later the basic storage element of digital circuitry.

László Fejes Tóth (1953). Lagerungen in der Ebene, auf der Kugel und im Raum. Berlin: Springer.

László Fejes Tóth (1964). Regular Figures. Oxford: Pergamon Press.

Jean-Christophe Filliâtre, Léon Gondelman and Andrei Paskevich (2014). The spirit of ghost code. Computer Aided Verification (CAV 2014), Lecture Notes in Computer Science 8559, pp. 1–16. DOI: 10.1007/978-3-319-08867-9_1.

Nicholas Griffin, ed. (2003). The Cambridge Companion to Bertrand Russell. Cambridge University Press. G. G. Berry as a junior librarian at the Bodleian, p. 63.

Jean-Pierre Marquis (1996, substantive revision 2019). Category Theory. Stanford Encyclopedia of Philosophy. Entry. The borrowed words “category” and “functor”.

Ross M. McConnell, Kurt Mehlhorn, Stefan Näher and Pascal Schweitzer (2011). Certifying algorithms. Computer Science Review 5(2), pp. 119–161. DOI: 10.1016/j.cosrev.2010.09.009.

Jason Megill. The Lucas-Penrose Argument about Gödel's Theorem. Internet Encyclopedia of Philosophy. Article.

Glen Mével, Jacques-Henri Jourdan and François Pottier (2019). Time credits and time receipts in Iris. ESOP 2019, LNCS 11423, pp. 3–29. DOI: 10.1007/978-3-030-17184-1_1.

Jeff Miller. Earliest Uses of Symbols of Set Theory and Logic. MacTutor History of Mathematics. Page. Peano's epsilon for *est*, 1889.

J. J. O'Connor and E. F. Robertson (n.d.). The four colour theorem. MacTutor History of Mathematics. Entry. (De Morgan's letter to Hamilton of 23 October 1852 is quoted from here.)

Panu Raatikainen (2013, revised 2025). Gödel's Incompleteness Theorems. Stanford Encyclopedia of Philosophy. Entry. Source for the cautions about what the Gödel sentence says.

Erich Reck (2008, substantive revision 2025). Dedekind's Contributions to the Foundations of Mathematics. Stanford Encyclopedia of Philosophy. Entry. Peano acknowledged Dedekind's priority for the axioms.

Neil Robertson, Daniel Sanders, Paul Seymour and Robin Thomas (1997). The four-colour theorem. Journal of Combinatorial Theory, Series B 70(1), pp. 2–44.

Jürgen Schmidhuber (2003). Gödel machines: self-referential universal problem solvers making provably optimal self-improvements. Technical Report IDSIA-19-03. arXiv: cs/0309048.

Marcel-Paul Schützenberger (1961). On the definition of a family of automata. Information and Control 4(2–3), pp. 245–270.

Dana Scott (1962). A system of functional abstraction. Lectures delivered at the University of California, Berkeley, 1962–63; unpublished. The origin of the Scott encoding, as given in Mogensen–Scott encoding, Wikipedia, Article.

Dana Scott. Carnegie Mellon University's School of Computer Science, faculty Turing Award recipients: the 1976 award jointly with Michael Rabin for their work on nondeterministic machines. His doctorate at Princeton under Alonzo Church, domain theory and models of the lambda calculus: Wikipedia, Dana Scott.

Emin Gün Sirer, Willem de Bruijn, Patrick Reynolds, Alan Shieh, Kevin Walsh, Dan Williams and Fred B. Schneider (2011). Logical attestation: an authoriza-

tion architecture for trustworthy computing. SOSP 2011, pp. 249–264. DOI: 10.1145/2043556.2043580.

Ken Thompson. Heidelberg Laureate Forum, laureate profile: the 1983 Turing Award with Dennis Ritchie for operating systems theory and the implementation of Unix. Thompson's design and implementation of the original Unix at Bell Labs: Wikipedia, Ken Thompson.

Alfred North Whitehead and Bertrand Russell (1910). Principia Mathematica, vol. I. Cambridge University Press. Introduction, chapter II.

Robin Wilson (2002). Four Colours Suffice? Gresham College lecture, 21 October 2002. Lecture. De Morgan's letter of 23 October 1852 to the Irish mathematician William Rowan Hamilton.

31.4 Correlations and environmental records

M.-C. Chen et al. (2019). Emergence of classical objectivity of quantum Darwinism in a photonic quantum simulator. Science Bulletin 64, p. 580.

M. A. Ciampini, G. Pinna, P. Mataloni and M. Paternostro (2018). Experimental signature of quantum Darwinism in photonic cluster states. Physical Review A 98, 020101.

Mauro Dorato and Jan Faye (2026). Niels Bohr as a Physicist of Principle. arXiv: 2609.04297. Quotes Bohr's Unity of Knowledge (1954) on irreversible amplification and permanent marks on a photographic plate, citing p. 73.

Lluís Masanes (2003). Necessary and sufficient condition for quantum-generated correlations. arXiv: quant-ph/0309137v1. The paper gives his affiliation as the Universitat de Barcelona.

Miguel Navascués, Stefano Pironio and Antonio Acín (2007). Bounding the Set of Quantum Correlations. Physical Review Letters 98, 010401. DOI: 10.1103/PhysRevLett.98.010401; arXiv: quant-ph/0607119v2.

Miguel Navascués, Stefano Pironio and Antonio Acín (2008). A convergent hierarchy of semidefinite programs characterizing the set of quantum correlations. New Journal of Physics 10, 073013. arXiv: 0803.4290v1.

Marcin Pawłowski, Tomasz Paterek, Dagomir Kaszlikowski, Valerio Scarani, Andreas Winter and Marek Żukowski (2009). Information causality as a physical principle. Nature 461, pp. 1101–1104. Pawłowski's affiliation, the University of Gdańsk, is on the paper: arXiv: 0905.2292.

Erwin Schrödinger. Wikipedia. Article. Austrian physicist; the wave equation of 1926.

B. S. Tsirel'son (1987). Quantum analogues of the Bell inequalities. The case of two spatially separated domains. Journal of Soviet Mathematics 36(4), pp. 557–570. DOI: 10.1007/BF01663472.

T. Unden, D. Louzon, M. Zwolak, W. H. Zurek and F. Jelezko (2019). Revealing the emergence of classicality using nitrogen-vacancy centers. Physical Review Letters 123, 140402.

COLOPHON

This book is set in Charter, drawn by Matthew Carter in 1987 for the laser printers of its day, 10.5 on 14 point, on a page 189 by 246 millimetres. It was built from its source with pdfTeX. The cells are drawn in TikZ.

Its own count, taken by the build: 116 numbered statements, of which 73 are closed by a proof on the page, and the rest stated and not proved: Theorem 16.3, Theorem 16.4, Theorem 16.5, Theorem 16.6, Theorem 16.8, Theorem 16.9, Theorem 16.10, Theorem 16.11, Theorem 16.13, Theorem 16.14, Theorem 16.15, Theorem 16.16, Proposition 16.17, Theorem 16.18, Proposition 16.19, Theorem 16.20, Theorem 16.21, Theorem 21.1, Theorem 21.3, Theorem 21.4, Theorem 21.5, Theorem 21.6, Theorem 21.7, Theorem 21.8, Theorem 21.9, Theorem 21.10, Theorem 21.11, Theorem 21.12, Theorem 21.13, Theorem 21.14, Theorem 21.15, Theorem 23.1, Theorem 25.1, Theorem 25.2, Lemma 25.3, Theorem 25.4, Theorem 25.5, Lemma 25.6, Corollary 25.7, Theorem 25.8, Lemma 25.9, Theorem 25.10, Theorem 25.12, Theorem 25.13. One cell for each, in the order they come, filled where the proof closed:

